CAS Core 8: A Compatibility and Evidence Kernel for Auditable Cognitive-Architecture Software

Little Henderson

Research monograph v0.3, 2026-08. Artifact status: DRAFT / OPEN / RING 4 / NOT LICENSED / HELD. This monograph expands the corresponding v0.2 research paper over the same Project Coherence Gate 1 / Gate 4 source artifacts; every status label, proof status, hash identity, and open obligation of those sources governs this text. Drafting this monograph promotes no claim; independent review remains open. Drafting was assisted by large-language-model tools; all content was reviewed by the author, who is solely responsible for it.

Contents

Chapter 1. Introduction 1
1. A kernel that cannot be talked into overclaiming 1
2. The first failure class: semantic drift 2
3. The second failure class: unauthorized claim promotion 5
4. The design-inversion thesis 8
5. Scope and non-goals 11
6. Objections and replies 12
7. Reading guide 13

Chapter 2. Background: Integrity, Canonicalization, Provenance, and Authorization 15
1. Cryptographic hashing and content addressing 15
2. Canonical serialization 18
3. Reproducible builds and supply-chain integrity 22
4. Provenance: entities, activities, derivations 24
5. Integrity versus authorization 25
6. Threat modeling 27
7. Process isolation versus sandboxing 28
8. Synthesis: from background to kernel 30

Chapter 3. Semantic Identity 33
1. The five-field structural reference 33
2. Definition-hash binding and definitional drift 35
3. The namespace catalog 36
4. Link kinds: relating without identifying 38
5. The injective-rendering invariant 40
6. The exact-type equality rule 41
7. The validation pipeline 43
8. How the layer composes with the rest of the kernel 43
9. Worked examples 44
10. What the identity layer does not claim 45

Chapter 4. The Research Boundary Object 47
1. The problem the boundary object answers 47
2. The boundary triple and its carrier 47
3. Joint consistency and construction-time rejection 49
4. The recheck-not-trust invariant 51
5. Status change without mutation 53
6. Tamper evidence versus authority 54
7. How boundaries thread through the kernel 55
8. Worked misuse scenarios 56

9. What the boundary object does not do 58

Chapter 5. Canonical Evidence Encoding 59
1. Why digests are load-bearing 59
2. The threat model for encodings 60
3. The rule set 60
4. Deterministic UTF-8 JSON with sorted keys 61
5. The rejection set: what cannot be evidence 63
6. Tagged exact values 64
7. One serialization route per public type 66
8. Bounded, cycle-aware canonicalization 67
9. Byte identity versus comparison identity 68
10. Identifier profiles 69
11. Comparison with RFC 8785 (JCS) 70
12. A failure-mode gallery 72
13. Placement in the evidence pipeline 74
14. Testing discipline 75
15. Summary and boundaries 75

Chapter 6. Registry Integrity 77
1. Registries as the kernel's memory 77
2. Exact-byte backing 77
3. The reconstruct-and-recheck loading rule 78
4. The rejection catalog 81
5. Recursive immutability 84
6. Per-record digests bound to registry bytes 85
7. Two registries, one discipline 86
8. Metadata is not authority 86
9. Limits and obligations 88

Chapter 7. The Legacy Compiler Compatibility Protocol 89
1. The profile and its refusal 89
2. Step 1: source-hash verification and pre-execution compiler-byte hashing 90
3. Step 2: the forced Python-only engine profile and the frozen policy 91
4. Step 3: rejection of `use` imports 92
5. Step 4: the fresh `python -I -S -B` worker and the sanitized environment 93
6. Step 5: pre-execution runtime binding without probing 95
7. Interlude: the worker wire protocol 97
8. Interlude: freezing the legacy artifact 98
9. Step 6: advisory-lock serialization of worker launches 98
10. Step 7: the receipt 99
11. Step 8: fresh replay before any structural decision 101
12. The isolation boundary: what this protocol is not 102
13. Reading a receipt: a reviewer's walkthrough 102
14. The protocol as a whole 104

Chapter 8. Concurrency and the Failure Taxonomy 105
1. Failures are evidence too 105
2. The advisory lock discipline 105

3. The no-retry principle 108
4. Worker transport failure records 110
5. Dependency-profile disagreement 111
6. The fractal bridge rule 113
7. The failure codes, collected 115
8. The taxonomy as a designed surface 116

Chapter 9. The Runtime Decision Model 117
1. The last layer 117
2. Typed actions: the end of stringly-typed intent 117
3. The structural replay comparison 119
4. The two constant-false invariants 122
5. The future-authorization design constraints 125
6. The denial vocabulary, collected 127
7. Coda: the keystone property 128

Chapter 10. Threat Model 130
1. Method and framing 130
2. Assets and trust boundaries 131
3. T1: Namespace conflation 131
4. T2: Identifier collision via delimiter injection 132
5. T3: Equality spoofing via subclass override 133
6. T4: Digest instability and type puns 134
7. T5: Newline rewriting 135
8. T6: Compiler-identity drift 135
9. T7: Receipt laundering 136
10. T8: Registry substitution 137
11. T9: Replay evasion 138
12. T10: Lock-based denial 139
13. T11: Dependency-profile divergence between workers 140
14. Threat interactions and compound scenarios 141
15. Coverage summary and severity assessment 142
16. What the model does not cover 143

Chapter 11. Integration Contracts with the Companion Candidates 145
1. The dependency discipline 145
2. CAS PCBA 8 148
3. CAS-Ohm 1.0 149
4. TKS v8.0 151
5. Noetic Topos 2.0 153
6. Companion conformance obligations 154
7. Cross-companion coherence 155

Chapter 12. Evidence, Obligations, and Program 156
1. Reading a bounded receipt 156
2. The bounded Gate-4 receipt, exactly 156
3. What the suite exercises 159
4. The six open obligations 161
5. Program: sequencing the obligations 165

6. Related work 166
7. Conclusion: the design inversion 167

Appendix A. Reference Tables 169
1. Semantic-reference fields 169
2. Boundary fields and valid combinations 170
3. Canonical-encoding rules 172
4. Compatibility-receipt fields 173
5. Worker-failure taxonomy 174
6. Registry-loader rejection catalog 174
7. Decision-model enumerations 175

Appendix B. Glossary 176

Bibliography 182

CHAPTER 1

Introduction

1. A kernel that cannot be talked into overclaiming

This monograph is about a small piece of software with an unusual design goal. CAS Core 8 is a compatibility and evidence kernel for the Cognitive Architecture System (CAS) software family. It does not score anything, prove anything about cognition, or translate anything into new semantics. What it does is narrower and, we will argue, prior to all of those ambitions: it makes source identity, semantic namespace, compatibility profile, registry integrity, action intent, and claim boundaries explicit enough to test, and it does so in a way that is deliberately hostile to its own tendency—and the tendency of the surrounding research program—to overclaim.

The phrase "cannot be talked into overclaiming" deserves unpacking, because it names the central engineering stance of the entire kernel. Research software accumulates claims the way sediment accumulates in a river delta: each individual deposit is small, locally reasonable, and apparently harmless. A test passes, and someone writes "verified" in a status file. A hash matches, and someone writes "authentic." A generated report contains the word "licensed," and a later reader—human or automated—treats the word as if it carried the force of the review process that the word is supposed to summarize. None of these steps is a lie. Each is a small promotion of evidence into a stronger register than the evidence supports, and the composition of many small promotions is a system whose stated properties nobody can actually defend.

The TKS/CAS research program from which CAS Core 8 emerged has direct, documented experience of this dynamic, in two specific and recurring forms. The program's own audit history—not a hypothetical threat model, but its record of voided work products and withdrawn claims—identifies two failure classes that no amount of downstream testing repairs, because both failures occur *upstream* of any test: they corrupt what the tests are about. This chapter develops both failure classes as extended narratives (Sections 2 and 3), states the design thesis that answers them (Section 4), fixes the scope and non-goals of the kernel and of this monograph (Section 5), and closes with a reading guide (Section 7).

A word on status before anything else, because the kernel's own discipline requires it of the prose that describes it. Everything discussed in this monograph is a Ring-4 research candidate: admission state OPEN, product state NOT LICENSED, public disclosure withheld. The bounded software receipts we will cite in later chapters are historical evidence about tested software behavior under exact fixtures; they are not experiments, not independent review, and not authority. The monograph promotes no claim. Where a property of the kernel has been exercised by its test suite we will say so; where a property is a design intention that has not been independently reviewed we will say that instead; and where the governing sources are silent we will say that the sources are silent rather than inventing a fact to fill the gap.

2. The first failure class: semantic drift

2.1. An ecology of ontologies. The research program in which CAS Core 8 lives is unusually dense in ontologies. This is not an accident of sloppy design; it is a structural consequence of the program's history. The program spans, at minimum, the following families of named semantic objects, each with its own authority, its own versioning history, and its own definitional texts.

- *TKS Foundations.* A family of seven positions in the TKS source material, indexed positionally, with definitional texts fixed in the source corpus. The Foundations cross with a second axis to yield a larger derived family; the details belong to the TKS corpus itself and are not restated here.
- *TKS Worlds.* A four-element axis in the same source material, crossing with Foundations and with the Noetic family to produce derived grids. The Worlds are positional, ordered, and defined by frozen source text.
- *CAS Drives.* Seven motivational primitives of the Cognitive Architecture System, named F1 through F7, with their own definitional texts in the CAS corpus, their own inversion-pair structure, and their own crossing with CAS Domains.
- *CAS Domains.* Four processing tiers of CAS, with a total precedence order, defined in the CAS corpus.
- *CAS Registers.* Four operational-character values introduced in a later CAS revision as an orthogonal axis over existing primitives.
- *CAS States.* The forty cognitive states obtained by crossing the ten CAS Noetics with the four Domains.
- *Noetic Topos objects.* Objects of the categorical formulation of CAS, which live in a mathematical setting with its own identity discipline and are related to, but not the same things as, the operator and state families above.

The crucial fact about this ecology is that its families are related by *history* and by *position*, but not by identity. The CAS Drives were developed with explicit awareness of the TKS Foundations; there is a positional correspondence between the two seven-element families, and that correspondence is analytically important. The CAS Domains stand in a similar positional relation to the TKS Worlds. The CAS Register axis was introduced precisely because a four-valued axis present in the TKS cross-products had been lost in an earlier translation and was restored in secular terms. In every one of these cases the relationship is real, documented, and load-bearing for the program's own understanding of itself—and in every one of these cases the related elements are *different semantic objects*, defined by different texts, carrying different entailments, governed by different canons, and versioned on different schedules.

Two entries in the list deserve a closing note each, because they mark the edges of the ecology. The Noetic Topos objects are the odd family out: they live in a mathematical formulation, where identity is already disciplined—objects are individuated by the category they live in, and nobody familiar with the formalism would conflate an object with the informal operator it models. One might conclude that this family, at least, needs no protection. The conclusion is exactly backwards, and the reason illuminates the whole design. The mathematical formulation is disciplined *inside mathematics*; the drift risk arises at the crossing, where prose, code comments, dashboards, and generated summaries refer to "the operator" and "the object" interchangeably because the informal register has no way to mark the difference. The formal family needs a namespace not to protect it from mathematicians but to protect the boundary between registers, which is where every other family's incidents have

also occurred. Conversely, the Register family marks the ecology's youngest edge: an axis restored in a recent revision after having been lost in translation from the source material. Its recency means comparatively few artifacts refer to it yet—which makes it the family for which getting identity right *now* is cheapest, and the clearest illustration of the timing argument made later in this chapter.

A research program with one ontology can be sloppy about identity and survive, because there is nothing to conflate. A program with seven or more interlocking families, several of them positionally aligned seven-element or four-element sets, is a machine for producing conflation. The alignment that makes the families intellectually productive is exactly the alignment that makes them dangerous: whenever two families have the same cardinality and a meaningful positional correspondence, every artifact that names an element of one family is one careless step away from being read as naming the corresponding element of the other.

2.2. Positional relabels are not paraphrases. The distinction the program's canon draws—and the distinction whose violation has actually voided work—is between a *positional relabel* and a *paraphrase*. A paraphrase restates the same semantic object in different words: the object's identity survives, and any entailment licensed by the original definition is licensed by the paraphrase. A positional relabel maps an element of one family to the same-indexed element of another family: the index survives, but the identity does not, and entailments do *not* transfer. The fifth TKS Foundation and the CAS Drive F5 occupy corresponding positions in aligned seven-element families. Statements true of one are not thereby true of the other. The definitional text of one is not the definitional text of the other. A derivation that consumes the definition of one and silently emits conclusions about the other has not paraphrased; it has changed the subject while keeping the index.

Adverse Finding 1.1 (The Drive/Foundation conflation incident). A prior work product in the program treated positionally corresponding Drive and Foundation names as interchangeable—as paraphrases of one another rather than as positional relabels across distinct families. The conflation was caught in review, and the work product was voided. The incident is recorded in the program's canon precisely because nothing in the artifacts involved had resisted the conflation: the names were plain strings, the strings looked alike, and every tool that touched them was happy to treat equal-looking strings as equal things.

Finding 1.1 is the single most important historical datum behind the kernel's semantic-identity mechanism, and *why* the conflation was so easy is the instructive question. The failure was not a failure of knowledge. The people and tools involved had access to canon documents stating explicitly that the correspondence was positional. The failure was a failure of *representation*: the artifacts in which the work was carried out had no way to say, in a machine-checkable form, "this token names a TKS Foundation, under this version of this source, with this defining text" as opposed to "this token names a CAS Drive, under that version of that source, with that defining text." Where the representation cannot express a distinction, the distinction survives only as long as every reader's discipline does—and across enough readers, enough sessions, and enough automated rewrites, some reader's discipline always fails.

2.3. How untyped strings hide drift. Consider the string `"F5"`. Within the program's corpora this three-character token can plausibly name at least three different things: a TKS Foundation in positional notation, a CAS Drive, or an incidental label in some third context (a spreadsheet column, a keyboard key, a paper's figure numbering). Nothing in the string distinguishes these readings. String equality—the only identity test an untyped

representation supports—answers "are these the same characters?" when the question that matters is "are these the same semantic object?" The two questions have different answers in both directions:

- *Equal strings, different objects.* `"F5"` as Foundation and `"F5"` as Drive are string-equal and semantically distinct. An untyped pipeline will merge them; every merge is a silent conflation of the kind in Finding 1.1.
- *Different strings, same object.* The same Drive may appear as `"F5"`, `"Drive 5"`, `"Agency"`, or a renamed successor in a later revision. An untyped pipeline will treat these as unrelated; provenance fragments, and cross-references quietly dangle.

Untyped strings hide a second, subtler kind of drift as well: *definitional* drift. Even when a token unambiguously names one family and one element, the defining text of that element can change between source versions—a canon correction, an expanded scope note, a translation fix. An artifact produced against the old definition and an artifact produced against the new one will use the same token in good faith and mean measurably different things. String identity is blind to this; so is any identity scheme that names concepts without binding the bytes that define them. The program has direct experience of definitional drift too: its canon documents exist in versioned editions precisely because definitions have been corrected, and its own rules require rechecking derived artifacts when a governing definition changes. A representation in which references do not carry the digest of their defining text makes that recheck obligation undiscoverable: nothing in the artifact records which definition it was built against, so nobody can enumerate what a definitional change invalidates.

Finally, untyped strings hide drift *socially.* A reviewer confronted with a typed reference—namespace, identifier, version, source, definition digest—can check each field against the canon. A reviewer confronted with `"F5"` can only check that the surrounding prose seems right, which means the review inherits every ambiguity of the prose. Drift that a typed representation would surface as a red field in a diff instead surfaces, if at all, as a subtle wrongness in paragraph seventeen of a document nobody rereads. The program's audit history says this is not a theoretical concern: prior audits found positional relabels treated as paraphrases and cross-namespace conflations voiding otherwise sound work products, and in each case the conflation had passed through multiple hands before review caught it.

2.4. A taxonomy of drift. It is useful, before leaving the first failure class, to separate the distinct mechanisms that the single word "drift" covers, because the kernel's semantic-identity design answers each of them with a different field of its reference tuple, and the mapping is easiest to see when the mechanisms are named individually.

Token conflation is the crudest form: two different semantic objects share a surface token, and a pipeline that keys on tokens merges them. The `"F5"` example above is token conflation waiting to happen. Its signature is silent merging—counts that are too high, joins that succeed when they should have failed—and it is answered by namespacing: when every reference carries its family as a typed field, equal tokens in different families simply are not equal.

Positional relabel drift is subtler: two families are genuinely aligned position-by-position, the alignment is analytically real, and an artifact slides from using the alignment to presupposing identity. This is the mechanism of Finding 1.1. Its signature is entailment leakage—properties licensed by one family's definitions appearing in conclusions about the other—and it is answered not by any single field but by the link-kind rule: the alignment is representable only as an explicit, typed, non-identity link, so every use of the correspondence is a visible act with a recorded justification.

Definitional drift occurs within a single family: the defining text of an element changes between editions of the canon, and artifacts built against different editions diverge while using identical names. Its signature is irreproducibility that appears only across time—two careful analyses of "the same" element disagreeing because they read different definitional bytes—and it is answered by binding the digest of the exact defining text into the reference, so that a definitional change changes the reference and the divergence becomes a visible inequality instead of a latent ambiguity.

Source and version drift are the bookkeeping forms: the same element, under the same definition, cited from different corpora or different editions, with downstream tooling unable to tell whether the citations corroborate each other or merely repeat each other. They are answered by the version and source-identifier fields, which make the provenance of a reference part of its identity rather than a comment beside it.

Rendering collision, finally, is drift introduced by the representation itself: a flattening of a structured reference into a string such that two distinct references flatten to the same string. This is the one form of drift that a naive structured design can *create* while solving the others—a reference tuple joined with an unescaped delimiter is a collision generator—and it is answered by the injective-rendering invariant, which requires that the string form be uniquely parseable back to the tuple. The semantic-identity chapter gives the design argument and the regression fixtures.

Five mechanisms, five answers, one tuple. The taxonomy also explains why the kernel treats identity as a five-field structure rather than adopting any single-field fix: each field earns its place against a named drift mechanism. Two of the five are documented in the program's audit record—token conflation and positional-relabel drift, jointly instantiated by Finding 1.1—and definitional drift is attested by the canon's own corrected editions. Source and version drift and rendering collision are, on the record the sources supply, anticipated rather than observed: the first is the bookkeeping exposure those corrections create, and the second cannot yet have occurred, since the program's prior artifacts were untyped strings with no structured rendering to collide. A representation that answers fewer than all five leaves the unanswered mechanisms as the new path of least resistance.

The design conclusion the kernel draws is stated here informally and developed fully in the semantic-identity chapter: identity must be *structural* (a typed tuple, not a string), *namespaced* (the families are disjoint by construction), *versioned and source-bound* (which edition of which corpus), and *definition-bound* (the digest of the exact defining bytes travels with the reference). And cross-family correspondences—the positional alignments that make the ecology productive—must be representable only as explicitly typed, non-identity links, so that using them costs a deliberate act instead of a careless one.

3. The second failure class: unauthorized claim promotion

3.1. Structural receipts and what they actually establish. The second failure class is orthogonal to the first. Semantic drift corrupts *what artifacts mean*; unauthorized claim promotion corrupts *what artifacts are entitled to assert*. Its raw material is the structural receipt: any machine-generated artifact that records a fact about software behavior. A passing test run is a structural receipt. A SHA-256 digest of a file is a structural receipt. A generated report listing sixty-one passing test methods, a replay comparison that came back equal, a registry snapshot whose loader accepted it—all structural receipts.

Structural receipts establish real things, and the kernel's design takes them seriously; most of this monograph is about making them exact. What a structural receipt establishes

is a conditional, historical fact of the form: *under these fixtures, these bytes, this interpreter, and this profile, this behavior was observed at this time.* That is genuine evidence, and for some questions—did the encoder reject NaN? did the loader reject the duplicate key?—it is exactly the evidence needed.

What no structural receipt establishes is any of the following: that a qualified reviewer examined the work; that a legal or organizational authority accepted a claim; that an empirical proposition about the world (as opposed to about the software's behavior on fixtures) is true; that a product may be shipped, disclosed, or licensed. These are facts about people, institutions, and review processes. They live outside the software, and no computation performed *by* the software can conjure them, because the software has no access to the trust roots—authenticated reviewer identities, legal authority, admission processes—in which such facts are grounded. A receipt can be perfectly authentic as a record of behavior and perfectly void as an authorization, in the same way that a photograph of a contract is evidence the paper existed and no signature at all.

Claim promotion is what happens when this boundary erodes: when a structural receipt is read, cited, or mechanically consumed as if it were an authorization. The erosion is rarely a single dramatic forgery. It is a gradient of small, individually defensible promotions—"the suite is green" becomes "the module is verified" becomes "the mechanism is sound" becomes "the claim is established"—each step performed by a different artifact, a different summary, a different reader, until the final assertion is separated from its evidential basis by half a dozen paraphrases and nobody can say where the authority was supposed to have entered. In a program that uses AI agents as producers of both code and prose, the gradient steepens: an agent that generates ten thousand words of confident summary around one green test run manufactures promotion surface faster than any human review process consumes it.

3.2. The self-hashed licensing receipts. The program's history contains a sharp, concrete instance of this failure class, and the kernel's design treats it the way the semantic-identity mechanism treats the Drive/Foundation incident: as the canonical exhibit.

Adverse Finding 1.2 (The self-hashed licensing receipts). At an earlier point in the program's history, AI-authored artifacts were produced that had the *form* of licensing receipts: they asserted a licensed or approved status, and they carried hashes computed over their own content, lending them an air of cryptographic solidity. Every element of these artifacts was self-supplied. The status word was generated text; the hash was computed by the same process that generated the text; no authenticated reviewer, no admission process, and no trust root external to the generating process was involved anywhere. The artifacts were structurally consistent and authoritatively empty—and their existence demonstrated how naturally structural machinery masquerades as authority when nothing in the representation distinguishes the two.

The self-hashed receipt is the purest possible specimen of the failure class. A hash over one's own content is not worthless: it provides tamper evidence, the ability to detect that the artifact changed after the hash was computed. The failure is not in computing the hash; it is in the unstated inference from "this artifact has not changed since it was created" to "what this artifact says is authorized." Tamper evidence answers *has this changed?*; authority answers *who stands behind this?* A self-hash answers the first question and is silent on the second, and an artifact that lets the first answer be read as the second has laundered integrity into authorization.

The general lesson, which the kernel elevates to its central invariant, is that *structural consistency is never authorization.* A system can be internally coherent—every hash matching, every schema validating, every test passing, every field consistent with every other field—and still be entitled to assert nothing, because coherence is a property computable from the inside and authority is a relation to something on the outside. Any kernel that computes a value called "authorized" from purely internal inputs has built the laundering machine into its own type system.

3.3. Anatomy of a promotion gradient. Because the laundering dynamic is easiest to dismiss in the abstract ("surely nobody confuses a checksum with a license"), it is worth walking the gradient step by step, in the schematic form the program's history instantiates. No single step below is dishonest; that is the point.

Step one: the receipt. A test suite runs; sixty-one methods pass; the run is recorded with digests of the suite's test identifiers and the artifacts under test. This is exact, bounded, and true. The receipt says: these tests, over these bytes, passed on this date.

Step two: the summary. A status document is written—by a human, or increasingly by an AI agent asked to "summarize progress." The document says the module is "fully tested and verified." The word "verified" has silently widened: the receipt verified behavior on fixtures; the summary implies verification of the module as such. The fixture-bound conditional has been dropped, because conditionals make for poor status documents.

Step three: the aggregation. A higher-level report aggregates many summaries. "All components verified" becomes "the system is sound." At this altitude nobody is looking at receipts; the inputs are summaries whose promotions are inherited and compounded. If any summary overstated, the aggregation now overstates transitively, with no marker of where the overstatement entered.

Step four: the status word. Someone—or some process—needs a machine-readable status, and writes LICENSED, "approved," or "ready" into a field, citing the aggregation. The field is now a plain string in a file, indistinguishable in kind from a string written by review and a string written by a generator. If the file also carries a hash of itself, it has acquired the costume of Finding 1.2: tamper evidence dressed as authority.

Step five: the mechanical consumer. A downstream tool reads the status field and unlocks behavior—publishes, promotes, enables— because the field says what it says. The tool is behaving correctly with respect to its inputs. The chain as a whole has manufactured an authorization out of a test run, and every individual link can defend itself.

The kernel's design can be read as a point-by-point refusal of this gradient. Receipts are exact and stay exact (canonical encoding, byte-backed registries). Summaries are outside the kernel's boundary, but the boundary object refuses to let any summary alter ring, license, or disclosure state. Status words are typed enums validated for joint consistency, not free strings. And the mechanical consumer at step five finds, in v0.1, that the field it wants to read is constant false, with no code path that sets it otherwise—so the gradient, wherever else it runs, cannot discharge through the kernel.

3.4. Why downstream testing cannot repair either failure. We claimed at the outset that neither failure class is repaired by downstream testing, and the reason is now visible. Tests check that software behaves as specified on the meanings the specification assigns. Semantic drift corrupts the assignment of meanings: a test suite exercising a conflated ontology verifies, with great rigor, the wrong proposition. Claim promotion corrupts the status of the results: a test suite whose green output is subsequently promoted into authority makes the promotion *more* convincing, not less, because the receipt being laundered is genuine. In

both cases the failure lives in the representational and evidential frame around the tests, which is exactly the layer a kernel can own. That is the argument for building the frame first, as its own artifact, with its own review surface—before any v8 semantics exists to be framed.

4. The design-inversion thesis

4.1. Subtraction versus gated addition. There is a default shape that ambitious software takes: build the capable system first, then subtract the dangerous behaviors—add validation here, a permission check there, a warning banner on the features that frighten the maintainers. Call this the *subtractive* posture. Its weakness is structural, not incidental: in a subtractive system the dangerous behavior is representable, reachable, and often the path of least resistance, and safety depends on an enumeration of guards being complete. Every new feature, every refactor, every integration adds states faster than the guard list grows. The two failure classes above are what the subtractive posture looks like from inside a research program: conflation was representable, so it happened; self-authorized receipts were representable, so they happened.

CAS Core 8 inverts the posture. The kernel is built so that, within its boundary, the dangerous behaviors are either *unrepresentable*—no value of any kernel type expresses them—or *fail-closed*—the attempt to construct or execute them is rejected at the boundary, loudly, before any downstream artifact exists. Capability starts near zero and is added only through separately reviewed gates. The kernel's answer to "may I?" is *no* by construction, and every future *yes* must arrive as a reviewed change to the kernel itself, with its own review surface, rather than as a runtime discovery.

Concretely, the inversion instantiates as five mechanisms, each of which receives a chapter-length treatment later in the monograph and each of which answers named threats drawn from the two failure classes:

(1) *Structural semantic references over disjoint namespaces*, with an injective-rendering guarantee and a hard rule that positional or historical correspondences are typed as translations, alignments, or hypotheses—never silently as identities. Under this mechanism the Drive/Foundation conflation of Finding 1.1 is not forbidden; it is *unrepresentable*. The strongest statement the types can express is an explicitly non-identity link carrying its own source and note.

(2) *A research-boundary object* whose ring, license, and disclosure fields must be jointly consistent at construction and are derived and rechecked by every public operation, so that no shared mutable "status" object and no recomputable self-hash can stand in for the boundary.

(3) *A deterministic canonical evidence encoding*—sorted-key UTF-8 JSON; rejection of unordered containers, binary floats, NaN, and infinity; tagged exact decimals; one serialization route per public type; bounded, cycle-aware canonicalization—so that hash equality of evidence means what it appears to mean, and digest-collision and type-pun paths are closed by rejection rather than by convention.

(4) *A hash-verified, process-isolated legacy compiler compatibility profile*, with a deterministic-closure protocol, an advisory-lock concurrency discipline, an exact worker-failure taxonomy, and—critically for the claim discipline—a stated non-sandbox boundary carried as explicit *false* capability fields in every receipt, so that no consumer can honestly read a receipt as hermetic-execution evidence.

(5) *Exact-byte-backed claim and theorem registries* whose loaders reject every structural irregularity and whose records carry deterministic digests bound to the registry bytes, so that parsed content cannot be substituted while raw identity is retained.

4.2. Unrepresentable versus fail-closed. The inversion has two distinct enforcement modes, and keeping them separate sharpens both the design and its review. A behavior is *unrepresentable* when no value of any kernel type expresses it: there is no way to write down an identity link between references in different namespaces, because the construction that would express it rejects at creation, and no other construction exists. Unrepresentability is the stronger mode—the dangerous state is not guarded against but absent—and it is available only where the kernel owns the vocabulary, which is why the semantic-identity layer leans on it hardest.

A behavior is *fail-closed* when it is representable as an attempt but the attempt is rejected at a boundary, with an error, before any downstream artifact exists. Canonicalization is the model case: a caller can certainly pass a float, a set, or a cyclic structure to the encoder; the encoder's job is to refuse loudly rather than guess. Fail-closed is the right mode wherever the kernel receives input from outside its own type system—bytes from disk, values from callers, records from ledgers—because there the dangerous state arrives from a vocabulary the kernel does not control.

The review consequence is worth stating: unrepresentability is reviewed by reading type definitions and constructors (is there really no path?), while fail-closure is reviewed by reading rejection tests (does every bad input class have a fixture that is refused?). The bounded receipts cited later in the monograph speak to the second mode directly—the suite's rejection cases are enumerated—and to the first mode only via constructor tests. Both reviews remain open obligations in the independent sense; the monograph reports what the receipts exercised, which is not the same thing.

4.3. The constant-false authorization stance. Threaded through all five mechanisms is the single commitment already named: structural consistency is never authorization. In the v0.1 candidate this commitment takes its most extreme available form. There is *no code path that grants authorization.* The runtime decision model's field `authorization_granted`, and the compatibility spelling `allowed`, are constant false; claim-bearing intent is denied even when a locally constructed ledger contains the text LICENSED, because the kernel has no authenticated reviewer and no legal trust root, and a self-hash or a Python-local sentinel is not issuance provenance. The most a nonclaim research operation can obtain is a structural replay comparison—a statement that two hash-verified executions agreed—which is deliberately typed as evidence, not as permission.

It is worth being precise about what this stance does and does not assert, because "the kernel always says no" can be misread as either paranoia or theater. The stance does not assert that authorization is impossible in principle, that review can never license a claim, or that the program intends to remain forever unlicensed. It asserts that *this* artifact, lacking a trust root, is not the place where authorization can originate, and that encoding this fact as a constant is strictly safer than encoding it as a configurable check. A configurable check can be misconfigured, mocked in testing and left mocked, or satisfied by laundered inputs; a constant with no writing code path can only be changed by changing the kernel, which is a reviewed act. The governing sources also state minimum design constraints that any future claim-authorization implementation must bind—an exact trusted registry/schema manifest, an authenticated authority and trust root, reviewer and admission receipts, evidence-packet

identities and verification status, exact claim identity, and the code, model, environment, semantic, and calibration identities of the requested operation—and until a separately reviewed design binds all of them, the kernel's answer remains no.

4.4. What the inversion costs. Honesty about the thesis requires stating its price. A fail-closed kernel is inconvenient in exactly the ways it is safe. Rejection-first canonicalization means ordinary Python values—floats, sets, ad hoc dicts with non-string keys—do not silently flow into evidence; every producer must adapt to the encoding, not the reverse. Disjoint namespaces mean cross-ontology analysis requires constructing explicit link objects with sources and notes, which is more work than typing the same string in two columns. Constant-false authorization means the kernel cannot, in its current form, be placed on any route that needs a yes. The design accepts these costs deliberately, on the argument—developed across the threat analyses of the later chapters—that each convenience foregone corresponds to a historically observed failure mode, and that the program has already paid, in voided work products, the larger price of the convenient alternative. Whether the balance is struck correctly is a question for the independent review that the kernel's own status block says remains open.

4.5. Why a kernel, and why now. One further design question deserves an answer in the introduction, because it shapes how every later chapter should be read: why is this a *kernel*—a small, separate artifact—rather than a discipline applied inside the program's existing codebases?

The first reason is reviewability. The properties the kernel claims are of the kind that can only be established by reading everything: an unrepresentability claim is a claim about the absence of a path, and absence is audited by exhaustion. Exhaustive reading is feasible for a handful of small modules with no dependencies on the semantics they police; it is not feasible for a research monorepo. The kernel's smallness is therefore not modesty but a precondition of its central claims. Every capability added to the kernel dilutes this property, which is one more reason capabilities arrive only through reviewed gates.

The second reason is dependency direction. The kernel is designed as the shared substrate of a set of companion research candidates—board modeling, typed physical quantities, a finite typed-arrow kernel, and a categorical evidence layer among them—and the dependency arrow is strict: companions consume the kernel's contracts; the kernel depends on none of their semantics. This directionality is what lets the kernel be reviewed in isolation and what prevents the classic failure in which the integrity layer quietly imports the domain layer and every audit of the former becomes an audit of both. A discipline diffused through the codebases would have no such direction; it would be everywhere and auditable nowhere.

The third reason is timing. The program is at a stage where a future CAS v8 is a candidate, not a codebase—which means the frame can still be built *before* the semantics that will inhabit it. Retrofitting identity and evidence discipline onto a living system means competing with every existing convenience and migrating every existing artifact; building the frame first means the first v8 artifact born is born inside it. The two historical findings of this chapter are, on this reading, the program's tuition payments: the conflation incident and the self-hashed receipts each demonstrated, at the cost of voided work, what the absence of the frame permits. Building the kernel now, while there is nothing yet to overclaim about, is the cheap moment.

None of this argues that the kernel is sufficient—the open-obligation set says plainly that it is not—only that it is correctly shaped and correctly timed for the narrow job it claims: to be the part of the system that everything else must go through, small enough to read,

strict enough to trust as far as structural properties go, and honest enough to say "no" about everything beyond them.

5. Scope and non-goals

The kernel's credibility depends on the exactness of its boundary at least as much as on its mechanisms, so we state the boundary here in full and will restate the relevant portions wherever a chapter's content might otherwise invite an inflated reading.

Claim Boundary 1.3 (Not a rewrite of CAS v7.7). CAS Core 8 is deliberately not a rewrite of CAS v7.7. The v7.7 framework —its operators, domains, drives, registers, notation, and canon— continues to exist and evolve on its own track. The kernel neither reimplements v7.7 semantics nor supersedes any v7.7 artifact. Its only contact with legacy semantics is the compatibility profile, which is explicitly `legacy_only`: it executes hash-verified legacy compiler bytes for comparison purposes and produces no CAS v8 semantic translation. No v8 parser and no full semantic migration is claimed anywhere in the candidate.

Claim Boundary 1.4 (Not on production routes). The kernel is not installed on any production route. Nothing a user touches is served by, gated through, or dependent on CAS Core 8 in its v0.1 form. The kernel is a research candidate whose correct venue is review, and its receipts describe test and research executions only.

Claim Boundary 1.5 (No hermetic execution, no semantic correctness, no authority). Three specific overclaims are disclaimed globally, here and throughout. First, the compatibility profile achieves Python-process isolation, not an operating-system sandbox; the host's native libraries and operating system are not content-addressed, and every receipt carries these limitations as explicit false capability fields. No statement in this monograph is a claim of hermetic execution. Second, nothing in the kernel establishes the semantic correctness of any CAS content—the kernel checks identity, encoding, and structural consistency, never meaning. Third, nothing in the kernel or in this monograph constitutes claim authority: admission state is OPEN, product state is NOT LICENSED, disclosure is withheld, and independent software, formal-methods, semantics, and product-safety review remains an open obligation.

Beyond these three, the governing sources enumerate further open obligations that scope the kernel from the other side: a hermetic subprocess and toolchain bundle is not established by Python-level isolation alone; Python/Rust engine conformance is not established and the engine profiles remain separate pending a differential-testing campaign; no import-DAG manifest for multi-file programs is admitted, so multi-file legacy programs are outside the compatibility profile; and no claim-authority implementation is admitted. Where a later chapter touches one of these, it will cite the obligation rather than quietly narrowing it.

Remark 1.6 (Relation to the wider program). Because the kernel polices vocabulary drawn from the wider program's canon, it is easy to misread it as adjudicating that canon. It does not. The authoritative definitions of the program's semantic families—which texts define which elements, which names are canonical, which correspondences are positional rather than identical—are fixed by the program's canon documents and their own governance, entirely outside the kernel. The kernel consumes the canon's facts as inputs: a reference's definition digest is the digest of bytes the canon supplies; a crosswalk's non-identity typing encodes a distinction the canon draws. If the canon changes, references bound to superseded definitions become visibly stale, which is the intended behavior—but the kernel has no opinion about

what the canon should say, and nothing in this monograph should be read as a gloss on, or a rival authority to, any canon document. The division of labor is exact: the canon owns meaning; the kernel owns the bookkeeping that keeps meaning from being silently swapped.

Two scope remarks about the monograph itself. First, the monograph's evidential basis is fixed: the candidate specification, the v0.2 research paper that supersedes the v0.1 white paper, the kernel implementation sources, and the bounded Gate-4 receipts they cite. The monograph adds exposition, organization, and argument; it adds no facts. Where the exposition needs a fact the sources do not supply, the text says the sources do not supply it. Second, the monograph inherits the kernel's claim discipline. Words like "verified," "guaranteed," and "proved" are used only for properties enforced by construction or exercised by the cited receipts, always with their fixture-bound character stated; design intentions are called design intentions.

6. Objections and replies

Three objections recur when the kernel's stance is presented, and answering them here sharpens what the monograph is and is not arguing.

Objection: the kernel is bureaucracy—the real work is the semantics, and this is paperwork around an empty center. The reply is the program's own ledger. The failures that voided work were not semantic errors that better semantics would have caught; they were frame errors—conflated identities, promoted receipts—that corrupted sound work. Paperwork is the pejorative for provenance maintained after the fact; the kernel is the opposite bet, that identity and evidence discipline maintained *by construction* is what lets future semantic work accumulate instead of dissolving under audit. Whether the bet pays is an empirical question the program will answer; that the alternative was already tried and failed is a matter of record.

Objection: constant-false authorization is theater—if the kernel can never say yes, nothing real depends on it, so its refusals cost nothing and prove nothing. Half of this is correct and accepted: in v0.1, no production behavior depends on the gate, which is precisely the claim of Boundary 1.4. What the objection misses is where the value of a constant-false period lies: it is the interval in which every *other* part of the kernel—identities, encodings, receipts, registries—hardens under real use while the authorization question is held safely trivial. The alternative sequencing, standing up a configurable yes/no gate first and hardening its inputs later, is exactly the shape of system in which the historical receipts incident occurred. The constant is not the end state; the stated future-authorization constraints are. The constant is the refusal to improvise the end state.

Objection: the fail-closed posture will not survive contact with users—friction accumulates, and someone will build the convenient bypass. This is the serious one, and the monograph does not pretend the kernel is immune; a bypass built beside the kernel is outside its boundary and no type system prevents it. Two things can be said. Structurally, the kernel minimizes the bypass surface by being a substrate rather than a checkpoint: companions consume its contracts as their data model, so bypassing it means forgoing the data model, not just skipping a check. Institutionally, the kernel makes the bypass *visible*: artifacts that do not carry its typed identities and boundaries are identifiable as non-conforming at review. Beyond that, the objection names a governance problem, and the kernel's honest position is that governance problems are not solvable by kernels—which is, after all, its thesis.

7. Reading guide

The monograph is organized so that a reader can enter at the level of generality they need and descend.

The background chapter that follows is a tutorial survey of the standard material the kernel builds on: cryptographic hashing and content addressing; canonical serialization and the reasons it is genuinely hard, with the JSON Canonicalization Scheme as the reference point; reproducible builds and supply-chain integrity; provenance models built on entities, activities, and derivations; the integrity-versus-authorization distinction as security engineering has long drawn it; threat-modeling method; and the distinction between process isolation and sandboxing. None of that material is project novelty, and the chapter is explicit about this: the kernel's relation to the background is adaptation and composition, not invention. Readers fluent in these topics can skim the chapter for its comparison tables and its closing map from background concepts to kernel mechanisms.

The semantic-identity chapter develops the first mechanism in full: the five-field structural reference and the rationale for each field, with particular attention to definition-hash binding as the detector for definitional drift; the disjoint namespace catalog; the link-kind system for cross-ontology correspondences and the historical incident that motivates its refusal of identity links; the injective-rendering invariant with its design argument and its delimiter fixtures; the exact-type equality rule and the spoofing scenario it blocks; and worked examples of legal and illegal reference usage, grounded in the reference implementation.

Subsequent chapters treat, in order: the research-boundary object and its recheck discipline; the canonical evidence encoding as a closed rule set with its threat analysis; the legacy compiler compatibility profile, its deterministic-closure protocol, its concurrency and failure disciplines, and its non-sandbox boundary; the byte-backed claim and theorem registries; the runtime decision model and the constant-false authorization invariant, together with the stated design constraints on any future authorization; the consolidated threat analysis pairing each historically observed failure with the mechanism that now blocks it; the kernel's relation to its companion candidates and the strict direction of dependency that keeps the kernel reviewable in isolation; the bounded software evidence, stated with its exact limits, including the fact that the repository-wide collector was not clean at the bound receipt and that the focused suites therefore support no repository-wide claim; and the complete open-obligation set, which functions as the monograph's honest ending: a list of the things this kernel does not yet establish, maintained with the same care as the list of things it does.

Three reading paths may help three kinds of reader. A reviewer assessing the kernel's safety story should read this introduction, the semantic-identity chapter, and the threat-analysis material, then audit the boundary statements of every other chapter. An engineer intending to build on the kernel's contracts should read the background chapter's canonicalization sections and then the mechanism chapters in order, treating every fail-closed rejection rule as part of the interface. A reader from the wider research program who mainly needs to know what the kernel does *not* do can read Sections 3–5 of this chapter and the open-obligation material, and take away the one sentence the whole design is built to keep true: nothing in this kernel, and nothing in this monograph, converts a receipt into an authority.

7.1. How to read the evidence statements. Because later chapters cite the bounded Gate-4 receipts repeatedly, the reader should calibrate once, here, what those citations can and cannot carry. The receipts record, at a fixed date and commit, a focused suite of sixty-one passing test methods over the kernel, identified by a digest of the suite's test identifiers,

within an integrated run of two hundred eight methods and two hundred seventy-two parameterized subtests over fifty-four hash-bound artifacts. Three limits govern every citation of these numbers. First, they are *historical*: they describe runs that happened, not a standing property of the codebase; any change to the code or fixtures after the receipt is outside their reach. Second, they are *fixture-bound*: a rejection test establishes that the tested bad input was refused, and supports an inductive expectation about its class only to the degree the class is well represented—a judgment the receipts themselves cannot make. Third, they are *scope-bound*: at the bound receipt the repository-wide test collector was not clean (four pre-existing package-path namespace collisions after some twelve hundred collected items), so the focused suites support no repository-wide verification claim, and the monograph will never use them to make one. When a later chapter writes "exercised by the receipts," it means exactly this much and no more. The reader who internalizes this calibration once can then read the mechanism chapters without the text having to re-hedge every sentence—though the boundary environments will re-state the limits wherever an inflated reading would be most tempting, on the theory that redundancy about limits is the one form of redundancy this subject rewards.

It is also worth saying what would count as *more* evidence, since the monograph's claim discipline can otherwise read as a counsel of despair. Independent re-execution of the suites from the hash-bound artifacts would upgrade the receipts from reported to reproduced. Independent review of the type definitions and constructors would address the unrepresentability claims, which no test can fully carry. Adversarial fixture construction—an external attempt to produce colliding renderings, type-punned encodings, or laundered authorizations—would test the fail-closed surfaces where they matter. And the separately stated review obligations (software, formal-methods, semantics, product-safety) name the institutional version of all three. The kernel's stance is not that such evidence is unobtainable; it is that until obtained, the kernel and its monograph say so.

Remark 1.7 (Terminology). Throughout the monograph, "the kernel" means the CAS Core 8 v0.1 research candidate as fixed by its specification and implementation sources; "the program" means the surrounding TKS/CAS research effort; "receipt" means a structural, machine-generated record of observed software behavior; "canon" means the program's authoritative definitional documents; and "Ring 4" abbreviates the joint boundary condition—admission OPEN, product NOT LICENSED, public disclosure withheld—that every artifact discussed here carries. Where a term of art from the background literature is used, it is defined at first use in the background chapter.

CHAPTER 2

Background: Integrity, Canonicalization, Provenance, and Authorization

This chapter is a tutorial survey of the standard technical material on which the CAS Core 8 kernel builds: cryptographic hashing and content addressing, canonical serialization, reproducible builds and supply-chain integrity, provenance modeling, the integrity-versus-authorization distinction, threat-modeling method, and the distinction between process isolation and sandboxing. Everything in this chapter is established background from the security, systems, and standards literature. None of it is project novelty, and the chapter's framing is deliberately impersonal: the kernel appears only in forward pointers, as a consumer of these ideas, never as their source. The kernel's specific contribution—discussed in later chapters—is the composition of standard mechanisms into a fail-closed substrate for a particular research program, together with a particular design stance about authorization; the mechanisms themselves are the community's.

Remark 2.1 (How to read this chapter). Readers fluent in applied cryptography and security engineering can skim: the load-bearing artifacts are the canonicalization comparison axes of Section 2.4, the isolation ladder of Section 7, and the closing map from background concepts to kernel mechanisms in Section 8. Readers new to the material should read linearly; each section is written to be self-contained, defines its terms at first use, and states not only what each mechanism provides but—because the kernel's design turns on these negatives—what it conspicuously does not provide.

1. Cryptographic hashing and content addressing

1.1. Hash functions and their contract. A cryptographic hash function is a deterministic function H mapping arbitrary-length byte strings to fixed-length digests, engineered so that the mapping behaves, for all practical adversarial purposes, like a random function. Three properties make up the standard contract. *Preimage resistance*: given a digest d, it is computationally infeasible to find any message m with $H(m) = d$. *Second-preimage resistance*: given a message m_1, it is infeasible to find a distinct m_2 with $H(m_1) = H(m_2)$. *Collision resistance*: it is infeasible to find any pair $m_1 \neq m_2$ with $H(m_1) = H(m_2)$. Collision resistance is the strongest and most fragile of the three: by the birthday bound, a digest of n bits offers at most on the order of $2^{n/2}$ work against collisions, and historical functions (MD5, SHA-1) fell to structural attacks far below even that bound. Current practice therefore standardizes on functions with comfortable margins; SHA-256, the function used throughout the kernel's sources, produces 256-bit digests, giving a nominal 2^{128} birthday bound and, at the time of writing, no publicly known structural attacks that materially degrade it.

Alongside the adversarial contract sits a humbler property that does much of the practical work: the *avalanche* behavior by which any single-bit change to the input flips, in expectation, half the digest bits. Avalanche is what makes a digest useful as a fingerprint: near misses do not look near. Combined with determinism, this yields the basic evidence pattern of every

integrity system: if two computations of H over allegedly identical bytes disagree, the bytes differ, full stop; if they agree, the bytes are identical unless the adversary has broken the function.

Two consequences of the contract deserve emphasis because they are routinely elided in casual use. First, a hash function is defined over *bytes*, not over meanings. H knows nothing about JSON, about Unicode, about numbers, about the intent of the producer. Two encodings of the same abstract value hash differently unless the encodings are byte-identical—which is the entire subject of Section 2. Second, a digest carries no information about who computed it, why, or with what authority. Anyone can hash anything. A digest is a statement about content, never a statement about endorsement; the machinery that turns content statements into origin statements (signatures, authenticated channels, trust roots) is separate machinery, surveyed in Section 5, and conflating the two is one of the classic errors of applied security engineering [4].

1.2. Function lifecycle and digest agility. Hash functions age. MD5 went from standard to broken-for-collisions within a decade of wide deployment; SHA-1 followed, first with theoretical attacks and then with demonstrated collisions; and systems that had woven either function into long-lived identifiers faced expensive migrations because a digest, once used as a name, is referenced everywhere and controlled nowhere. The standard engineering responses are worth recording as background because any system that persists digests inherits the problem. First, *margin*: choose a function whose security comfortably exceeds present need, since migration cost dwarfs the marginal cost of a longer digest. Second, *explicitness*: record which function produced each digest—in a field, a prefix, or a fixed system-wide declaration—so that a future migration can distinguish old names from new mechanically rather than by guesswork. Third, *planned coexistence*: migrations happen by dual-hashing during a transition window, and schema designs that permit a second digest field beside the first migrate gracefully, while designs that hard-wire "the" digest do not. Fourth, a caution against overreach: *agility* in the protocol-negotiation sense (letting peers choose among functions at run time) has its own failure modes—downgrade attacks, cross-function confusion where the same content has multiple live names—and evidence systems generally prefer a single fixed function per era with explicit migration events over continuous negotiation. The practical upshot for digest-bearing records: fix one function, name it, validate digest syntax strictly (length, alphabet, case convention) at every intake boundary, and treat any future function change as a schema-visible event rather than a silent substitution.

1.3. Digests as names: content addressing. Once digests are cheap and collision-resistant, a powerful naming discipline becomes available: use $H(\text{content})$ *as the name of the content*. A store keyed this way is called content-addressed. The discipline has properties that location-based naming (file paths, URLs, database keys) cannot offer:

- *Self-verification.* Given a name and alleged content, anyone can check the binding by rehashing. Retrieval integrity needs no trusted server: a corrupt or malicious store can fail to produce the content, but cannot produce the wrong content undetected.
- *Immutability of reference.* A content address names one byte string forever. There is no "the file changed under the same name"; change produces a new name. Version history becomes a first-class object rather than an overwrite.
- *Deduplication and convergence.* Identical content produced independently receives the same name, so replicas can be reconciled and duplicates collapsed without coordination.

- *Composability via hash trees.* Structures whose nodes name their children by digest—Merkle trees and directed acyclic graphs— extend the discipline to aggregates: the root digest commits to the entire reachable structure, and any tampering anywhere invalidates the root. Version-control systems, transparency logs, and package-distribution systems are all elaborations of this one idea.

1.4. Hash trees, logs, and the mechanics of aggregation. The hash-tree idea deserves a slightly longer treatment, because the kernel's registry and receipt structures are degenerate cases of it and the general form explains the design pressure. In a Merkle tree, each leaf is the digest of a data block and each internal node is the digest of its children's digests; the root digest therefore commits to every leaf, and membership of any leaf can be proven with a logarithmic-size path of sibling digests. Three practical systems elaborate the idea in three directions. Version-control systems make the tree a DAG of commits, each naming its parents and its content tree by digest, so that a single commit identifier fixes an entire history; the append-only, tamper-evident character of such histories is a direct corollary of content addressing. Transparency logs make the tree an append-only sequence with signed tree heads, so that a log operator can prove both membership and append-only consistency to auditors, and equivocation—showing different histories to different observers—becomes detectable by gossip between observers. Package ecosystems make the tree a manifest: a release names its files by digest, a lockfile names its dependencies by digest, and installation verifies the closure.

Two mechanical details recur in all three and matter for any system that records digests in evidence. First, *domain separation*: when digests of digests are computed, leaf and node encodings must be distinguishable (by a prefix byte or a typed encoding), or an attacker can present an internal node as a leaf and forge memberships; the general lesson is that even hashing needs typing. Second, *presentation discipline*: a 256-bit digest has many textual presentations—upper and lower hexadecimal, base64, prefixed and truncated forms—and systems that compare digests as strings must fix one presentation or normalize before comparison, since case-mismatched hexadecimal is byte-different while digest-identical. These are small points, but evidence systems live and die by small points; a registry that stores lowercase digests and receives uppercase ones will either fail closed (annoying, recoverable) or compare case-insensitively somewhere and exactly somewhere else (a latent inconsistency of exactly the kind canonical disciplines exist to forbid). Fixing a single normalized digest presentation at every interface is the standard remedy.

The limits are the duals of the properties. Content addressing binds names to bytes, so it inherits byte-sensitivity: a semantically irrelevant byte change (a rewritten line ending, a reordered map key) produces a new name and severs every reference. Systems therefore face a standing temptation to normalize content before hashing—and every normalization is a decision about which distinctions are "irrelevant," a decision that itself needs to be explicit, deterministic, and recorded, or the cure reintroduces the disease. The kernel's two-identity discipline for line endings (raw digest and normalized comparison digest carried as separate, non-substitutable fields) is a forward pointer here: it is one project-specific answer to a completely standard tension.

A second limit: content addressing authenticates content, not context. Knowing that bytes match a digest says nothing about whether those bytes are the *right* bytes for a purpose—the current version, the reviewed version, the version this receipt was actually computed over. Binding digests to purposes is the job of provenance (Section 4) and of typed records that carry digests in named fields with declared roles.

2. Canonical serialization

2.1. The problem. Digest-based evidence has a hidden premise: that equal abstract values produce equal bytes. Serialization formats violate this premise by default. The same logical map, list, or number admits many legal encodings, and any process that re-serializes—a different library, a different version, a different platform—may lawfully produce different bytes and hence a different digest. The result is the worst kind of failure for an evidence system: *false negatives* (identical evidence appearing to differ, breaking replay comparisons and cache keys) and, more insidiously, *false positives* via type punning (distinct values encoding to identical bytes, so that different evidence hashes equal). A canonical serialization is a function from abstract values to bytes that is total on a declared domain, deterministic, and injective on that domain: one value, one byte string; distinct values, distinct byte strings.

Definition 2.2 (Canonical encoding). Let V be a declared domain of abstract values. A *canonical encoding* for V is a function $c : V \to B$ into byte strings that is (i) total on V, (ii) deterministic across implementations, platforms, and runs, and (iii) injective: $c(v_1) = c(v_2)$ implies $v_1 = v_2$ under the domain's declared equality. A system adopting c additionally declares its *out-of-domain behavior*: what happens when a value outside V is presented.

Under Definition 2.2, digest evidence acquires its intended meaning: for $v_1, v_2 \in V$, $H(c(v_1)) = H(c(v_2))$ holds exactly when $v_1 = v_2$, up to hash collisions. The three clauses fail independently in practice, and each failure has its own signature: partiality surfaces as crashes or silent omissions on exotic values; nondeterminism surfaces as false difference; and non-injectivity surfaces as false identity, the quietest and most dangerous of the three. The subsections that follow survey the standard obstacles to each clause, because each obstacle corresponds to a real divergence observed across mainstream serializers.

2.1.1. *Key order.* JSON objects and most map types are semantically unordered, but their serializations are sequences; some order must be chosen. Libraries default variously to insertion order, hash-iteration order (which can vary across runs, processes, and library versions, and in some languages is deliberately randomized), or sorted order under some collation. Any digest computed over an encoding with unspecified key order is nondeterministic in the fleet sense: it may be stable on one machine today and differ on another tomorrow. A canonical form must fix a total order on keys and say precisely over what it sorts—bytes of the UTF-8 encoding, UTF-16 code units, Unicode code points—since these orders differ once strings leave ASCII. A related trap is duplicate keys: JSON's grammar permits them, parsers disagree on whether the first or last wins, and an encoder that tolerates them has already lost injectivity at the parsing boundary. Strict systems reject duplicates outright.

2.1.2. *Numbers.* Numbers are the richest source of canonicalization failure. Binary floating point admits multiple decimal renderings of the same value (`1E2`, `100.0`, `1.0e+2`), and conversely, distinct values with renderings that careless formatting collapses. The modern remedy for rendering is the shortest-round-trip rule: render the fewest digits that parse back to the identical binary value. But even with rendering fixed, deeper puns remain. Is the integer 1 the same value as the float 1.0? Many serializations say yes by encoding both as `1`; many type systems say no. Is -0 the same as 0? IEEE 754 distinguishes them; most decimal renderings do not. `NaN` and the infinities have no JSON representation at all, so libraries variously reject them, encode `null`, or emit non-standard tokens—three behaviors, three different digests, one input. Exact decimal quantities (the natural carrier for measured and monetary values) add a further layer: the decimal value 1.0, the string `"1.0"`, and the integer 1 are three distinct typed values that naive encodings can collapse into one byte string.

An evidence encoding must decide, explicitly, which numeric distinctions are real, and then either represent them distinguishably (for example by tagging exact decimals as typed objects) or exclude the ambiguous carriers from the domain entirely (for example by rejecting binary floats). Both are defensible; the indefensible option is the default one, silence.

2.1.3. *Strings and Unicode.* Unicode offers multiple byte sequences for perceptually and even canonically equivalent text: precomposed versus decomposed forms (NFC/NFD), compatibility variants (NFKC/NFKD), invisible format controls, bidirectional override characters, and unpaired surrogates that some decoders pass through and others reject. Escaping rules multiply encodings further: JSON permits `"/"` and `"\/"`, permits any character to be `\u`-escaped, and permits both short and long escape forms. A canonical form must pick one escaping policy and one normalization stance. There is also a security dimension: invisible and bidirectional characters enable identifiers that render identically but differ in bytes—a spoofing vector whenever humans review machine identifiers. The conservative response, standard in review-critical systems, is to split the string domain: machine identifiers are restricted to a declared ASCII profile (trimmed, normalized, drawn from an explicit alphabet), while unrestricted prose travels in separate fields that never participate in identifier comparison.

2.1.4. *Aggregates, sharing, and cycles.* Serialization formats describe trees; program values are graphs. Two structural questions follow. First, *unordered containers*: a set has no traversal order, so any linearization imposes one; unless the element type carries a total order that the encoder applies, set encodings are nondeterministic, and strict systems simply reject sets from the evidence domain. Second, *sharing and cycles*: a value reachable twice can be encoded twice (losing sharing) or by reference (requiring a reference scheme), and a cyclic value cannot be tree-encoded at all—a naive recursive encoder will not terminate. Robust canonicalizers are therefore explicitly bounded: they track visited nodes, fail on cycles, and enforce declared depth, node-count, and output-size limits, converting resource-exhaustion attacks into clean rejections. Boundedness is not a luxury; an encoder that can be driven to unbounded recursion or output by a crafted input is a denial-of-service primitive sitting in the middle of the evidence path.

2.1.5. *Type identity.* Finally, serialization erases types unless told otherwise. A tuple and a list, an enumeration member and its underlying value, a domain-specific wrapper and its payload—each pair collapses to the same encoding in most formats. Whether that collapse is acceptable depends entirely on whether the type distinction is load-bearing. In an evidence system where enumerated statuses are precisely the values that must not be forgeable by strings, the collapse is not acceptable, and the standard remedy is type tagging: the encoding of a typed value embeds a reserved marker naming its type, and the reserved marker is excluded from ordinary maps so that data cannot impersonate tags. Tagging trades a little verbosity for injectivity across type boundaries, and it must be paired with the exclusion rule—a tagging scheme whose tags can be spoofed from plain data has merely moved the pun.

2.1.6. *A worked divergence.* To make the failure concrete, consider one modest abstract value: a record with a name field, a count, and a measured ratio—say the name `"café"`, the count one, and the ratio one-tenth. A survey of mainstream serializers, run naively, can lawfully produce all of the following encodings, each yielding a different SHA-256 digest: keys in insertion order versus sorted order; the name with the accented character raw (in either NFC or NFD form) versus escaped (in short or long escape form); the count as `1` versus `1.0` (an integer widened by a numeric tower) versus `"1"` (stringified by a lossy channel); and the ratio as `0.1` versus `0.10000000000000001` (a formatter printing seventeen digits)

versus `1E-1`. That is at least several dozen byte-distinct encodings of one value, produced by correct, widely deployed software making defensible local choices. An evidence pipeline in which any two components disagree on any one of these choices will report difference where there is identity. Worse, run the survey in reverse: the string `"1"` and the integer `1`, or a decimal one-tenth and a binary double one-tenth, can each arrive at the *same* bytes through some component pair, reporting identity where there is difference. Every rule in a strict canonical encoding can be read off from this example: fix the key order, fix the string form and reject the invisible cases, split the numeric types and reject the ambiguous carrier, tag what remains, and refuse everything else.

2.1.7. *Lessons from canonicalization history.* The difficulty is also attested historically, and one episode is instructive enough to summarize as background. The XML signature stack of the early 2000s required canonicalization (C14N) so that signatures survived innocuous re-serialization, and the resulting specifications—canonical XML, exclusive canonicalization, and their interaction with signature transforms—became notorious for complexity: namespace inheritance, attribute ordering, whitespace in mixed content, and processing-instruction handling all demanded rules, and implementations diverged in the corners. The security literature subsequently documented signature-wrapping attacks in which the signed subtree and the processed subtree were different nodes of the same document—an exploitation not of the hash or the signature but of the gap between what was canonicalized and what was consumed. Two standard lessons are drawn from that experience. First, canonicalization complexity is attack surface: the more transformations sit between the value the producer meant and the bytes the verifier hashes, the more room for the two to diverge. Second, the safest designs canonicalize *once*, at a single declared boundary, and consume exactly the bytes they verified, rather than re-deriving the value downstream. JSON-era designs, including JCS [1] and the stricter evidence encoders surveyed below, are deliberately minimal in reaction: small value domains, few transformations, one route from value to bytes. The kernel's one-serialization-route-per-type rule (later chapters) is this lesson in project form.

2.2. RFC 8785: the JSON Canonicalization Scheme. The reference point for JSON canonicalization is the JSON Canonicalization Scheme (JCS), published as RFC 8785 [1]. JCS targets the interoperable subset of JSON (I-JSON): data that fits in UTF-8, with numbers representable as IEEE 754 double precision and no duplicate object keys. Within that subset, JCS fixes: object keys sorted by their UTF-16 code-unit sequences; number serialization per the ECMAScript shortest-round-trip algorithm; a minimal, fixed string escaping; no insignificant whitespace; and UTF-8 output. The design philosophy is deliberate reuse: by matching ECMAScript's own serialization behavior, JCS lets a large installed base produce canonical output with modest effort, and it achieves its goal of byte-stable digests for I-JSON data across conforming implementations.

Two of the scheme's choices repay a closer look, since they illustrate how a canonicalization standard turns the obstacles of the preceding subsections into decisions. The key-ordering rule sorts object keys by comparing their UTF-16 code-unit sequences. For ASCII keys this coincides with byte order, but outside the basic multilingual plane the choice is visible: characters encoded with surrogate pairs in UTF-16 compare differently under code-unit order than under code-point or UTF-8 byte order, so the same object can sort differently under a plausible-but-wrong implementation and produce a plausible-but-wrong digest. The rule is a reminder that "sorted keys" is not yet a specification; *sorted by what* is. The number rule adopts the ECMAScript serialization algorithm, which emits the shortest decimal string that round-trips to the same IEEE 754 double, with fixed rules for when exponent notation

appears and with negative zero serialized as `0`. This buys cross-implementation byte stability for every representable double, at the declared price of the domain restriction: values needing more than double precision, and the non-finite values, are simply outside I-JSON and hence outside the scheme's promises. Systems whose evidence includes such values must carry them in strings or tagged structures by their own convention—which is precisely the pattern the strict encoder family adopts for exact decimals.

Equally instructive is what JCS deliberately does not do. It does not extend the value domain: sets, cycles, and shared structure are outside JSON and hence outside JCS. It does not distinguish numeric types: within I-JSON, a number is a double, so the integer/decimal/float distinctions of Section 2.1.2 are collapsed by the domain choice itself, and values outside double range or precision are simply not I-JSON. It does not normalize Unicode content: strings are canonicalized in escaping, not in normalization form, so NFC and NFD spellings of the same text remain distinct values. It does not tag types, bound resources, or take a stance on rejection versus coercion for out-of-domain values—those questions fall to the systems that adopt it. None of these are defects; they are scope decisions appropriate to JCS's interoperability goal. But they mean that JCS is a *component* of an evidence discipline, not the discipline itself: a system whose evidence includes exact decimals, typed enumerations, or adversarial inputs must layer domain restrictions, tagging, and bounds on top of—or instead of—the interoperability-oriented core.

2.3. The rejection stance. The out-of-domain axis deserves a short argument of its own, because it is where evidence encoders and general-purpose serializers part company most sharply. A general-purpose serializer is judged by how much it can encode; faced with a set, it sorts or lists it; faced with a NaN, it emits a null or a token; faced with an unknown object, it calls a fallback hook. Every one of these accommodations is a silent semantic decision made on the producer's behalf, and in an evidence setting each becomes a pun generator: the sorted set is now indistinguishable from a sorted list; the null-for-NaN is indistinguishable from a genuine null; the fallback-encoded object is indistinguishable from a hand-built map that happens to match. The coercion that makes the serializer convenient makes the digest ambiguous, and digest ambiguity is unrecoverable downstream—no consumer of the digest can reconstruct which of the collapsed values was meant.

The rejection stance inverts the judgment: an evidence encoder is judged by how little it encodes without instruction. Everything outside the declared domain fails closed, with an error naming the offending value and the rule it violates, at the moment and place where the producer—who alone knows what was meant—can fix it. The cost profile is the mirror image of coercion's: friction is paid up front, visibly, per producer, while the benefit (every digest in the system means exactly one thing) accrues silently, system-wide, forever. Which stance is right is not absolute but positional—the same organization may correctly run coercing serializers at its logging edges and a rejecting encoder at its evidence core—and the classic failure is stance drift: an evidence path that, under delivery pressure, imports the convenient serializer "temporarily." The stance is therefore best enforced structurally (a single blessed encoder for the evidence domain, no alternative import path), which is the design shape later chapters describe for the kernel; the argument for it, as this section has tried to show, is entirely general.

2.4. Comparison axes for canonical encodings. Surveying the design space, canonical encodings can be compared along seven axes, summarized in Table 1 (the number-policy axis occupies two rows there, one for finite and one for non-finite treatment).

(1) *Value domain*: which abstract values are encodable at all—full host-language values, a format's native model, or a deliberately restricted evidence domain.
(2) *Number policy*: single numeric type versus distinguished integers, decimals, and floats; treatment of non-finite values.
(3) *String policy*: escaping rules; Unicode normalization stance; identifier profiles versus free prose.
(4) *Ordering*: what is sorted, and over what collation.
(5) *Out-of-domain behavior*: coerce (make something encodable), best-effort (encode with caveats), or reject (fail closed). This axis, more than any other, separates interoperability formats from evidence formats: coercion maximizes the chance of producing output; rejection maximizes the meaning of the output that is produced.
(6) *Type identity*: erased types versus tagged types, and whether tags are spoofable from data.
(7) *Resource bounds*: unbounded recursion versus declared depth, node, and size limits with cycle detection.

TABLE 1. Comparison axes for canonical serialization designs. "JCS" is RFC 8785 [1]; "strict evidence encoders" names the family of reject-and-tag designs of which the kernel's encoder (later chapters) is one member.

Axis	JCS (RFC 8785)	Strict evidence encoders
Value domain	I-JSON	declared, restricted
Numbers	IEEE double only	int/decimal split; floats rejected
Non-finite values	outside I-JSON	rejected explicitly
Strings	escape canonicalization	profiles; controls rejected
Ordering	UTF-16 code-unit key sort	sorted string keys
Out-of-domain	out of scope	fail closed
Type identity	erased	reserved-tag typed values
Resource bounds	unspecified	depth/node/size bounds; cycle fail

The table should be read as a contrast of goals, not of quality. JCS optimizes for the interoperability of independently implemented producers over benign data; strict evidence encoders optimize for the evidentiary meaning of a digest over potentially adversarial data. The first wants maximal producers; the second wants minimal surprise. A mature system frequently needs both, in different places—an interchange layer speaking a JCS-like dialect at its edges and a strict encoder at its evidentiary core—and confusing the two roles in either direction (strictness at the interoperability edge, tolerance at the evidence core) produces respectively friction and laundering.

3. Reproducible builds and supply-chain integrity

3.1. The reproducibility contract. A build is *reproducible* when, in the Reproducible Builds project's canonical formulation, "given the same source code, build environment and build instructions, any party can recreate bit-by-bit identical copies of all specified artifacts" [3]. The motivation is a trust gap at the heart of software distribution: users run binaries, but review—when it happens at all—happens on sources. Nothing in a conventional build pipeline connects the two: the binary's contents are determined not only by the source but by the compiler, its version and flags, the build machine's state, and whatever compromised

any of them. Reproducibility closes the gap by making the source-to-binary function deterministic, so that independent rebuilders can each compile the source and compare digests. Agreement among independent parties establishes—to the strength of the hash function and the independence of the rebuilders—that the distributed binary is what the reviewed source produces. A single malicious or compromised builder can no longer substitute a backdoored artifact without every honest rebuilder's digest disagreeing.

Achieving bit-for-bit determinism is an exercise in hunting down incidental environment leakage, and the catalog of leaks doubles as a catalog of what "environment" even means: embedded timestamps and dates; absolute build paths baked into debug info; locale- and timezone-dependent formatting; filesystem readdir order and archive member order; parallelism-dependent link order; uninitialized memory serialized into outputs; randomized identifiers and, notably, randomized hash-iteration order in the build tooling itself. The remedies are correspondingly standardized: pinned toolchains, normalized metadata, forced stable orderings, and declared epochs for anything time-shaped. The parallel with Section 2 is exact and not coincidental: reproducible builds are canonical serialization applied to the compilation function, and both disciplines reduce to the same maxim—every source of nondeterminism is either eliminated, pinned and recorded, or it silently voids digest comparison.

3.2. Pinning, lockfiles, and the verification economy. Between the source and the build sits the dependency graph, and the reproducibility discipline extends to it through pinning: a dependency manifest that names not version *ranges* but exact artifacts, each with its digest, so that resolution is a lookup rather than a negotiation. The lockfile pattern—a generated, committed file recording the fully resolved, hashed closure of the dependency graph—is the ecosystem-standard instantiation. Its evidentiary reading is the important one here: a lockfile converts "the build used library X" from a claim about intentions into a claim about bytes, checkable by rehashing. Unpinned resolution, by contrast, makes every build a function of the repository state of third parties at build time, which is exactly the kind of unrecorded input that voids digest comparison; the supply-chain attack literature is largely a catalog of ways this openness is exploited (dependency confusion between internal and public names, typosquatted packages, maintainer-account compromise followed by a poisoned patch release), and pinning-plus-hashing is the standard first line against all of them.

The verification side has its own economics, worth stating because they explain why reproducibility is a *social* mechanism and not merely a technical one [3]. One reproducible build verified by nobody provides nothing; the security benefit scales with the number and independence of rebuilders who actually recompute and compare, because the attacker must now compromise all of them—or the comparison channel—rather than one build server. Independence is doing the work in that sentence: rebuilders sharing the same compromised toolchain image reproduce each other's compromise. The design consequences that generalize to any evidence system are two. Cheap verification is a feature to be engineered deliberately—the whole pipeline from source to comparable digest must be runnable by an outsider at tolerable cost, or independence stays theoretical. And comparison results are themselves evidence needing the full discipline of this chapter: a rebuild receipt has content, provenance, and (no) authority, exactly like any other receipt.

3.3. Limits and the supply-chain frame. Reproducibility, like hashing, has sharply bounded semantics. Bit-for-bit agreement establishes that binary corresponds to source; it establishes nothing about whether the source is any good. Reproducibly built malware is still malware, reproducibly built bugs are still bugs, and a reproducible pipeline whose declared

toolchain is itself compromised reproduces the compromise faithfully—the classic trusting-trust regress, answerable only by diversifying or bootstrapping the toolchain itself. Reproducibility is thus one link in a supply-chain integrity story that also needs source integrity (version control with content-addressed history), dependency integrity (pinned, hashed dependency manifests), and distribution integrity (signed, logged releases). Each of these links has, since roughly 2021, acquired a standardized and widely deployed instantiation: the in-toto attestation framework and the SLSA provenance levels [6, 7] give build provenance a signed, machine-readable statement form, and the Sigstore infrastructure [8]—keyless signing recorded in an append-only transparency log—now backs release signing and provenance at major open-source package registries. The survey retains the generic vocabulary because the kernel adopts the underlying ideas rather than these frameworks, but the frameworks are where current practice lives. The general supply-chain lesson for any evidence system is the one the kernel's compatibility profile takes as a premise: an execution result is only as meaningful as the identity of everything that participated in producing it, and each participant—source bytes, interpreter, environment, dependencies—must be either content-addressed and recorded, or honestly declared out of scope. Declaring the out-of-scope set is not an admission of failure; it is what makes the in-scope claims exact. The dishonest alternative is a receipt that is silent about what it did not pin.

4. Provenance: entities, activities, derivations

4.1. The PROV model. Where hashing answers "is this the same content?" and reproducibility answers "does this artifact correspond to that source?", provenance answers "where did this come from, and through what?" The W3C PROV ontology [2], which expresses the PROV data model in OWL, standardizes the vocabulary. Its core is a typed graph over three node kinds. *Entities* are things "with some fixed aspects" [2]—documents, datasets, files, a particular version of a particular artifact. *Activities* are occurrents—things that, in the standard's words, "occur over a period of time and act upon or with entities" [2]. *Agents* are things that "bear some form of responsibility" [2]—people, organizations, or software to which activity is attributed. The core edge kinds connect them: an activity *used* an entity; an entity *was generated by* an activity; an entity *was derived from* another entity; an entity *was attributed to* an agent; an activity *was associated with* an agent. Derivation is the load-bearing relation for evidence systems: it asserts that one entity's existence and content depend on another's, and chains of derivation compose into an auditable ancestry for any artifact.

Two modeling disciplines in PROV repay attention. First, *entities are versions, not names*: the model's "fixed aspects" requirement means that a mutable thing is represented as a sequence of entities, one per state—which harmonizes exactly with content addressing, where each state has its own digest and hence its own natural entity identifier. Second, *activities are first-class*: the run that produced an artifact is itself a node, with its own attributes (start and end, parameters, environment), not an annotation squeezed into the artifact. An evidence record that identifies its generating activity—which suite, which profile, which interpreter, which date—is a PROV generation edge in typed-record form, and the kernel's execution receipts (later chapters) are readable as exactly that.

4.2. Granularity, bundles, and anti-patterns. Applying the model well is mostly a matter of granularity decisions, and the standard guidance can be compressed into three choices and three anti-patterns. The choices: *entity grain* (is the entity the file, the record, or the field? finer grains support sharper invalidation when an ancestor changes, at higher

recording cost); *activity grain* (one activity per pipeline run, per stage, or per operation? the right grain is the one at which parameters and environment are actually uniform, since an activity's attributes should be true of all of it); and *assertion bundling* (PROV allows provenance about provenance—who asserted this graph, when—which is the hook by which provenance records themselves enter the integrity-and-authority stack rather than floating above it).

The anti-patterns are the ones audit experience keeps rediscovering. *Provenance by narrative*: ancestry recorded as prose in a report rather than as typed edges over identified entities— unqueryable, unverifiable, and quietly divergent from the artifacts it describes. *The mutable entity*: a provenance graph whose nodes name mutable locations ("the config file") rather than fixed states, so that the graph's truth decays as the locations change; the remedy is the content-addressed entity identifier already discussed. *The absent activity*: derivation edges recorded without the generating activity, so that *what was run*—the question every serious audit asks first—is unanswerable; systems that generate receipts as a side effect of execution, rather than reconstructing them afterward, avoid this by construction. Each anti-pattern has appeared, in some form, in the wider research program's history, which is why its evidence tooling records typed, digest-anchored records generated at execution time; the background point is that these are the textbook remedies, not innovations.

4.3. What provenance is not. Provenance is descriptive, not normative. A complete, honest provenance graph for a worthless artifact is perfectly possible; so is a beautiful ancestry terminating in a poisoned root. Provenance assertions are, moreover, themselves just data—anyone can claim that an entity was derived from another—so provenance without integrity (content-addressed endpoints) is unverifiable narrative, and provenance with integrity but without authorization (authenticated asserters) is verifiable narrative of unknown authority. The three layers stack: integrity fixes *what* the nodes are, provenance fixes *how* they relate, and authorization—the subject of the next section—fixes *who stands behind* the assertions. Systems get into trouble precisely when a lower layer is dressed as a higher one: a derivation chain read as an endorsement chain, or a generation record read as a review record. Keeping the layers typed and separate is the design discipline; the vocabulary of [2] exists so that the separation has standard names.

5. Integrity versus authorization

5.1. Two different questions. Security engineering has long insisted on separating two questions that casual system design merges [4]. *Integrity*: is this artifact the artifact it claims to be—unmodified, matching its digest, consistent with its schema? *Authorization*: is the action this artifact licenses actually permitted—has a principal with the authority to permit it done so? The questions have different verifiers, different failure modes, and different machinery. Integrity is verified by computation over content: rehash, recheck, reparse. Authorization is verified by reference to a *trust root*: some principal or key whose authority is established outside the system under discussion—by legal arrangement, organizational decision, or physical control— and from which chains of delegation descend. A message authentication code or a bare hash provides integrity; only a construction bound to an authenticated principal—a signature verifiable against a key whose holder is known and whose authority over this action is established—begins to provide authorization, and even then only relative to the policy that gives that key its meaning.

The distinction is sharpest in its failure cases, which security engineering catalogs at length [4]. A system that checks integrity and concludes authorization is vulnerable to *laundering*: any principal able to produce internally-consistent artifacts—which is every principal, since consistency is computable—can mint permissions. The archetype is the self-signed assertion consumed as if countersigned: a document that hashes correctly, validates against its schema, and says "approved," where every one of those three facts was produced by the same unaccountable process. Conversely, a system that checks authorization but not integrity is vulnerable to substitution: a genuinely authorized decision applied to swapped content. Sound designs therefore verify both, separately, and are explicit about which mechanism carries which burden. Policy names who may do what; mechanism enforces it; and the enforcement of a policy must never be inferable from properties that any party can self-supply.

5.2. The mechanism inventory. This section inventories, still as background, the standard mechanisms that occupy the space between bare integrity and full authorization, because their differences are exactly the distinctions the kernel's records must not blur [4].

A *bare hash* binds nothing to nobody: anyone can compute it, so it provides integrity only relative to a trusted copy of the digest itself. A *message authentication code* binds content to possession of a shared secret: it proves origin-or-tampering to the other holder of the secret, but—because both parties can compute it—provides no third-party-verifiable attribution and no non-repudiation. A *digital signature* binds content to a private key: any holder of the public key can verify, and only the private-key holder can sign, which yields attribution and non-repudiation *relative to the binding between key and principal*. That binding is itself the next problem up: a *certificate* is a signed assertion by one key about the identity or attributes of another, chains of certificates reduce many bindings to few, and the chains terminate in roots distributed out of band and defended by procedure—key ceremonies, hardware custody, audit. Above all of this sits *policy*: a valid signature by an authentic key still authorizes nothing unless the governing policy assigns that key authority over that action; the verifier's real question is never "is this signed?" but "is this signed by a principal whom policy entitles to license this act?"

Laid out this way, the inventory exposes the category error at the heart of the second failure class in the introduction: a self-hashed artifact sits on the *bottom* rung of this ladder—below the MAC, since not even a shared secret is involved—while presenting visual credentials (long hexadecimal strings, "verified" vocabulary) associated with the top. Nothing about the rungs is subtle to a security engineer; the failure mode is that systems without an explicit ladder let artifacts drift up it rhetorically. Typed records that say which rung they occupy—this is a digest, this is a signature, this key's authority derives from that policy—are the standard countermeasure.

5.3. Trust roots and the boundary of self-certification. The deep point beneath the distinction is that authorization cannot be bootstrapped from inside. Every authorization chain terminates in a root that the verifying system takes on grounds external to computation: a key ceremony, a legal identity, an organizational appointment. A system with no such root can compute forever without producing a single authorized bit; the most it can honestly produce is tamper-evident, well-formed, thoroughly-described *candidate* material awaiting external review. This is not a defect to be engineered around; it is the definition of the boundary. The honest design consequence, where no trust root exists, is to make the absence structural: represent authorization as a type the system cannot inhabit, rather than as a flag the system could set. The kernel's constant-false authorization stance, described in the introduction and developed in the runtime chapter, is this standard consequence applied

without exception; the background point here is that it is an application of textbook doctrine [4], not an invention.

6. Threat modeling

6.1. Method. Threat modeling is the discipline of systematically asking what can go wrong, before and during design rather than after deployment [5]. The method's canonical loop has four questions. *What are we building?*—answered with a model of the system: its components, data flows, and, critically, its trust boundaries, the surfaces where data or control crosses between principals of different privilege or trustworthiness. *What can go wrong?*— answered by structured enumeration over the model. *What are we going to do about it?*—answered by mitigations: eliminate the threatened element, add a control, or explicitly accept and document the risk. *Did we do a decent job?*—answered by validation: checking the model against the built system and the mitigations against the threats.

The best-known enumeration structure is STRIDE, a mnemonic over six threat categories: *Spoofing* (claiming an identity one does not have), *Tampering* (modifying data or code), *Repudiation* (denying an action absent evidence), *Information disclosure* (exposing data to those not entitled to it), *Denial of service* (degrading availability), and *Elevation of privilege* (gaining capability beyond one's authorization) [5]. Applied per element—each process, data store, and data flow in the model is examined against each applicable category—STRIDE converts the open question "what can go wrong?" into a checklist with known blind spots rather than an unbounded brainstorm. Its output is a set of *threat statements*: concrete, reviewable sentences of the form "an actor with access X performs Y, causing Z," each paired with a mitigation, an acceptance, or an open item.

6.2. Trust boundaries and model hygiene. The modeling step deserves its own comment, because the quality of everything downstream is bounded by it. The standard modeling artifact is the data-flow diagram: external entities, processes, data stores, and the flows between them, with trust boundaries drawn wherever a flow crosses between principals that should not fully trust each other—user to service, service to database, parent process to spawned worker, checked-out repository to executing toolchain. Threat enumeration then concentrates on the crossings, because that is where assumptions change hands: data validated on one side is raw on the other; an identity authenticated in one zone is a plain string in the next. Two hygiene rules from practice [5] transfer directly to evidence systems. First, *model the system you have, not the one you intend*: a diagram omitting the debug endpoint, the cache, or the "temporary" shared mutable status object will produce a threat list omitting their threats, and audits repeatedly find that the omitted element is where the incident lived. Second, *boundaries are claims to be enforced, not facts to be assumed*: drawing a line between worker and parent means something only if a mechanism (a process boundary, a sanitized channel, a validation gate) holds it, and every drawn boundary should be traceable to its mechanism. A complementary enumeration structure, attack trees—root goal decomposed into conjunctive and disjunctive subgoals—is often used alongside STRIDE when a single high-value failure ("an unauthorized claim is consumed as authorized") needs its paths enumerated exhaustively; the two structures answer different questions (coverage per element versus depth per goal) and mature models use both.

6.3. Threats as review artifacts. Two features of the method matter for how this monograph is written. First, threat statements are *design inputs*, not post-hoc documentation: a mechanism justified by a threat statement can be reviewed for whether it actually

blocks the stated threat, which is a much sharper question than whether it is "secure." Second, the method treats *explicit risk acceptance* as a first-class outcome: a threat that is out of scope is written down as out of scope, with the reasoning, rather than silently omitted—the threat-model analog of the supply-chain rule that unpinned inputs be declared. The later chapters of this monograph follow both practices: each kernel mechanism is presented with the named threats it answers (drawn from the program's actually-observed failure classes, which serve as the threat catalog's empirical core), and the non-goals—non-sandbox, non-hermetic, non-authoritative—are presented as accepted, documented boundaries in exactly the sense of [5]. Where a STRIDE category maps cleanly onto a kernel concern, the mapping is the expected one: spoofing maps to identity and equality attacks, tampering to digest and registry substitution, repudiation to receipt exactness, elevation of privilege to claim promotion. The mapping is noted here once so that later chapters can use the program's own vocabulary without re-deriving it.

6.4. Validation: closing the loop. The fourth question of the loop—did we do a decent job?—is the one most often skipped, and its standard content is worth a paragraph because the monograph's evidence chapters are, in effect, one project's answer to it [5]. Validation has two halves. Model validation checks that the diagram still matches the system: every component present, every flow drawn, every boundary's enforcing mechanism still in place—a check that must recur, since systems drift and threat models rot quietly. Mitigation validation checks that each threat statement's mitigation actually engages: for a fail-closed rule, that a test presents the threatening input and observes the refusal; for an unrepresentability claim, that review of the types finds no constructor path; for an accepted risk, that the acceptance is still visible in the artifacts consumers see (the false capability field is present in the receipt, not merely in the design notes). The discipline of pairing each threat with its validation evidence —and of distinguishing "mitigated by design," "mitigated and tested," and "accepted and documented" as three different statuses—is what turns a threat model from a design-time essay into a maintained contract. Where later chapters tabulate the kernel's threats against its mechanisms and its receipts, they are following this template, and the same template says exactly what is missing when a validation cell is empty: an open obligation, to be listed as such.

7. Process isolation versus sandboxing

7.1. A ladder of containment. "The code ran in isolation" is among the most overloaded sentences in systems engineering, and unpacking it matters wherever execution receipts will be consumed as evidence. It is clarifying to arrange the common meanings as a ladder, each rung adding a boundary the previous rung lacks.

Rung 1: in-process hygiene. The code runs in the same process, with discipline: fresh module namespaces, controlled globals. The weakest rung: it shares the entire address space, interpreter state, and permissions of its host, and any misbehavior is total.

Rung 2: process isolation. The code runs in a separate operating-system process, with a fresh address space and explicit communication channels. For interpreted languages the rung includes interpreter hygiene: launching the fresh interpreter with flags that suppress inherited path injection, site customization, and stale bytecode, and with a sanitized environment rather than the parent's. Process isolation protects the *parent's* state from the child and makes the child's inputs enumerable. It does *not* constrain what the child may do to the world: the child holds the same user privileges as its parent—files, network, devices—and shares the host's native libraries and kernel.

Rung 3: OS sandboxing. The child's privileges themselves are reduced: system-call filtering, namespace and container isolation of filesystem and network views, mandatory access control, resource limits. A sandboxed child that misbehaves is constrained by the operating system, not merely observed by its parent. Sandboxing is a policy mechanism and is only as strong as the policy and the kernel attack surface beneath it.

Rung 4: virtualization and hardware isolation. The boundary moves below the operating system: virtual machines, and further down, separate physical hardware. Stronger again, with its own residual channels (hypervisor bugs, side channels).

Orthogonal: hermeticity. Cutting across the ladder is the question of Section 3: is everything that participates in the execution content-addressed? A hermetic execution pins and records the identity of the interpreter, native libraries, and OS surface, so that the receipt names its entire closure. Hermeticity is orthogonal to containment—a strongly sandboxed execution can be non-hermetic (unpinned toolchain inside a strong box) and a weakly contained one can be partially hermetic (pinned interpreter bytes, shared host).

7.2. Interpreter-level isolation in practice. Because the kernel's compatibility profile lives at rung 2, the interpreter-hygiene details of that rung are worth spelling out in their standard, non-project form; the discussion is phrased for Python, whose mechanisms are representative. A child interpreter launched naively inherits far more from its parent than the process boundary suggests: environment variables that inject code paths or site customizations; the working directory and script directory prepended to the module search path, so that a file named like a standard library module shadows it; user and system site-packages directories, whose contents vary by machine and time; startup files designated by environment variables; and stale compiled bytecode caches that can desynchronize from their sources. Each inheritance channel is a way for the execution's behavior to depend on unrecorded host state—precisely the supply-chain sin of Section 3—and each has a standard closure: isolated mode (`-I`) removes the script-directory and user-site injection and ignores the environment variables that alter interpreter behavior; suppressing site initialization (`-S`) removes machine-dependent path customization; disabling bytecode writing (`-B`) prevents cache pollution; and an explicitly constructed, minimal environment replaces the inherited one. What remains, and must be either recorded or declared unpinned, is the interpreter binary itself, its standard library, and every native library it loads— the recording of which is content addressing again, and the honest declaration of which (when not recorded) is the capability-honesty practice below.

One further rung-2 practice appears wherever isolated workers share a host: concurrency discipline. Two workers serializing through a shared resource—a cache directory, a lock file, a device—need a mutual-exclusion mechanism, and *advisory* locks (locks that constrain only cooperating processes that choose to check them) are the common tool. The standard honesty requirements transfer: an advisory lock's guarantees should be stated as bounded (acquisition timeouts, polling intervals) and unclaimed properties—fairness, protection against non-cooperating processes—should be stated as unclaimed. A receipt that says "serialized via advisory lock, without fairness claims" is a rung-2 receipt telling the truth about rung 2.

7.3. Why the rung must be stated. The engineering point of the ladder is not that higher is always better—each rung has real costs in complexity, portability, and performance, and the right rung depends on the threat model (Section 6). The evidentiary point, which is the one this monograph needs, is that *a receipt must state its rung.* An execution receipt that says "isolated" without saying which boundary held invites the consumer to assume the strongest reading; over time, the strongest reading becomes the folklore, and the folklore

becomes a false premise in someone's safety argument. This is claim promotion (in the introduction's sense) applied to containment. The standard remedy is capability honesty: receipts carry explicit fields for the boundaries they did *not* establish—sandboxing absent, hermeticity absent—as first-class negative facts, machine-readable, so that no downstream consumer can honestly overread the rung. The kernel's compatibility receipts, which expose their non-sandbox and non-hermetic status as explicit false capability fields, instantiate this remedy; the remedy itself is simply Section 6.3's documented-acceptance practice transplanted to execution evidence.

8. Synthesis: from background to kernel

The seven bodies of material surveyed here compose into a single picture, assembled here once before the mechanism chapters descend into detail.

Hashing and content addressing (Section 1) supply the primitive: bytes can be named by content, and equal names mean equal bytes. Canonical serialization (Section 2) makes the primitive apply to structured values: with a total, deterministic, injective encoding on a declared domain, equal digests mean equal values and distinct values mean distinct digests—provided the domain, number policy, string policy, type tagging, and bounds are all fixed, which is why the strict reject-and-tag family exists alongside interoperability-oriented designs like JCS [1]. Reproducible-build discipline (Section 3) extends the same determinism from values to executions: pin or record every participant, and declare what is not pinned. Provenance (Section 4) supplies the vocabulary for relating the resulting artifacts—entities generated by activities, derived from entities, attributed to agents—without ever confusing ancestry with endorsement. The integrity-versus-authorization doctrine (Section 5) then draws the line that the whole stack must respect: everything so far is computable from content and therefore self-suppliable, so none of it, however elaborate, amounts to permission; permission requires a trust root that computation cannot conjure. Threat modeling (Section 6) provides the method by which a design argues, mechanism by named threat, that its boundaries hold and that its accepted risks are stated rather than hidden. And the isolation ladder (Section 7) applies the same honesty to execution: say which boundary held, and carry the negative capabilities in the receipt.

Table 2 records the correspondence between this background and the kernel mechanisms that later chapters develop. The right-hand column is a map of forward pointers, not a claim list; in every row, the idea is standard and the kernel's role is adoption and composition under a fail-closed stance.

Table 2. Background concepts and the kernel mechanisms that adopt them.

Background concept	Kernel adoption (later chapters)
Content addressing	definition digests; hash-bound artifacts
Canonical encoding (strict)	evidence encoder; tagged decimals
Injectivity	injective reference rendering
Reproducibility discipline	compiler-byte pinning; replay rule
Provenance (generation/use)	execution receipts; registry digests
Integrity vs. authorization	constant-false authorization
Trust roots	stated future-authorization constraints
Threat modeling	per-mechanism threat statements
Isolation ladder honesty	false capability fields in receipts

8.1. On the choice of references. A brief note on why these five citations, and not fifty, anchor the survey. Each was chosen as the consensus entry point to its literature rather than as a research frontier. RFC 8785 [1] is the operative standard for JSON canonicalization—short, normative, and implementable from its text, which makes it the right fixed point for comparing stricter designs against. The reproducible-builds literature is summarized through the practitioner survey of Lamb and Zacchiroli [3], which consolidates a decade of project experience—the failure catalog, the rebuild-and-compare verification model, and the deployment history—in citable form; for the adjacent supply-chain attestation practice, the operative standards are the in-toto attestation framework, the SLSA provenance levels, and the Sigstore signing infrastructure [6, 7, 8], named in Section 3.3. The W3C PROV ontology [2] is the standardized vocabulary for provenance, and citing the standard rather than the surrounding research keeps the chapter's usage checkable against a fixed document. Anderson's *Security Engineering* [4] is the standard synthesis for the integrity/authentication/authorization architecture this chapter leans on; the doctrine predates the book, but the book is where it is assembled with the failure-case evidence that gives it teeth. Shostack [5] plays the same role for threat modeling: the four-question frame and the per-element STRIDE practice as taught to working engineers. Topics for which the chapter cites nothing by name—hash-function internals, XML canonicalization history, container isolation mechanics—are treated at the level of common professional knowledge, and readers wanting depth should consult the standard texts of those areas; the chapter's obligations are to represent the consensus accurately and to claim none of it, and both are easier to audit when the reference list is short enough to actually read.

8.2. Composition pitfalls. A final piece of background: the ways in which correct components of this stack compose incorrectly, since several kernel rules exist to block specific compositions rather than specific components. Four pitfalls recur in the literature and in audit experience.

The normalization sandwich. Content is canonicalized, hashed, then re-serialized by a downstream component with different rules, and the re-serialization is later re-hashed for comparison. Both serializers are individually deterministic; the composition is not. Remedy: hash once at a declared boundary and transport the hashed bytes, or re-canonicalize with the identical, versioned encoder— never with a different one.

The identity switch. One component compares raw digests, another compares normalized digests (line endings, Unicode form, case), and records flow between them. Each comparison is locally sound; jointly they partition the artifact space differently, and an artifact pair can be "equal" in one component and "different" in the next—an inconsistency an adversary can steer. Remedy: carry each identity as its own named field, declare which field each comparison uses, and forbid substitution—the two-identity discipline noted in Section 1.3.

The authority creep. An integrity artifact is consumed by a chain of components, each of which adds a small interpretive step, until the last consumer treats it as authorization—the promotion gradient of the introduction, restated as a composition failure. Remedy: type the rungs (Section 5.2) so that each component's input and output declare their rung, and any rung-raising step requires the explicit machinery (a signature, a policy check) rather than a paraphrase.

The boundary mirage. A containment or validation boundary is drawn in the design and assumed by downstream components, but the mechanism enforcing it was weakened or removed—the sandbox that became a process, the schema check that became a warning. Consumers inherit a false premise. Remedy: capability honesty (Section 7.3): receipts carry

the enforced boundary explicitly, including its negative fields, so that a weakened boundary changes the receipts and the change is visible.

The common shape of all four is that composition erases context— which encoder, which identity, which rung, which boundary—and the common remedy is to make the context travel with the artifact as typed fields that fail closed on mismatch. That sentence is, in compressed form, the design brief the kernel sets itself; the mechanisms of the following chapters are its elaboration.

Claim Boundary 2.3 (Non-novelty of this chapter). Nothing surveyed in this chapter is original to the CAS Core 8 project. Cryptographic hashing, content addressing, canonical serialization and RFC 8785 [1], reproducible builds [3], the PROV provenance model [2], the integrity/authorization doctrine of security engineering [4], threat-modeling method [5], and the process-isolation/sandboxing distinction are community property, cited here as standard background. Where later chapters describe kernel mechanisms in this vocabulary, the reader should attribute the vocabulary and the underlying techniques to this literature, and to the kernel only the particular composition, the particular domain restrictions, and the particular refusals it makes—each of which remains, per the kernel's own boundary, an unreviewed Ring-4 research candidate and not a licensed or authoritative artifact.

CHAPTER 3

Semantic Identity

This chapter develops the first and most foundational of the kernel's five mechanisms: the semantic-identity layer. Its subject is the question that the introduction's first failure class showed the program cannot afford to leave implicit—*when do two mentions name the same thing?*—and its method is to make the answer a matter of typed structure rather than of string coincidence or reader discipline. The chapter proceeds from the five-field structural reference and the rationale for each field (Section 1), through the definition-hash binding that detects definitional drift (Section 2), the disjoint namespace catalog (Section 3), the link-kind system for cross-ontology correspondence and the historical incident that motivates its central refusal (Section 4), the injective-rendering invariant (Section 5), and the exact-type equality rule (Section 6), to worked examples of legal and illegal usage (Section 9) and a closing statement of what the layer does not claim (Section 10).

Throughout, the exposition is grounded in the reference implementation: the module `src/cas_core_v8/semantics.py`, which defines the reference type, the namespace enumeration, the crosswalk type, and the exact-match predicate, and the module `src/cas_core_v8/identity.py`, which supplies the validation primitives the reference type applies to its fields. Where the chapter states what the code does, it reports the code as read; where it states why, it reconstructs the design argument from the governing specification and paper; and where the sources are silent on a rationale, the chapter says so rather than supplying one.

1. The five-field structural reference

Definition 3.1 (Semantic reference). A *semantic reference* is the structural tuple

$$(\text{namespace},\ \text{local_id},\ \text{version},\ \text{source_id},\ \text{definition_sha256}),$$

where the namespace is drawn from the closed catalog of Section 3, `local_id` and `source_id` are validated ASCII machine identifiers, `version` is a non-blank canonical text, and `definition_sha256` is the SHA-256 digest, in normalized uppercase hexadecimal, of the exact defining text of the identified element in the identified source. Identity of references is structural equality of the full tuple; no proper subset of the fields identifies.

In the implementation, Definition 3.1 is the frozen dataclass `SemanticRef` with exactly these five fields, declared with `frozen=True` and `slots=True` and validated exhaustively in `__post_init__`: the namespace must be an exact `SemanticNamespace` member; `local_id`, `version`, and `source_id` must be non-blank, trimmed strings free of forbidden Unicode categories; `local_id` and `source_id` must additionally pass the ASCII-identifier profile; and `definition_sha256` is normalized through the strict digest validator. A value that fails any check does not exist as a `SemanticRef`: construction raises `SemanticValidationError`. There is no partially valid reference, no reference with a pending or empty field, and no mutation path after construction. The docstring's one-line summary is accurate enough to quote in paraphrase: one versioned semantic object, bound to its defining source bytes.

The remainder of this section walks the five fields individually, because each is there to kill a specific drift mechanism from the introduction's taxonomy, and the design is best understood field-by-threat.

1.1. Namespace: which family. The namespace field answers the token-conflation problem directly. The string `"F5"` is not a reference; a reference says *which family's* `F5`, as a typed enumeration member, not as a naming convention. Because the field is an enumeration and the constructor requires `type(namespace) is SemanticNamespace`, there is no way to hold a reference whose family is a guess: strings that merely spell a namespace's name do not pass, and neither do subclass or look-alike enum members. Two references with different namespaces are unequal regardless of every other field, which is the formal rendering of the canon fact that the families are disjoint. The deeper effect is on tooling: any join, index, or cache keyed on references inherits the namespace distinction for free, so the silent cross-family merge that untyped strings invite becomes structurally impossible at every downstream site, not just at disciplined ones.

1.2. Local identifier: which element. The `local_id` names the element within its family. It is deliberately a *machine* identifier, validated against the ASCII profile of Section 7: non-blank, trimmed, ASCII-only, NFKC-stable, and drawn from an explicit identifier alphabet. The profile exists because local identifiers are exactly the strings humans review in diffs, registries, and receipts, and the review channel is where invisible-character and confusable spoofing does its damage: an identifier containing a zero-width character or a bidirectional override renders identically to its honest twin while being byte-distinct. The profile excludes the entire class by construction—control, format, and surrogate categories are rejected, non-ASCII is rejected, and NFKC-instability is rejected—so that two local identifiers that look the same to a reviewer are the same bytes. Prose names, by contrast, are not local identifiers and do not belong in this field; the layer's contract is that unrestricted text lives in separate fields (such as the crosswalk note of Section 4) and never participates in identity.

1.3. Version: which edition. The version field carries the edition of the element's definition within its source—the answer to version drift. Its validation is lighter than the identifier profile: the implementation requires a non-blank, trimmed string free of forbidden Unicode categories, but does not force the ASCII identifier alphabet, so version designations follow the source's own versioning conventions. The governing sources do not further constrain the version field's format, and this chapter accordingly claims nothing about comparability, ordering, or interpretation of version strings: two references with different versions are simply different references. That refusal to interpret is itself the point. A version field with built-in ordering semantics would tempt tooling to treat "newer" as "interchangeable-with"; the structural rule that any version difference breaks identity forces every cross-version inference through an explicit, reviewable act instead.

1.4. Source identifier: which corpus. The `source_id` names the corpus or document in which the defining text lives, under the same ASCII identifier profile as `local_id`. Its role is provenance at the identity layer: the same element name, at the same version string, cited from two different sources, is two different references. This is the field that makes corroboration distinguishable from repetition—two analyses grounded in the same source are visibly so—and it is also the hook by which references connect to the program's content-addressed artifact discipline, since a source identifier can be resolved, in registries and receipts, to hash-bound source bytes. The sources governing this chapter do not fix a global catalog of source identifiers; the field's validity is syntactic, and the binding of identifiers to actual

corpora is registry business outside this layer, a division of labor the chapter returns to in Section 10.

1.5. Definition digest: which exact words. The fifth field binds the reference to the SHA-256 digest of the exact defining text of the element in the identified source. It is validated as exactly sixty-four hexadecimal digits and normalized to uppercase, so that presentation-case differences cannot make equal digests unequal. Its rationale is important enough to carry its own section, next.

1.6. Why five fields, and not fewer or more. The tuple's arity is worth defending directly, because each of the obvious alternatives fails against a named drift mechanism from the introduction's taxonomy.

One field (the bare name) is the status quo ante and the proven failure: it answers none of the drift mechanisms and is the representation in which the program's incidents occurred.

Two fields (namespace and name) kill token conflation and nothing else: definitional, version, and source drift all remain invisible, and the flattened form remains collidable without a rendering discipline.

An opaque surrogate (a UUID or database key) is superficially attractive—globally unique, cheap to compare—but it identifies a *row*, not a *commitment*. Nothing in a surrogate is checkable against the world: two parties cannot verify, from the identifier alone, that they mean the same element as defined by the same bytes, and every question about family, edition, or source requires a resolution service that becomes a new trust dependency and a new drift locus. The five-field tuple is deliberately self-describing: every field is either directly meaningful or directly checkable by rehashing, with no resolver in the loop for the identity question itself.

A single content hash of the definition (dropping the naming fields) fails in the other direction: it commits to bytes but cannot say what the bytes are *for*. Two sources quoting the same definitional sentence would collapse into one reference; elements whose defining texts coincide (short definitions do collide textually) would be conflated; and the reference would be unreadable in review without a reverse lookup. Names without hashes drift; hashes without names are mute; the tuple carries both because identity here must serve both machines and reviewers.

More fields were available—a digest of the whole source corpus, a canon-edition field, a human-readable gloss—and the governing sources do not record a rationale for each exclusion; the chapter notes only the structural pattern in what was included: every field present participates in identity and is validated by a named grade of the string discipline, whereas glosses and corpus digests belong to registries and provenance records, which already carry them. Widening identity widens every equality comparison in the system; the tuple is the minimal set that answers the taxonomy, and minimality here is a reviewability feature.

2. Definition-hash binding and definitional drift

2.1. Naming a concept as defined by particular bytes. The first four fields make a reference precise about family, element, edition, and corpus. The fifth makes it precise about something subtler: the actual words. A reference does not merely name a concept; it names a concept *as defined by particular bytes*, and it carries a commitment to those bytes that any party can check by rehashing the defining text.

To see why the first four fields are not enough, consider what they leave open. A version string is an assertion by the source's editors that a text is a particular edition; it is not a

function of the text. Sources get re-released with silent corrections under unchanged version labels; working copies diverge from published copies; a definitional sentence is touched in one checkout and not another; a transcription introduces a variant that nobody flags as an edition. In each of these cases, two parties can hold references agreeing on all four leading fields while reading different defining texts—which is precisely the definitional drift of the introduction: identical names in good faith, divergent meanings in fact. The digest closes the gap because it is computed *from* the text rather than asserted *about* it. If the defining bytes differ anywhere, the digests differ, the references are unequal, and the drift that was invisible as a latent ambiguity becomes visible as a plain inequality between two records.

2.2. Staleness as a feature. The binding has a consequence that looks like a cost and is actually the mechanism working: when a canon definition is corrected, every reference bound to the superseded text becomes *stale*—its digest no longer matches what the current source says. Nothing breaks silently; rather, every artifact carrying a stale reference is now mechanically enumerable, which is exactly the recheck obligation that untyped representations make undiscoverable. The program's canon maintenance already imposes the obligation in prose (derived artifacts are to be rechecked when a governing definition changes); the digest field is what turns that prose obligation into a query. It is worth being precise about what staleness means and does not mean: a stale reference is not invalid—it truthfully names the element as defined by the old bytes, which is often exactly what a historical artifact should say—but it is visibly non-current, and any pipeline that requires currency can enforce it by comparing digests against the live source. The layer itself takes no position on whether stale references are acceptable in a given context; it only guarantees that the question is decidable.

2.3. What the digest does not do. Three non-claims, in the spirit of the monograph's discipline. First, the digest binds text, not meaning: two byte-identical definitions are one definition, but the layer has no opinion on whether two byte-different definitions are semantically equivalent paraphrases—that is canon governance, not bookkeeping. Second, the digest does not by itself prove that the named text is really the defining text of the named element in the named source; a reference constructed with a digest of the wrong passage is well-formed and wrong. Verifying the binding requires resolving the source and rehashing, which is registry and review business. Third, the field inherits the general limits of hashing from the background chapter: it is evidence about content, never about authority, and its strength is the strength of SHA-256's collision resistance, which the kernel consumes as standard background rather than re-arguing.

3. The namespace catalog

3.1. The catalog and its disjointness. The governing specification names seven disjoint namespace families: TKS Foundation, TKS World, CAS Drive, CAS Domain, CAS Register, CAS State, and Noetic Topos object. The implementation's enumeration, `SemanticNamespace`, declares nine members:

- `TKS_FOUNDATION`, `TKS_WORLD`, and `TKS_NOETIC` on the TKS side;
- `CAS_DRIVE`, `CAS_DOMAIN`, `CAS_REGISTER`, `CAS_NOETIC`, and `CAS_STATE` on the CAS side; and
- `NOETIC_TOPOS_OBJECT` for the categorical formulation.

The two extra members relative to the specification's prose list, `TKS_NOETIC` and `CAS_NOETIC`, give the Noetic family *separate namespaces on each side of the TKS/CAS divide*. The governing sources do not state the rationale in so many words, and the chapter reports the

natural reading as a reading: the TKS Noetics and the CAS Noetics stand in exactly the kind of positional, historically loaded correspondence that the layer exists to keep honest, so folding them into one namespace would build the layer's own failure mode into its catalog. Under the nine-member enumeration, relating a TKS Noetic to its CAS counterpart requires a typed crosswalk like any other cross-family relation. Readers comparing specification and implementation should treat the enumeration as the operative catalog; the specification's seven-family list is not contradicted by it, since the specification's "Noetic Topos object" family remains its own namespace and the Noetic split refines rather than reshuffles the prose taxonomy.

Table 1 records the operative catalog with the family each member carries. The descriptions are orientation, not definition: what each family's elements *are* is fixed by the program's canon documents, outside this layer, and the table deliberately says only enough to keep the members apart.

TABLE 1. The operative namespace catalog (`SemanticNamespace`). Descriptions are orientational; the canon owns the definitions.

Member	Family carried
`TKS_FOUNDATION`	TKS Foundation positions
`TKS_WORLD`	TKS World axis elements
`TKS_NOETIC`	Noetic family, TKS side
`CAS_DRIVE`	CAS motivational Drives
`CAS_DOMAIN`	CAS processing Domains
`CAS_REGISTER`	CAS Register axis values
`CAS_NOETIC`	Noetic family, CAS side
`CAS_STATE`	CAS Cognitive States
`NOETIC_TOPOS_OBJECT`	objects of the categorical formulation

Disjointness needs no runtime machinery: it is a property of the type. An enumeration member belongs to exactly one enumeration, every reference carries exactly one member, and equality of references requires equality of members. There is no subtyping among namespaces, no namespace wildcard, and no reference that is "in" two families. The catalog is also *closed*: callers cannot mint namespaces at run time, because the enumeration is the type, and extending it is a source-level, review-visible change to the kernel. Closure is a deliberate cost. A research program discovers new families, and when it does, the kernel must change through its gate rather than accept an open-world namespace string—the design-inversion thesis applied to vocabulary.

3.2. Namespaces are not a hierarchy. One further negative deserves emphasis because the program's ontologies invite it: the catalog encodes no relations among namespaces. CAS States are, in the program's own canon, generated by crossing Noetics with Domains; nothing in the namespace layer represents that fact, and `CAS_STATE` is not a composite, child, or product namespace of `CAS_NOETIC` and `CAS_DOMAIN` as far as identity is concerned. This is intentional flattening. Structural relations among families are semantic content, owned by the canon and representable—where needed—by explicit typed links or by future reviewed extensions; baking them into identity would make the identity layer a theory of the domain, which is exactly what it must not be. The layer's job is to keep names apart, not to explain them.

4. Link kinds: relating without identifying

4.1. The crosswalk object. Disjoint namespaces solve conflation and immediately create a new need: the program's cross-family correspondences are real and analytically load-bearing, so the layer must offer a way to state them—one that makes stating them a deliberate, reviewable act. The implementation's carrier is the frozen dataclass `SemanticCrosswalk`, with fields: a validated ASCII `crosswalk_id`; `source` and `target`, each an exact `SemanticRef`; a `relation` drawn from the enumeration `CrosswalkRelation`; a `source_artifact_sha256` digest identifying the artifact that grounds the asserted relation; a mandatory non-blank `note` of canonical text; and a research `boundary` that the constructor requires to be exactly the frozen Ring-4 open value—admission OPEN, product NOT LICENSED, disclosure false—via the exact-match predicate `is_exact_ring4_open`.

Three of these fields enforce, at construction time, that a crosswalk is an evidence-bearing assertion rather than a bare arrow. The digest field means every crosswalk names the bytes of the artifact on whose authority the correspondence is asserted: there is no crosswalk "because everyone knows." The mandatory note means every crosswalk carries a human-readable justification; an empty or whitespace note is a construction error, so the class of undocumented links is empty by type. And the pinned Ring-4 boundary means no crosswalk can present itself as licensed or disclosable: whatever a correspondence is, in this kernel it is research material, and the object says so in a machine-checkable field that the constructor refuses to let vary.

4.2. The relation kinds. The enumeration `CrosswalkRelation` has four members: `IDENTITY`, `TRANSLATION`, `ALIGNMENT`, and `HYPOTHESIS`. The governing sources specify their intended use at the level that matters for safety: a positional or historical correspondence is represented as TRANSLATION, ALIGNMENT, or HYPOTHESIS, and is *never* silently represented as IDENTITY. The sources do not supply formal definitions distinguishing the three non-identity kinds from one another, and the chapter therefore offers only their natural readings, marked as exposition: a *translation* link records a documented rendering of an element of one system in another system's terms; an *alignment* records a structural or positional correspondence between systems without asserting that either renders the other; and a *hypothesis* records a conjectured relation awaiting review. What the layer enforces is not this gloss but the machine-checkable part: the kind is a typed field, it is part of the record, and no kind is coercible into another or into identity. A consumer that treats a `HYPOTHESIS` link as if it were a `TRANSLATION` is misusing a visibly typed record —an auditable error, not a silent one.

RULE 3.2 (No silent identity). Cross-namespace identity is unrepresentable. The constructor of `SemanticCrosswalk` permits the relation `IDENTITY` only when the exact-match predicate `semantic_refs_match` holds between source and target— that is, only when the two endpoints are the very same versioned reference, field for field. Since references in different namespaces never match exactly, every cross-namespace crosswalk must carry a non-identity kind, and the strongest cross-family statement the type system can express is an explicitly typed, noted, source-backed, Ring-4 link.

The implementation's error message for a violation is worth reporting because it shows the historical failure written directly into the code's contract: an `IDENTITY` crosswalk whose endpoints differ raises a validation error stating that identity is permitted only for the exact same versioned reference, and naming the two canonical temptations—TKS Foundations related to CAS Drives, and TKS Worlds related to CAS Domains—as pairs that must use a non-identity relation. The degenerate self-link that remains legal (an `IDENTITY` crosswalk

from a reference to itself) asserts nothing beyond what structural equality already gives, and its admission keeps the enumeration honest: identity exists as a kind, but the only thing it can ever relate is a thing to itself.

4.3. The crosswalk as a review artifact. Read the crosswalk against the provenance background of the survey chapter: the object is best understood as a provenance record specialized to one assertion kind. In provenance vocabulary, a crosswalk is an entity asserting a relation between two other entities (the endpoint references), derived from a grounding entity (the artifact named by `source_artifact_sha256`), with the asserter's justification carried in the note. What the kernel adds to the generic pattern is fail-closed completeness: the generic anti-patterns—derivation without a grounding entity, assertion without justification, narrative provenance—are each excluded at construction rather than discouraged in guidance.

This shape makes the crosswalk reviewable in a specific, checklist-like way, and spelling the checklist out shows what the type does and does not automate. A reviewer confronted with a crosswalk record can ask: (i) do the endpoint references resolve —do their sources exist and do their definition digests match the cited passages? (ii) does the grounding artifact, fetched by its digest, actually assert or evidence the claimed correspondence? (iii) is the relation kind the strongest the grounding artifact licenses, and in particular, would the grounding artifact license more than `HYPOTHESIS`? (iv) does the note fairly summarize the grounds? Questions (i) and the digest half of (ii) are mechanical; questions (iii) and (iv) are judgment. The type's contribution is that every judgment question arrives with its materials attached and its scope bounded—one relation, one grounds, one note—rather than diffused through a document. The layer does not review; it makes review tractable, which is the same division of labor the kernel draws everywhere between structural consistency and human authority.

A related operational point: because crosswalks are immutable values, a correspondence that is later revised—upgraded from `HYPOTHESIS` to `TRANSLATION` after review, or withdrawn—is represented by a *new* crosswalk (and, where registries track it, a status change at the registry layer), not by mutation of the old record. The old assertion remains what it was, permanently citable as what was once claimed on what grounds. The governing sources do not specify a crosswalk versioning or supersession scheme beyond this immutability, and the chapter accordingly leaves lifecycle policy to the registry chapters, noting only that immutability is what makes any such policy auditable.

Adverse Finding 3.3 (The motivating incident). The rule's justification is historical before it is theoretical. A prior work product in the program conflated positionally corresponding Drive and Foundation names—treating the correspondence as paraphrase rather than as relabel—and was voided in review. The introduction analyzed why the artifacts of the time could not resist the error: the names were untyped strings, and no representation existed in which the correspondence could be stated *without* being stated as identity. The crosswalk type is the direct answer. Under it, the same analytical move the incident attempted is expressible—as an `ALIGNMENT` or `TRANSLATION` with a note and a source artifact—and the move the incident actually made is not expressible at all.

Threat 3.4 (Namespace conflation). An agent or tool treats a positional correspondence between families as identity and propagates entailments across the boundary. *Blocked by* Definition 3.1 and Rule 3.2: distinct namespaces make the endpoint references unequal, and the crosswalk constructor refuses the identity kind for unequal endpoints, so the conflation has no representation; the nearest representable statement is a typed non-identity link carrying its own note, source digest, and research boundary.

5. The injective-rendering invariant

5.1. The problem with flattening. Structured references must sometimes travel as strings: as keys in stores, as identifiers in logs and receipts, as line items in review documents. Flattening is where a structurally sound design can quietly reintroduce collision. The cautionary construction is the naive delimiter join: render the tuple as `namespace:local_id:version:source_id:dige` and two distinct tuples collide whenever a component contains the delimiter—the pair (`a`, `b:c`) and the pair (`a:b`, `c`) flatten identically. An attacker who controls any component can manufacture such collisions deliberately, aliasing one reference's records to another's; an unlucky honest value can do it by accident. Either way, every property the tuple design earned is forfeited at the string boundary.

Invariant 3.5 (Injective rendering). The rendered string form of a semantic reference is injective on structural tuples: two distinct tuples render to distinct strings, and delimiter-containing components cannot cause the rendering of one tuple to equal the rendering of another.

5.2. The design argument. The implementation renders a reference through the property `qualified_id`, which serializes the five components, in fixed order, as a JSON array with compact separators and non-ASCII preserved, and prefixes the result with the versioned scheme tag `semantic-ref.v1:`. The injectivity argument has three steps, each carried by a specific feature of this construction.

Proposition 3.6 (Injectivity of the rendering). *The map from valid `SemanticRef` tuples to their `qualified_id` strings is injective.*

Design argument. First, *escaping*: JSON string encoding escapes every delimiter-significant character—quotation marks, backslashes, and the structural commas and brackets appear inside components only in escaped or quoted positions—so a component's content can never terminate its own string early or open a neighboring one. This is the length-prefixing role played generically: an encoding in which component boundaries are unforgeable from component content. Second, *fixed arity and order*: the rendering is always a five-element array in declared field order, so no reading of the string can reassign bytes across field boundaries or between differently-shaped tuples. Third, *unique parseability*: JSON array syntax parses unambiguously, so the composite admits exactly one decomposition into components; two tuples differing in any field therefore differ at the first divergent component of the unique parse, and their renderings differ as strings. The versioned prefix adds a fourth, forward-looking guarantee: any future rendering scheme carries a different tag, so strings from different schemes can never be confused as one another. The property is enforced by construction and is regression-tested with delimiter-bearing component fixtures—components containing quotation marks, backslashes, colons, commas, and bracket characters—whose pairwise renderings the suite checks for distinctness.

5.3. The design space: escaping, length-prefixing, hashing. Three standard techniques achieve injective flattening, and the choice among them is instructive. *Length-prefixing* encodes each component as its byte length followed by its bytes; boundaries are unforgeable because they are computed, not delimited. It is the technique of canonical binary formats, maximally simple to prove correct, and minimally readable: a length-prefixed identifier is opaque to a human reviewer, which matters in a layer whose strings appear in diffs and review documents. *Escaping* keeps the composite readable—components appear as themselves,

delimiters are visible—at the price that correctness now depends on the escaping rules covering every delimiter-significant character, in every implementation that ever parses the form. Hand-rolled escape schemes are historically where this goes wrong; adopting a ubiquitous, rigorously specified escape discipline (here, JSON string encoding) buys the readability of escaping while outsourcing the hard part to a format with decades of cross-implementation hardening and existing conformance suites. *Hashing the tuple* (rendering as a digest of a canonical encoding) is injective up to collision resistance and produces fixed-length names, but is doubly opaque—neither the fields nor their boundaries survive—and forces every consumer through a resolution step; it is the right shape for storage keys, not for review-facing identifiers.

The implementation's choice—JSON-escaped, fixed-arity, version-prefixed—sits deliberately at the readable end: a `qualified_id` can be eyeballed in a receipt, its fields recognized, its scheme tag checked, and, when needed, parsed back losslessly with a stock parser. The versioned prefix is the hedge that keeps the choice revisable: a future scheme (length-prefixed, hashed, or otherwise) would carry a different tag, and Proposition 3.6's cross-scheme clause keeps the generations from colliding. The governing sources do not record a comparison of alternatives; the analysis above is the chapter's reconstruction of the standard trade, offered so that a reviewer can check the choice against its space rather than take the space on faith.

Two remarks temper the proposition properly. The argument is a design argument over the encoding, not a mechanized proof; the cited fixtures exercise the adversarial component classes the design anticipates, and the general claim inherits the usual fixture-bound limits stated in the introduction. And injectivity is claimed for the rendering of *valid* references—the validation layer has already excluded control characters and unpaired surrogates from components, which removes the classes of input on which JSON string encoding across implementations is historically least uniform.

THREAT 3.7 (Identifier collision). Crafted components cause two distinct references to render to one identifier, aliasing one reference's records, receipts, or registry rows to another's. *Blocked by* Invariant 3.5 via Proposition 3.6: component content cannot forge component boundaries, so the aliasing string does not exist.

6. The exact-type equality rule

6.1. Virtual equality as an attack surface. In an object-oriented host language, equality is a method, and methods can be overridden. This ordinarily benign fact becomes an attack surface the moment equality carries security or evidence weight. Consider a subclass of the reference type that overrides equality and hashing: instances can report themselves equal to any honest reference—matching it in every comparison, colliding with it in every hash-keyed store—while carrying arbitrary divergent field values and arbitrary overridden behavior. Every downstream check that asks "is this the reference I expected?" through the virtual equality protocol receives the answer the impostor chooses. The same construction threatens boundary objects: a subclass of the research-boundary type whose accessors report OPEN and NOT LICENSED while its stored state says otherwise would pass any check that trusts the protocol. Nothing about this requires exotic machinery; it is a dozen lines in any dynamic language, and it defeats naive structural checking completely.

RULE 3.8 (Exact-type identity). Equality-overriding subclasses are not admitted as identity-bearing values. Identity-bearing types reject subclass instances at construction, and every load-bearing comparison of identity or boundary values is performed by exact-type predicates

that check concrete runtime types and primitive fields directly, without dispatching through overridable equality.

6.2. Both halves in the implementation. The rule has two halves, and the implementation carries both. The first half closes the front door: `SemanticRef`, `SemanticCrosswalk`, and the research-boundary type each begin `__post_init__` by checking `type(self)` against the exact class and raising if a subclass is being constructed; the artifact-identity type in `identity.py` does the same. One cannot obtain an instance of a subclass of any identity-bearing kernel type through its own constructor.

The second half assumes the front door has failed—an impostor object arrived from somewhere—and makes comparisons immune anyway. The predicate `semantic_refs_match` compares two candidate references by checking, in order: that each operand's concrete type `is` exactly `SemanticRef`; that each namespace's concrete type is exactly the namespace enumeration, comparing their primitive values; and that each remaining field is exactly a `str` on both sides and equal as such. At no point does the predicate invoke `==` on the composite objects; it never dispatches through anything an impostor could override. The module applies the same pattern at its other identity boundary (`artifact_identities_match` for byte identities, field by field with type checks), and the crosswalk constructor uses the exact predicate—not virtual equality—when enforcing Rule 3.2, so the no-silent-identity rule cannot be satisfied by an object that merely claims equality.

6.3. The discipline beyond references. The exact-type discipline is not local to the semantic layer; it is the kernel's uniform equality posture, and seeing its second instantiation clarifies the pattern. The byte-identity type `ArtifactIdentity` in `identity.py`—the carrier of the raw-digest and normalized-digest pair that the background chapter's two-identity discipline requires—applies the same two halves: its constructor rejects subclass instances, and its comparison predicate, `artifact_identities_match`, walks every primitive field with an exact `type(...) is` check on both operands before comparing values, never invoking virtual equality. The predicate is verbose by design—seven fields, each with two type checks and one comparison—and the verbosity is the audit trail: a reviewer can confirm, line by line, that no comparison in the function dispatches anywhere overridable.

The pattern generalizes to a rule of thumb the later chapters will keep instantiating: *wherever a comparison gates anything, the comparison is a named, exact-type predicate owned by the kernel.* The alternatives each fail a review question. Virtual equality fails "who controls this comparison?"—the answer is whichever operand's class got there first. Structural equality via serialization (compare the encodings) fails "what does inequality mean?"—an encoding difference localizes poorly and imports the whole encoder into every equality question. Field-by-field comparison without type checks fails the impostor case. The named-predicate pattern answers all three: the kernel controls the comparison, inequality localizes to a field, and impostors fail at the type gate. Its cost, beyond verbosity, is that adding a field to an identity-bearing type requires touching its predicate—which the kernel's authors evidently accepted as a feature, since a field silently excluded from comparison is exactly the kind of drift the discipline exists to prevent; the constructor-side validation and the predicate enumerate the same field lists, and a divergence between them is the sort of defect the focused suites' fixtures are positioned to catch.

Threat 3.9 (Equality spoofing). An adversarial or buggy subclass overrides equality and hashing to impersonate an honest reference or boundary object, passing downstream identity checks while carrying divergent state. *Blocked by* Rule 3.8: construction of the subclass

through kernel constructors raises; and even a foreign-built impostor fails every load-bearing comparison, because the exact-type predicates check concrete types and primitive fields and never call the overridable protocol.

REMARK 3.10 (Cost of the rule). The rule spends real flexibility: well-behaved subclassing—a convenience wrapper, a memoizing variant—is rejected along with the malicious kind, and the exact-type predicates are verbose compared to `==`. The trade is the design inversion again: the kernel prefers a closed, verifiable equality discipline over an open, convenient one, because the open one's safety would rest on global knowledge that no subclass anywhere overrides equality dishonestly—an unreviewable premise in exactly the way the introduction's laundering premise was.

7. The validation pipeline

The reference type's guarantees bottom out in three validation primitives from `identity.py`, shared with the rest of the kernel; the layer is best understood with their exact contracts in view.

`require_canonical_text` accepts only exact `str` values and rejects any character in Unicode categories `Cc` (controls), `Cf` (format characters, including zero-width and bidirectional controls), or `Cs` (surrogates). It is the floor under every textual field in the layer: even free-text fields like the crosswalk note, which accept arbitrary languages and scripts, exclude the invisible-character classes wholesale.

`require_ascii_identifier` layers the machine-identifier profile on that floor: the value must be non-blank and equal to its own trim; ASCII-only; a fixed point of NFKC normalization (excluding compatibility characters that normalize to something else than themselves); and a full match of the identifier alphabet `[A-Za-z0-9][A-Za-z0-9._:/@+\-]*` —an alphanumeric head followed by alphanumerics and a small punctuation set. This is the profile applied to `local_id`, `source_id`, and `crosswalk_id`.

`require_sha256` accepts only exact `str` values matching sixty-four hexadecimal digits after uppercasing, and returns the uppercase normalization; the stored digest field is therefore case-canonical, and the presentation-discipline pitfall of the background chapter (case-mismatched hexadecimal comparing unequal) is closed at intake.

The division of labor among the three primitives reflects the layer's string philosophy: three grades of string—digest, machine identifier, canonical prose—each with a named validator, each field of each type assigned exactly one grade, and no load-bearing comparison ever performed on a string of the prose grade.

8. How the layer composes with the rest of the kernel

Before the worked examples, a short tour of the layer's consumers, since several of its design choices only make full sense in composition.

Canonical encoding. References and crosswalks are evidence values, and both types expose the single declared serialization route the encoding chapter requires: a `to_dict` view whose fields are exactly the validated primitives (plus, for references, the `qualified_id` rendering). The namespace and relation enumerations pass through the encoder's enum tagging, so their encodings are type-distinct from the plain strings that spell their values—a reference whose namespace is the enumeration member and a hand-built map whose namespace is the string `"CAS_DRIVE"` cannot hash equal. The validation grades of Section 7 also pre-satisfy the encoder's text rules: by the time a reference exists, its strings already contain no control or

format characters, so a valid reference is always encodable and the encoder never becomes a second, later point of identity failure.

Receipts and registries. Execution receipts and registry records carry references in their identity fields, which is where the layer's review-facing choices pay: a receipt naming its semantic profile by `qualified_id` is auditable by eye and parseable by tooling, and a registry row keyed by rendered reference inherits Invariant 3.5 as a no-aliasing guarantee for its keying. Conversely, the layer leans on the registries for everything it deliberately does not do: resolving `source_id` values to hash-bound corpora, tracking crosswalk supersession, and enumerating stale references after a canon change are registry operations over the immutable values this layer mints.

Boundaries and the runtime gate. Every crosswalk carries the exact Ring-4 boundary, and the runtime decision model treats identity-layer values as it treats all structural evidence: sufficient to support nonclaim comparisons, never sufficient to support claim-bearing intent. The layer thus supplies the vocabulary for the kernel's refusals as well as its assertions—when the gate denies a claim-bearing action, the record of the denial names, in this layer's terms, exactly which semantic objects were involved.

9. Worked examples

The following examples are expository constructions in the implementation's vocabulary; digests are abbreviated with ellipses for readability and would be full sixty-four-digit values in real records.

EXAMPLE 3.11 (Legal: two references and a typed correspondence). An analyst needs to relate the fifth TKS Foundation, as defined in a hash-bound edition of the TKS source corpus, to the CAS Drive F5, as defined in a hash-bound edition of the CAS corpus. Two references are constructed:

```
SemanticRef(namespace=TKS_FOUNDATION, local_id="F5", version="corpus-1.0", source_id="tks-
SemanticRef(namespace=CAS_DRIVE, local_id="F5", version="v7.7", source_id="cas-canon", def
```

The equal `local_id` strings are harmless: the references are unequal in namespace, source, and digest, and no kernel operation will merge them. The correspondence is then stated as:

```
SemanticCrosswalk(crosswalk_id="xw-f5-drive5-0001", source=<the TKS ref>, target=<the
CAS ref>, relation=ALIGNMENT, source_artifact_sha256="7D10...EA", note="Positional
correspondence per the translation key; not a paraphrase.")
```

The crosswalk constructs successfully: endpoints are exact references, the kind is non-identity, the note is non-blank, the grounding artifact is named by digest, and the default boundary is the exact Ring-4 open value. The record asserts precisely what the canon licenses—a documented positional alignment—and nothing stronger.

EXAMPLE 3.12 (Illegal: the conflation, replayed). The same analyst—or an automated agent summarizing the analyst's work—attempts the historical error: the same two references with `relation=IDENTITY`. Construction raises `SemanticValidationError`; the message states that identity is permitted only for the exact same versioned reference and names the Foundation/Drive and World/Domain pairs as required non-identity cases. There is no flag to pass, no permissive mode, and no alternative constructor. The error is not a policy lookup that could be misconfigured; it is the type refusing to hold the value.

EXAMPLE 3.13 (Illegal: malformed fields). Each of the following constructions fails with a validation error naming the offending field: a namespace supplied as the plain string `"CAS_DRIVE"` rather than the enumeration member (namespace must be a `SemanticNamespace`);

a `local_id` of `" F5"` or `"F5 "` (surrounding whitespace), `""` (blank), or `"F 5"` with a zero-width space between the characters (forbidden format category—the case that is invisible in review and exists precisely to be caught here); a `local_id` containing non-ASCII letters or failing NFKC stability; a `definition_sha256` of the wrong length, with non-hex characters, or absent. A lowercase but otherwise well-formed digest, by contrast, is *accepted and normalized* to uppercase—case is presentation, not identity.

Example 3.14 (Illegal: the impostor). A subclass `class LooksLikeRef(SemanticRef)` overriding `__eq__` to return true against any reference cannot be constructed: the base class's `__post_init__` raises on the subclass type before any field is validated. An impostor built without the constructor still fails every kernel comparison, because `semantic_refs_match` tests `type(left) is SemanticRef` before touching any field. The spoof has no purchase at either surface.

Example 3.15 (Legal but delimiter-bearing: rendering under stress). A version string is prose-grade text and may lawfully contain JSON-significant characters—say `version='1.0 "corrected", second printing'`. The `qualified_id` rendering escapes the quotation marks within the JSON string encoding, and no choice of other components in any other reference can produce the same rendered string: the delimiter fixtures of Proposition 3.6 exercise exactly this class. The naive colon-join of Section 5.1 would have been ambiguous here; the versioned JSON rendering is not.

Example 3.16 (Definitional drift, detected end to end). A final worked scenario exercises the digest field's purpose across time. In month one, an analyst binds a reference to a canon element: namespace `CAS_DRIVE`, `local_id "F2"`, version `"v7.7"`, source `"cas-canon"`, and the digest `"51BC...D3"` of the element's defining sentence as it stands in the analyst's hash-bound checkout. The reference enters a dozen analysis records and one crosswalk. In month three, the canon receives a corrected printing: the defining sentence of the same element is amended—one clause tightened—without a change to the version designation, an event the program's history shows is realistic. A colleague working from the corrected corpus constructs the "same" reference and obtains digest `"E907...2A"`.

Nothing fails, and that is the design: both references are valid, each truthfully names the element as defined by particular bytes, and they are *unequal*. The inequality surfaces exactly where it should: a replay comparison between the analyst's records and the colleague's reports a reference mismatch localized to the digest field; a registry query for records bound to the old digest enumerates precisely the dozen artifacts owing a recheck; and the crosswalk, whose endpoint no longer matches the live source, is flagged stale by the same comparison. Under a four-field design, every one of these records would have continued to match while meaning different words; the month-three divergence would have been discovered, if ever, as a subtle analytical disagreement with no mechanical cause. The digest field converts it into bookkeeping —which is the entire thesis of the layer in one scenario.

10. What the identity layer does not claim

Claim Boundary 3.17 (Limits of semantic identity). The layer guarantees, to the strength of its design arguments and its cited fixtures: that references are structural, disjointly namespaced, definition-bound, injectively renderable, and comparable only by exact type and field; and that cross-namespace identity is unrepresentable. It does not and cannot guarantee: that a reference's digest was computed over the correct defining passage (a registry and review obligation); that any crosswalk's asserted correspondence is true (the crosswalk

records an assertion and its grounds, not a fact); that the namespace catalog is complete for the program's future needs (extension is a reviewed kernel change); or that any use of the layer confers semantic correctness, empirical validity, or authority of any kind. Every value the layer produces carries, or is governed by, the Ring-4 research boundary—admission OPEN, product NOT LICENSED, disclosure withheld—and nothing in this chapter's mechanisms alters that boundary. The bounded receipts that exercise this layer (reference identity, rendering distinctness under delimiter fixtures, crosswalk rejection cases, exact-type rejections) are historical, fixture-bound software evidence in the sense fixed by the introduction, and support no claim beyond tested behavior under those fixtures.

REMARK 3.18 (The adversary model, stated plainly). A reader may reasonably ask who the adversary of this chapter's threat statements is supposed to be, since a research kernel is not an internet-facing service. The honest answer, consistent with the program's history, is threefold, in ascending order of intent. First, *entropy*: honest tools and honest authors making locally reasonable choices—a different serializer, a convenient string join, a subclass for a quick experiment—whose composition produces conflation and aliasing with no adversary anywhere; most of the layer's rules earn their keep against this adversary alone. Second, *the program's own generative tooling*: AI agents that produce code, records, and summaries at scale, whose failure mode is not malice but fluent overreach— constructing the plausible-looking link, the confident identity, the tidy rendering—faster than review can police; fail-closed construction converts this pressure into visible errors at the point of generation. Third, *a deliberate adversary*: the spoofing and collision constructions of this chapter are written as attacks because design against the deliberate case covers the accidental ones, and because a kernel meant to underwrite future claims must assume its inputs will one day be crafted, not merely careless. The threat statements name the third adversary; the first two are the ones the program has actually met, and they are covered a fortiori.

The layer's contribution to the kernel is therefore best stated in the design-inversion vocabulary: it does not detect conflation, it *disarms* it, by making the conflating statement inexpressible and the honest statement cheap. Every mechanism in the chapters that follow—boundaries, encodings, compatibility receipts, registries—consumes references and crosswalks in the forms fixed here, which is why the layer comes first: it is the vocabulary in which the rest of the kernel is prevented from lying.

CHAPTER 4

The Research Boundary Object

1. The problem the boundary object answers

Chapter 1 identified unauthorized claim promotion as one of the two failure classes that motivate the kernel. The failure has a characteristic shape: an artifact that is merely *consistent*—a green test run, a self-computed hash, a well-formed receipt, a ledger row containing the word LICENSED—is read by some downstream consumer as *authorized*, and a research candidate crosses into product territory without any reviewer, authority, or legal act having occurred. The program's own history contains the canonical instance: AI-authored, self-hashed "licensing receipts" that were structurally impeccable and epistemically worthless. Nothing in those artifacts was false in the narrow sense; the failure was that a status which can only be conferred by an external act was inferred from internal structure.

The research boundary object is the kernel's answer to this failure class at the level of representation. Its design premise is that the status of an artifact—where it sits in the admission process, whether it may back a product claim, whether it may be publicly disclosed—must be (i) explicit machine-visible data rather than convention, (ii) constrained so that the historically observed contradictory combinations cannot be constructed at all, and (iii) rechecked at every point of use rather than trusted from any earlier point of construction. The present chapter develops each of these three commitments, then dwells at length on the distinction that makes the whole design coherent: the difference between *tamper evidence* and *authority*. The chapter closes with worked scenarios of boundary misuse and the exact mechanism that blocks each one.

Throughout, the claim discipline of the monograph applies. Everything described here is a Ring-4 research candidate: admission OPEN, product NOT LICENSED, disclosure withheld. The chapter describes what the v0.1 candidate's code enforces under its bounded test fixtures; it does not claim security against arbitrary in-process adversaries, does not claim that any boundary object confers or withholds legal status, and does not promote any artifact past the boundary it carries.

CLAIM BOUNDARY 4.1 (Scope of this chapter). The boundary object is a representational and procedural discipline inside a single Python process. It is not an access-control system, not a signature scheme, and not a legal instrument. Statements of the form "the kernel rejects X" mean that the v0.1 candidate raises a typed validation error on the fixture family exercising X in the bound Gate-4 suite; they are historical software receipts, not guarantees over untested inputs.

2. The boundary triple and its carrier

2.1. The triple. The governing specification defines the research boundary as a triple.

DEFINITION 4.2 (Research boundary triple). A *research boundary* is the triple

$$(\text{admission state},\ \text{product state},\ \text{public disclosure}),$$

where the admission state ranges over {OPEN, IN-REVIEW, ACCEPT}, the product state over {NOT LICENSED, LICENSED}, and public disclosure over {false, true}.

The three components answer three different questions, and the design treats the questions as genuinely independent axes that are then coupled by explicit consistency rules rather than being collapsed into a single status enum.

The *admission state* answers: where does this artifact sit in the review process? OPEN means no review act has been recorded; IN-REVIEW means a review is underway; ACCEPT means an admission decision has been recorded. Nothing about the admission state says what the artifact may be used for.

The *product state* answers: may this artifact back a product claim? The only value that permits it is LICENSED, and—as the consistency rules below make precise—that value is constructible only in a configuration the v0.1 kernel never occupies. NOT LICENSED is not a pejorative; it is the honest default of every research artifact.

The *public disclosure* flag answers: has disclosure of this artifact been authorized? The flag is deliberately boolean rather than an enum of disclosure channels: v0.1 needs only the distinction between "no disclosure authorization exists" and "a disclosure authorization has been recorded," and a richer disclosure model would require exactly the authenticated trust root that v0.1 deliberately lacks.

2.2. The carrier and the ring index. In the v0.1 implementation the triple is carried by a frozen record with four fields, because the consistency rules are indexed by a fourth datum: the admission ring.

DEFINITION 4.3 (Boundary carrier). The boundary carrier is the immutable record

(`admission_ring`, `status`, `license_status`, `public_disclosure_authorized`),

where `admission_ring` is an integer in $\{1, 2, 3, 4\}$, `status` is an admission-state enumeration value, `license_status` is a product-state enumeration value, and `public_disclosure_authorized` is a boolean. Each field is validated for exact runtime type at construction: the ring must be exactly an integer (a boolean, though an integer subtype in the host language, is excluded by the registry layer's explicit boolean–integer check, Chapter 6; the enumeration checks here govern the status and license fields), the two states must be exactly members of their enumerations—not strings that happen to spell the same word—and the disclosure flag must be exactly a boolean.

The ring is the program's admission-severity index inherited from the Project Coherence process: Ring 1 and Ring 2 are reserved for reviewed mathematical material, Ring 3 for artifacts that have passed an admission decision, and Ring 4 for open research candidates. The kernel itself assigns no meaning to the rings beyond the consistency constraints below; in particular, nothing in the kernel can move an artifact between rings, because the carrier is immutable and every v0.1 construction site requires the exact Ring-4 configuration.

Two further construction rules deserve emphasis before the consistency rules, because they recur throughout the kernel and were motivated in Chapter 3.

RULE 4.4 (Exact carrier type). A subclass of the boundary carrier is rejected at construction: the constructor checks that the runtime type of the object under construction is exactly the carrier type, not a derived type. Consequently no object with overridden equality, overridden field access, or additional behavior can pose as a boundary value at any kernel interface.

RULE 4.5 (Exact-field predicate). The kernel's boundary predicate—"is this the exact Ring-4 open boundary?"—does not call any overridable equality. It checks, field by field, that the value has exactly the carrier type, that the ring is exactly the integer 4, that the status is exactly the OPEN enumeration member (by identity, not by string comparison), that the product state is exactly NOT LICENSED, and that the disclosure flag is exactly the boolean false.

Rule 4.5 may look paranoid in isolation. Its point is architectural: every consumer of a boundary decides for itself, from primitive fields, what the boundary says. There is no delegation to a method the value itself supplies, because a value that answers questions about its own trustworthiness is exactly the anti-pattern the kernel exists to eliminate. This is the first appearance of a theme that Section 4 makes into an invariant.

3. Joint consistency and construction-time rejection

3.1. The three consistency rules. The four fields of the carrier are not independent. Three rules couple them, and the constructor enforces all three, so that a violating combination is never a value at all—it is a raised validation error.

RULE 4.6 (Ring-4 closure). If `admission_ring` = 4, then the status must be OPEN, the product state must be NOT LICENSED, and the disclosure flag must be false. Equivalently: a Ring-4 artifact is open, unlicensed, and undisclosed, with no exceptions representable.

RULE 4.7 (License precondition). If the product state is LICENSED, then `admission_ring` = 3 and the status is ACCEPT. Equivalently: a licensed product state is representable only for an artifact that has a recorded admission decision at Ring 3.

RULE 4.8 (Disclosure precondition). If the disclosure flag is true, then the status is ACCEPT. Equivalently: no disclosure authorization is representable for an artifact that has not passed admission.

Each rule encodes one historically observed contradiction as an unrepresentable state. Rule 4.6 makes the phrase "a licensed Ring-4 candidate" a type error rather than a category the program has to argue about. Rule 4.7 makes "licensed but never admitted" unrepresentable—this is precisely the shape of the historical self-issued licensing receipts, which asserted a product status with no admission act behind it. Rule 4.8 makes "disclosed but still open" unrepresentable, which blocks the quieter failure in which a draft escapes into public circulation and the disclosure is then rationalized after the fact.

It bears stating plainly what these rules do *not* say. Rule 4.7 does not say that Ring 3 plus ACCEPT *makes* an artifact licensed; it says only that outside that configuration a licensed product state cannot even be written down. Whether a Ring-3 accepted artifact is actually licensed is a question about acts in the world—reviewer identity, legal authority—about which the carrier carries no evidence whatsoever. The consistency rules are necessary-condition filters, never sufficient-condition grants. This asymmetry is the entire content of Section 6, and it is why the kernel's runtime gate (Chapter 9) denies claim-bearing intent even when handed a structurally valid LICENSED boundary.

3.2. Enumerating the state space. Because the component sets are small, the effect of the three rules can be stated exactly.

PROPOSITION 4.9 (Valid boundary count). *The unconstrained carrier state space has* $4 \times 3 \times 2 \times 2 = 48$ *combinations. Exactly* 15 *of them satisfy Rules 4.6–4.8; the remaining* 33 *are rejected at construction.*

CASE ANALYSIS. Consider each ring. For ring 4, Rule 4.6 admits exactly one of the twelve combinations: (OPEN, NOT LICENSED, false). For rings 1 and 2, Rule 4.7 excludes the product state LICENSED entirely (six combinations per ring); among the six remaining unlicensed combinations, Rule 4.8 excludes a true disclosure flag for OPEN and IN-REVIEW (two combinations per ring), leaving four per ring, hence eight. For ring 3, the unlicensed half contributes the same four; the licensed half is admitted by Rule 4.7 only with status ACCEPT, for which Rule 4.8 permits either disclosure value, contributing two. Hence $1+8+6 = 15$. □

Table 1 organizes the thirty-three rejected combinations into families by the rule that rejects them; a combination violating more than one rule is listed under the first rule that fires in the constructor's check order.

Family	Description	Count
R4/status	ring 4 with status IN-REVIEW or ACCEPT	8
R4/license	ring 4, status OPEN, product LICENSED	2
R4/disclosure	ring 4, otherwise closed, disclosure true	1
Lic/ring	LICENSED at ring 1 or 2	12
Lic/status	LICENSED at ring 3 without ACCEPT	4
Disc/status	disclosure true without ACCEPT (rings 1–3, unlicensed)	6

TABLE 1. The 33 invalid carrier combinations by rejecting family. Counts reflect the constructor's check order, which tests the Ring-4 closure first; the R4/status family therefore absorbs ring-4 combinations that also violate the license or disclosure preconditions.

The counting exercise is not an ornament. It is the concrete meaning of the design slogan "invalid states are unrepresentable": the program's historically dangerous statuses correspond to identifiable cells of a finite table, and each such cell is a constructor exception, testable one fixture at a time. The Gate-4 suite exercises representatives of each family; the receipt for that suite is bounded software evidence in the sense of Chapter 12, no more.

3.3. The distinguished constructor. The kernel provides a single distinguished factory: the Ring-4 open boundary, producing the unique valid ring-4 carrier (OPEN, NOT LICENSED, false). Every artifact type in the v0.1 kernel—semantic crosswalks, registry records, registries, compatibility receipts—defaults its boundary field to a *fresh* instance from this factory. Two properties of this arrangement matter.

First, freshness: the default is a factory invoked per construction, not a module-level shared instance. No two artifacts share a boundary object, so no single mutation—accidental or otherwise—can silently change the recorded status of a population of artifacts. Sharing a mutable status object across artifacts would create precisely the "shared mutable trust" that Invariant 4.11 forbids.

Second, bias: the distinguished factory is the *most restrictive* boundary, and it is the only one with a privileged constructor. Constructing any other valid boundary—say, a Ring-3 accepted licensed one—requires spelling out every field explicitly. The kernel does not make the dangerous configurations impossible to write down when they are internally consistent (Rule 4.7 deliberately leaves the Ring-3 licensed shape representable, because a future reviewed design will need it); it makes them impossible to *stumble into*, and—as the next section shows— impossible to *use* anywhere in v0.1.

3.4. Formal summary. It is convenient to fix notation used in the rest of the chapter. Let

$$\mathcal{B} \;=\; R \times S \times L \times D, \qquad R = \{1,2,3,4\},\ |S| = 3,\ |L| = 2,\ |D| = 2,$$

be the unconstrained carrier space, and let $\mathcal{V} \subseteq \mathcal{B}$ be the subset satisfying Rules 4.6–4.8, so that $|\mathcal{B}| = 48$ and, by Proposition 4.9, $|\mathcal{V}| = 15$. The constructor computes the characteristic function $\chi_{\mathcal{V}}$ and raises on the complement; the use-site predicate of Rule 4.5 computes the characteristic function of a much smaller set—the singleton containing the Ring-4 element—together with the exact-type conditions that $\chi_{\mathcal{V}}$ cannot express because they concern the carrier's runtime representation rather than its field values.

COROLLARY 4.10 (Uniqueness of the Ring-4 configuration). *The fiber of $\mathcal{V}$ over ring* 4 *is a singleton:* $\mathcal{V} \cap (\{4\} \times S \times L \times D) = \{(4, \text{OPEN}, \text{NOT LICENSED}, \text{false})\}$. *Consequently "a valid Ring-4 boundary" is a definite description: any two valid Ring-4 carriers agree on every field, and the use-site predicate needs no tolerance, range, or case analysis—it tests one point.*

The corollary is why the v0.1 discipline is cheap enough to run everywhere. A predicate that tested membership in a fifteen-element set with exceptions would invite caching and shortcuts; a predicate that tests a single known point, field by field, costs a handful of comparisons and admits no interesting optimization, so there is no engineering pressure to skip it. Representational austerity purchases procedural discipline.

4. The recheck-not-trust invariant

4.1. Statement.

INVARIANT 4.11 (Recheck, do not trust). Every public operation of the kernel derives the boundary it enforces from its own inputs and rechecks that boundary at the point of use. No operation trusts a boundary because some earlier operation checked it; no operation consults a shared mutable object to learn a status; and no operation accepts a recomputable self-hash as evidence that a boundary is still what it was at construction.

The invariant has three clauses, and each is worth unpacking, because each closes a distinct loophole through which construction-time validation alone would leak.

4.2. Derive and recheck at use. Construction-time rejection (Section 3) guarantees that no invalid combination is ever a value. It does not guarantee that a value checked yesterday is valid today, for two reasons. The first is mundane: the host language provides no in-process memory protection. The carrier is a frozen record, which resists *accidental* mutation—ordinary attribute assignment raises—but a determined caller in the same process can bypass the freeze through the language's low-level attribute machinery. The kernel does not pretend otherwise. The second reason is architectural: values flow. A boundary object constructed in one component is embedded in an artifact, passed across an interface, stored, retrieved, and eventually consulted by a component that never saw its construction. If validity were checked only at construction, the checking component and the relying component would be different, and the reliance would be exactly the kind of unexamined trust the kernel is built to avoid.

The v0.1 discipline is therefore that every use site re-runs the exact-field predicate of Rule 4.5. A crosswalk validates its boundary when constructed *and* the registry validates the boundaries of its records when loaded *and* the registry re-validates them when a record is resolved *and* the record re-validates its own boundary inside every integrity check. The pattern is deliberately redundant. Redundant checking of cheap predicates is the kernel's

substitute for the trusted intermediary it refuses to have: instead of one blessed component whose word is final, there are many unprivileged components each of which verifies what it relies on. The cost is a few field comparisons per operation; the benefit is that a corrupted boundary is caught at the first use after corruption, with a typed error naming the artifact, rather than persisting silently inside a structure that was "already checked."

REMARK 4.12. The recheck discipline also changes what a successful attack would have to look like. Against a check-once design, an attacker mutates a value after its single validation and wins. Against a check-at-every-use design, the attacker must either mutate the value into another *valid* configuration—but the only valid configurations reachable from Ring-4 open change ring, status, or license fields that every v0.1 use site pins to exact Ring-4 values, so the mutated object fails the very next predicate— or subvert the predicate itself, which is a plain function over primitive fields with no dispatch through the attacked value. The design cannot stop an adversary who can rewrite the kernel's own code in memory; no in-process scheme can. It arranges instead that anything short of that produces a loud, typed failure. This is a containment posture, not a security claim, and the Gate-4 receipts exercise only its fixture-level behavior.

4.3. No shared mutable trust. The second clause forbids a pattern so convenient that it must be named to be resisted: a module-level "current status" object that components consult, and that privileged code updates when status changes. Such an object is shared mutable trust. It has three defects. It is a single point of silent failure: one mutation, however caused, changes the effective status of every artifact that consults it. It inverts the evidence relation: artifacts no longer carry their status; they borrow it from ambient state, so the same artifact can be "licensed" at one moment and not at another with no change to the artifact itself, and no record of why. And it is unauditable: a consultation of ambient state leaves no trace in the artifact of what was consulted or when.

The kernel's arrangement is the opposite on all three points. Status is a field of the artifact, immutable, validated at construction, and rechecked at use. Two artifacts never share a carrier. There is no ambient status to consult and therefore no ambient status to corrupt. When Chapter 6's registries serialize an artifact, the boundary goes with it, byte for byte, into the digested evidence—so the status an artifact claimed is part of what its digest commits to.

4.4. No recomputable self-hash as trust. The third clause addresses a subtler temptation, and it is the hinge to Section 6. Suppose the carrier computed a digest of its own fields at construction and stored it alongside them, and suppose use sites verified the stored digest instead of re-running the field predicate. This looks like an integrity mechanism, and against *accidental* corruption it is one: a stray mutation of a status field would, with overwhelming probability, disagree with the stored digest.

Against an adversary it is nothing at all. The digest is a public function of the fields. Whoever can rewrite the fields can rerun the function and rewrite the digest in the same act. A stored self-digest therefore cannot distinguish "unchanged since construction" from "changed by someone who also updated the digest"—which is to say, it detects exactly the adversaries who are not trying. The program has direct historical experience of the resulting failure mode: self-hashed receipts whose hashes were perfectly valid, because of course they were; the entity that fabricated the receipt computed them.

The kernel accordingly assigns self-digests a strictly limited role. Where they appear (Chapter 5's canonical encodings give every evidence record a deterministic digest), they

serve comparison and deduplication and the detection of accidental divergence between independently held copies. They are never consulted *in place of* field-level revalidation, and they are never, anywhere, treated as evidence of authorization. What separates a digest that means something from a digest that does not is possession by a party the mutator cannot rewrite—an external pin, in the vocabulary of Chapter 6, where the registry loader receives its expected hashes from the caller precisely so that the loaded bytes cannot vouch for themselves.

5. Status change without mutation

A natural question about an immutable status carrier is how status ever changes. An artifact that is OPEN today may, one day, pass review; the disclosure of a held result may one day be authorized. If the carrier cannot be updated, where does the change live?

The kernel's answer is that it does not live in the kernel at all, and this is a design position rather than a gap. The v0.1 kernel contains no transition function on boundaries: no operation takes a boundary and returns a successor boundary, no operation rewrites a boundary in place, and no artifact's boundary field is assignable after construction. A change of status, when the surrounding process one day produces one, is an *act*—a review, an admission decision, a disclosure authorization—and acts are recorded the way the program records everything else: as new artifacts. The reviewed successor of a Ring-4 candidate is a new artifact version carrying a new boundary, constructed under Rules 4.6–4.8 like any other, with the admission act itself recorded as data in the ledgers of Chapter 6. The predecessor persists, unchanged, with the status it always had.

Three properties fall out of this arrangement, each of which an in-place transition model would forfeit.

History is retained by construction. Because status change means new-artifact-with-new-boundary, the trajectory of an artifact through the admission process is a sequence of immutable records rather than the final state of an overwritten field. An auditor reconstructing "when did this become reviewed, and what did it look like before?" reads the sequence; nothing must be recovered from logs of destroyed states. This is the provenance instinct—keep the entity, the activity, and the derivation distinct and durable [**2**]—applied to status.

No time-of-check races on status. A mutable status field invites the classic race: one component checks the boundary, another mutates it, the first acts on stale information. With immutable carriers the checked value cannot change between check and act; what can happen is only that a *newer artifact version* exists, which is a visible, queryable fact rather than a silent invalidation of a value in hand.

No demotion laundering. Transitions run both ways, and the dangerous direction is not always upward. A program that can promote a boundary in place can also quietly *demote* one—or, more insidiously, revert a demotion—leaving no trace that the elevated status ever existed or was ever withdrawn. In the new-artifact model a withdrawal is itself an act producing a record; the demoted predecessor's history does not vanish because history is never stored in a mutable cell.

REMARK 4.13 (The transitional states). The admission-state vocabulary includes IN-REVIEW, and Proposition 4.9 counts valid configurations containing it at rings 1 through 3. In v0.1 those configurations are representable but unconstructed: no kernel artifact carries them, because every v0.1 construction site pins the exact Ring-4 configuration. Their presence in the vocabulary is forward provision, not dead code—a future admission process needs a representable "review in progress" that is distinct from both OPEN and ACCEPT, and Rule 4.8

already couples it correctly (no disclosure during review). The kernel ships the vocabulary and the consistency rules now, so that the future process inherits its state space from a reviewed design instead of improvising one.

6. Tamper evidence versus authority

6.1. Two questions. The distinction this section develops is the load-bearing conceptual distinction of the entire kernel, and the boundary object is where it lives. It can be put as two questions that sound similar and share no evidence.

Definition 4.14 (Tamper evidence). A mechanism provides *tamper evidence* for an object if it answers the question: *has this object changed since a reference point?* The answer is relative to a reference (a construction event, a pinned digest, a retained byte string) and is available to anyone holding the reference.

Definition 4.15 (Authority). A mechanism provides *authority* for a status if it answers the question: *who conferred this status, and were they entitled to?* The answer requires an authenticated link to a principal outside the object—a reviewer, an institution, a legal act—and is unavailable to any mechanism whose entire input is the object itself.

The two questions are orthogonal in both directions. An object can be perfectly tamper-evident and wholly unauthorized: the historical self-issued receipts were bit-for-bit stable and hash-consistent from the moment of fabrication, and fabricated. An object can be authorized and tamper-corrupted: a genuinely reviewed record, later damaged in storage, still *was* authorized; what has been lost is the ability to demonstrate which bits the authorization covered. Integrity mechanisms and authorization mechanisms are different technologies with different trust roots, and treating the first as the second is among the oldest failure patterns in security engineering [4].

6.2. What a self-digest answers. Applied to the boundary carrier, the distinction yields a precise account of what each mechanism in this chapter does and does not establish.

The consistency rules of Section 3 establish *coherence*: the recorded status is not self-contradictory. They answer neither question. A coherent status can be both changed and unauthorized.

A digest of the carrier's canonical encoding, held by an external party or pinned in an external record, establishes *tamper evidence* relative to that pin: if the object later hashes differently, it changed. It answers Definition 4.14's question exactly and Definition 4.15's question not at all. Nothing in a hash says who was entitled to create the hashed bytes. Content addressing is a naming discipline, not an endorsement; the reproducible-builds literature is built on precisely this separation—bit-identical artifacts establish that a build was reproduced, while who ought to trust the build's inputs is a separate, socially anchored question [3].

A digest stored *inside* the object itself establishes less: tamper evidence against accidental mutation only, for the reasons of Section 4. The kernel's documentation is deliberately blunt on this point, and the monograph repeats it as an invariant because experience shows it is exactly the point that erodes.

Invariant 4.16 (A self-digest is not authority). A self-digest detects accidental or malicious mutation after construction relative to an externally held reference; a self-held digest detects accidental mutation only. In neither role is a digest a signature, a reviewer identity, or an authority grant. No kernel operation treats hash validity as authorization.

6.3. What would answer the authority question. It is worth stating positively what *would* answer Definition 4.15's question, because the v0.1 kernel's refusal to fake an answer is a design decision, not an oversight. An authority mechanism requires a trust root: some principal whose statements are accepted axiomatically, plus an authenticated channel from that principal to the artifact—in practice a signature scheme with managed keys, an enrollment process binding keys to reviewer identities, and revocation. It further requires provenance structure: a record not merely that a status holds but of the acts and agents that produced it, in the sense in which provenance models distinguish an entity from the activity that generated it and the agent that bears responsibility [2]. The v0.1 kernel has none of this, on purpose. A half-built authority mechanism is worse than none, because it manufactures artifacts that look like answers to the authority question while answering only the tamper question. The kernel's stance—developed as the constant-false authorization invariant in Chapter 9—is that until a reviewed design supplies a real trust root, the authority question has a standing answer: *no*.

Remark 4.17. The two-question analysis also explains a choice that otherwise looks strange: why the kernel bothers with elaborate integrity machinery at all if integrity confers no authority. The answer is that the authority question is answerable only *about* specific bytes. A reviewer who one day signs a claim record signs particular content; if the content is not fixed—if parses can drift from bytes, encodings can vary by platform, statuses can mutate in place—then even a genuine authorization act fails to bind anything definite. Tamper evidence is not authority, but it is the substrate on which any future authority must stand. Chapters 5 and 6 build that substrate; this chapter's boundary object ensures that until authority exists, the substrate cannot be mistaken for it.

7. How boundaries thread through the kernel

The boundary object would be decoration if it were carried only by some artifacts, or carried but not enforced. The v0.1 design threads it through every public artifact type, with the same exact-field predicate at every seam. This section walks the seams; the companion chapters treat each artifact in depth.

7.1. Semantic crosswalks. A semantic crosswalk (Chapter 3) records a typed, non-identity relation between references in different namespaces. Every crosswalk carries a boundary, defaulted fresh to Ring-4 open, and the crosswalk constructor rejects any boundary that is not exactly Ring-4 open. The consequence is that no cross-namespace correspondence in v0.1 can so much as suggest a reviewed status: even a correspondence the program's authors consider obviously correct is constructible only as an open, unlicensed, undisclosed research object. The historical Drive–Foundation conflation could not have been laundered through a v0.1 crosswalk with an elevated status, because no elevated status is constructible on that artifact type.

7.2. Registry records and registries. Chapter 6's claim and theorem registries enforce the boundary twice over, at two different layers that must not be confused. The *registry artifact* —the loaded, validated, in-memory view—carries a kernel boundary, pinned exactly to Ring-4 open like every other v0.1 artifact. The *records inside the ledger*, by contrast, carry their own status fields as data: a claim record has a product-claim status that may spell LICENSED, an admission ring that may be 3, a review state that may be REVIEWED. The loader validates these data fields against the record contracts—a LICENSED claim row lacking

reviewers, evidence packets, or a review date is rejected as contractually malformed—but validation of the data is not adoption of the data. The registry view of an all-LICENSED ledger is still a Ring-4 open artifact. The kernel reads ledgers; it does not become what it reads. This layering is the metadata-is-not-authority invariant of Chapter 6, and it is the boundary object that makes the layering representable: without a kernel-level boundary distinct from record-level status fields, "the registry says licensed" and "the kernel asserts licensed" would be one statement instead of two.

7.3. Compatibility receipts. Every compatibility execution under the deterministic-closure protocol (Chapter 7) records the research boundary in its receipt, alongside the compiler identities, worker identities, and result digests. The boundary is recorded so that a receipt is self-describing about its claim status: a consumer holding only the receipt can see—inside the digested content, not in a detachable wrapper—that the execution it evidences was a Ring-4 research act. A receipt cannot be re-presented as evidence of licensed behavior without altering its bytes, which the digest pin makes evident. This is tamper evidence in service of honest labeling: the receipt does not become more authoritative by carrying its boundary, but it becomes harder to relabel.

7.4. The runtime gate. Chapter 9's decision model consumes boundaries at the last seam: an action request carries typed intent, and the gate distinguishes nonclaim research evidence from claim-bearing intent. There the boundary discipline reaches its point. Structural validity of every artifact in the chain—references, crosswalks, encodings, receipts, registry records—feeds the gate; and the gate still answers *no* to claim-bearing intent, constantly, because the authority question has no affirmative answer in v0.1. The boundary object is what lets the gate's refusal be principled rather than arbitrary: the gate can point to the exact field, on the exact artifact, that records the absence of authorization.

7.5. Direction of dependence. One structural property of the threading deserves note. The boundary type is defined in the kernel's semantic layer and consumed by every other layer; no layer above redefines or extends it (Rule 4.4 forecloses extension by subclassing). Companion candidates—the PCBA board model, the measurement stack, the topos layer—import the same carrier and the same predicate. Status vocabulary is therefore global and singular: there is exactly one way, program-wide, to say "Ring-4 open," and it is checked the same way everywhere. Vocabulary fragmentation—each subsystem with its own status enum, related by convention—is how contradictory statuses arise between subsystems without any single subsystem being internally inconsistent.

8. Worked misuse scenarios

This section exercises the design against concrete misuse attempts. Each scenario names the actor's move and the mechanism that blocks it. The scenarios are fixture-shaped on purpose: each corresponds to a family of tests in the bound Gate-4 suite, and the blocked-by clauses are statements about tested constructor and predicate behavior, within the evidence bounds of Chapter 12.

EXAMPLE 4.18 (Forged elevated status at construction). An agent constructs a boundary with ring 4 and product state LICENSED, intending to attach it to a crosswalk so that downstream tooling displays the crosswalk as licensed. *Blocked at the first step*: the carrier constructor raises on the R4/license family of Table 1; no such value exists to attach. If the agent instead constructs an internally consistent Ring-3 ACCEPT LICENSED boundary, the

value exists—and the crosswalk constructor rejects it, because every v0.1 artifact constructor requires the exact Ring-4 open boundary. The forgery fails at whichever seam it first touches.

EXAMPLE 4.19 (Impersonation by subclass). An agent defines a subclass of the carrier that reports ring 4 and OPEN through overridden attribute access while storing elevated values, or that overrides equality so that comparisons against the distinguished Ring-4 boundary succeed. *Blocked by* Rule 4.4 at construction of the subclass instance itself, and independently by Rule 4.5 at every use site, since the predicate's first conjunct tests exact runtime type and never dispatches through the candidate value's own methods.

EXAMPLE 4.20 (Post-construction mutation). An agent constructs a legitimate Ring-4 boundary, embeds it in an artifact, and then uses low-level attribute machinery to overwrite the product state in place, hoping the artifact's earlier validation carries forward. *Blocked by* Invariant 4.11: the next operation that touches the artifact re-runs the exact-field predicate and raises a typed boundary error. The mutation is not prevented—no in-process scheme prevents it—but it cannot be *used*: every path from the mutated object to an effect passes through a recheck. If the agent also updates a self-held digest to match, nothing improves for them: Invariant 4.16 says no kernel path consults the self-digest in place of the field predicate.

EXAMPLE 4.21 (Ambient status injection). An agent arranges for a shared "process status" object—a module global, an environment flag, a configuration singleton—to indicate a licensed context, expecting kernel operations to consult it. *Blocked by* absence of the seam: no kernel operation reads ambient status. The boundary enforced by an operation is derived from the operation's own inputs (Invariant 4.11, first clause). There is nothing to inject into.

EXAMPLE 4.22 (Receipt laundering). An agent presents a structurally valid artifact chain—green Gate-4 suite, hash-consistent receipts, a coherent boundary—to a human or downstream tool as evidence that the artifact is approved for product use, exploiting the reader's inference from consistency to authorization. *Blocked within the kernel by* the constant-false authorization posture threaded through Section 7: no kernel output states an authorization, every receipt embeds its Ring-4 boundary in digested content, and the runtime gate denies claim-bearing intent regardless of the chain's structural quality. *Not blocked outside the kernel*: a human reader can still be talked into the inference. The kernel's contribution is that every artifact the launderer can exhibit carries, in its own digested bytes, the statement that it is open, unlicensed research—so the laundering requires actively contradicting the artifact, not merely decorating it.

EXAMPLE 4.23 (Disclosure flip). An agent flips the disclosure flag on an otherwise-open boundary to make an artifact appear publishable. *Blocked by* Rule 4.8 at construction (disclosure requires ACCEPT) and, for the mutation variant, by the recheck discipline as in Example 4.20. The subtle point is that the flag's semantics are conjunctive with status: there is no representable state in which disclosure is authorized for an artifact that has not passed admission, so "leak first, regularize later" has no target state to regularize into at Ring 4.

EXAMPLE 4.24 (Cross-artifact status borrowing). An agent pairs a Ring-4 artifact with a genuinely elevated artifact (say, a Ring-3 record loaded from a ledger) and presents the pair so that the elevated status appears to cover both. *Blocked by* per-artifact carriage: status is a field of each artifact, embedded in each artifact's digested encoding; no kernel operation aggregates boundaries across artifacts or computes a "session status." The registry layering of Section 7 is the systematic form of this defense: even the registry that *contains* elevated records is itself Ring-4 open.

Exercise 4.25 (Check-order absorption). Table 1 attributes each invalid combination to the first rule that fires in the constructor's check order. Verify the counts by re-deriving the table under the opposite order (license precondition checked first, Ring-4 closure last), and identify the combinations that migrate between families. Conclude that the *partition* of the 33 invalid combinations is an artifact of check order while the invalid *set* is not—and explain why fixtures should therefore assert rejection, and may assert the error family only where the check order is itself part of the documented contract.

Check Class 4.26 (Boundary fixture families). The boundary behavior described in this chapter corresponds to the following fixture families in the bound Gate-4 suite: (i) construction- rejection fixtures covering representatives of each invalid family of Table 1; (ii) exact-type fixtures rejecting subclassed carriers and enumeration look-alikes (strings spelling enumeration words); (iii) predicate fixtures exercising each conjunct of the exact-field predicate independently; (iv) recheck fixtures mutating a constructed carrier and asserting typed failure at next use; and (v) seam fixtures asserting that each v0.1 artifact constructor (crosswalk, registry record, registry) rejects every valid non-Ring-4 boundary. Passing receipts for these families are historical software evidence under the bounds of Chapter 12; they license no claim beyond tested behavior on the fixtures.

Threat 4.27 (Residual: kernel-code subversion). An adversary who can modify the kernel's own code or the interpreter in memory can disable the predicate, the constructors, and the gate together. No mechanism in this chapter addresses that adversary, and no claim is made otherwise. The design target is the historically observed failure class— honest components and agents drifting into overclaim through convenient representations—plus low-effort in-process tampering, not a hostile runtime.

9. What the boundary object does not do

The chapter closes by listing, without euphemism, the questions the boundary object leaves open; they are drawn into the obligation set of Chapter 12.

First, it confers nothing. Fifteen valid configurations are representable; in v0.1 exactly one is constructible on kernel artifacts, and none is evidence of any status beyond the artifact's own assertion about itself.

Second, it authenticates nothing. There is no reviewer identity, no signature, no trust root; the authority question of Definition 4.15 has no affirmative answer anywhere in the kernel, and every mechanism that resembles an answer (digests, receipts, coherent statuses) has been shown to answer a different question.

Third, it does not survive a hostile runtime (Threat 4.27), and its tamper evidence is only as good as the external pins that hold the digests—pins whose discipline is the subject of Chapters 5 and 6.

Fourth, its ring semantics are inherited, not defined: the kernel enforces the consistency rules over the ring index but does not define what review at each ring consists of. That definition lives in the Project Coherence process documents, outside the kernel, and a future authorization design (Chapter 9) must bind to it explicitly.

What the boundary object does do is narrow: it makes status explicit, coherent, immutable-in-use, rechecked at every seam, and honest about the difference between "unchanged" and "authorized." The remaining chapters build outward from that narrow guarantee: Chapter 5 fixes the bytes that boundaries and everything else are committed to; Chapter 6 fixes the ledgers that carry status vocabulary as data; and Chapter 9 shows the gate that turns the whole discipline into a standing, principled *no*.

CHAPTER 5

Canonical Evidence Encoding

1. Why digests are load-bearing

Every mechanism described in this monograph eventually reduces a question about meaning to a question about bytes. A semantic reference binds a concept to the digest of its defining text (Chapter 3). A registry record carries a digest bound to the ledger bytes it was parsed from (Chapter 6). A compatibility receipt binds compiler bytes, worker bytes, and a result digest (Chapter 7), and the replay requirement compares fresh digests against recorded ones before any structural decision can pass (Chapter 9). In each case the operative comparison is hash equality of encoded values: two artifacts are treated as the same evidence exactly when their canonical encodings hash identically.

This reduction is what makes the kernel auditable—byte equality is a question any party can settle without trusting the kernel—but it places the entire weight of the system on one function: the encoder that turns in-memory values into the bytes that get hashed. If that function is nondeterministic, platform-dependent, or many-to-one in the wrong places, then every downstream comparison is silently invalid, and no amount of care in the layers above repairs it. The failure is maximally quiet: hashes still compute, receipts still verify against themselves, tests still pass on the machine that wrote them, and the divergence surfaces—if it surfaces at all—as an irreproducible mismatch between two honest parties, months later, with the original context gone.

Definition 5.1 (Canonical evidence encoding). The *canonical evidence encoding* is the kernel's single function from admissible in-memory values to byte strings, with the contract that (i) equal admissible values encode to equal bytes on every platform and every run (determinism), (ii) values the kernel treats as semantically distinct encode to distinct bytes (injectivity on the admitted domain), and (iii) every value outside the admissible domain is rejected with a typed error rather than encoded by best effort (fail-closed totality).

The three clauses are jointly the definition of "canonical" as this monograph uses the word. Clause (i) rules out dependence on iteration order, hash seeds, locale, word size, and library version. Clause (ii) rules out *type puns*: pairs of distinct values that encode identically, so that evidence about one can be replayed as evidence about the other. Clause (iii) is the clause most often skipped in practice and the one the kernel is most insistent about: an encoder that guesses on hard inputs converts every hard input into a latent comparison bug, whereas an encoder that rejects them converts the same inputs into visible errors at the moment of encoding, when the context needed to fix them still exists.

The design follows the same instinct as the reproducible-builds discipline [3]: identity is a function of bytes, so byte production must be an engineered, specified, testable process rather than an incidental property of whatever serializer was handy. And, per the standing distinction of Chapter 4, none of this confers authority. A canonical digest says *these bytes*; it never says *these bytes are approved*. The encoding chapter is about making the first statement

exact so that the second statement, if a future reviewed design ever makes it, has a definite object.

Claim Boundary 5.2 (Chapter scope). This chapter specifies the v0.1 encoding profile and the rationale for each rule, with worked encodings. Statements that the kernel "rejects" or "produces" particular bytes are statements about the v0.1 candidate's tested behavior on the Gate-4 fixture families (acceptance and rejection cases for sets, floats, NaN, infinity, decimal tagging, cycles, and depth bounds are among the enumerated suite contents); they are bounded software receipts in the sense of Chapter 12, not conformance claims over untested inputs, and the profile itself remains a Ring-4 research candidate.

2. The threat model for encodings

Before the rules, the failures they answer. Encoding failures come in two dual families, and it is worth having both clearly in view because several rules exist to block one family without reintroducing the other.

Threat 5.3 (Digest instability: same evidence, different bytes). Two encodings of the same value differ—across runs, platforms, library versions, or processes—so hash comparison reports difference where there is none. Consequences: replay verification fails spuriously; deduplication splits; honest parties cannot reproduce each other's digests; and, worst, operators learn to treat digest mismatches as noise, at which point the mismatch that matters is waved through. Classic causes: unordered container iteration, hash-seed-dependent ordering, float formatting drift, locale- dependent case mapping, newline rewriting in transit, Unicode normalization applied by one layer and not another.

Threat 5.4 (Type puns: different evidence, same bytes). Two values the system treats as semantically distinct encode to the same bytes, so a digest computed over one validates the other. Consequences: an evidence record can be constructed to collide with a different record; quantities can be silently reinterpreted (an exact decimal read back as a string, a tuple as a list, an enumeration member as its underlying value); and any downstream logic keyed on digests inherits the confusion. Classic causes: encoders that erase type distinctions on output (most JSON encoders erase tuple/list, and many erase decimal/float/string distinctions), and container coercions applied before encoding.

The two threats pull in opposite directions. Aggressive normalization (lowercase everything, sort everything, coerce everything to strings) fights instability and manufactures puns; aggressive type preservation fights puns and, done casually, reintroduces instability (embedding a language-specific repr string preserves type at the cost of platform dependence). The rule set below is best read as a sequence of decisions about exactly where to normalize and exactly where to distinguish, with each decision justified against both threats at once.

3. The rule set

Rule 5.5 (Canonical encoding, v0.1 profile). The canonical evidence encoding:

(1) is deterministic UTF-8 JSON with sorted string keys and minimal separators, terminated by a single line feed;
(2) rejects unordered containers (sets and frozen sets), binary floating- point values, NaN, and infinities;
(3) tags exact decimals, tuples, and enumeration members as typed objects, so that Decimal("1.0"), the string "1.0", and the integer 1 have pairwise distinct encodings, and reserves the tag key so payload data cannot forge a tagged value;

(4) gives each public type exactly one declared serialization route; and
(5) is bounded and cycle-aware: it enforces declared depth, node-count, byte-size, and integer-width limits, and fails closed on any reference cycle.

Before the clause-by-clause development, the profile's central property can be stated once, in the vocabulary of Definition 5.1.

PROPOSITION 5.6 (Determinism and injectivity on the admitted domain). *On the domain of values the profile admits, the canonical encoding is a function of value content alone—independent of construction history, platform, process, and run—and is injective up to the profile's declared identifications: two admitted values encode to the same bytes if and only if they present the same canonical content under their declared serialization routes.*

Design argument. Determinism: every source of environmental variation is either eliminated (sorted keys remove ordering dependence; minimal separators remove formatting dependence; UTF-8 with direct non-ASCII emission removes escaping choices) or excluded from the domain (unordered containers, floats, and locale-sensitive text transformations are rejected, so no admitted value's encoding consults them). Injectivity: the JSON grammar is uniquely parseable; distinct admitted scalars receive distinct token forms, with the JSON-inexpressible distinctions (decimal, tuple, enumeration) carried by tagged objects under a key that is reserved and therefore cannot arise from payload data; and route uniqueness (Rule 5.12) ensures no value has two derivations. The argument is a design argument, checked by the acceptance, rejection, and determinism fixture families of the bound suite (Section 14); it is not a machine-checked proof, and the open obligation to formalize it is recorded in Chapter 12.

Each clause now gets its own section, with the rationale, the worked encodings, and the specific incident or attack it answers.

4. Deterministic UTF-8 JSON with sorted keys

4.1. The base grammar. The encoding's surface grammar is JSON: objects with string keys, arrays, strings, integers, booleans, and null. JSON is chosen not for expressiveness — the whole point of the profile is to restrict it—but for auditability: the encoded bytes are readable by a human reviewer and parseable by independent tooling in any language, so a digest dispute can be settled by inspection. Three determinism decisions turn the grammar into an encoding.

Key order. Object keys are emitted in sorted order, so that the bytes do not depend on insertion order, hash seeding, or any other artifact of how the in-memory mapping was built. Two mappings with the same key–value pairs encode identically regardless of construction history. The sort is over the key strings themselves as sequences of code points; Section 11 returns to the subtlety that this ordering choice is not the same as RFC 8785's.

Separators and whitespace. The encoding uses minimal separators—a comma between items, a colon between key and value, no spaces, no indentation—so that formatting conventions cannot create spurious byte differences. Pretty-printing is a display concern; display transformations of evidence are computed from the canonical bytes, never the reverse.

Text encoding and terminator. The bytes are UTF-8 with non-ASCII characters emitted directly rather than escaped, and the encoding is terminated by exactly one line feed. The terminator is a profile decision worth flagging: it makes an encoded evidence record a well-formed single-line text file on POSIX conventions, at the cost of a deliberate, documented divergence from encoders that emit no terminator (Section 11).

Example 5.7 (Worked base encoding). The mapping with pairs `b` ↦ 2, `a` ↦ 1, `c` ↦ [`true`, `null`] encodes as the bytes

```
{"a":1,"b":2,"c":[true,null]}
```

followed by one line feed—regardless of the order in which the pairs were inserted, the platform, or the process's hash seed. The same mapping built by parsing, by literal construction, or by merging two mappings encodes to the same twenty-nine bytes plus terminator, and therefore to the same SHA-256 digest.

4.2. Text admissibility. Determinism of the byte layer is necessary but not sufficient: the *content* of strings can smuggle instability and confusion. The profile therefore restricts admissible text before it restricts anything else. Every string entering the encoding—keys and values alike—is validated to exclude three Unicode categories: control characters, invisible format controls, and surrogate code points. Control characters make encoded evidence unsafe to display and diff; invisible format controls (zero-width spaces, directional overrides) make two visually identical strings byte-distinct, which is Threat 5.3 weaponized against human review; and surrogates are not characters at all—their presence means the value was manufactured through an encoding error, and encoding them produces byte sequences that independent UTF-8 parsers legitimately reject or repair differently.

Rejection, not repair, is the deliberate choice. A profile that silently strips a directional override produces clean bytes from dirty input—and thereby makes the dirty input and its cleaned twin encode identically, a pun between a string that renders deceptively and one that does not. The kernel's position is that deceptive text is not admissible evidence, and the place to discover that is at encoding time, with an error naming the field.

4.3. The hash layer. The encoding exists to be hashed, and the hash layer has its own small profile, stated here once so the rest of the monograph can say "canonical digest" without ambiguity. The digest function is SHA-256 over the canonical encoding's bytes—terminator included—and nothing else: no salt, no key, no truncation, no double hashing. The choice is conservative and deliberately uninteresting: a single, universally implemented, unbroken cryptographic hash, so that any party with any toolchain can recompute a digest from bytes and so that a digest value in an evidence record has exactly one possible derivation.

Digest *values*, when they appear as fields in payloads and records, are governed by the identifier discipline of Section 10 in its strictest form: exactly sixty-four hexadecimal digits, carried in uppercase—the profile's declared case, matching the reference layer's digest normalization (Chapter 3)—with case normalization applied at the field boundary so that a digest arriving in lowercase is normalized on intake rather than stored in mixed conventions. The alternative—tolerant, case-insensitive digest comparison scattered across consumers—is a standing invitation for one consumer to forget the tolerance, at which point the same digest exists as two unequal strings on two sides of a comparison.

One caveat belongs to this layer and is stated here rather than repeated: every identity claim of the form "equal digest, therefore equal bytes" holds up to the collision resistance of SHA-256. The kernel makes no contribution to, and no claims about, that cryptographic property; it inherits the standard assumption and records it as an assumption. What the kernel does ensure is the converse direction and the preimage discipline around it: equal admitted values always produce equal digests (Proposition 5.6), and every digest in a kernel record names bytes that are retained or pinned somewhere a verifier can reach, so the assumption is only ever invoked about bytes that exist, never about bytes taken on faith.

5. The rejection set: what cannot be evidence

Clause (2) of Rule 5.5 names four rejections. Each is a value family that mainstream serializers accept, and each is rejected here because it is *poisonous to digests*: its presence anywhere in a payload makes the encoding either nondeterministic or many-to-one. This section takes them in turn.

5.1. Unordered containers. A set has no canonical element order, so any encoding of a set must choose one. Every available choice is bad. Iteration order is the worst: in the host language it depends on insertion history and, for strings, on a per-process hash seed, so the same set encodes differently in the same program across runs—pure Threat 5.3. Sorting the elements repairs determinism for homogeneous sets of totally ordered elements and fails for everything else: mixed-type sets have no total order, and imposing an artificial one (sort by type name, then value) builds a subtle language-specific convention into the byte layer, which is instability displaced, not removed. Worse, sort-based set encoding manufactures a pun: the set $\{1, 2\}$ and the *list* $[1, 2]$ become byte-identical, though one is an unordered membership assertion and the other an ordered sequence—evidence with different semantics.

The kernel refuses the choice entirely: sets and frozen sets are rejected with a typed error. A caller holding set-shaped data must decide, at the call site and visibly in code review, what ordered artifact the evidence actually is—usually a sorted list under an ordering the caller can state and defend. The decision is forced to the layer that has the semantics, instead of being made silently and identically-for-everyone by the encoder, which is the one layer guaranteed not to know what the set means.

5.2. Binary floats. Binary floating-point values are rejected outright—not merely NaN and infinities, but every float, including innocent-looking ones. The rationale has three independent layers, any one of which would suffice.

First, representation: most decimal literals are not representable in binary floating point. The value written `0.1` is stored as the nearest double, $0.1000000000000000055511151231257827\ldots$; the value written `0.3` is not the sum of the stored `0.1` and `0.2`. Evidence records exist to carry exact commitments, and a float is an inexact commitment wearing exact-looking digits.

Second, formatting: turning a double back into digits is a rendering algorithm with a history of divergence—shortest-round-trip rendering versus fixed precision, differences across languages, libraries, and versions. Two systems holding bit-identical doubles can emit different digit strings, and two different doubles can emit the same short string under loose formatting. That is both threats of Section 2 at once, seated in the innermost byte layer.

Third, arithmetic provenance: floats invite computation before encoding, and float computation is order-sensitive ($(a + b) + c \neq a + (b + c)$ in general), platform-tunable, and non-reproducible by default. A digest over a float-bearing payload is a digest over the accumulated rounding history of whatever machine produced it.

The kernel's alternative is Section 6's tagged exact decimals: values that carry precisely the digits the recorder committed to, no more and no fewer. The rejection of floats is enforced by exact type check, which has a small but important corollary in the host language: booleans, which the language implements as an integer subtype, are tested for *before* integers, so `true` encodes as a boolean and can never be punned with the integer 1—a miniature instance of the same discipline.

5.3. NaN. NaN would be rejected even if floats were not, and it is worth recording why, because NaN is uniquely corrosive to an evidence system. NaN is not a value; it is the propagating absence of one. It is the only widely deployed datum that is unequal to

itself, so its presence breaks the reflexivity that every comparison-based mechanism in this monograph assumes: a record containing NaN cannot even be said to equal *itself*, and "same evidence, same digest" loses meaning at the semantic layer no matter what the byte layer does. Serialization compounds this: the JSON grammar has no NaN token, so encoders that accept NaN emit a nonstandard extension token that independent parsers variously accept, reject, or translate—meaning the encoded bytes are not, in fact, JSON, and cross-tool audit fails on exactly the payloads containing the least meaningful data. A NaN in an evidence payload is always a symptom: some upstream computation failed and the failure was allowed to flow. The kernel's fail-closed contract requires surfacing that failure at its origin, not laundering it into a hashed record whose digest then faithfully commits to nothing.

5.4. Infinities. Positive and negative infinity share NaN's grammatical problem—no JSON token, nonstandard extensions, parser divergence—without its reflexivity pathology, and they add a subtler one: they encode a claim ("this quantity exceeds every finite bound") that is essentially never the fact a recorder means to commit to. In practice an infinity in a payload records an overflow, a division by zero, or a sentinel convention ("no limit") that belongs in the schema as an explicit optional field, not in the number line. Because the kernel's exact-decimal carrier is finite by construction—the decimal type admits non-finite values, and the encoder checks finiteness and rejects them—the profile makes "every encoded number is a finite, exact decimal or a bounded integer" an invariant a consumer can rely on without reading any schema.

Invariant 5.8 (Finite exact numerics). Every numeric value in a canonical evidence payload is either an integer within the declared width bound or a finite tagged decimal. No encoded payload contains a binary float, a NaN, or an infinity; fixtures for each rejection are part of the bound suite.

6. Tagged exact values

6.1. The pun problem, concretely. Clause (3) is the profile's answer to Threat 5.4, and the motivating instance is numeric. Consider three values a program might hold: the exact decimal constructed from the digits `1.0`; the five- character string `"1.0"`; and the integer 1. A naive encoder maps the decimal to the JSON number token `1.0`, the string to `"1.0"`, and the integer to `1`—three distinct byte strings, which looks fine until one notices that the decimal's token collides with what a float encoder would emit, that many decimal encoders "simplify" `1.0` to `1` and collide with the integer, and that string-typed numeric fields (ubiquitous in ledger data) collide with stringified decimals. Across real serializer configurations, some pair of these three values collides somewhere. An evidence system cannot tolerate that: a quantity asserted as an exact decimal with one decimal place of recorded precision is a different commitment from the bare integer and from a piece of prose that happens to contain digits.

6.2. The tagging scheme. The kernel encodes every type-distinct scalar that JSON cannot natively distinguish as a small tagged object under a reserved key. The tag key is `$cas$type`; its value names the carried type; the remaining fields carry the value in a deterministic rendering.

Example 5.9 (The three encodings differ). Under the v0.1 profile:

- the exact decimal Decimal(`"1.0"`) encodes as {`"$cas$type":"decimal","value":"1.0"`};
- the string `"1.0"` encodes as `"1.0"`;

- the integer 1 encodes as `1`.

The three byte strings are pairwise distinct, so the three digests are pairwise distinct (up to hash collision), and no evidence about one value can validate a record about another. The decimal's digits are rendered in fixed-point form from the decimal's own exact state—never through a binary float—so the rendering is a function of the committed digits alone.

REMARK 5.10 (Precision is part of the value). Because the rendering preserves the decimal's stored exponent, Decimal(`"1.0"`) and Decimal(`"1.00"`) encode differently: `"1.0"` versus `"1.00"` in the tagged value field. This is deliberate. In measurement contexts the recorded precision is part of the assertion—a companion candidate records calibrated quantities where one decimal place versus two is a different claim about the instrument. Callers for whom trailing zeros are *not* significant must normalize before encoding, visibly, at the call site; the encoder will not decide for them. The same one-way principle as Section 5.1: normalization is a semantic act and belongs to the layer that knows the semantics.

The same scheme carries two further distinctions. Tuples—used in the kernel for fixed-shape positional data—encode as {`"$cas$type":"tuple","items":[...]`}, distinct from JSON arrays, which carry variable-length sequence data; a loader can therefore reject a sequence where a fixed shape was committed, and vice versa. Enumeration members encode as a tagged object carrying both the enumeration's fully qualified type name and the underlying value, so the member OPEN of the admission-status enumeration can never be punned with the bare string `"OPEN"`—which matters directly for Chapter 4's boundary carrier, whose digested form must not be forgeable from string-typed data that happens to spell the same words.

EXAMPLE 5.11 (Worked enumeration encoding). Let OPEN be a member of an admission-status enumeration whose fully qualified type name is `cas_core_v8.semantics.AdmissionStatus` and whose underlying value is the string `"OPEN"`. Encoded directly, the member produces (with keys shown sorted, as emitted):

```
{"$cas$type":"enum",
"enum_type":"cas_core_v8.semantics.AdmissionStatus",
"value":"OPEN"}
```

while the bare string `"OPEN"` produces the six bytes `"OPEN"` (with quotes). The tagged form binds three facts the bare string cannot: that the value is an enumeration member at all, *which* enumeration it belongs to, and what its underlying value is. Two enumerations that happen to share the word OPEN—an admission status and, say, a disclosure status—therefore encode distinctly, and a digest over a payload containing one can never validate a payload containing the other. The enumeration check runs before the scalar checks precisely because scalar-backed enumerations would otherwise decay to their underlying values and lose all three facts. Note also the dual reading: a *declared view* may choose to project an enumeration to its plain string (Chapter 4's boundary view does exactly this), and under Rule 5.12 that projection is stable and unambiguous because it is the type's only route; what the tagging forbids is not projection but *collision* —the projected string lives inside a view whose shape is fixed, not loose in a position where a raw enumeration member could also appear.

6.3. Reserving the tag. A tagging scheme creates its own forgery surface: if payload data may contain the tag key, then a mapping in user data with key `$cas$type` and value `"decimal"` is byte- indistinguishable from an encoder-produced tagged decimal, and the pun returns one level up. The profile closes this by reservation: the tag key is forbidden as

an ordinary mapping key anywhere in admissible payloads, enforced at encoding time (and independently at registry-load time, Chapter 6). Within the admitted domain, an encoded object bearing the tag key was produced by the encoder's own tagging path—there is no other way for those bytes to arise—which is what makes clause (ii) of Definition 5.1 (injectivity) hold across the tagged extension: the encoding remains one-to-one because the tagged and untagged worlds cannot collide by construction.

7. One serialization route per public type

Clause (4) is a rule about the codebase rather than about any single byte string, and it exists because multi-route encoding is how determinism dies in mature systems even when every individual route is deterministic.

Consider a public type—say, the artifact-identity record of Section 9—that can reach bytes two ways: a hand-written view that emits selected fields in a chosen shape, and a generic reflective path that walks the record's fields mechanically. Both are deterministic. They are also different: the hand-written view may include a derived field the reflective path omits, order or name fields differently, or render a nested value through a different sub-route. Every digest computed on route A is incomparable with every digest computed on route B, and *nothing fails*: each route round-trips its own output, each verifies its own digests, and the system contains two parallel, internally consistent evidence universes. The divergence surfaces only when a record digested on one route is verified on the other—typically at the worst seam, between two components maintained by different people, each of whom is provably "correct."

Rule 5.12 (Single-route encoding). Each public type has exactly one declared serialization route into the canonical encoding. Types with a declared evidence view are encoded through that view and only that view; the generic structural path is a fallback for types without one, and the encoder consults the declared view first, so no admissible value has two derivations. Adding a second route to an existing type is a breaking change to evidence comparability and is treated as such.

The v0.1 encoder implements the rule by a fixed dispatch order: tagged scalars first, then declared views, then the generic structural walk for plain field-record types, with the reflective walk deliberately reimplemented inside the bounded traversal of Section 8 rather than delegated to a library helper whose recursion is neither cycle- nor depth-bounded. The rule's payoff is a property worth stating:

Proposition 5.13 (Route uniqueness implies comparability). *Under Rule 5.12, for any two encoded payloads of the same public type, digest inequality implies value distinction under the type's declared view, and digest equality implies byte-identical canonical content. Without route uniqueness, neither implication holds: digest inequality may reflect route choice, and digest equality on one route says nothing about encodings on another.*

7.1. A worked end-to-end encoding. The rules compose; a full example shows how. Consider the Ring-4 research boundary of Chapter 4, whose declared view presents four fields: the admission ring (integer), the admission status and license status (enumerations, projected by the view to their plain string values), and the disclosure flag (boolean). The encoder consults the declared view (Rule 5.12), obtains a four-key mapping, validates each key and each string value as admissible text, sorts the keys, and emits:

```
{"admission_ring":4,"license_status":"NOT LICENSED",
"public_disclosure_authorized":false,"status":"OPEN"}
```

followed by the terminating line feed. Every profile decision is visible in these bytes. The keys appear in sorted order regardless of the view's construction order. The ring is a bare JSON integer— admissible because bounded—and could not have been a float; had the view held the ring as a float 4.0, encoding would have failed with a typed error rather than emitting `4` or `4.0`. The two status strings are the view's declared projections; they are ordinary strings here because the view says so, and the same enumeration members encoded *outside* this view would take the tagged form of Example 5.11—no ambiguity arises because each public type has one route and the route fixes the shape. The boolean is `false`, not 0: exact-type dispatch keeps the boolean/integer distinction. There are no spaces after separators, no escaping of the space inside `"NOT LICENSED"` (UTF-8 direct emission), and exactly one terminator byte at the end. The SHA-256 of these bytes is the carrier's canonical digest; by Proposition 5.6 any process on any platform that holds an equal boundary value computes the same digest, and no admissible value other than this boundary's view content computes it (up to hash collision, per Section 4.3).

The example also illustrates the division of labor between this chapter and Chapter 4: the encoding guarantees that these bytes faithfully and uniquely name the carrier's content; it is the boundary discipline, not the encoding, that determines what the content *means*—and, per the standing invariants, the digest of these bytes evidences their integrity, never anyone's authorization of them.

8. Bounded, cycle-aware canonicalization

Clause (5) addresses a family of failures that are not about which bytes come out but about whether the encoder terminates and stays within resource bounds when handed adversarial or accidentally pathological structure. An encoder that is part of a fail-closed evidence path is itself an attack surface: if hashing a payload can be made to hang, recurse without bound, or allocate without bound, then the evidence path is a denial-of-service lever against every operation that must hash before proceeding—which, in this kernel, is every operation.

The v0.1 profile enforces four declared limits and one structural check.

Depth. Traversal beyond a declared nesting depth (64 in v0.1) fails closed. Legitimate evidence in this program is shallow; a payload nested hundreds deep is either a bug or a crafted recursion bomb, and the encoder refuses both identically.

Node count. Traversal visits at most a declared number of nodes (10,000 in v0.1). This bounds work even for wide, shallow structures that evade the depth limit.

Byte size. Accumulated text is metered during traversal and the final encoding is checked against a declared output bound (8 MiB in v0.1), so a payload cannot balloon past what downstream storage and review can handle. The in-flight metering matters: it stops runaway text at the point of accumulation instead of after a gigabyte-scale join.

Integer width. Integers beyond a declared bit width (2,048 bits in v0.1) are rejected. Arbitrary-precision integers are otherwise a quiet amplification channel—a small in-memory object whose decimal rendering is enormous—and no legitimate evidence quantity in this program approaches the bound.

Cycles. The traversal maintains the set of composite objects on the current path; revisiting an active object fails closed immediately with a cycle error. The alternative behaviors are both unacceptable: infinite recursion is a crash at best, and silent cycle-breaking (encode the first visit, elide the second) makes the encoding depend on traversal entry point—two references to the same cyclic structure would encode differently depending on where the walk began, which is Threat 5.3 in structural clothing. Note the check is scoped to the *active path*:

acyclic sharing (the same immutable value referenced twice in different branches) is legal and encodes by value at each occurrence, which is the correct evidence semantics—the payload commits to content, not to in-memory object graph shape.

The four values themselves are engineering margins, not measured quantities: the governing sources record no payload census from which they were derived. The honest statement is that each bound sits far above every payload the bound suites exercise while remaining cheap to enforce, and that a legitimate payload ever approaching a bound is, by Remark 5.14, a forced profile-version event rather than a quiet tuning change.

Remark 5.14 (Limits are part of the profile). The four numeric limits are declared constants of the profile, not tuning parameters: two implementations of the encoding agree only if they enforce the same bounds, since a payload accepted by one and rejected by the other breaks fail-closed interoperability exactly at the margin. Any future revision of the limits is therefore a version change of the encoding profile itself, visible in the profile identity that receipts record.

9. Byte identity versus comparison identity

9.1. Two identities, both recorded. Everything so far concerns values the kernel encodes. A second identity problem attaches to artifacts the kernel merely *reads*: source files, ledgers, schemas, compiler bytes. For these, the bytes already exist, and the question is what "the same file" means across platforms whose text conventions differ. The program's operating history makes this concrete: files authored on one platform arrive on another with carriage-return/ line-feed sequences rewritten to bare line feeds or vice versa—by version-control autocrlf settings, editor defaults, or transfer tooling. After such a rewrite the file's raw digest has changed while, on the program's declared comparison convention, its content has not.

Collapsing this situation in either direction loses information that incidents have shown to matter. Recording only the raw digest makes every platform transit look like tampering, flooding review with false mismatches until reviewers stop looking—the desensitization failure of Threat 5.3. Recording only a normalized digest makes genuine byte differences invisible whenever they hide in line endings, and byte-exactness is precisely what Chapter 6's registries and Chapter 7's compiler verification pin.

Rule 5.15 (Dual identity). Every artifact identity records both a *raw* identity—digest and byte count of the exact bytes—and a declared *comparison* identity—digest and byte count after the single declared normalization, CRLF-to-LF, and no other transformation. The two are separate fields with separate roles; neither substitutes for the other, and no kernel operation silently accepts one where the other is required.

The normalization is deliberately minimal and closed: replace each two-byte CRLF sequence with LF, touch nothing else. It is not trailing- whitespace trimming, not encoding transcoding, not Unicode normalization. A one-transformation comparison identity can be reasoned about exactly; an open-ended "semantic" normalization cannot.

9.2. The line-ending census. Alongside the two digests, the identity records a three-way census of the artifact's line-ending sequences: the count of CRLF sequences, of lone line feeds, and of lone carriage returns. The three counts are never conflated into a single "line count," because their joint pattern is diagnostic in a way no aggregate is. All-CRLF with zero lone characters is a consistent foreign-convention file; all-LF is a consistent native one; nonzero counts in more than one column mean a *mixed* file—almost always the fingerprint

of a partial rewrite, a bad merge, or an editor that converted only the lines it touched; and lone carriage returns, rare on modern platforms, flag either ancient tooling or bytes that were never text. The identity derives a summary profile (CRLF, LF, CR, mixed, or no line endings) from the census for human display, but the counts themselves are the recorded evidence.

The census also underwrites an internal consistency check that turns the dual identity from two assertions into one verifiable structure: since the only transformation is CRLF-to-LF and each application shortens the artifact by exactly one byte, the normalized byte count must equal the raw byte count minus the CRLF count. An identity record violating this arithmetic is rejected at construction—it could not have been produced by measuring any real byte string, so it is either corrupt or fabricated.

Proposition 5.16 (Census arithmetic). *For any byte string, the CRLF-to-LF normalized length equals the raw length minus the number of CRLF sequences, and the total of* $2 \cdot (\textit{CRLF count}) + (\textit{lone LF count}) + (\textit{lone CR count})$ *cannot exceed the raw length. Both facts are enforced as construction-time checks on identity records, so a record's five numeric fields are mutually corroborating rather than independently assertable.*

9.3. The incidents this answers. Three recurring incident shapes, all previously observed in the program's operations, are each made visible-instead-of-silent by the dual identity. A cross-platform checkout changes raw digests en masse while comparison digests hold: the pattern "raw differs, LF-normalized matches, CRLF census changed" names the event exactly, and review can wave it through *as that event* rather than as an unexplained mismatch. A genuine content change hidden in a reformatting commit shows the opposite signature—comparison digests differ—which no amount of newline noise can mask. And a partial conversion produces a mixed census where either consistent convention stood before, surfacing the half-rewritten file that per-digest comparison alone would report only as "different," without the why.

10. Identifier profiles

Text in evidence records serves two masters. Machine identifiers—record identities, namespace names, version strings, hash fields—must be comparable, greppable, unambiguous in logs, and safe across every tool in the audit chain. Prose—titles, notes, definitions quoted from source documents—must be carried faithfully, including in languages and scripts the identifiers' rules would forbid. A single admissibility rule for both either cripples prose or opens identifiers to confusable-character and normalization attacks. The profile therefore declares two tiers.

Rule 5.17 (Two-tier text). Review-visible machine identifiers satisfy a declared restricted profile: nonblank, trimmed, ASCII-only, drawn from an explicit identifier alphabet (alphanumerics plus a short list of structural punctuation), and a fixed point of compatibility normalization—the identifier must already equal its own NFKC normal form. Prose fields carry unrestricted Unicode subject only to the global text admissibility of Section 4 (no controls, no invisible format characters, no surrogates). Prose never participates in identifier comparison, and identifiers are compared as exact byte sequences with no case folding or locale participation.

Each clause of the restricted profile closes a specific hole. ASCII-only removes the entire homoglyph space—no Cyrillic `а` posing as Latin `a` in a record identity. The NFKC fixed-point requirement is subtler: rather than normalizing identifiers on intake (which would make two

visually distinct inputs collide post-hoc, a pun manufactured by the kernel itself), the profile requires that the identifier *already be* normal, rejecting anything that normalization would change. The kernel thus never rewrites an identifier; it only refuses non-canonical ones—the same reject-don't-repair stance as Section 4, applied one tier up. Hash-valued fields get a stricter micro-profile still: exactly sixty-four hexadecimal digits, normalized to uppercase, the declared case, at the boundary, so that digest comparison is byte comparison everywhere downstream and a mixed-case digest can never split into two apparently different values.

11. Comparison with RFC 8785 (JCS)

The obvious question about any bespoke canonical JSON profile is why it is not simply the standard one. RFC 8785, the JSON Canonicalization Scheme [1], solves an overlapping problem: deterministic bytes for JSON data, for exactly the hashing-and-signing use cases this chapter cares about. The kernel's profile agrees with JCS on the load-bearing architecture and departs from it in documented, deliberate ways; this section states both sides precisely, because "roughly JCS" is exactly the kind of claim the monograph's discipline forbids.

11.1. Where the profile follows JCS. The agreements are the structural ones. Both use UTF-8 as the sole byte encoding. Both emit objects with sorted keys, so member order carries no information. Both use minimal separators with no insignificant whitespace. Both emit non-ASCII text directly rather than ASCII-escaped. Both are designed so that hashing the output is meaningful: same data, same bytes, same digest. A reader who knows JCS can read the kernel's encoded evidence without relearning anything at the surface level.

11.2. Where the profile is stricter. Every departure is a restriction or an extension motivated by the evidence threat model, and each is flagged in Table 1.

Numbers. The deepest divergence. JCS inherits I-JSON's number model: numbers are IEEE-754 doubles, serialized by the ECMAScript shortest-round-trip algorithm. This is the right call for JCS's goal— canonicalizing arbitrary interoperable JSON—and it is unacceptable here, for every reason in Section 5.2: doubles are inexact carriers, and shortest-round-trip rendering, while deterministic, commits evidence to binary-rounding artifacts. The kernel rejects floats entirely and carries exact quantities as tagged decimals. In JCS terms: the kernel's numeric domain is integers (bounded) plus a tagged-object extension, and no value in the kernel's domain exercises JCS's double-serialization machinery at all.

Key ordering. JCS sorts object members by the UTF-16 code units of their names; the kernel's encoder sorts by Unicode code points. The two orders agree on all-ASCII keys—which covers every machine identifier under Rule 5.17—but can differ when keys mix supplementary-plane characters with code points in the upper basic multilingual plane, because surrogate-pair code units interleave differently than code points. The kernel documents this as a divergence rather than hiding it: its encoded bytes for such keys are *not* JCS bytes. The practical exposure is narrow (non-ASCII keys are rare in kernel payloads and forbidden in identifier positions), but narrow is not zero, and the profile says so. The governing sources do not record why code-point order was preferred over adopting JCS's code-unit sort outright; the natural reading, offered as reconstruction rather than record, is that code-point comparison is the host language's native string order, so the canonical sort needs no hand-written UTF-16 transcoding step—one fewer transformation in exactly the place where this chapter counts transformations as attack surface—and byte-for-byte JCS compatibility was already forfeited by the terminator and the tagged extension.

Type discrimination. JCS canonicalizes JSON's data model and therefore cannot distinguish what JSON cannot: it has no tuples, no decimals-as-distinct-from-numbers, no enumerations. The kernel's tagged extension (Section 6) is strictly outside JCS's scope— JCS would happily canonicalize the tagged objects as ordinary objects, which is compatible but meaningless without the reservation rule that makes tags unforgeable.

Admissibility. JCS canonicalizes whatever valid JSON it is given; it has no rejection set. The kernel rejects sets and floats at the host- language boundary, rejects control and format characters inside strings, rejects the reserved tag key in data, and enforces the resource bounds of Section 8, none of which JCS contemplates. In the same spirit, the kernel's registry layer rejects duplicate keys at parse time (Chapter 6), where JCS—operating downstream of a parser—assumes I-JSON well-formedness rather than enforcing it.

Output framing. The kernel terminates encodings with a line feed; JCS output is unterminated. A JCS digest and a kernel digest of the same object therefore differ even when every byte before the terminator agrees. This is the one divergence with no semantic content, retained because the kernel's encoded records live as files in a line-oriented audit toolchain; it is called out so that no consumer expects cross- verification against a JCS implementation to succeed byte-for-byte.

Dimension	RFC 8785 (JCS)	CAS Core 8 v0.1 profile
Byte encoding	UTF-8	UTF-8 (same)
Key order	UTF-16 code units	code points (differs off-ASCII)
Whitespace	none	none (same)
Numbers	IEEE doubles, ES6 rendering	floats rejected; tagged decimals
NaN / infinity	inexpressible in I-JSON	rejected with typed error
Sets / tuples / enums	out of scope	rejected / tagged / tagged
String admissibility	any JSON string	controls, format chars, surrogates rejected
Duplicate keys	assumed absent (I-JSON)	rejected at parse (registry layer)
Resource bounds	none	depth, nodes, bytes, integer width
Terminator	none	single LF

TABLE 1. Agreement and divergence between the JCS canonicalization and the kernel's evidence-encoding profile. "Rejected" means a typed error at encoding time; the kernel is a restricted profile with a tagged extension, not a JCS implementation.

REMARK 5.18 (Relation to I-JSON). JCS is defined over the I-JSON message profile—the interoperable subset of JSON that excludes duplicate keys, surrogates, and numbers outside double precision [1]. The kernel's admitted domain is, by construction, inside I-JSON's spirit and stricter than its letter: duplicate keys are actively rejected rather than assumed absent, surrogate and format-control code points are rejected rather than discouraged, and the number question is mooted by the float rejection. One divergence runs the other way and should be flagged: I-JSON bounds integers to the double-representable range, while the kernel admits integers up to its declared 2,048-bit width, carried as exact JSON integer tokens. A consumer parsing kernel evidence with a double-backed JSON library would silently round such integers; the kernel's own parse layer does not, and the profile documents that evidence consumers must use exact-integer parsing. In practice kernel payloads carry digests as strings

and quantities as tagged decimals, so wide integers are rare—but rare is not absent, and the profile says which side of the divergence it stands on.

The summary judgment: the kernel follows JCS wherever JCS's choice is about determinism, and overrides JCS wherever JCS's choice is about *tolerance*—of floats, of arbitrary strings, of unbounded structure—because an evidence kernel's job is precisely to be intolerant there. The profile should be read as "JCS's architecture, minus JCS's number model, plus a typed extension and a rejection set," and receipts that name the encoding profile name the kernel's, never JCS, so no consumer can mistake one for the other.

12. A failure-mode gallery

The chapter closes with a gallery of concrete failures, each stated as it would occur in an undisciplined encoder, with the clause of the profile that blocks it. The gallery serves the same role as Chapter 4's misuse scenarios: it is the rule set exercised against realistic trouble, and each entry corresponds to a rejection or determinism fixture family in the bound suite.

Example 5.19 (Hash-seed shuffle). A payload containing a set of identifier strings encodes in iteration order; the process's string-hash seed differs per run; the same evidence hashes differently on every execution, and replay verification fails intermittently in a way that vanishes under debugging (fixed seeds). *Blocked by* the set rejection of Section 5.1: the payload never encodes; the caller must supply a sorted list.

Example 5.20 (The point-one trap). A measurement of `0.1` units is stored as a double and encoded as a number. A second system recomputes the quantity as `0.30000000000000004`-style arithmetic residue, or renders the same double with different formatting, and the digests diverge—or, under loose formatting, two genuinely different stored quantities render to the same short string and the digests collide. *Blocked by* the float rejection of Section 5.2 and the tagged-decimal carrier: the recorder must commit to exact digits, and those digits are what is hashed.

Example 5.21 (NaN equals nothing). An upstream failure yields NaN in a numeric field; the encoder emits a nonstandard token; one downstream parser errors, another accepts, a third reads null. Meanwhile the record is unequal to itself under the very comparison semantics the registry relies on. *Blocked by* the NaN rejection of Section 5.3: the upstream failure surfaces at encoding time as a typed error naming the field.

Example 5.22 (Decimal–string–integer pun). A ledger field holding the string `"1.0"` is compared, via digest, against a record whose author stored the decimal 1.0; an untagged encoder emits identical bytes for both, and evidence about a prose field validates a numeric commitment. *Blocked by* the tagging of Section 6: the encodings differ (Example 5.9).

Example 5.23 (Tag forgery). An author embeds `{"$cas$type":"decimal","value":"2.0"}` as ordinary data in a payload, crafting bytes identical to an encoder- produced tagged decimal. *Blocked by* the reservation rule: the tag key is rejected in data positions at encoding time and again at registry load, so the forged bytes are not producible through the admitted path.

Example 5.24 (Two-route drift). A convenience helper serializes a public record type reflectively for a cache, while the evidence path uses the type's declared view; a field is added to the view only; cache digests and evidence digests silently fork, and a verification months later compares digests from different universes. *Blocked by* Rule 5.12: one route per public type, with route addition treated as a breaking change.

Example 5.25 (Autocrlf masquerade). A checkout rewrites a pinned source file from LF to CRLF; raw digests change across the whole tree; a reviewer, habituated to platform noise, approves a diff in which one file's change is *not* line-ending noise. *Blocked by* the dual identity and census of Section 9: the noise files show "raw differs, LF matches, census flipped"; the real change shows "LF differs," a signature no newline rewrite can produce.

Example 5.26 (Invisible ink). An identifier contains a zero-width character; it renders identically to its clean twin in every review surface, while comparing unequal and hashing differently—two records that every human reads as one. *Blocked by* text admissibility (format controls rejected) and the ASCII identifier profile of Section 10.

Example 5.27 (Compatibility confusable). An identifier uses a Unicode compatibility character (a full-width Latin letter, a ligature) that NFKC would fold to the plain form; depending on which layer normalizes, the identifier intermittently equals its folded twin. *Blocked by* the NFKC fixed-point requirement: the non-normal form is rejected at intake, and the kernel itself never folds, so there is no layer-dependent equality.

Example 5.28 (Recursion bomb and cycle). A crafted payload nests two thousand levels deep, or contains a mapping that transitively contains itself; a naive encoder overflows the stack or recurses forever inside the evidence path, denying service to every operation that hashes first. *Blocked by* the bounds and active-path cycle check of Section 8: depth and node limits fail closed with typed errors; the cycle is detected at first re-entry.

Example 5.29 (Big-integer balloon). A payload carries 2^{100000} in a field where no legitimate quantity exceeds a few hundred bits; the decimal rendering alone is tens of kilobytes, and a stream of such records exhausts storage metered by record count. *Blocked by* the integer-width bound; quantities that genuinely need arbitrary width must be redesigned as explicit structures, visibly, rather than smuggled through the number path.

Example 5.30 (Locale case surprise). A pipeline "normalizes" identifiers by lowercasing them, and one stage runs under a locale in which case mapping differs from the ASCII expectation (the classic instance maps the letter `I` outside the ASCII pair), so the same identifier lowercases to different byte strings on different hosts. *Blocked by* the identifier profile of Section 10: identifiers are compared as exact bytes with no case folding anywhere in the kernel, and the one sanctioned case normalization—hexadecimal digest case—is locale-independent by construction. A transformation that is not in the profile does not happen; a transformation that is in the profile is total and deterministic.

Example 5.31 (Escape-style fork). Two encoders serialize the same non-ASCII string, one emitting raw UTF-8 and the other emitting ASCII escape sequences; both outputs parse back to the same string, but the bytes—and therefore the digests— differ, and a digest computed by one toolchain fails verification in the other. *Blocked by* profile fixity: the canonical encoding emits non-ASCII directly, always, and receipts name the encoding profile, so a consumer either implements the declared emission or is not computing canonical digests at all. Parse-equivalence is explicitly not the profile's identity relation; byte equality is.

Example 5.32 (Terminator mismatch). An independent implementation reproduces every rule but omits the final line feed; all of its digests differ from the kernel's on every payload, and the divergence is invisible to any inspection of the pre-terminator content. *Blocked by* documentation and fixtures rather than by code: the terminator is a declared clause of Rule 5.5, Table 1 flags it as a deliberate JCS divergence, and the determinism fixture family includes

known-answer encodings whose recorded digests any conforming implementation must reproduce byte-for-byte. A known-answer test is the cheapest possible cross-implementation contract, and the terminator is exactly the kind of detail it exists to catch.

13. Placement in the evidence pipeline

Before the testing discipline, this section walks the encoder's actual consumers, because the profile's rules were each motivated by a threat in the abstract and each lands somewhere concrete in the kernel.

Semantic references (Chapter 3) carry a definition digest: the hash of the exact defining text of the referenced concept in its identified source. The dual-identity discipline of Section 9 governs how that source text is identified across platforms, and the identifier profile governs the reference's namespace and identifier components. A reference is thus a consumer of this chapter twice over—its target is named by a digest, and its own rendered form obeys the identifier tier.

Boundary carriers (Chapter 4) are encoded through their declared view, as worked in Section 7.1; their digests appear inside every artifact that embeds a boundary, which is every artifact. The tagging rules guarantee that a boundary's digested content cannot be counterfeited from look-alike string data, and the single-route rule guarantees that two components digesting the same boundary agree.

Registry records (Chapter 6) are the heaviest consumers: every record carries the canonical digest of its payload, recomputed and compared during the reconstruction rule at every load, resolution, and export. The registry layer additionally enforces the parse-side counterparts of this chapter's rules—duplicate-key rejection, text admissibility of ledger strings, reserved-tag rejection in ledger data—so that content entering by parse meets the same bar as content entering by construction.

Compatibility receipts and replay (Chapters 7 through 9) reduce to digest comparison end to end: compiler bytes are hashed before execution, raw and LF compiler identities are recorded as separate fields per Rule 5.15, result payloads are canonically digested, and the replay requirement compares fresh digests against recorded ones before any structural decision passes. Here the encoding carries its largest responsibility: a nondeterministic encoder would make honest replays fail and—far worse—a punning encoder could make a dishonest replay pass. The entire force of the replay gate rests on Proposition 5.6.

Threat 5.33 (Profile drift across versions). A future revision of the encoding profile—a changed limit, a new tagged type, a different key ordering—silently invalidates every stored digest computed under the old profile, or, worse, coexists with it so that the same value has two "canonical" digests in circulation. *Answered by* treating the profile itself as an identified artifact: the profile's rules and limits are part of the kernel's specified surface (Remark 5.14), receipts record the profile identity alongside their digests, and any profile change is a version change that new receipts name explicitly. Old digests remain verifiable under the old profile, new digests under the new, and no digest is ever compared across profiles. The known-answer fixtures of Section 14 are the tripwire: a profile change that was not intended as one fails the recorded encodings immediately.

The pipeline view also clarifies a division of responsibility that recurs in review discussions. The encoder guarantees that equal values produce equal bytes; it does not guarantee that the *right* values were encoded. A receipt that digests the wrong compiler bytes, or a record that faithfully encodes a false statement, passes every rule in this chapter. Those failures belong to the layers that choose what to encode—the protocol disciplines of Chapter 7 and

the registry contracts of Chapter 6—and the encoding's contribution is to make their choices exact, durable, and comparable, so that a wrong choice is at least a definite, citable, refutable one.

14. Testing discipline

The encoding profile is only as credible as the fixtures that pin it down, and the bound Gate-4 suite organizes them into families that mirror the rule set. The families are recorded explicitly here, both because they are the concrete referent of every "blocked by" clause above and because their structure is itself a design artifact: each family tests one clause in isolation, so a future regression names its rule.

Check Class 5.34 (Encoding fixture families). (i) *Acceptance and determinism*: admissible payloads encoded repeatedly and across permuted construction orders, asserting byte-identical output, plus known-answer encodings with recorded digests. (ii) *Rejection, per clause*: sets and frozen sets; floats; NaN; infinities (including non-finite decimals); non-string keys; the reserved tag key in data positions; control, format, and surrogate code points in text; oversized integers. (iii) *Tagging*: the decimal/string/integer triple of Example 5.9; precision-preserving decimals; tuple-versus-list; enumeration-versus- value; boolean-versus-integer. (iv) *Bounds and cycles*: payloads at and just beyond the depth, node, and byte limits; self-referential and mutually referential structures asserting the typed cycle error; shared acyclic substructure asserting by-value encoding. (v) *Dual identity*: artifacts in CRLF, LF, CR, mixed, and no-line-ending forms, asserting raw and comparison digests, census counts, the census arithmetic of Proposition 5.16, and profile classification. (vi) *Identifier profiles*: admissible and inadmissible identifiers across the ASCII, NFKC-fixed-point, and alphabet clauses, and digest-case normalization. The Gate-4 receipt records these families within the 61-method focused suite; per the evidence bounds of Chapter 12, they establish tested behavior on the fixtures, not conformance over all inputs.

Two testing-discipline points generalize beyond this chapter. First, rejection tests are first-class citizens, equal in number and care to acceptance tests: a fail-closed system's contract *is* mostly its rejections, and a suite that only exercises the happy path verifies the least important half of the design. Second, determinism is tested by adversarial construction rather than by repetition alone—the same logical payload built through different insertion orders, merge histories, and container provenances—because repetition under one construction path can pass forever while a second path diverges. Both points are inherited by the registry fixtures of Chapter 6, which layer parse-level rejections on top of this chapter's encoding-level ones.

15. Summary and boundaries

The canonical evidence encoding is the kernel's answer to a simple observation with unforgiving consequences: every comparison the kernel ever makes is a comparison of bytes, so the value-to-bytes function must be engineered to the same standard as any other trust-bearing component. The v0.1 profile achieves this by restriction and explicitness—sorted deterministic JSON as the surface; a rejection set (unordered containers, floats, NaN, infinities, dirty text, oversize structure) that converts every source of nondeterminism or punning into a typed error; a tagged extension that keeps exact decimals, tuples, and enumerations distinct where JSON would conflate them; a single declared route per public type; and a dual raw/comparison identity with a line-ending census for the artifacts the kernel reads

rather than writes. Where the profile follows RFC 8785 it does so knowingly, and where it departs—numbers above all —the departure is a stated consequence of the evidence threat model rather than an accident of implementation.

Claim Boundary 5.35 (What the encoding does not establish). Determinism and injectivity on the admitted domain are properties of the encoding function, evidenced by the bounded Gate-4 fixtures; they are not proofs over all inputs, not a cryptographic analysis of SHA-256, and not statements about layers above or below the encoder. A canonical digest identifies bytes. It does not validate the semantics of what the bytes say, does not establish who produced them, and—per Invariant 4.16—never functions as authorization. Those limits are not defects of the profile; they are its honest perimeter, and the registry and decision layers of Chapters 6 and 9 are built against exactly that perimeter.

CHAPTER 6

Registry Integrity

1. Registries as the kernel's memory

The kernel components of the preceding chapters are stateless disciplines: references, boundaries, and encodings constrain values in flight. The claim and theorem registries are where the program's assertions come to rest. A *claim registry* is the ledger of empirical hypotheses and product claims the program tracks, each with its scope, falsifier, evidence packets, review state, and product-claim status; a *theorem registry* is the ledger of mathematical statements—axioms, definitions, theorems, propositions, lemmas, corollaries, conjectures, models—each with its proof status, migration disposition, novelty status, and admission ring. Both ledgers are governed by the Project Coherence record contracts, which fix the field inventory, the enumeration vocabularies, and the conditional requirements that couple them.

Registries are therefore the highest-value target in the kernel's threat model. Every downstream consumer that wants to know "what does the program claim, and with what status?" reads a registry; an adversary—or an honest bug—that can make a registry *appear* to say something it does not say has subverted the program's memory without touching any other mechanism. And the historical failure class of Chapter 1 applies with full force: a ledger row containing the word LICENSED is precisely the kind of structural artifact that gets read as authority. The registry design must accordingly do two jobs at once that pull in opposite directions: bind the in-memory view to exact bytes so tightly that no substitution is representable, while ensuring that even a perfectly bound, perfectly parsed LICENSED row authorizes nothing.

This chapter develops both jobs. Sections 2 and 3 give the exact-byte backing and the reconstruct-and-recheck loading rule, including the constructor- substitution attack the rule exists to defeat. Section 4 walks the full rejection catalog. Sections 5 and 6 cover recursive immutability and per-record digests. Section 8 states the metadata-is-not-authority invariant, which is where the chapter's two jobs meet.

CLAIM BOUNDARY 6.1 (Chapter scope). As throughout the monograph, statements that the loader "rejects" or "verifies" describe the v0.1 candidate's tested behavior on the bound Gate-4 fixture families (registry loader rejection cases are an enumerated component of the suite). They are bounded software receipts, not guarantees over untested inputs. The registries themselves, and every view the kernel constructs of them, are Ring-4 research artifacts: admission OPEN, product NOT LICENSED, disclosure withheld.

2. Exact-byte backing

2.1. Bytes first, views second. The foundational design decision is about what a loaded registry *is*. The conventional answer—a parsed data structure, with the source file forgotten once parsing succeeds—is exactly wrong for evidence, because it makes the parse the artifact of record. Two different parsers, or two versions of one parser, can produce

different views of the same bytes; a view can be constructed with no bytes behind it at all; and once the bytes are gone, no consumer can distinguish a faithful view from a fabricated one. The kernel inverts this: the artifact of record is the byte string, and a loaded registry is a *claim about bytes* that carries the bytes with it.

Definition 6.2 (Byte-backed registry). A loaded registry is a value comprising: the retained raw ledger bytes and the retained raw schema bytes; the dual artifact identities (raw and CRLF-to-LF comparison digests with line-ending census, Chapter 5) computed from each; two externally supplied pinned digests, one per byte string; the parsed header, schema document, and record tuple; a canonical digest for the schema document and for each record; and the kernel research boundary. Construction fails unless every derived component reconstructs from the retained bytes and every identity matches its external pin.

Two clauses of this definition carry the weight. First, the bytes are *retained*, not merely read: the loaded value holds the exact ledger and schema bytes for its entire lifetime, so any later verification can go back to ground truth without touching the filesystem, whose content may have changed since load. Second, the expected digests are *externally supplied*: the loader's caller passes the pinned raw digest of the ledger and of the schema as arguments, and the loader verifies the bytes it read against those pins before anything else.

Rule 6.3 (External pins are trust inputs). The pinned digests a registry is loaded against come from the caller— ultimately from a reviewed manifest outside the loaded artifact—and never from the loaded bytes themselves. A ledger cannot vouch for its own identity: a digest stored inside the ledger would be recomputed by whoever rewrote the ledger, per the self-hash analysis of Chapter 4. The loader's contract is conditional and honest: *if* the caller's pin identifies the intended bytes, *then* the loaded view is bound to those bytes; the kernel takes no position on whether the pin itself is correct, which is a provenance question outside v0.1's scope [2].

2.2. Both ledger and schema are pinned. It deserves emphasis that the schema document—the machine-readable record contract the ledger is validated against—is pinned and retained with exactly the same discipline as the ledger. An unpinned schema is a laundering channel: an adversary who cannot touch the ledger could substitute a permissive schema, and previously invalid content would validate. The v0.1 loader reads the schema bytes, checks them against the caller's schema pin, validates the schema's own structure (it must be a closed-object contract: the canonical draft dialect, the canonical identifier for its kind, additional properties forbidden, and required and declared fields exactly matching the contract embedded in the kernel), and only then validates records. The kernel thus checks the schema against its own embedded expectations as well as against the pin—a deliberate redundancy, so that neither a wrong pin alone nor a drifted embedded contract alone can silently widen what validates.

3. The reconstruct-and-recheck loading rule

3.1. The rule. Byte retention makes strong verification *possible*; the loading rule makes it *mandatory and repeated*.

Rule 6.4 (Reconstruct and recheck). Every parsed component of a loaded registry—header, schema document, and each record—is reconstructed from the retained raw bytes and compared, under canonical encoding, against the component the registry presents. The full reconstruction runs at construction time and again at every consuming operation (record

resolution, serialization, integrity query). A registry whose presented components do not reconstruct from its retained bytes is rejected with a typed raw-binding error naming the component that failed.

Concretely, the recheck re-derives the artifact identities from the retained ledger and schema bytes and compares them field-by-field (including the line-ending census) against the stored identities and the external pins; re-decodes the ledger bytes with the full duplicate-key- rejecting parse; re-validates the root fields, header, and schema document; re-parses and re-validates every record from the decoded ledger; recomputes every record's canonical digest; and compares the freshly reconstructed record tuple against the presented one—kind, identity, version, canonical payload digest, and boundary, record by record. Only if every comparison agrees does the operation proceed.

This is the registry-layer instance of Chapter 4's recheck-not-trust invariant, and the same rationale applies: validation performed once, at load, protects nothing against in-process mutation afterward, and protects nothing against values that never went through the load path at all. The second case is the interesting one, and it is the subject of the next subsection.

3.2. The substitution attack. The load path—read bytes, verify pins, parse, validate, construct—is not the only way to obtain a value of the registry type in a memory-safe- but-not-capability-safe host language. The registry and record types are ordinary records with public constructors, and the language provides generic copy-with-changes machinery for such records. This creates the *substitution attack*: construct a registry value directly, or copy an honestly loaded one field-by-field, keeping the retained bytes, the identities, and the pins from the honest artifact while substituting the parsed content—a record tuple with one extra claim, one altered status field, one relaxed schema document. The forged value would present impeccable byte credentials (the bytes *are* the honest bytes) around a payload the bytes do not contain. Any consumer that trusted the credentials and read the payload would be reading fiction with a valid pedigree.

Threat 6.5 (Parsed-content substitution). An agent constructs a registry or record value through the public constructor or generic dataclass-copy machinery, retaining an honest artifact's raw bytes, identities, and pinned digests while substituting parsed header, schema, or record content. The presented view diverges from the retained bytes while every byte-level credential verifies.

3.3. The defense. The defense is that Rule 6.4 runs *inside construction itself*, not merely inside the blessed load path. The registry type's initializer—executed identically whether the value is built by the loader, by a direct constructor call, or by dataclass-copy (which routes through the same initializer)—performs the full reconstruction: it re-parses the retained bytes and compares the result against whatever parsed content the constructor was handed. The attack is therefore self-defeating:

Proposition 6.6 (Substitution is unrepresentable). *No value of the registry type can simultaneously retain byte strings B and present a parsed component that is not the canonical parse of B. Construction with substituted content fails the raw-binding comparison; construction with substituted bytes fails the pin comparison; and post-construction mutation of either side is caught by the same reconstruction rerun at the next consuming operation. The only values of the registry type that exist are those whose views are recomputed-and- matched parses of their own retained, pinned bytes.*

Design argument. The initializer receives bytes and parsed components as separate inputs and derives the comparison itself; there is no constructor parameter that suppresses the check, and the copy machinery cannot bypass the initializer. A forger must therefore supply bytes whose canonical parse equals the forged content—which is not substitution but honest authorship of a different ledger, and that ledger's raw digest will not match the external pin the consumer supplies. The residual path, in-memory mutation after construction, does not create a stable forged value: every consuming operation reruns the reconstruction (Rule 6.4), so the mutated value fails at first use. As always, an adversary who rewrites the kernel's own code is out of scope (Chapter 4).

The corollary for reviewers is a pleasant inversion of the usual audit burden: to check what a v0.1 registry value says, it suffices to check what its retained bytes say, because no representable value of the type can disagree with its bytes. The bytes are reviewable with independent tools; the binding is the kernel's job and is exercised by the bound fixtures; the meaning of the bytes is the record contract's job, to which we now turn.

3.4. The load procedure. For reference in the rejection catalog, the complete load path is worth setting out as an ordered procedure; each step names the class of rejection it can raise, and the catalog of Section 4 expands each class.

Protocol 6.7 (Registry loading). Loading a registry of a declared kind, against caller-supplied expected raw digests for ledger and schema, proceeds:

(1) resolve the kind to its embedded contract (canonical schema-version string, record-schema name, record-identifier grammar, field inventories); an unsupported kind fails closed;
(2) validate both caller-supplied pins as well-formed digest values before any input is read, so a malformed pin cannot degrade into a skipped comparison;
(3) read the exact ledger bytes; compute the dual artifact identity; reject on unreadable input, non-UTF-8 content, duplicate JSON keys, or a non-object root;
(4) compare the ledger's raw digest against its pin; mismatch fails closed before any content-level validation runs, so no diagnostic about unpinned content is ever emitted;
(5) validate the root field set exactly; validate the embedded schema identifiers against the kind's contract; extract and validate the header;
(6) read the schema bytes (from the declared companion location or a caller-specified path); compute their identity; compare against the schema pin; validate the schema document as the closed-object contract for the kind;
(7) parse and validate every record against the field inventories, shape rules, enumeration vocabularies, identifier grammar, and conditional contracts; compute each record's canonical digest; construct each record value (whose own initializer revalidates);
(8) construct the registry value, whose initializer enforces kind homogeneity, identity uniqueness, boundary exactness, and then runs the full reconstruction of Rule 6.4 against the retained bytes before the value ever escapes to the caller.

The step order embodies a policy: identity before content. The pins are checked before any semantic validation, so every error message the loader can produce is a statement about bytes the caller intended to load; the loader never leaks a detailed parse diagnosis of an artifact that was not the pinned one, which would otherwise be a convenient oracle for probing unpinned content through error channels.

4. The rejection catalog

The loader's validation is deliberately exhaustive, and this section enumerates it. The organizing principle is a closed-world reading of the record contracts: everything not explicitly admitted is rejected, every rejection carries a typed error code naming the violated rule, and no rejection is downgraded to a warning. The catalog is presented in the order the loader encounters each class.

4.1. Byte and parse layer. *Non-retained or non-UTF-8 bytes.* The decode layer requires exact byte strings (not paths, not streams, not text already decoded by an unknown convention) and rejects content that is not valid UTF-8. This anchors everything downstream: there is exactly one byte string in play, and its decoding is unambiguous.

Duplicate JSON keys. The parse installs a pair-collision hook so that an object with a repeated key is rejected at parse time. Standard JSON parsers silently keep the last binding, which is a smuggling channel: a ledger containing `"product_claim_status"` twice—once benign, once elevated— would validate against schema and read differently depending on parser convention. The kernel refuses to let parser convention decide evidence content; a duplicate key is an error, full stop.

Non-object roots. A ledger whose top level is not a JSON object is rejected before any field logic runs.

4.2. Root and header layer. *Unknown or missing root fields.* The ledger root must contain exactly the six canonical fields—schema version, record-schema name, document status, as-of date, completeness statement, and the record array—no more and no fewer. The error for a mismatch reports both the missing and the surplus field names. Surplus rejection is the closed-world principle at the root: an unrecognized root field is not "extra information" but a place where content invisible to the contract could ride along inside pinned bytes.

Wrong schema identifiers. The root's schema-version and record-schema fields must equal, exactly, the canonical strings for the registry's kind (claim ledger versus theorem ledger, each with its own versioned identifier). A ledger of one kind cannot load as the other, and a ledger authored against a different contract version fails loudly rather than best-effort parsing.

Malformed header. The header triple extracted from the root is validated on its own: document status and completeness must be nonblank strings of admissible canonical text, and the as-of field must be a real calendar date in ISO form—pattern-checked *and* calendar-checked, so `2026-02-30` fails even though it matches the pattern.

4.3. Schema-document layer. As Section 2 described, the schema document must declare the canonical JSON-Schema dialect, carry the exact canonical identifier for its kind, describe a closed object (additional properties forbidden), declare required and optional fields exactly matching the kernel's embedded contract (duplicate entries in the required list are rejected), and contain its conditional-contract clauses. A schema document that is valid JSON Schema but not *this* contract is rejected.

4.4. Record layer: shape. *Unknown and missing record fields.* Each record is checked against the exact required and optional field inventories for its kind—sixteen required fields for a claim (identity, version, title, statement kind, claim text, source references, scope, controls, falsifier, uncertainty, evidence packets, review state, product-claim status, admission

ring, disclosure status, reviewers), seventeen for a theorem—with the same two-sided error reporting as the root: missing fields are named, surplus fields are named, and either is fatal.

Wrong shapes. Every field has an exact shape requirement: nonblank strings where prose is required; arrays of strings for reference lists (with per-kind minimums—at least one source reference always); the claim scope must be an object with exactly the five canonical subfields (population, models, architectures, contexts, time window) of the exact required types; timestamp fields must be RFC 3339 date-times *with an explicit timezone*, pattern-checked and parse-checked. Shape errors name the record and field.

Wrong enumerations. Every status field is validated against its closed vocabulary: claim statement kinds (empirical hypothesis or product claim); review states; product-claim status (LICENSED, NOT LICENSED, or OPEN); disclosure status (open, held, or flagged for human review); theorem statement kinds; the ten migration dispositions; the nine proof statuses; the five novelty statuses. A value outside the vocabulary—including a case variant or a padded string—is rejected. The admission ring is checked as an exact integer in the admitted range for its kind (claims: 3 or 4; theorems: 1 through 4), with the host language's boolean-integer conflation explicitly excluded, so `true` cannot pose as ring 1.

Conditional contracts. The contracts couple fields across a record, and the loader enforces the couplings: a claim whose product status reads LICENSED must also be in review state REVIEWED, at admission ring 3, with nonempty evidence packets, nonempty reviewers, and a present review timestamp—a LICENSED row missing any of these is not a weaker license but a contract violation, rejected outright; a theorem marked formally verified must be reviewed and cite proof artifacts; a theorem admitted at ring 1 or 2 must be reviewed and dated. These conditions encode, at the data layer, the same necessary-condition discipline as Chapter 4's boundary rules: the loader ensures that elevated statuses cannot be *written* without their structural prerequisites, while conferring nothing when they are (Section 8).

4.5. Closed vocabularies and migration audit. The theorem registry's vocabularies deserve a paragraph beyond their role as rejection triggers, because their structure encodes the program's migration posture. The *migration disposition* vocabulary distinguishes ten outcomes for a legacy statement under review— unassessed, in review, inherited, clarified, corrected, conditional, downgraded, deprecated, refuted, replaced—and the point of the closed enumeration is that a migrating statement must land in exactly one of them, explicitly. The historically dangerous alternatives are the open ones: free-text dispositions, where "mostly carried over" and "carried over" blur; or absence, where a statement's survival into a new version is inferred from its continued presence in a document. The same logic governs the *proof status* vocabulary, which separates gradations that overclaim tends to collapse—an informal proof present is not proved; a standard result cited is not a new result proved; a counterexample present coexists in the vocabulary with proof statuses precisely so that refutation is a recordable state rather than a deletion; and *independent replication required* exists so that "proved here" and "established" remain distinguishable. The *novelty status* vocabulary does the same for the claim of originality, separating standard results and standard specializations from framework couplings and candidate new results, with novelty-unassessed as the honest default. None of these vocabularies is enforced semantically by the kernel—the loader cannot check that a proof exists, only that the status field is a member of the closed set and that the conditional contracts hold (a formally verified entry must cite artifacts and be reviewed). The kernel's contribution is structural: every migration question the program has historically answered vaguely now has a field that must be filled from a finite list, and a review of the registry is a review of those explicit answers.

4.6. Record layer: identity. *Invalid identifiers.* Record identifiers must match the declared identifier grammar for their kind—a fixed uppercase prefix (`CLM-` for claims, `THM-` for theorems) followed by an alphanumeric-initial body over an explicit uppercase alphabet. The grammar is a machine-identifier profile in the sense of Chapter 5: ASCII, closed alphabet, no invisible characters, no confusables.

Identity–payload agreement. The record value's identity and version fields must equal the corresponding fields inside its payload; a record cannot present one identity while carrying another.

Duplicate and ambiguous identities. Within a registry, the (identifier, version) pair must be unique; a duplicate is rejected at load. Resolution is correspondingly strict: resolving an identifier that matches no record is a not-found error, and resolving an identifier that matches multiple versions without specifying which is an *ambiguity* error, not a default to latest. "Latest" is a convention, and conventions are where readers and writers diverge silently; the kernel requires the caller to say which version of a claim it means whenever the question has more than one answer.

Missing metadata. Finally, the absence-of-metadata cases: required fields absent (caught by the inventory check), required timestamps absent where a conditional contract demands them, empty reference arrays where minimums apply, and blank strings where nonblank prose is required. The loader treats missing metadata identically to wrong metadata—rejection with a named field—because an evidence ledger with holes is not a smaller ledger but an unvalidatable one.

Remark 6.8 (Why warnings are absent). The catalog contains no warning tier: every violation is fatal to the load. The rationale is the desensitization argument of Chapter 5 applied to validation: a warning tier trains consumers to proceed past violations, and the one violation that matters will arrive dressed like the hundred that did not. A registry either satisfies its contract exactly or is not a registry; partial credit is a property of the authoring workflow, which can run the loader and read the typed errors, not of the loaded artifact.

4.7. The catalog at a glance. Table 1 summarizes the catalog by rejection family and the layer at which each fires. Every family carries a typed error class whose code names the family (the implementation prefixes these `E-REGISTRY-`, for example `E-REGISTRY-DUPLICATE-KEY` for repeated JSON keys and `E-REGISTRY-AMBIGUOUS` for underspecified version resolution), so that fixtures and operational logs can assert the family, not merely the fact, of a failure.

4.8. Worked rejections. Three of the families repay concrete illustration, because the inputs that trigger them look innocent.

Example 6.9 (Duplicate-key smuggling). A crafted claim record contains, in its serialized bytes, the pair `"product_claim_status":"NOT LICENSED"` followed later in the same object by `"product_claim_status":"LICENSED"`. Under a last-binding parser the record reads as licensed; under a first-binding parser, as not licensed; a validator using one convention and a consumer using the other disagree about the same pinned bytes—the worst possible property for evidence. The kernel's parse hook rejects the object at the first repeated key, before validation begins, so the bytes are simply not a loadable ledger under any convention.

Example 6.10 (Ambiguity is not latest). A ledger legitimately carries two versions of one claim—say versions `1.0` and `1.1` of `CLM-EXAMPLE`—as the record contracts permit. A consumer resolves `CLM-EXAMPLE` without a version, expecting "the current one." The kernel raises the ambiguity error instead of choosing, because the two plausible conventions (latest

Layer	Rejection family	Representative trigger
Bytes	unreadable / non-bytes input	path unreadable; non-byte payload
Bytes	encoding	ledger not valid UTF-8
Parse	duplicate JSON key	repeated key in any object
Parse	root shape	top level not an object
Pin	ledger hash mismatch	raw digest differs from pin
Pin	schema hash mismatch	schema digest differs from pin
Root	root field set	missing or surplus root field
Root	schema identifiers	wrong kind or contract version
Header	header shape / date	blank status; invalid as-of date
Schema	contract mismatch	open object; field set drift
Record	field inventory	missing or unknown record field
Record	shape	wrong type; malformed scope; bad timestamp
Record	enumeration	value outside closed vocabulary
Record	ring	non-integer or out-of-range ring
Record	conditional contract	licensed row without reviewers
Record	identifier grammar	id fails prefix/alphabet rule
Record	identity agreement	id/version differ from payload
Registry	duplicate identity	repeated (id, version) pair
Registry	kind homogeneity	foreign-kind record in registry
Registry	boundary	non-Ring-4 kernel boundary
Resolve	not found	no record for cited id
Resolve	ambiguity	multiple versions, none specified
Binding	raw reconstruction	view diverges from retained bytes

TABLE 1. The registry rejection catalog by layer. Each family is fatal; there is no warning tier. "Binding" failures are raised by the reconstruction rule at construction and at every consuming operation.

by version-string order, latest by review date) can disagree with each other and with the author's intent, and a citation minted under one convention would verify differently under the other. The consumer must cite `CLM-EXAMPLE` at an exact version; the citation quadruple of Definition 6.13 then names one record forever.

EXAMPLE 6.11 (Schema swap). An agent leaves a pinned ledger untouched but supplies a widened schema document—additional properties permitted, an enumeration extended—hoping previously rejected content will validate in a future load of a modified ledger. The swap fails twice before it can matter: the schema bytes fail their own external pin (step 6 of Protocol 6.7), and even a schema that somehow carried a correct pin is validated against the kernel's *embedded* contract, which rejects an open object or a drifted field set outright. The double check means a schema-layer attack must subvert the pin manifest and the kernel code together, not either alone.

5. Recursive immutability

A loaded registry is shared: multiple components hold references to it, and its records are handed out on resolution. If the parsed values were ordinary mutable structures, every recipient could alter what every other recipient sees—an aliasing channel that would undermine

the byte binding from inside, since Rule 6.4 catches mutation only at the next consuming operation, and an in-place mutation between operations could mislead a reader that consulted the structure directly.

The kernel therefore freezes loaded values recursively at construction. Every parsed mapping is wrapped in a read-only mapping view; every parsed array is converted to an immutable sequence carrier that preserves JSON-array semantics; and the freeze descends through the entire structure, so there is no depth at which a mutable container survives. Scalars are validated as they are frozen—strings must be admissible canonical text (the control- and format-character rules of Chapter 5 apply to ledger content as to everything else), and the reserved canonical type tag is rejected wherever it appears in ledger data, which closes the tag-forgery channel of Chapter 5 at the registry layer as well as at the encoder.

Two design details deserve note. First, the immutable array carrier is a distinct type, not the host language's tuple: the canonical encoding distinguishes tuples from JSON arrays (Chapter 5), so an array frozen into a tuple would change its canonical digest—immutability must not alter identity. The freeze machinery accordingly rejects bare tuples in registry data as a shape error rather than guessing which identity was meant. Second, thawing—recovering plain structures for revalidation— is the exact inverse and is used only inside the reconstruction rule, where the recovered structure is immediately compared against a fresh parse of the retained bytes; frozen and thawed forms are two views of one canonical content, with equality of canonical digests as the invariant connecting them.

INVARIANT 6.12 (Frozen views, stable digests). The recursive freeze applied at load, and the thaw applied during reconstruction, preserve canonical content: the canonical digest of a record's payload is identical before freezing, after freezing, and after a freeze–thaw round trip. Immutability changes who can write; it never changes what the bytes say.

6. Per-record digests bound to registry bytes

Consumers rarely cite a whole registry; they cite a record—*this* claim, at *this* version. The kernel therefore gives each record its own deterministic digest: the canonical-encoding digest (Chapter 5) of the record's complete payload, computed from content that Rule 6.4 has bound to the retained ledger bytes. The record digest inherits the byte binding transitively: it is not an independent assertion about the record but a compressed name for content whose provenance is the pinned ledger.

DEFINITION 6.13 (Record citation). A record citation is the quadruple (registry kind, record identifier, record version, record canonical digest), optionally accompanied by the registry's pinned raw digest. A consumer holding a citation can verify, against any independently obtained copy of the ledger bytes, that the cited record exists in those bytes with exactly the cited content— without trusting the kernel instance that issued the citation.

This is the registry layer's contribution to the program's evidence chain: downstream artifacts (compatibility receipts, evidence packets, the runtime gate's decision records) refer to claims by citation, and the citation is checkable end-to-end—ledger pin to ledger bytes, bytes to parse, parse to record, record to digest—with each link exercised by the reconstruction rule. Per the standing distinction, the whole chain establishes tamper evidence and reference integrity only. A correct citation of a LICENSED record proves that the ledger says LICENSED; what that is worth is the next section's question.

6.1. Serialized views re-verify. The registry and its records export serialized views—for display, for inclusion in receipts, for cross-process transfer—and the export path is disciplined the same way as every other consuming operation: a record's view export first re-runs the record's full integrity check, and a registry's view export first re-runs the complete reconstruction of Rule 6.4. An exported view therefore cannot be produced from a value that has drifted from its bytes; the last act before serialization is a fresh proof of binding. The exported view itself is ordinary canonical-encodable content (Chapter 5), so a consumer can digest it, and the record's embedded canonical digest gives the consumer a cross-check that survives the export boundary: recompute the digest of the exported payload and compare it to the digest the export carries. The pattern generalizes: every time registry content crosses a boundary—load, resolve, export—the crossing is preceded by reverification and followed by content that carries the means of its own recheck. What the pattern never produces, per the next section, is content that carries the means of its own authorization.

7. Two registries, one discipline

The division into a claim registry and a theorem registry is itself a design decision worth a section, because it enforces at the ledger layer a distinction the program has historically blurred: the difference between an empirical assertion about systems in the world and a mathematical assertion about defined structures. The two kinds have different falsification semantics (a claim names its falsifier and its uncertainty as required fields; a theorem names its assumptions, dependencies, and proof artifacts), different status vocabularies (product-claim status and disclosure status on one side; proof status, migration disposition, and novelty status on the other), and different admission-ring ranges (claims are admitted only at rings 3 and 4—an empirical claim is never "reviewed mathematics"—while theorems span rings 1 through 4). A single mixed ledger would invite category slippage: an empirical result drifting into citation as established theory, or a theorem's proof status being read as evidence for an empirical deployment claim. Separate kinds, separate contracts, separate identifier prefixes, and a loader that rejects a record of one kind in a registry of the other make the slippage a shape error.

The kinds are nonetheless linked, deliberately and in one direction: a claim record may carry theorem references among its optional fields, citing the mathematical statements it depends on, and a theorem record carries its dependencies on other theorems. Both are citation lists over the identifier grammars—strings, validated for shape, not resolved by the loader. The v0.1 kernel does not chase references across registries, does not verify that a cited theorem exists, and does not compute transitive dependency closures; those are integrity properties of the *corpus*, not of a single pinned artifact, and checking them requires fixing which set of registries constitutes the corpus—exactly the trusted-manifest question that Rule 6.3 defers to the process layer. The kernel's per-artifact discipline is designed so that a future corpus-level checker has sound material to work with: every reference it would chase is grammatically valid, every target it would resolve is byte-pinned and citable by digest, and every version it would need is explicit because ambiguity is unresolvable (Example 6.10). Cross-registry closure is recorded as an open obligation in Chapter 12.

8. Metadata is not authority

8.1. The invariant. Everything in this chapter so far makes registries harder to falsify. Nothing in it makes registries *authoritative*, and the design goes out of its way to keep the two properties separate.

INVARIANT 6.14 (Metadata is not authority). The text LICENSED inside a registry record is necessary metadata, never sufficient authority. Loading, validating, digesting, and citing a record whose product-claim status reads LICENSED establishes only that the pinned ledger bytes carry a contractually well-formed row with that status. No kernel operation treats that fact as authorization for any claim-bearing action; in v0.1 the runtime gate's answer to claim-bearing intent is constant denial regardless of registry content, because the kernel has no authenticated reviewer and no legal trust root against which a LICENSED row could be verified as an act rather than a string.

The invariant's phrasing—necessary, never sufficient—is exact. The conditional contracts of Section 4 make well-formed LICENSED metadata a *necessary* condition for any future authorization: a future reviewed design will require, at minimum, a reviewed, ring-3, evidence-backed, reviewer-named, dated record, and the loader guarantees that nothing less can even be written. But the sufficiency direction requires what v0.1 deliberately lacks: a trust root. The reviewers field of a record is a list of names—strings, authored by whoever authored the ledger. Nothing binds a name to a person, a person to a review act, or a review act to an entitlement. Verifying such bindings is the function of authentication infrastructure—signatures, enrolled identities, revocation—which is a separate technology from integrity checking and cannot be improvised out of it [4]. A kernel that pretended otherwise would be manufacturing the historical failure this monograph opened with: structurally impeccable receipts read as authority.

8.2. Descriptive queries are marked as such. The kernel does provide a convenience query that reports whether a claim record's payload satisfies the full licensed-metadata contract (status LICENSED, state REVIEWED, ring 3, nonempty packets and reviewers, dated). Its existence is worth a remark because of what it is careful *not* to be: it is a descriptive predicate over ledger content, documented as such, returning a fact about bytes; it feeds display and reporting, and no decision path consumes it as a permission. The runtime gate (Chapter 9) does not call it to decide anything, because the gate's decision on claim-bearing intent is not a function of ledger content at all in v0.1—it is the constant *no*. Keeping the descriptive query and the decision path visibly disconnected is a small design choice with a large purpose: the day a future design connects them, the connection must be an explicit, reviewed edge binding a trust root, not the quiet reuse of a boolean that was lying around.

8.3. Worked scenario: the perfect forged ledger. The chapter's disciplines compose into a useful worst-case exercise. Suppose an agent authors, from scratch, a complete claim ledger in which every record is contractually perfect and every status reads LICENSED: correct fields, valid enumerations, ring 3, REVIEWED, plausible reviewer names, dated, with evidence-packet identifiers that resolve to artifacts the agent also authored. Every mechanism in this chapter behaves as designed: the bytes parse; the records validate; digests bind; citations verify. What has the agent obtained?

Within the kernel: nothing. The loaded registry is a Ring-4 open artifact like every other (its kernel boundary is pinned to Ring-4 open regardless of what its records say—the layering of Chapter 4); the descriptive query truthfully reports that the ledger says LICENSED; and the runtime gate denies claim-bearing intent exactly as it would against an empty ledger, because no code path consults ledger content for authorization. Outside the kernel: the agent still needs a caller to load the forged ledger, which requires the caller to supply its digest as a pin—and pins, by Rule 6.3, come from reviewed manifests, not from the artifact's author. The forgery thus fails twice over: it cannot pass the pin gate without subverting the

manifest process, and even loaded, it authorizes nothing. The residual risk is the one the kernel cannot reach and does not claim to: a human reader shown the forged ledger outside any kernel discipline may believe it. The kernel's contribution is that belief would have to be formed *against* the kernel's every output, not through it.

9. Limits and obligations

The chapter closes by naming what the registry design leaves open; the items join the obligation set of Chapter 12.

First, the pin-provenance gap: the loader verifies bytes against caller-supplied pins, and the discipline governing the pins themselves —the reviewed manifest, its own integrity, its update process—is a process-layer obligation outside the v0.1 kernel. The registry machinery is exactly as trustworthy as the weakest pin its caller supplies.

Second, the authority gap, stated throughout: no authenticated reviewer identity, no legal trust root, no authorization semantics for LICENSED rows. The future-design constraints (an exact trusted registry and schema manifest among them) are recorded with the runtime decision model in Chapter 9.

Third, the coverage bound: every "rejects" in this chapter is evidenced by the bound fixture families of the Gate-4 receipt, which exercise representatives of each rejection class, not the space of all malformed inputs. The reconstruct-and-recheck rule reduces what a consumer must trust to the correctness of one reconstruction function; it does not prove that function correct, and independent review of it remains an open obligation.

CHECK CLASS 6.15 (Registry fixture families). The bound suite's registry families track Table 1: (i) pin-mismatch fixtures for ledger and schema; (ii) parse-layer fixtures (duplicate keys, non-UTF-8 bytes, non-object roots); (iii) root, header, and schema-contract fixtures covering missing, surplus, and drifted fields; (iv) per-field record fixtures for each shape rule, each closed vocabulary, the identifier grammar, and each conditional contract, for both claim and theorem kinds; (v) identity fixtures (payload disagreement, duplicate and ambiguous identities, not-found and underspecified resolution); (vi) substitution fixtures constructing registry and record values with swapped parsed content, swapped bytes, and subclassed types, asserting the typed binding failures of Proposition 6.6; and (vii) immutability and digest fixtures asserting frozen views, freeze–thaw digest stability (Invariant 6.12), and citation verification against independently re-read bytes. As throughout, passing receipts for these families are bounded historical evidence, not conformance claims.

Within those limits, the registry layer completes the kernel's structural story: bytes are pinned, views are reconstructed and rechecked, every irregularity in the catalog is a typed rejection, loaded values are frozen, records are citable by digest—and at the end of all that machinery, the word LICENSED is still just a word. The next chapters take up the execution side of the kernel: the hash-verified legacy compiler profile (Chapter 7), its concurrency and failure disciplines (Chapter 8), and the decision model (Chapter 9) where the constant-false authorization invariant turns this chapter's distinction between metadata and authority into an enforced runtime answer.

CHAPTER 7

The Legacy Compiler Compatibility Protocol

1. The profile and its refusal

The mechanisms of the four preceding chapters—structural semantic references, the research boundary object, the canonical evidence encoding, the byte-backed registries—all concern artifacts at rest. This chapter concerns the one place in CAS Core 8 where something actually *runs*: the legacy compiler compatibility profile, under which the kernel executes the frozen CAS v7.7 Python notation compiler against caller-supplied source text and returns a hash-bound record of what happened.

The profile is named `legacy_only`, and the name is a commitment, not a description. The profile binds the exact bytes of the active Python CAS-notation compiler—the single compiler artifact listed in the Gate 4 record, identified by a raw digest and a newline-normalized content digest—and it executes those bytes, and only those bytes, for comparison purposes. What it refuses to do is equally definite: it produces *no* CAS v8 semantic translation. The compilation record carries a field `v8_semantic_translation` whose only admissible value is the null value; a constructor that receives anything else raises `E-COMPAT-V8-TRANSLATION` and the record never exists. There is no configuration, flag, or subclass through which a legacy compilation acquires v8 meaning. If a future v8 semantics wants to say something about a legacy program, it must do so through a separately reviewed translation gate that does not yet exist, and the compatibility record is deliberately shaped so that its output cannot be mistaken for that gate's output.

This refusal is the first thing to understand about the protocol, because it explains the protocol's otherwise puzzling austerity. A compatibility execution is not trying to be useful in the ordinary sense—it is not an interpreter service, a build system, or a language server. It is an *evidence procedure*: a way of establishing, to the standard the kernel's canonical encoding can support, the statement "these exact source bytes, compiled by these exact compiler bytes, on this exactly described interpreter, produced this exact artifact." Every step below exists because some component of that statement can be silently falsified in an ordinary execution environment, and the protocol's job is to close each falsification channel or to record, honestly, that it remains open.

Definition 7.1 (The frozen compiler identity). The admitted compiler is identified by three constants: a raw digest over the on-disk bytes as they exist in the working tree (historically carrying CRLF line endings, digest beginning `065DC438`), a canonical digest over the LF-normalized content (digest beginning `8169188A`), and a source-authority identifier `SRC-ACTIVE-COMPILER-CURRENT`. The identity policy accepts exactly two raw variants—the CRLF form and the LF form—and requires that the LF-normalized content digest equal the single frozen canonical value in every case.

The two-variant acceptance is not laxity; it is the byte-identity discipline of the canonical encoding chapter applied to a real repository. A checkout on Windows and a checkout on a POSIX host may legitimately hold different raw bytes for the same compiler because of

newline rewriting, and the program has been burned before by conflating the two identities. The policy therefore admits either raw form but anchors both to one LF content identity, and—critically—the receipt records *both* digests separately, so a reviewer can always tell which physical variant actually executed. Neither digest substitutes for the other, and no other raw variant is admissible: a compiler file whose LF normalization matches but whose raw digest is neither frozen variant is rejected, because an unexplained third physical form is itself evidence of something the profile does not understand.

Remark 7.2. Throughout this chapter, error identifiers such as `E-COMPAT-SOURCE-HASH` name the typed failure codes raised by the reference implementation. They are quoted because the taxonomy is load-bearing: Chapter 8 argues that *which* failure occurred must survive into evidence, and that requires failures to be enumerable rather than prose.

The remainder of the chapter develops the eight-step deterministic-closure protocol, one section per step. The protocol statement itself is short; the argument for each step is where the content lives.

Protocol 7.3 (Deterministic closure, restated). A compatibility execution: (1) verifies the requested source hash and reads and hashes the compiler bytes before execution; (2) forces the Python-only engine profile under an exact frozen policy; (3) rejects `use` imports, because no import-DAG manifest is admitted; (4) executes the verified compiler bytes in a fresh `python -I -S -B` worker with a sanitized environment; (5) samples and binds the worker runtime before compiler code executes, without WMI or subprocess probing; (6) serializes worker launches through a profile-bound advisory lock with a 90-second acquisition bound, 10-millisecond poll, and 10-millisecond post-release yield, never retrying a failed worker and never claiming fairness; (7) records the full receipt—raw and LF compiler identities, interpreter and worker bytes, import search paths, loaded-module inventory, exact syntax, semantic, and engine profiles, result digest, diagnostics, and research boundary; and (8) requires fresh replay against the same source and compiler identity before any structural decision can pass.

2. Step 1: source-hash verification and pre-execution compiler-byte hashing

The first step establishes *what* is about to run, before anything runs.

The caller does not hand the protocol a file path and a promise. Source text arrives inside a typed envelope carrying the source string, a declared version, a declared syntax profile, and a SHA-256 digest of the source's UTF-8 encoding. The protocol's first act is to recompute that digest from the retained source bytes and compare it to the declared value; a mismatch raises `E-COMPAT-SOURCE-HASH` and nothing executes. The declared version must be one of the two accepted version strings and the declared syntax profile must be exactly `cas-v7.7`; anything else fails closed with `E-COMPAT-VERSION` or `E-COMPAT-PROFILE`.

Symmetrically, the compiler is read from disk *once*, as bytes, and both of its identities—raw and LF-normalized—are computed from those retained bytes before the worker is launched. The retained bytes, not the path, are what the worker later receives: the compiler source is transmitted to the child process over standard input, base64-encoded inside the worker request, together with its raw digest. The worker independently recomputes the digest of what it received and refuses to execute on mismatch. The path from which the bytes were read plays no further role in the evidence.

Threat 7.4 (Time-of-check to time-of-use substitution). An adversary—or, far more commonly, an innocent concurrent process such as an editor, a formatter, or a version-control operation—modifies the compiler file between the moment its hash is computed and

the moment it is executed. A protocol that hashes a path and then executes the path certifies the wrong bytes.

Step 1 blocks this threat structurally rather than by racing faster. Because the bytes that are hashed are the same in-memory bytes that are serialized into the worker request, there is no window in which the file system can substitute content: the file could be rewritten a nanosecond after the read and the evidence would still be exactly correct, because the evidence describes the retained bytes, not the file. The compilation record then retains those compiler bytes for its whole lifetime, and its constructor re-derives the identity from the retained bytes and compares it against the recorded identity—so even a record deserialized or reconstructed later cannot claim an identity its own retained bytes contradict (`E-COMPAT-COMPILER-IDENTITY`).

The same pattern applies to the source. Note the doubled check: the envelope verifies its own digest at construction, and the compilation constructor re-verifies both that the envelope's source still hashes to the envelope's declared digest and that the compilation's top-level `source_sha256` equals the envelope's. Redundant verification of this kind is a deliberate stylistic commitment of the kernel. Each object recomputes, from primitive retained data, every identity it asserts, rather than trusting that some earlier layer already did. The cost is a few extra hash computations; the benefit is that no single bypassed check silently propagates a false identity into evidence.

Why hash *before* execution rather than after? Because a post-execution hash certifies at most what the file looked like after the run, and an execution can modify its own inputs—or an adversary can restore a tampered file after the run completes. Pre-execution hashing over retained bytes is the only ordering under which the digest provably describes what executed. This is the reproducible-builds discipline [3] applied at the granularity of a single tool invocation: the artifact's identity is fixed at the moment of capture, and everything downstream refers to the capture, not to the mutable world.

Rule 7.5 (No path authority). A file path is a lookup hint, never an identity. Every identity claim in the compatibility profile is over retained bytes, and every consumer of the claim can recompute it from data the record itself carries.

3. Step 2: the forced Python-only engine profile and the frozen policy

The second step establishes *what kind of thing* is allowed to do the running.

The kernel admits exactly one engine profile in v0.1: `cas-v7.7/python-only`. The constant is not a default that a caller may override; it is checked at three independent points. First, the frozen policy object—compiler source authority, accepted versions, accepted syntax profiles, both raw digest variants, and the canonical LF digest—is reconstructed at every compilation and verified field-by-field against the module constants; if the policy object and the constants have drifted apart in any component, the compilation fails with `E-COMPAT-POLICY` before any worker launches. Second, the worker's returned artifact must itself declare `engine` equal to `python`; a legacy compiler that dispatched to any other execution engine, however it might have been induced to, fails the run with `E-COMPAT-ENGINE`. Third, the compilation record's constructor refuses any engine profile string other than the frozen constant, so the restriction survives into every downstream copy of the evidence.

What failure does this triple check block? Three distinct ones.

Configuration drift. A policy expressed once, at a distance from its enforcement point, tends to decay: constants get edited for a test and not reverted, a refactor renames a profile string, a merge resolves a conflict wrongly. By reconstructing and re-verifying the policy

at each compilation, the protocol converts silent drift into a loud, typed failure at the next execution. The policy is not configuration in the operational sense at all—there is nothing to configure—it is a self-checking restatement of the profile's identity.

Engine substitution. The CAS program contains, and intends to contain, more than one execution engine: the monorepo's architecture designates a Rust engine as the eventual production scoring core, and cross-implementation conformance between Python and Rust is explicitly *not established*—it is an open obligation, awaiting a differential-testing campaign. Until that campaign exists, a receipt that said only "the compiler ran" would be ambiguous across engines whose agreement no one has demonstrated. Forcing the Python-only profile makes the receipt's scope exact: this evidence speaks about the Python execution path and nothing else. A future native-engine profile would be admitted, if at all, as a *separately hash-bound* profile with its own identity constants—never as a widening of this one.

Semantic scope creep. The semantic profile string recorded in every receipt is `CAS v7.7 operational compatibility only`. The phrase is chosen to be un-misreadable: the profile asserts operational compatibility—the frozen compiler, run under the frozen policy, produced this artifact—and nothing about the correctness, meaning, or v8 interpretation of the compiled notation. A receipt is not a semantics. Freezing the semantic profile string, and rejecting receipts that carry any other, prevents the gradual rewording by which scope claims usually grow.

INVARIANT 7.6 (One profile, three checks). In v0.1 the engine profile is a single frozen constant verified independently at policy reconstruction, at worker-artifact inspection, and at record construction. No admitted code path executes legacy source under any other engine profile, and no admitted record asserts one.

4. Step 3: rejection of `use` imports

The third step bounds *how much* is allowed to run: exactly one source unit.

The CAS notation language includes a `use` construct by which one program imports another. The compatibility profile rejects it entirely. After the worker returns, the protocol walks the returned abstract syntax tree and fails the compilation with `E-COMPAT-UNBOUND-IMPORT` if any node of kind `UseStmt`, `UseStatement`, or `ModuleImport` appears anywhere in it. Multi-file CAS programs are, in v0.1, simply outside the profile.

The reason is not that imports are dangerous in themselves; it is that the profile has no honest way to *identify* what an import would bring in. Every other input to a compatibility execution is content-addressed: the source is hash-verified, the compiler is hash-verified, the interpreter and worker and loaded Python modules are inventoried and digested. An import statement, by contrast, names a module by a resolvable name, and resolution is a function of search paths, file-system state, and timing. To admit imports honestly, the profile would need an *import-DAG manifest*: a caller-supplied, hash-bound closure enumerating every transitively imported source unit with its digest, verified before execution exactly as the root source is, so that the identity "this program" expands to "this exact finite set of byte strings." No such manifest format has been designed, reviewed, or admitted. Until one is, an execution containing an import would be an execution whose full input is unidentified—and an evidence record over an unidentified input is precisely the kind of artifact this kernel exists to make unrepresentable.

THREAT 7.7 (Ambient-resolution smuggling). A source program imports a module whose content differs between the compilation that produced a receipt and a later replay, or differs between two hosts, or is chosen adversarially via search-path manipulation. The receipt

appears to bind "the program" while an unbounded portion of the program's meaning rides in un-hashed ambient state.

The rejection blocks this threat at the grammar level, which is the strongest available place: no analysis of resolution behavior is needed, because the construct that would invoke resolution is not admitted. Notice also *where* the check runs—on the returned AST, after compilation—rather than by pre-scanning the source text. Textual pre-scanning would have to reimplement enough of the lexer to distinguish a real `use` statement from the word appearing inside a string or comment, and any divergence between the scanner and the actual compiler grammar becomes a bypass. Inspecting the compiler's own parse output means the check sees imports exactly where the compiler sees them.

The AST walk is complemented by a second, independent emptiness check: the worker reports an `external_cas_imports` list in its isolation record, and both the protocol and the execution-profile constructor require that list to be empty (`E-COMPAT-UNBOUND-IMPORT` again). Two mechanisms, one from the artifact side and one from the runtime side, must agree that nothing external was touched.

Claim Boundary 7.8 (Out of profile, not impossible). The rejection of multi-file programs is a scope statement, not a capability judgment. Admitting an import-DAG manifest is listed among the kernel's open obligations; nothing in this chapter should be read as claiming that multi-file compatibility evidence cannot be built—only that it has not been, and that the profile fails closed rather than approximating it.

5. Step 4: the fresh `python -I -S -B` worker and the sanitized environment

The fourth step controls *where* the run happens: a freshly created Python process whose interpreter-level state is as close to a function of its declared inputs as CPython permits.

The worker is launched as `python -I -S -B -u` on the resolved path of the currently selected interpreter, with the packaged worker script as its program, the request delivered over standard input, a freshly created temporary directory as its working directory, a 30-second wall-clock timeout, and an environment constructed from scratch rather than inherited. Each flag closes a specific channel.

Isolated mode, `-I`. This is the broadest of the three flags and subsumes several older ones. It removes the script's own directory (and the current working directory) from the module search path; it ignores all `PYTHON*` environment variables that would otherwise steer the interpreter (`PYTHONPATH`, `PYTHONSTARTUP`, and the rest); and it disables the per-user site-packages directory. The channels it closes are exactly the classic interpreter-hijack channels: a `PYTHONPATH` entry that shadows a standard-library module, a startup file that runs arbitrary code before the worker's first line, a user-site package that monkey-patches `json` or `hashlib`. Under `-I`, the interpreter's import behavior is a function of the installation, not of the invoking user's ambient configuration.

No site initialization, `-S`. Even without user-site directories, CPython ordinarily imports the `site` module at startup, which executes `sitecustomize` and any `.pth` files found in site directories—and `.pth` files may contain executable lines. `-S` skips site initialization entirely. This matters for evidence because `.pth`-injected code is close to invisible: it runs before any user code, appears in no import statement, and varies per installation. A worker that ran site initialization could have its module namespace altered by content the receipt never inventoried.

No bytecode writing, `-B`. The worker must not write `.pyc` files. This is partly hygiene—the worker should not leave residue keyed to its inputs on the shared host—but it is also

an evidence measure: cached bytecode is a second physical representation of source whose staleness semantics are timestamp-based, and a future run that silently loaded a stale cache would execute bytes that no receipt hashed. Refusing to write the cache in the worker removes the worker as a source of such artifacts. (The unbuffered flag `-u` is operational rather than evidential: it ensures the single JSON response is not stranded in a stdio buffer when the process exits.)

The environment is rebuilt, not filtered. The construction starts from an empty dictionary and copies in an enumerated whitelist: on all platforms `SystemRoot`, `WINDIR`, and `COMSPEC` where present (Windows process creation and parts of the standard library misbehave without them), plus—on Windows only—the two processor-architecture variables discussed in Step 5; and then sets exactly three Python variables: `PYTHONHASHSEED` to `0`, `PYTHONIOENCODING` to `utf-8`, and `PYTHONUTF8` to `1`. Fixing the hash seed removes the one deliberate source of run-to-run nondeterminism CPython injects (randomized string hashing perturbs any code that accidentally depends on set or dict iteration order in unhashed ways); fixing the I/O encoding and UTF-8 mode removes locale-dependent text behavior, so the worker's byte-level output does not vary with the host's regional settings. The execution profile later verifies that the worker environment was *exactly* this constructed mapping—sorted, duplicate-free, subset of the whitelist, value-for-value equal—so an extra inherited variable is not merely unused but structurally rejected (`E-COMPAT-ENVIRONMENT`).

Why is inherited `sys.modules` an evidence hazard? Consider the alternative the fresh process replaces: executing the compiler in the calling process, or in a forked copy of it. The calling process is a long-lived research host. By the time a compilation is requested it may have imported hundreds of modules—test frameworks, notebook kernels, plugin systems, earlier versions of CAS code—any of which may have monkey-patched builtins, registered import hooks, installed codecs, or cached state in module globals. `sys.modules` is a process-global, mutable map, and Python's import system consults it *before* the file system: a module already present under a given name is simply returned, whatever its content. An in-process execution of the hash-verified compiler bytes would therefore run against a module namespace assembled by unhashed history. The receipt would bind the compiler bytes exactly and yet describe almost nothing about what actually executed, because the semantics of every `import` inside the compiler would be determined by parent-process state that appears nowhere in the evidence. Worse, the contamination is bidirectional and cumulative: each compilation would alter the very state the next compilation runs under, so receipts would not even be comparable with each other. The fresh process resets `sys.modules` to the interpreter's minimal startup contents, and the receipt records two boolean facts—`fresh_process` true, `parent_sys_modules_inherited` false—that the runtime gate later refuses to pass without (`DENY_NONISOLATED_PYTHON_PROCESS`).

Two further execution conditions complete Step 4, and each has a rationale of the same kind as the flags.

The disposable working directory. The worker's current working directory is a freshly created temporary directory, unique to the launch and removed when the launch completes. The current directory is one of the oldest ambient channels in computing: relative paths resolve against it, some tools consult it for configuration, and—the historical Python-specific hazard—older interpreter configurations put it on the import path, so that a file named like a standard module in the invoker's directory could shadow the real one. The `-I` flag already removes the import-path exposure; the disposable directory removes the rest. A worker whose current directory is empty, fresh, and about to be deleted can neither find anything there nor leave anything behind that a later run could find: the directory contributes exactly

nothing to the execution, which is the only contribution an unhashed location is permitted to make. It also means two compilations never share a working directory, closing the channel by which one run's residue could become another run's input—a small-scale instance of the same reasoning that motivates the fresh process itself.

The 30-second timeout. The worker is bounded at 30 seconds of wall clock, enforced by the parent. The bound is not a performance target—the frozen compiler parses a single source unit in milliseconds—but a liveness backstop with an evidence rationale. Without a bound, a worker wedged by any of the host pathologies this chapter and the next catalog (launch storms, file-system stalls, interpreter-initialization anomalies) would hang its compilation, and its caller, indefinitely; the failure would manifest as a silent stall somewhere up the stack rather than as a typed record at the point of fault. With the bound, a wedged worker becomes a worker-start failure with a transport record, attributable and countable like every other failure. The value is generous by three orders of magnitude relative to the expected work precisely so that it never fires on legitimate compilations: a timeout that trips on slow-but-honest runs would create a selection pressure correlated with host load, and Chapter 8's argument against biased receipt populations applies to timeouts exactly as it applies to retries. Thirty seconds is small enough to keep research pipelines live and large enough that its firing is, in practice, always news about a fault rather than about load.

Rule 7.9 (State must be inventoried or absent). Any interpreter state that can influence the compilation must either appear, content-addressed, in the execution profile, or be structurally absent from the worker. Ambient state that is present but uninventoried is not admitted in any form.

The worker's module namespace after execution is then *inventoried*: every entry of `sys.modules` is reported with its origin kind (file, builtin, frozen, or virtual), and every file-backed module must resolve to an absolute path inside the selected Python installation—or be the worker script itself—and must match a recorded digest. A file-backed module outside those roots fails the run with `E-COMPAT-DEPENDENCY-ESCAPE`. This is the closing of the loop opened by `-I -S`: the flags make it *unlikely* that anything outside the installation is loaded, and the inventory check makes it *fail closed* if it happens anyway.

6. Step 5: pre-execution runtime binding without probing

The fifth step identifies *what machine and interpreter* the evidence describes—and does so by reading, never by asking.

Before any compiler code executes, the worker samples its runtime identity: the interpreter implementation name (mapped to a display form—`CPython`, `PyPy`, and so on—from `sys.implementation.name`), the three-component version from `sys.version_info`, and a platform pair. On Windows the platform name is derived from `sys.platform` and the machine label from the `PROCESSOR_ARCHITEW6432` or `PROCESSOR_ARCHITECTURE` environment variables—which is precisely why those two variables are in the environment whitelist of Step 4: they are promoted from ambient state to explicit, profile-hashed *inputs*. On POSIX hosts the pair comes from `os.uname`. If neither source is available the label is an explicit `unknown-machine` rather than a guess. Alongside these labels, the worker reports the resolved path, byte count, and raw SHA-256 digest of its own interpreter binary and of the worker script file, and the parent verifies both against fresh reads of the same paths.

What the protocol pointedly does *not* do is probe. It does not query WMI, it does not shell out to `uname` or `systeminfo` or `wmic`, it does not import `platform` machinery that

lazily spawns subprocesses, and it performs no lazy imports at all in the identity path. This restraint is itself a security and evidence decision, for four reasons.

First, *probing executes code that the profile does not bind.* A WMI query traverses a service, a provider stack, and a COM infrastructure; a subprocess probe executes whatever binary the search path resolves. None of those components are content-addressed in the profile, so their outputs would be assertions by unidentified software—exactly the class of input the kernel refuses everywhere else. A runtime label obtained through an unhashed intermediary has the evidentiary standing of hearsay.

Second, *probing widens the attack surface at the worst moment.* The identity sample runs inside the trusted window between process start and compiler execution. Every additional mechanism invoked in that window—service connections, subprocess creation, dynamic library loading—is an additional mechanism whose compromise or misconfiguration can alter the run before the compiler's first instruction, and an additional reason two otherwise identical runs might differ.

Third, *probing is a nondeterminism generator.* WMI calls time out, return transient errors, and can be slow under load; subprocesses inherit scheduling jitter and can fail for reasons unrelated to the machine's identity. The deterministic-closure property that Step 8 will test—two independent executions must agree exactly—would be hostage to infrastructure that fails sporadically. The chosen sources (`sys` fields, `os.uname`, two whitelisted environment variables) are plain memory reads within the already-running interpreter: they cannot hang, cannot spawn, and return the same bytes every time on the same host.

Fourth, and most structurally: *the profile must be a function of its recorded inputs.* The execution-profile validator recomputes the expected runtime identity from the same sanitized environment that the worker received and requires the worker's report to match it exactly (`E-COMPAT-RUNTIME`). That recomputation is only possible because every input to the identity derivation is itself part of the recorded profile. A probe result cannot be validated this way—there is nothing recorded from which to recompute it—so it could only be trusted, and the kernel does not trust.

PROPOSITION 7.10 (Runtime identity is replayable). *Under the profile's derivation rules, the runtime identity quadruple (implementation, version, platform, machine) is a deterministic function of the interpreter binary and the sanitized environment mapping, both of which are content-addressed in the execution profile. Two runs whose profiles agree on those inputs agree on the quadruple.*

Design argument. The derivation reads only `sys.implementation.name`, `sys.version_info`, `sys.platform` or `os.uname`, and the two whitelisted architecture variables. The first three are constants of the interpreter build identified by the executable digest; the last are values of the recorded environment mapping. No file-system, network, service, or subprocess input participates. The validator enforces agreement by recomputing the quadruple and rejecting mismatches, so a profile asserting a non-derivable identity cannot be constructed.

The binding is *pre-execution* for the same reason the compiler hash is: an identity sampled after compiler code has run is an identity the compiled code could have influenced. The worker's protocol is strictly staged—verify received compiler digest, sample runtime, execute, inventory, report—and the parent's shape checks on the response (exact key sets at every level, rejected on any deviation) make a worker that reordered or omitted stages structurally distinguishable from a conforming one.

7. Interlude: the worker wire protocol

Steps 4 and 5 described the worker's execution conditions; this interlude describes the conversation between parent and worker, because the wire format is itself part of the evidence discipline and several of its choices block failures that would otherwise be invisible.

The exchange is one request and one response, both single JSON documents. The request travels over standard input and carries exactly five fields: the protocol identifier `cas_core_v8.isolated_compiler_worker.v0.1`, the base64-encoded compiler source bytes, the compiler's raw SHA-256, the legacy source text, and the source SHA-256. The request is serialized through the kernel's canonical encoding, so the bytes the worker receives are the same bytes any auditor would reproduce from the recorded inputs. Delivering the compiler over standard input rather than by path is the mechanical realization of Rule 7.5: the worker never opens the compiler file, so no file-system race, symlink trick, or path-resolution surprise can interpose between verification and execution. The worker recomputes the digest of the decoded bytes and refuses on mismatch, giving the pair a two-sided check—the parent verified what it sent, the worker verifies what it received.

The response is read from standard output, with three parsing rules that each close a specific hole.

First, the parent parses the response with a duplicate-key rejecting hook: a JSON object in which the same key appears twice fails the run with `E-COMPAT-WORKER-PROTOCOL`. Ordinary JSON parsers silently keep the last occurrence, which means two parsers can extract different values from the same bytes—a classic parser-differential channel through which a crafted or corrupted response could tell the shape-checker one thing and the value-consumer another. Rejecting duplicates outright removes the ambiguity instead of adjudicating it.

Second, floating-point literals are parsed directly into exact decimals rather than binary floats. The canonical evidence encoding rejects binary floats; parsing them into `Decimal` at the wire boundary means a numeric value in a compiler artifact never transits through a lossy binary representation on its way into evidence, and two hosts with different float-formatting behavior cannot produce receipts that differ for that reason alone.

Third, every level of the response is checked against an *exact* key set—not a superset check but set equality. A success response must have exactly the keys `protocol`, `ok`, `artifact`, `runtime`, `dependencies`, and `isolation`; the runtime object exactly its eight keys; the flags object exactly its six; an error response exactly `protocol`, `ok`, `error_code`, and `message`. An extra field is as fatal as a missing one. This is unusual—most protocols tolerate unknown fields for forward compatibility—but forward compatibility is precisely what an evidence protocol must not have by accident. An unknown field is unaudited content riding inside a hash-bound record; tolerated today, it becomes load-bearing tomorrow without ever passing review. Version evolution in this kernel happens by changing the protocol identifier, loudly, not by widening what a fixed identifier accepts.

The response's outcome logic is similarly strict in both directions. If the worker reports failure, the parent raises the worker's own typed error code. If the worker reports success but exited nonzero, that contradiction is itself a failure (`E-COMPAT-WORKER-EXIT`): a process that claims success while signaling failure is not half-trusted on the favorable half. If the worker wrote *anything* to standard error, the run fails with `E-COMPAT-WORKER-STDERR` even if standard output parsed perfectly—because a conforming worker is silent on stderr, and unexplained diagnostics are unexplained behavior. In every one of these failure paths, the transport receipt of Chapter 8—exit code, byte counts, digests of both streams—is embedded

in the error, so the failure record identifies the exact observable behavior without reproducing its content.

RULE 7.11 (Exact shapes only). Worker messages are validated by set equality of keys at every level, duplicate keys are rejected, and any deviation—extra field, missing field, wrong protocol identifier, stderr output, success-with-nonzero-exit—fails the compilation closed. There is no lenient parse mode.

8. Interlude: freezing the legacy artifact

Between the worker's response and the receipt of Step 7 stands one more transformation that deserves explicit treatment: the compiler's output is *frozen* into exact, immutable evidence values before it is hashed or stored. The freezing pass walks the entire artifact and applies a closed set of rules; anything the rules do not cover fails the run with one of the typed `E-COMPAT-ARTIFACT-SHAPE`, `E-COMPAT-ARTIFACT-TEXT`, or `E-COMPAT-ARTIFACT-NUMBER` errors collected in Table 1.

Null, booleans, and integers pass through unchanged. Strings must satisfy the kernel's canonical-text requirements. Exact decimals must be finite—a NaN or infinity in compiler output is not representable in evidence, so it fails closed rather than serializing as some encoder's best guess. Any binary float that survived to this layer is converted to the exact decimal it denotes, with the same finiteness requirement. Mappings must have string keys, each canonicalized, and are frozen into read-only views; a mapping that uses the encoder's reserved type-tag key is rejected outright, because an artifact that could impersonate the encoding's own tagging machinery could forge type distinctions—the type-pun attack of the encoding chapter, attempted from inside the data. Lists are frozen into immutable list values; bare tuples are rejected with an instruction to use the immutable list type, so that there is exactly one array representation in evidence rather than two that compare unequal. After freezing, the artifact's root must be a mapping whose `valid` field is a genuine boolean.

Why freeze at all? Two reasons, one about time and one about identity. The time reason: the compilation record outlives the function call that created it, and its digest is computed once. If the artifact remained mutable, any later code touching the record could alter the artifact after digesting, and the record would carry a digest of something it no longer contains—the constructor's re-verification (the recomputed artifact digest must equal the recorded one, else `E-COMPAT-ARTIFACT-HASH`) would catch this only at reconstruction boundaries, not in a live object. Immutability makes the digest permanently truthful. The identity reason: the freezing rules are exactly the canonical encoding's admission rules, applied eagerly. Freezing at the boundary means a compilation record can never hold a value that the encoder would later reject; there is no artifact that is storable but unhashable, and therefore no evidence object whose identity is undefined.

REMARK 7.12. The freezing pass is also where the artifact's diagnostics are derived: the frozen AST is walked once for fractal literals and overloaded RPM calls, producing the typed diagnostic codes of Step 7. Deriving diagnostics from the frozen form, rather than the raw response, guarantees that the diagnostics describe exactly the bytes the digest describes.

9. Step 6: advisory-lock serialization of worker launches

The sixth step governs *when* workers may start, and it is the protocol's one concession to the messiness of real operating systems.

The empirical problem is concrete: on Windows, when many isolated Python runtimes initialize concurrently, a child process can occasionally exit with a return code and *no* stdout or stderr at all—a transport-level failure with nothing to diagnose. A process-local mutex does not fix this, because independent test runners and research processes overlap on the same host. The protocol therefore serializes worker creation through two nested exclusions: a process-local thread lock, and a cross-process advisory file lock—a single byte locked via `msvcrt.locking` on Windows and `fcntl.flock` elsewhere—on a fixed-name file (`cas-core-v8-compatibility-worker.lock`) in the host temporary directory. The launch policy identifier `cross-process-exclusive-lock-v1`, the lock path, and all three timing parameters are recorded in every execution profile and verified against the frozen constants at profile construction; a profile asserting different timing is rejected (`E-COMPAT-ISOLATION`), so the concurrency discipline under which evidence was produced is itself part of the evidence.

The parameters: acquisition is bounded at 90 seconds, after which the compilation fails closed with `E-COMPAT-WORKER-LOCK`; while waiting, the acquirer polls the non-blocking lock at 10-millisecond intervals; after releasing, the holder sleeps 10 milliseconds before proceeding, as a deliberate scheduler yield so that a process already waiting on the shared lock gets an opportunity to acquire it. Chapter 8 develops the rationale for each number and for the advisory (as opposed to mandatory) choice in depth; here we record the two properties that belong to the closure protocol itself.

First, *no retry*. If a worker fails—for any reason, including the very launch-storm failures the lock exists to reduce—the compilation fails. The protocol never launches a replacement worker for a failed one. The full argument is in Chapter 8; the short form is that retrying a nondeterministic failure filters the receipt population by success and thereby converts flakiness into silent selection bias over evidence.

Second, *no fairness claim*. The advisory lock guarantees mutual exclusion of cooperating launchers; it does not guarantee that waiters acquire in order, that any particular waiter is not starved short of its 90-second bound, or anything about non-cooperating processes, which are not obliged to respect an advisory lock at all. The post-release yield is a mitigation, not a guarantee, and the protocol's documentation says so. This honesty is not modesty for its own sake: a fairness property, once asserted, becomes something downstream consumers may build on, and asserting a property the mechanism does not have is exactly the overclaiming failure mode the kernel exists to prevent—applied, this time, to the kernel's own description of itself.

THREAT 7.13 (Launch-storm evidence corruption). Concurrent isolated-interpreter initialization intermittently yields empty-transport child failures. Without serialization, a fraction of compatibility executions fail for reasons uncorrelated with their inputs; without the no-retry rule, that fraction would be silently resampled away, biasing the surviving receipts; without recorded lock parameters, the conditions under which any receipt was produced would be unknowable. The three mechanisms jointly keep the failure visible, bounded, and attributable.

10. Step 7: the receipt

The seventh step fixes *what is remembered*. Its principle: the receipt must contain every fact a skeptical reviewer needs in order to re-derive the run's identity, and it must state its own limitations as machine-readable fields rather than prose.

The compilation record and the compiler-validation receipt derived from it carry, at minimum:

Compiler identity, twice. Both the raw digest of the compiler bytes that executed (which of the two frozen physical variants it was) and the LF-normalized content digest. As argued in Step 1, neither substitutes for the other: the raw digest answers "which bytes ran," the LF digest answers "which content ran," and conflating them has historically hidden both real differences and harmless ones.

Interpreter and worker bytes. The resolved path, byte count, and SHA-256 digest of the Python executable, and the same for the worker script. The compiler is not the only code in the room; the interpreter that ran it and the harness that fed it are code too, and a receipt that hashed the compiler while leaving the interpreter anonymous would invite exactly the substitution it closes elsewhere.

Import search paths. The worker's complete `sys.path`, each entry required to be absolute, duplicate-free, and confined to the selected Python installation. The search path is the function from import statements to file contents; recording it makes the import behavior of the run auditable rather than assumed.

Loaded-module inventory. Every module present in the worker after execution, with origin kind, path where applicable, and content digest for every file-backed entry; the inventory must be sorted, duplicate-free, and must bind `__main__` to the worker script's own digest. This is the receipt's answer to "what code, in total, participated?"—not the code that was supposed to participate, but the code the interpreter actually loaded.

Exact profiles. The syntax profile (`cas-v7.7`), the semantic profile (`CAS v7.7 operational compatibility only`), the engine profile (`cas-v7.7/python-only`), and the runtime profile string derived in Step 5. These scope the receipt: any consumer can determine, from the receipt alone, precisely which compatibility claim is being evidenced and which are not.

Result digest and diagnostics. The canonical-encoding digest of the frozen compiler artifact, and a typed, nonempty diagnostic tuple. The diagnostics always include the profile marker `LEGACY_COMPATIBILITY_PROFILE`, and add `UNTYPED_FRACTAL_REQUIRES_BRIDGE` when the artifact contains fractal literals (whose v8 meaning only the explicit bridge of Chapter 8 may supply) and `OVERLOADED_RPM_REQUIRES_TYPE_CHOICE` when it contains RPM calls whose typing a human must resolve. Diagnostics are how the receipt marks the places where legacy content touches questions the profile refuses to answer.

The research boundary. Every record embeds the Ring-4 boundary triple—admission OPEN, product NOT LICENSED, disclosure false—and every constructor in the chain re-verifies it. A receipt whose boundary is anything else cannot be built.

Capability booleans, including the false ones. The receipt asserts `isolated_mode`, `ignore_environment`, `fresh_process` true and `parent_sys_modules_inherited` false, as established in Step 4—and it asserts `os_sandboxed` false and `host_os_dependencies_content_addressed` false, as Section 12 discusses. It also carries an explicit self-hash scope string, `CORRUPTION_DETECTION_ONLY_NOT_AUTHENTICA` so that the receipt's own integrity digest cannot be quietly promoted into an authentication claim even by documentation drift.

RULE 7.14 (Receipts state their own weaknesses). A capability the execution does not have is recorded as an explicit false field, verified at construction (a receipt asserting it true is rejected), and re-checked at assessment (a receipt asserting it true is denied with `DENY_FALSE_OS_ISOLATION_CLAIM`). Limitations are data, not caveats.

The design debt here is to provenance practice [2]: the receipt separates the entity (source and compiler bytes), the activity (the worker execution, with its policy and timing), and the environment of the activity (interpreter, modules, paths), rather than collapsing them into a single opaque success bit. And the receipt's whole content is serialized through the canonical

encoding of the earlier chapters [1], so its digest is stable and its comparison semantics are exact.

11. Step 8: fresh replay before any structural decision

The eighth step determines *when the evidence may be used*: never on faith in a stored record, always after fresh re-derivation.

The requirement operates at two nested levels. At the compilation level, every compile is already a determinism experiment: the protocol runs *two* independent fresh workers over the same retained source and compiler bytes and requires their frozen artifacts to have equal canonical digests (`E-COMPAT-NONDETERMINISTIC-RESULT` otherwise) and their execution profiles to have equal digests (`E-COMPAT-NONDETERMINISTIC-PROFILE`, with the bounded diagnostic of Chapter 8). A compilation record thus never represents a single run's say-so; it represents an agreement.

At the decision level, the requirement bites harder. When a compilation record is later presented as evidence for a structural decision—in the assessment and gate machinery of Chapter 9—the kernel does not compare the stored record against stored expectations. It first re-verifies that every file the record's execution profile binds (interpreter, worker, file-backed modules) still has its recorded bytes (`E-COMPAT-DEPENDENCY-RACE` on any drift), and then *re-executes the whole protocol*: same retained source bytes, same retained compiler bytes, two fresh workers, full verification, and requires the replay's complete canonical serialization—artifact, profile, diagnostics, everything—to be byte-identical to the supplied record (`E-COMPAT-REPLAY-MISMATCH` otherwise). Only the replay's receipt, not the stored one, feeds the decision.

What does this block? The stored-receipt attack surface in its entirety. A receipt is bytes; bytes can be fabricated, replayed from another host, generated by a tampered earlier run, or produced honestly under conditions that no longer hold. Signature schemes address some of this at the cost of a key-management trust root the kernel deliberately does not yet have. Fresh replay addresses it differently: the receipt is demoted from *proof* to *prediction*. Presenting a compilation record to the gate amounts to predicting, in exact canonical bytes, what a fresh deterministic-closure execution will produce; the gate runs the experiment and checks the prediction. A fabricated receipt fails because the fabricator must predict digests of an execution they cannot run in advance on the verifier's host; a stale receipt fails because some bound file has drifted; an honest receipt passes and, in passing, demonstrates that the claimed determinism actually holds *now*, on *this* machine, which is the only fact the decision needs.

Invariant 7.15 (No decision on stored evidence). No structural decision in the kernel is reachable from a stored compatibility record alone. Every path to a structural pass includes a fresh execution of the full closure protocol whose canonical serialization must equal the record byte-for-byte.

The cost is real: replay doubles-and-more the execution work, and confines structural decisions to hosts that hold the same interpreter and compiler bytes. The kernel accepts this cost knowingly. In a research program whose documented failure history includes self-generated artifacts masquerading as authority, cheap trust in stored records is precisely the resource an overclaim consumes. Making evidence expensive to *assert* and cheap to *check* is the correct asymmetry; replay makes assertion and checking the same act.

12. The isolation boundary: what this protocol is not

Everything above concerns Python-process isolation, and it is important to state exactly what that phrase does and does not mean. The worker is a fresh operating-system process with a sanitized environment, an isolated interpreter configuration, an inventoried module namespace, and a bounded lifetime. It is *not* sandboxed. It runs as the invoking user, with the invoking user's privileges, on the shared file system, with no syscall filter, no capability restriction, no network namespace, no resource jail beyond the 30-second timeout. A hostile compiler—which the frozen, hash-bound compiler is not, but the point of an evidence kernel is not to need that assumption—could read files, write files, or open sockets, and nothing in this protocol would prevent it. The protocol constrains and records the *Python* face of the execution; the operating-system face is constrained only by the host's ordinary rules.

Likewise, the content-addressing perimeter has a definite edge. Inside it: the source, the compiler, the interpreter binary, the worker script, every file-backed Python module, the environment mapping, the search paths, the launch policy. Outside it: the native libraries the interpreter loads (the C runtime, OpenSSL, the OS loader's dependency graph), the kernel and system services beneath them, and the hardware. Two receipts with identical profiles can therefore have been produced above different native substrates, and the receipt cannot tell you. For the deterministic-closure property this residual matters less than it might—the replay requirement re-runs on the actual current substrate, so a substrate-dependent divergence surfaces as a replay mismatch rather than passing silently—but for any claim of the form "this execution was hermetic," the receipt is not evidence, and it says so itself.

Claim Boundary 7.16 (Non-sandbox, machine-readable). The profile's `os_sandboxed` and `host_os_dependencies_content_addressed` fields are constant false in v0.1. The constructors reject a profile or receipt asserting either true (`E-COMPAT-ISOLATION`); the assessment denies such a receipt (`DENY_FALSE_OS_ISOLATION_CLAIM`); and every structural pass carries the two limitation codes `LIMITATION_NO_OS_SANDBOX` and `LIMITATION_HOST_OS_DEPENDENCIES_NOT_CONTENT` in its decision record. A downstream consumer cannot honestly read any artifact of this protocol as hermetic-execution evidence, because the artifact itself denies it in typed fields.

This is an unusual design choice and worth dwelling on. Most systems describe their guarantees; few encode their non-guarantees as validated, constant, machine-checked fields. The kernel does so because the alternative—limitations living in documentation—has a known failure mode: documentation and behavior drift apart, and the strongest claim anyone remembers becomes the claim in circulation. A false capability field cannot drift. If a future version of the kernel gains a real OS sandbox, admitting it will require changing validators that currently *reject* the true value—a loud, reviewable act—rather than quietly starting to say yes. The threat being modeled here is not an external attacker but the program's own future enthusiasm: the boundary field is a precommitment device against the kernel's maintainers, in the same spirit as the constant-false authorization fields of Chapter 9.

13. Reading a receipt: a reviewer's walkthrough

The steps above are stated mechanism-first. Walk the same ground once more from the opposite direction: a reviewer holds a compiler-validation receipt and wants to know what, exactly, they may conclude from it. The exercise makes the protocol's division of labor concrete and shows where each step's guarantee is consumed.

The reviewer's first question is *scope*: what kind of claim is this receipt even attempting? The answer is printed in the receipt itself. The receipt version string identifies the record

contract; the syntax, semantic, and engine profile fields identify the claim family as CAS v7.7 operational compatibility under the Python-only engine; the `legacy_only` flag (validated true) and the structural absence of any v8 translation field establish that no v8 meaning is being asserted; and the embedded Ring-4 boundary establishes that whatever the receipt shows, it is research evidence—admission open, product not licensed, disclosure withheld. A receipt cannot soft-pedal any of this, because each field is validated at construction and re-checked at assessment; the reviewer does not need to trust the surrounding prose, which is the point.

The second question is *identity*: what ran? The receipt names the source digest, both compiler digests, the interpreter digest, and the worker digest. The reviewer can check the compiler digests against the frozen constants of Definition 7.1 independently—the constants are published in the Gate 4 record—and can distinguish, from the raw digest, whether the CRLF or LF physical variant executed. If the reviewer holds the same artifacts, every one of these digests is recomputable from bytes; nothing requires taking the receipt's word.

The third question is *conditions*: under what regime did it run? The receipt's boolean block answers with validated facts rather than assurances: isolated mode, environment ignored, fresh process, no inherited module state—and, in the false fields, no OS sandbox and no host-OS content addressing. The execution profile behind the receipt adds the sanitized environment (names and values from the fixed whitelist), the confined search paths, the full module inventory, and the launch policy with its exact timing constants. A reviewer who suspects that some receipt was produced under looser conditions has a decidable question, not a judgment call: looser conditions cannot satisfy the profile validators, so either the receipt's digests do not verify, or the conditions were as stated.

The fourth question is *meaning*: what did the run produce, and what interpretive cautions attach? The result digest identifies the frozen artifact exactly; the diagnostics enumerate the caution flags. If `UNTYPED_FRACTAL_REQUIRES_BRIDGE` is present, the artifact contains fractal digit sequences whose v8 interpretation the receipt explicitly does not supply—only the typed bridge of Chapter 8, with a caller-owned catalog, may do that, and even then only as type-correct composition, never digit meaning. If `OVERLOADED_RPM_REQUIRES_TYPE_CHOICE` is present, a human type decision is pending and no downstream automation should pretend otherwise.

The fifth and decisive question is *currency*: is any of this still true, here, now? Here the reviewer's answer is procedural rather than inspective: present the record to the assessment machinery and let Step 8 replay it. If the replay passes, the reviewer has learned something no stored receipt could tell them—that the deterministic closure holds on the present host at the present moment. If it fails, the failure code says why: drifted dependency bytes, a changed interpreter, a nondeterministic compiler behavior surfacing, or a record that never corresponded to a real execution at all. In both outcomes the reviewer's conclusion rests on a fresh experiment, and the stored receipt served only as the experiment's exact specification.

Remark 7.17. Note what is absent from the walkthrough: at no point does the reviewer ask *who* produced the receipt. This is deliberate and double-edged. The protocol makes receipts verifiable without provenance of authorship—anyone with the bytes can check everything—but by the same token a verified receipt carries no authorship claim, no review status, and no authority. The question "who vouches for this?" has no answer inside the compatibility profile, and Chapter 9 shows that the kernel answers it, for now, with a constant: no one.

14. The protocol as a whole

Read together, the eight steps have a shape worth making explicit. Steps 1–3 fix the *inputs*: what runs (hash-verified source and compiler bytes), under what regime (one frozen engine profile), and how much (one source unit, no imports). Steps 4–6 fix the *execution*: where (a fresh, flag-isolated, environment-sanitized process), on what identified substrate (pre-execution runtime binding without probing), and when (serialized launches, bounded waits, no retries). Steps 7–8 fix the *output*: what is remembered (a receipt that re-derives, scopes, and self-limits) and when it may be believed (only upon fresh replay).

Adverse Finding 7.18 (Closure, in the profile's honest sense). Within the recorded perimeter—source, compiler, interpreter, worker, Python modules, environment, search paths, launch discipline—a compatibility execution is a deterministic function, and the protocol tests this determinism on every compile (two workers) and again at every decision (fresh replay). Outside the perimeter—native libraries, operating system, hardware—no closure is claimed, and the receipts carry that non-claim as data. Both the Gate 4 test evidence for this behavior and the profile itself are Ring-4 research artifacts: structural software receipts under exact fixtures, not certification, review, or authority.

The next chapter takes up the two components of this protocol that most repay closer analysis—the concurrency discipline of Step 6 and the failure taxonomy that Steps 4–6 feed—together with the one bridge by which legacy artifacts may touch v8 structure at all.

CHAPTER 8

Concurrency and the Failure Taxonomy

1. Failures are evidence too

The previous chapter treated the deterministic-closure protocol as a pipeline whose happy path ends in a receipt. This chapter treats the unhappy paths, and its governing observation is simple: in an evidence kernel, a failure is not an inconvenience to be engineered away—it is a data point about the world, and the way the kernel handles it determines whether the surviving body of receipts means what it appears to mean.

Three pressures act on any system that executes subprocess workers on shared research hosts. Concurrency: multiple independent processes—test runners, notebooks, agents—launch workers on the same machine at overlapping times, and the operating system's behavior under that load is imperfect. Nondeterminism: some failures do not reproduce, and the temptation to "just try again" is constant. Diagnosis: when something fails, someone must be able to find out why, which pulls toward recording more; but evidence records circulate, which pulls toward recording less. The kernel's answers—an advisory lock with honest limits, an absolute no-retry rule, and failure records that are bounded and hashed rather than verbose—form a coherent position on all three, and this chapter develops each in depth. The chapter closes with the fractal bridge rule: the single, narrow, typed doorway through which legacy artifacts may touch v8 structure, included here because it is the profile's remaining fail-closed surface and because its design applies the same discipline—explicit inputs, typed failures, no silent meaning—to a semantic rather than operational problem.

Everything in this chapter, as throughout the monograph, describes a Ring-4 research candidate: the behaviors are test-evidenced under exact fixtures at the Gate 4 receipt, and nothing here is a reliability certification or an authority claim.

2. The advisory lock discipline

2.1. The problem the lock answers. The lock exists because of an observed, concrete pathology, not a theoretical one. On Windows hosts, when many isolated Python interpreters initialize at the same time, a child process occasionally exits with a return code but with completely empty standard output and standard error. From the parent's perspective the worker simply vanished: no artifact, no error message, no diagnostics—an empty transport. The failure is intermittent, load-dependent, and uncorrelated with the inputs of any particular compilation.

For an ordinary build system this would be an annoyance. For an evidence procedure it is worse, because the deterministic-closure protocol runs *two* workers per compilation and compares them; a launch-storm failure in either worker kills the compilation, and a storm that strikes during replay (Step 8 of Protocol 7.3) kills a structural decision. The failure rate of compilations would become a function of what else the host happens to be doing—an unrecorded, uncontrolled variable modulating the production of evidence.

A process-local mutex is the obvious first move and is insufficient, for a stated reason: the processes contending for the host are not threads of one program. Independent test runners, developer shells, and research agents overlap on the same machine, and no in-process lock is visible across them. The serialization has to live somewhere all cooperating processes can see, which on a shared host means the file system.

2.2. Why advisory. The chosen mechanism is a one-byte advisory file lock—`msvcrt.locking` on Windows, `fcntl.flock` elsewhere—on a file of fixed name, `cas-core-v8-compatibility-worker.lock`, in the host temporary directory, taken in exclusive non-blocking mode inside a poll loop. "Advisory" means the lock constrains only processes that ask: a process that never attempts the lock is not prevented by the operating system from launching whatever it likes. Why accept that?

First, because the alternative does not exist in portable form. Mandatory file locking is a niche, platform-divergent capability—effectively unavailable in any form the kernel could rely on across Windows and POSIX—and OS-level mechanisms that could truly exclude non-cooperating processes (job objects, cgroups, privileged brokers) would require exactly the sandboxing infrastructure that Boundary 7.16 declares out of scope for v0.1. An advisory lock is what a user-privilege, cross-platform kernel can actually have; claiming more would be false capability, and this kernel's stance on false capability fields is already on record.

Second, because the threat model is coordination, not adversaries. The empty-transport pathology arises from *cooperating* software—the program's own test suites and research processes—colliding by accident. Advisory locking is precisely a coordination mechanism for cooperating parties. An adversarial process on the host could ignore the lock, but an adversarial process on the host can also do far worse things that no lock addresses (Chapter 7 was explicit that the worker is not sandboxed); defending against accident with a mechanism proportioned to accident is the honest scope.

Third, because advisory OS locks have the one liveness property this design cannot do without: *crash release*. The lock is held through an open file handle, and when the holding process dies—normally or abnormally—the operating system releases the lock. A lock-file convention based on file existence (create to acquire, delete to release) fails exactly here: a crashed holder leaves the file behind and every future launcher waits forever or implements fragile staleness heuristics. Under the OS-held advisory lock, a crashed holder stalls contenders for at most the remainder of their bounded wait, usually far less.

The kernel wraps the file lock in a second, inner discipline: a process-local thread lock acquired first, with the same bound. This is not redundancy. The file lock serializes *processes*; the thread lock serializes *threads within a process*, which would otherwise contend on the same file handle with less well-defined semantics, and it does so without file-system traffic for the common single-process case. The two locks share one deadline, measured on the monotonic clock—so a wall-clock adjustment mid-wait cannot extend or truncate the bound—and both failure modes surface as the same typed error, `E-COMPAT-WORKER-LOCK`, with a message distinguishing which level timed out.

Finally, the entire discipline is *profile-bound*. The launch policy identifier (`cross-process-exclusive-lock-` the canonical lock path, and all three timing constants are fields of the execution dependency profile, verified against the frozen module constants at profile construction and rejected on any deviation. The concurrency regime under which a receipt was produced is thus part of the receipt—a reviewer never has to guess which locking discipline was in force, and a build of the kernel with altered timing produces receipts that say so by failing validation.

2.3. The lock file itself. The physical lock deserves a paragraph of its own, because even here there are choices with failure modes behind them. The lock file lives at a resolved, canonical path in the host temporary directory—the one directory guaranteed writable to every cooperating process on every supported platform without configuration—under a fixed, namespaced filename. A configurable location would reintroduce the coordination problem the lock solves: two processes configured differently would lock different files and exclude nothing. The profile validator enforces canonicality, rejecting any profile whose recorded lock path is not the canonical resolution of the fixed location, so a receipt cannot claim the discipline while having used a private lock.

On first use, the acquirer opens the file in append mode—creating it if absent, never truncating it if present, since truncating a file another process holds a lock on invites platform-specific grief—and writes a single null byte if and only if the file is empty, because a byte-range lock needs at least one byte to range over. The locked region is exactly that one byte. Locking a single byte rather than the whole file keeps the operation identical across the two platform APIs, whose whole-file semantics differ, and makes the lock's extent independent of anything any process ever writes. The file's *content* carries no information at all: it is not a PID file, not a timestamp, not a queue. All state lives in the OS lock table, which is exactly the state the OS reliably cleans up on process death. Every convention that stores liveness information in lock-file content eventually confronts the staleness problem—is the recorded PID still the same process?—and the kernel sidesteps the entire genre by storing nothing.

Failure handling around the lock is fail-closed in both directions. An acquirer that cannot open the lock file (permissions, exotic tempdir configuration) fails immediately with the typed lock error rather than proceeding unserialized. And—less obviously—a holder that cannot *release* cleanly also raises: a release failure means the OS lock state is no longer what the discipline believes, and continuing silently would let a wedged lock manifest later as inexplicable timeouts in unrelated processes. The error surfaces at the holder, where the cause lives.

2.4. Parameter rationale. Three numbers govern the discipline: a 90-second acquisition bound, a 10-millisecond poll interval, and a 10-millisecond post-release yield. None is magic; each is an explicit trade, and recording the reasoning is part of keeping the profile reviewable.

The 90-second bound. The bound must be long enough that ordinary contention almost never trips it, and short enough that a wedged host fails visibly rather than hanging research pipelines indefinitely. The relevant scale is the holder's occupancy: a lock holder runs one worker under a 30-second timeout, and a full compilation runs two workers back-to-back, so a waiter behind one full compilation can legitimately wait on the order of a minute in the degenerate case where both of the holder's workers exhaust their timeouts. Ninety seconds covers that worst legitimate case with margin, while remaining small compared to the human timescales of a research session. The essential property is not the exact value but its *finiteness and fail-closed character*: acquisition either succeeds within the bound or raises the typed lock error, and no code path waits forever or proceeds unlocked. An unbounded wait converts a host anomaly into a silent hang; an unlocked fallback converts it into precisely the launch storm the lock exists to prevent. Both are worse than an honest, attributable failure.

The 10-millisecond poll. The file lock is taken in non-blocking mode inside a sleep loop rather than in blocking mode. Blocking acquisition would delegate the wait to the OS, which is cheaper, but it makes deadline enforcement platform-dependent and awkward, and it forfeits control over wakeup behavior. Polling at 10 milliseconds costs at most a hundred lock

attempts per second per waiter—negligible load against a file-system lock—while bounding the latency between a release and the next acquisition at one poll interval. A much smaller interval would buy latency no compilation notices (workers run for seconds) at the price of busier spinning; a much larger one would add human-perceptible lag to serialized test suites. Ten milliseconds sits where the marginal cost of both errors is small, and, being profile-bound, it is the same everywhere evidence is produced.

The 10-millisecond post-release yield. After releasing the file lock, the holder sleeps ten milliseconds before returning. This is the most unusual of the three and exists because of a scheduling asymmetry: the thread that just released is warm—running, cache-resident, and (in the common test-suite pattern) about to loop around and want the lock again—whereas a waiter in another process must be scheduled before its next poll can succeed. Without the yield, a tight local loop can reacquire the lock indefinitely ahead of a cross-process waiter, not through any queue-jumping in the OS but simply because it asks more often from a running state. The yield hands the scheduler an explicit opportunity to run someone else first.

2.5. What is honestly not claimed. It must be stated plainly, because the receipt-consuming layers of the kernel rely on statements being exact: this discipline claims *mutual exclusion among cooperating launchers within a bounded wait*, and nothing else. It does not claim fairness—waiters may acquire in any order, and the OS lock APIs used here maintain no ordered queue the kernel could appeal to. It does not claim starvation-freedom—an unlucky waiter can lose every race for the full 90 seconds and then fail, and the post-release yield is a mitigation that improves the odds, not a mechanism that bounds them. It does not claim anything about non-cooperating processes, which are invisible to an advisory lock by definition.

Why belabor non-claims? Because of how properties propagate. A downstream engineer who reads "serialized" as "fair" will eventually build something—a latency budget, a liveness argument, a test—on the fairness they inferred, and it will fail rarely, unreproducibly, and far from its cause. The program's documented failure history is largely a history of inferred properties hardening into assumed ones. The kernel's defense is to make its concurrency contract exactly as strong as its mechanism, in writing, in the profile: *bounded exclusive launch, unfair, advisory, fail-closed.* A future version that wants fairness must build a mechanism that has it, and change the policy identifier when it does.

Rule 8.1 (Concurrency contract). Cooperating worker launches are mutually exclusive under the profile-bound advisory lock; acquisition is bounded at 90 seconds with 10-millisecond polling and fails closed; a releasing holder yields 10 milliseconds. No fairness, starvation-freedom, or exclusion of non-cooperating processes is claimed, and no admitted receipt asserts any.

3. The no-retry principle

The protocol never retries a failed worker. Not on lock timeout, not on empty transport, not on nonzero exit, not on malformed output, not on the nondeterminism failures of Section 5. A failed worker fails its compilation, and the failure propagates, typed, to the caller. This section develops the argument for that rule in full, because it is the point where the kernel's evidence discipline conflicts most sharply with ordinary engineering instinct, and because the rule is load-bearing for the meaning of every receipt the kernel emits.

The instinct says: transient failures are noise; retrying filters noise; filtered results are cleaner. The instinct is correct for *operations* and wrong for *evidence*, and the difference is worth stating carefully.

Consider what a compatibility receipt is for. Chapter 7 defined the deterministic-closure property: within the recorded perimeter, compilation is a deterministic function, tested by running two workers per compile and replaying before every decision. The population of receipts is therefore not merely a set of useful outputs—it is the *measurement record* of that determinism claim. Every compilation is a trial; agreement is a confirming instance; disagreement or worker failure is an anomaly that bears on whether the closure property actually holds on real hosts.

Now introduce retry. A retry policy re-runs failed trials and keeps the successes. The surviving record then no longer estimates the behavior of the system; it estimates the behavior of the system *conditioned on eventual success*. Failures whose causes are real but intermittent—a dependency file being rewritten during runs, an interpreter behaving differently under memory pressure, a compiler bug that manifests one run in fifty, a host process interfering with workers—are exactly the failures retry masks best, because they are exactly the failures that pass on the second attempt. The receipts look clean; the closure property looks confirmed; and the anomalies that would have falsified it have been quietly deleted by the harness itself. This is selection bias in its textbook form—the analyst sees only the survivors—except that here the selection is applied automatically, invisibly, and before any analyst is involved. No field in a retried receipt says "this succeeded on attempt three"; and if such a field existed and were honest, the receipt would no longer be the receipt of a deterministic procedure at all.

The statistical analogy is optional stopping: re-running an experiment until it yields the desired outcome and reporting only the final run invalidates the reported result, even though every individual run was honestly conducted. Retry-until-success is optional stopping implemented in infrastructure. The kernel's refusal is the refusal to let its own harness perform the maneuver.

There is a second, independent argument from *identity*. A receipt binds one execution: these bytes, this profile, this artifact. Under retry, what would the receipt describe? The successful attempt only—with the failed attempts either discarded (evidence destroyed) or appended (in which case the record describes a composite procedure, "run until success within n tries," whose determinism is a different and weaker property than the one the kernel tests). Either way the crisp statement "a compatibility execution is a deterministic function of its recorded inputs" is lost. The two-worker agreement check makes this concrete: its entire evidential force comes from the fact that both workers are unconditional. Two workers that agree *after the harness discarded disagreeing pairs* would demonstrate nothing except the harness's persistence.

Third, there is an argument from *attribution*. The launch-storm pathology of Section 2.1 was discovered, characterized, and mitigated because it produced visible failures. A retry layer above the worker would have absorbed it years earlier at small per-run cost—and the cost would have compounded invisibly: an unserialized, retrying system fails more, retries more, loads the host more, and fails more again, with no signal reaching anyone. Fail-closed systems surface their pathologies while the pathologies are cheap; fail-retry systems surface them as capacity mysteries later. The advisory lock exists *because* the no-retry rule made the underlying failure impossible to ignore—the two disciplines are complements, the lock reducing the failure rate honestly (by removing a cause) rather than cosmetically (by hiding occurrences).

The objection writes itself: is a compilation that fails because the host was busy not simply wasted work? It is—and the waste is priced correctly. The caller sees a typed failure with a transport record; the caller may absolutely decide to run a new compilation, and

nothing forbids a human or an orchestration layer from doing so. The difference between that and in-protocol retry is where the decision is visible. A caller-level rerun is a new, independent compilation, producing its own complete evidence, with the failed attempt's record still existing wherever the caller's logs put it; the decision to rerun was made *above* the evidence layer by an accountable party. An in-protocol retry is made *inside* the evidence layer by the harness, and leaves no trace in the surviving record. The kernel's position is not that reruns are forbidden; it is that the evidence layer itself must be memoryless and unconditional, so that whatever selection happens, happens where it can be seen.

A final clarification distinguishes this rule from the retry discipline of distributed-systems practice, because the surface similarity misleads. In a distributed system, retrying an idempotent request against a replicated service is sound engineering: the request's *outcome* is the object of interest, the service's transient unavailability is genuinely irrelevant to that outcome, and the retry semantics are part of the declared contract—callers know results are at-least-once outcomes and design accordingly. None of the three conditions holds here. The compatibility protocol's object of interest is not the compiled artifact alone but the *execution as an event*: the receipt's value lies in describing what one run of the closed procedure did, which is exactly the thing retry redefines. Worker failures are not presumed irrelevant—the determinism claim makes every failure potentially relevant, and presuming otherwise is the bias. And no consumer of a receipt is in a position to "design around" hidden retries, because the receipt format asserts single-execution semantics; a retrying implementation would make every receipt a small lie. Retry is not wrong in general; it is wrong for artifacts whose meaning is "this is what happened," and the kernel's receipts have no other meaning.

Invariant 8.2 (Unconditional workers). Every worker launch admitted by the profile is unconditional: its outcome, success or failure, propagates to the caller as the outcome of its compilation. No admitted code path launches a worker because a previous worker failed.

4. Worker transport failure records

When a worker fails at the transport level—wrong exit status, undecodable output, stderr traffic, protocol-shape violation—the kernel must record something, and the question is what. Its answer is deliberately spare: the failure record embeds exactly the worker's exit code, the byte counts of standard output and standard error, and the SHA-256 digests of both streams. Five numbers and no content.

The exclusions are the design. Stream *content* is excluded for three reasons that compound. Bound: a misbehaving worker can emit arbitrarily much; a failure record that embeds content has no size bound, and failure records travel—into exceptions, logs, test output, evidence packets—so an unbounded record is an unbounded liability everywhere downstream. Sensitivity: stderr from a failing process is precisely the kind of text that leaks incidental host detail—user paths, environment fragments, library versions—and Section 5 will make the general principle explicit: evidence records must not exfiltrate host contents. Trust: the content of a failed worker's streams is, by construction, output of software in an unknown state; embedding it verbatim in records that conforming tools will parse and display extends the blast radius of whatever produced it.

What the five fields preserve is exactly what diagnosis and integrity need. The exit code classifies the failure at the OS level. The byte counts distinguish the signature cases at a glance—the launch-storm pathology is *exit code with zero bytes on both streams*, instantly recognizable—and separate "said nothing" from "said something malformed." The digests do the quiet heavy lifting: they make the failure record checkable against any copy of the

streams that does exist elsewhere (a caller that captured them can prove they are the same bytes the record describes), they make identical failures identical as records (two occurrences of the same misbehavior produce matching digests, so recurrence is detectable across time and hosts without content comparison), and they commit the record to a specific observed behavior without reproducing it. A transport record is thus a content-addressed *reference* to the failure, in the same sense that the rest of the kernel's evidence is content-addressed reference to artifacts.

The fail-closed casework around the transport record was described in Chapter 7 and is completed here from the failure side. An empty response is never treated as an empty success: no parseable protocol object means no worker claim at all, and the compilation fails with the transport record attached. A malformed response—undecodable bytes, duplicate keys, wrong shape, wrong protocol identifier—likewise fails closed; the parser's strictness (Rule 7.11) means there is no lenient path along which a damaged response could be partially believed. Contradictory responses fail on the contradiction itself: a worker that reports success but exits nonzero, or that produces a perfect success document while also writing to stderr, is a worker whose self-description is inconsistent, and inconsistency is grounds for rejection, not adjudication. In every case the error carries its typed code—`E-COMPAT-WORKER-EXIT`, `E-COMPAT-WORKER-PROTOCOL`, `E-COMPAT-WORKER-STDERR`, `E-COMPAT-WORKER-START`, `E-COMPAT-WORKER-LOCK`—so the failure taxonomy remains enumerable, and, per Invariant 8.2, none of them triggers a second attempt.

Rule 8.3 (Transport minimality). A worker transport failure is recorded as exit code, stdout and stderr byte counts, and SHA-256 digests of both streams—nothing else. Stream content never enters an evidence record. Empty and malformed responses fail closed and are never retried.

5. Dependency-profile disagreement

The subtlest failure in the taxonomy is the one where nothing failed. Both workers of a compilation launch cleanly, execute the verified compiler bytes, and return well-formed success responses—and their execution dependency profiles differ. Perhaps a Python module file changed between the two launches; perhaps the two processes resolved different search paths; perhaps something in the interpreter's initialization is not as deterministic as assumed. Whatever the cause, the premise of the receipt—that "the execution environment" is a single well-defined thing the receipt describes—has just been falsified by experiment, and the compilation must die: `E-COMPAT-NONDETERMINISTIC-PROFILE`.

(The sibling failure, in which the profiles agree but the compiled *artifacts* differ, is `E-COMPAT-NONDETERMINISTIC-RESULT`; it needs no special record design, because the artifacts' digests already identify the disagreement. The profile case is harder precisely because profiles describe the host.)

The error record for a profile disagreement contains: both profile digests, and a diagnostic listing *at most 32* keyed difference paths, with a boolean flag stating whether the list was truncated. The comparison runs over a transformed view of each profile in which every environment variable's value is replaced by its hash, and every dependency's file path is likewise replaced by its hash. Three design decisions—the bound, the keying, the hashing—each carry an argument.

Why the bound. A difference enumeration between two structured documents has no natural size limit: if the profiles diverge early and broadly—say, entirely different module

inventories—a naive diff enumerates hundreds of paths, and the diagnostic dwarfs the evidence it annotates. The failure record inherits every liability of unbounded records from Section 4, with one addition special to diagnostics: a huge diff is also *useless*, because a reviewer confronting three hundred difference paths learns less than one confronting the first thirty and a flag saying "and more." Thirty-two paths suffice to localize any plausible real-world divergence (a changed file, a differing path set, a drifted environment entry produce a handful of paths each); the truncation flag preserves honesty about what was cut; and the bound makes the worst-case record size a constant of the profile rather than a function of how strange the host was. The traversal itself is deterministic—dictionary keys visited in sorted order, lists by index with an explicit length-difference marker—so the *same* 32 paths are reported for the same disagreement, and two occurrences of one underlying cause produce comparable diagnostics.

Why keyed paths. The paths are expressed against keyed collections—environment entries addressed as `name=PYTHONUTF8`, dependencies as `module_name=json`—rather than positional indices. Positional addressing is fragile under exactly the perturbation being diagnosed: if one profile contains an extra module, every subsequent index shifts, and a positional diff reports dozens of spurious differences downstream of one real insertion. Keyed addressing pins each reported difference to the entity it concerns, so an inserted module is one path, not an avalanche.

Why hashes for values. Environment values and dependency paths are the two fields in a profile that describe the host rather than the computation, and they are exactly the fields excluded, as values, from the diagnostic. The reasons parallel Section 4 but sharpen. An environment value on a research host can contain usernames, machine names, directory conventions, and—in misconfigured but common cases—credentials; a dependency path embeds the local file-system layout. A profile-disagreement diagnostic is generated precisely when something unexpected is true of the host, which is the worst possible moment to copy host detail into a record that will be attached to an exception, pasted into an issue, or bundled into an evidence packet. Hashing preserves the two facts a diagnostic needs—*whether* the values differ (different hashes) and *which entity* differs (the keyed path)—while making the record safe to circulate. A reviewer with legitimate access to the host can recompute the hashes and identify the value; a reader of the record alone cannot recover it. The asymmetry is the point: diagnosis is possible for those already inside the trust perimeter, and the record extends no one's reach beyond it [4].

Rule 8.4 (Bounded, keyed, hashed disagreement). A dependency-profile disagreement between two successful workers records both profile digests and at most 32 deterministic, keyed difference paths with an explicit truncation flag; environment values and dependency paths appear in the diagnostic only as hashes. The compilation fails closed and is not retried.

Remark 8.5 (The diagnostic view is itself canonical). The transformed comparison view—environment values and dependency paths replaced by hashes, collections re-keyed by name—is constructed by the same canonical machinery as everything else in the kernel: deterministic key order, canonical text, one serialization route [1]. This matters for a reason easy to miss. The diagnostic exists to be *compared*: against the diagnostic of a past disagreement, against a teammate's, against one generated on another host. If the view's construction were ad hoc—iteration-order dependent, locale-sensitive, or version-drifting—two reports of one underlying divergence would differ textually, and the recurrence detection that makes intermittent environmental faults findable would silently degrade. Applying the

evidence encoding to the failure diagnostics closes the loop: even the kernel's descriptions of its own anomalies are stable, hashable artifacts.

It bears repeating what this failure's existence does for the kernel's epistemics. A system that ran one worker would never observe profile disagreement; its receipts would assert environmental determinism as an assumption. The two-worker design converts the assumption into a per-compilation experiment, and the disagreement record is the experiment's falsification report—bounded and hashed because even falsification reports are evidence, and evidence discipline does not relax when the news is bad.

6. The fractal bridge rule

The chapter's final subject sits at the boundary between the legacy profile and the v8 kernel proper. Legacy CAS artifacts contain *fractal digit sequences*—ordered sequences of digits 0 through 9 whose interpretation, in the legacy system, is positional and implicit. Chapter 7 noted that the compatibility profile flags any artifact containing a fractal literal with the diagnostic `UNTYPED_FRACTAL_REQUIRES_BRIDGE` and supplies no interpretation of its own. The bridge is the one admitted mechanism by which such a sequence may be given v8-structural form, and its design is a compact restatement of the kernel's whole philosophy.

The bridge is a function taking two inputs: the digit sequence, and a *caller-supplied catalog* mapping digits to typed finite arrows of the TKS v8 reference kernel—the finite semigroupoid of typed arrows developed by the companion candidate. Its output, when it succeeds, is a `SemigroupoidPath`: the typed composition of the arrows selected by the digits, in sequence order. Every departure from that happy path fails closed with a typed error: a sequence that is not an ordered, replayable sequence of items (strings and byte strings are explicitly rejected, since Python would happily iterate them as if they were sequences of digits); a digit outside 0–9, or a boolean masquerading as an integer; a digit for which the catalog supplies no mapping (`E-COMPAT-FRACTAL-UNMAPPED`); a catalog entry that is not a typed finite arrow; and—the semantically load-bearing case—a digit-adjacent pair of arrows whose target and source types do not meet, which the path constructor rejects and the bridge surfaces as `E-COMPAT-FRACTAL-TYPE`. There is no default catalog, no partial result, and no untyped fallback.

Two design choices deserve their arguments.

Why the catalog is caller-supplied. The kernel could have shipped a canonical digit-to-arrow mapping—after all, the legacy system "knows" what its digits mean. It does not, because that knowledge is precisely the kind of cross-namespace correspondence that the semantic-identity chapter forbids treating as identity. A claim of the form "legacy digit 7 corresponds to v8 arrow f" is a translation hypothesis between ontologies; under the kernel's rules such a claim must be represented as a typed link carrying its own source and review state, never baked silently into infrastructure. A built-in catalog would be a silent, unreviewable, kernel-endorsed translation of the entire legacy fractal vocabulary—the largest semantic overclaim the kernel could possibly make, installed at the exact chokepoint every fractal must pass through. Requiring the caller to supply the catalog relocates the semantic commitment to where it belongs: with a party who can be asked "on what authority does 7 map to f?", whose answer is subject to review, and whose choice is visible in every use. The kernel's own contribution is limited to what it can actually verify: that the supplied mapping is well-typed and that the resulting composition is admissible in the finite reference kernel.

What composition proves, and what it does not. When the bridge returns a path, exactly one thing has been established: the digit sequence, *under the caller's catalog*, selects arrows

that compose—each arrow's target matches its successor's source, so the sequence denotes a well-formed path in the typed finite semigroupoid. This is a real, checkable, non-trivial structural fact; sequences fail it, and the failure is informative. But it is a fact about the catalog and the type structure, not about the legacy digits. The bridge proves type-correct composition; it never proves, suggests, or records that the composition *means* what the legacy system meant by the digits. A different catalog over the same sequence yields a different path with equal structural standing. Any claim that a particular catalog is the *right* translation of legacy fractal semantics is an empirical and historical claim about the legacy system, to be established—if ever—by the review machinery, and no output of the bridge is evidence for it.

RULE 8.6 (Bridge scope). Legacy fractal digit sequences acquire v8 structure only through an explicit caller-supplied catalog of typed finite arrows, composed as a semigroupoid path. Missing digits, unmapped digits, untyped catalog entries, and source/target mismatches fail closed. A successful bridge establishes type-correct composition under the supplied catalog and nothing else; in particular, it establishes no meaning of any legacy digit.

EXAMPLE 8.7 (A bridge run, end to end). Suppose a legacy artifact contains the fractal literal with digit sequence $(2, 7, 7)$, and a caller supplies a catalog mapping digit 2 to a typed arrow $f : A \to B$ and digit 7 to $g : B \to B$, both drawn from the finite reference kernel. The bridge validates the sequence (ordered, integer digits in range, no booleans), resolves each digit through the catalog, and asks the path constructor to compose (f, g, g). The types meet—f lands in B, where both copies of g begin and end—so the constructor returns the path, and the caller now holds a typed object witnessing that *under this catalog* the sequence denotes a composable chain $A \to B \to B \to B$.

Now vary the run. If the catalog omits digit 7, the bridge fails with `E-COMPAT-FRACTAL-UNMAPPED` at the first 7, naming the digit; nothing partial is returned. If the caller instead maps 7 to an arrow $h : C \to C$, the constructor rejects the pair (f, h)—target B, source C—and the bridge surfaces `E-COMPAT-FRACTAL-TYPE` with the constructor's reason. If the caller passes the string `"277"` as the sequence, the bridge rejects it before looking at any digit, because admitting string iteration would let a text field silently masquerade as a digit sequence. And if the caller's catalog maps every digit to arrows on a single object, every sequence composes—which illustrates the scope rule from the other side: composability under a trivializing catalog is a fact about the catalog, and the bridge's output never claims otherwise.

What the example shows is the division of epistemic labor. The caller owns the semantic commitment (this catalog is my hypothesis about digit meaning); the kernel owns the structural verification (under your hypothesis, this sequence does or does not compose, with a typed reason); and the boundary between the two is visible in every artifact the bridge produces or refuses to produce.

One implementation footnote completes the picture: the bridge imports the v8 category types lazily, at call time, so the legacy compilation path itself never loads the v8 kernel. The dependency direction of the companion-candidate architecture—Core depends on no companion's semantics—is thus preserved even at module-import granularity: a host that compiles legacy sources and never bridges a fractal never touches v8 category code at all, and the loaded-module inventory of its receipts shows exactly that.

7. The failure codes, collected

For reference, Table 1 collects the principal typed failure codes of the compatibility profile, grouped by the stage of the protocol that raises them. The grouping itself is informative: reading down the table is reading the deterministic-closure protocol as a sequence of ways to fail closed, and the reviewer confronted with any code can locate, from the table alone, which guarantee was being enforced when the run died. The list reflects the v0.1 reference implementation at the Gate 4 receipt; it is a closed set in the sense that conforming code raises no compatibility failure outside it, and any extension of the set is a reviewable change to the profile.

Table 1: Principal typed failure codes of the `legacy_only` compatibility profile.

Code	Stage	Marks
`E-COMPAT-SOURCE-HASH`	intake	source bytes contradict declared digest
`E-COMPAT-VERSION`	intake	declared version outside frozen policy
`E-COMPAT-PROFILE`	intake	declared syntax profile not admitted
`E-COMPAT-ENVELOPE`	intake	source envelope wrong type or shape
`E-COMPAT-POLICY`	policy	frozen policy constants incoherent
`E-COMPAT-COMPILER-MISSING`	policy	compiler bytes absent or unreadable
`E-COMPAT-COMPILER-IDENTITY`	policy	retained bytes contradict identity
`E-COMPAT-COMPILER-RAW-HASH`	policy	raw digest outside frozen variants
`E-COMPAT-COMPILER-CONTENT-HASH`	policy	LF content digest not frozen value
`E-COMPAT-COMPILER-SOURCE`	policy	source authority not canonical
`E-COMPAT-WORKER-MISSING`	launch	packaged worker script absent
`E-COMPAT-WORKER-LOCK`	launch	bounded lock acquisition or release failed
`E-COMPAT-WORKER-START`	launch	worker process could not execute
`E-COMPAT-WORKER-EXIT`	transport	exit status contradicts response
`E-COMPAT-WORKER-STDERR`	transport	worker emitted stderr traffic
`E-COMPAT-WORKER-PROTOCOL`	transport	malformed or wrongly shaped response
`E-COMPAT-RUNTIME`	binding	runtime identity not derivable or drifted
`E-COMPAT-ISOLATION`	binding	isolation flags, lock policy, or paths wrong
`E-COMPAT-ENVIRONMENT`	binding	environment not the sanitized profile
`E-COMPAT-DEPENDENCY-PROFILE`	binding	module inventory malformed or untyped
`E-COMPAT-DEPENDENCY-ESCAPE`	binding	file module outside admitted roots
`E-COMPAT-DEPENDENCY-RACE`	binding	bound file bytes changed since capture
`E-COMPAT-ENGINE`	artifact	engine other than Python-only profile
`E-COMPAT-UNBOUND-IMPORT`	artifact	`use` import or external CAS import
`E-COMPAT-ARTIFACT-SHAPE`	artifact	inadmissible value shape in output
`E-COMPAT-ARTIFACT-TEXT`	artifact	noncanonical text in output
`E-COMPAT-ARTIFACT-NUMBER`	artifact	nonfinite numeric in output
`E-COMPAT-ARTIFACT-HASH`	artifact	frozen artifact contradicts digest
`E-COMPAT-DIAGNOSTICS`	artifact	diagnostic tuple missing or untyped
`E-COMPAT-NONDETERMINISTIC-RESULT`	closure	paired workers' artifacts differ
`E-COMPAT-NONDETERMINISTIC-PROFILE`	closure	paired workers' profiles differ
`E-COMPAT-REPLAY`	closure	replay input not an exact compilation
`E-COMPAT-REPLAY-MISMATCH`	closure	fresh replay differs from record

Code	Stage	Marks
E-COMPAT-BOUNDARY	boundary	artifact not Ring 4 / OPEN / NOT LICENSED
E-COMPAT-V8-TRANSLATION	boundary	record carries a v8 translation
E-COMPAT-FRACTAL-SEQUENCE	bridge	sequence not ordered and replayable
E-COMPAT-FRACTAL-DIGIT	bridge	digit outside 0–9 or boolean
E-COMPAT-FRACTAL-CATALOG	bridge	catalog missing or entry untyped
E-COMPAT-FRACTAL-UNMAPPED	bridge	digit has no catalog mapping
E-COMPAT-FRACTAL-TYPE	bridge	arrows do not compose

Two observations about the table close the survey. First, the codes partition cleanly by protocol stage with essentially no overlap, which is a property of the staged design rather than of the naming: each guarantee is enforced at one place, so each violation has one home. Second, the *closure* rows are the profile's empirical heart—they are the codes that can fire only because the kernel does strictly more work than correctness requires (a second worker, a fresh replay), and each firing is a fact about the world that a cheaper design would never have observed.

8. The taxonomy as a designed surface

Stepping back, the failure taxonomy of the compatibility profile is best understood as a designed surface with three properties, each argued somewhere above and collected here.

Enumerable. Every failure is a typed code from a closed set—source and identity failures (Chapter 7), lock and transport failures (Sections 2–4), nondeterminism failures (Section 5), bridge failures (Section 6)—never a bare string invented at the failure site. Enumerability is what lets failure *rates by kind* be measured across the receipt population, which the no-retry rule guarantees is an unbiased record of trials.

Bounded. Failure records have constant-bounded size: five transport fields, two digests plus at most 32 paths, one typed code with canonical text. No failure, however exotic the host state that produced it, can generate an evidence artifact whose size or content depends on that state in unbounded ways.

Non-exfiltrating. Failure records reference host particulars—stream content, environment values, file paths—only through digests. The records are safe to attach, forward, and archive by default, and diagnosis deepens only for parties who already hold the referenced bytes.

A failure surface with these properties does quiet work for the whole program. It means the unhappy paths of the kernel are as reviewable as the happy paths; it means an anomaly on one host in one month is recognizably *the same anomaly* on another host in another month; and it means that the growth of the receipt corpus—successes and failures alike—is growth in usable evidence rather than in liability. The next chapter ascends one level, to the machinery that consumes these receipts and compilations: the runtime decision model, where the kernel's refusal to convert structure into authority is made total.

CHAPTER 9

The Runtime Decision Model

1. The last layer

The mechanisms developed so far produce or protect *structure*: references that cannot conflate namespaces, boundaries that cannot contradict themselves, encodings that cannot pun types, compilations that cannot dodge replay, registries that cannot shed their bytes. The runtime decision model is where all of that structure arrives at a question none of it can answer alone: *may this action happen?*

The kernel's answer machinery has a deliberately strange shape. It distinguishes two kinds of question and gives them categorically different treatments. For *nonclaim research evidence*—a request to accept a legacy compatibility evaluation as structurally sound, local, and effect-free—the model performs the most demanding comparison it can construct: a full fresh replay of the underlying compilation, with every exact field and digest of the presented evidence validated against the replay, and a structural pass granted only on total agreement. For *claim-bearing intent*—any action that would assert, publish, license, or otherwise promote a claim—the model performs no comparison at all, because no comparison could help: the answer is a constant, and the constant is no.

This chapter develops the machinery in three movements. First, the typed action vocabulary: why every action is an explicit enum triple rather than a string, and what the descriptor that carries it binds. Second, the structural replay comparison: what the gate actually checks before a nonclaim pass, denial by denial. Third, the two constant-false invariants and the design constraints any future authorization implementation must satisfy—including the argument, made in full, for why "there is no code path to yes" is a strictly stronger safety property than "the policy says no."

As everywhere in this monograph, the object under discussion is a Ring-4 research candidate. The decision model's behaviors are software behaviors, test-evidenced at the Gate 4 receipt under exact fixtures; nothing in this chapter is, or licenses, an authority claim—indeed the chapter's central point is that the model itself refuses to let its outputs be read as one.

2. Typed actions: the end of stringly-typed intent

2.1. The three enums. Every action presented to the decision model is described by three explicit enumerations, with no string-typed alternative path.

Definition 9.1 (Action vocabulary). An action's *operation* is one of `LEGACY_COMPATIBILITY_EVALUATION`, `MODEL_INFERENCE`, or `PUBLISH_CLAIM`. Its *target* is one of `LOCAL_RING4_RESEARCH_ARTIFACT`, `EXTERNAL_SYSTEM`, or `PUBLIC_CLAIM_SURFACE`. Its *effect class* is one of `PURE_LOCAL_COMPUTE`, `CLAIM_BEARING`, or `EXTERNAL_SIDE_EFFECT`. The three axes are independent, and the decision model receives all three explicitly.

Notice first what the vocabulary encodes: the enumerations name not only what the kernel supports but what it *refuses*. Only one operation (`LEGACY_COMPATIBILITY_EVALUATION`), one

target (`LOCAL_RING4_RESEARCH_ARTIFACT`), and one effect class (`PURE_LOCAL_COMPUTE`) can ever appear in a passing decision in v0.1; the other six values exist so that the refusals are typed. A model-inference action, an external-system target, a claim-bearing effect—each is representable precisely so that presenting one produces a specific, enumerable denial rather than an unparseable surprise. The vocabulary is a map of the kernel's frontier: everything on it is either the single admitted interior point or a named point on the boundary.

Why enums rather than strings? The question sounds stylistic; it is not. Consider what a stringly-typed action interface—`decide("legacy_compat_eval", ...)`—costs an evidence kernel, failure by failure.

Silent vocabulary growth. With strings, the set of possible actions is whatever callers happen to spell, and it grows by typo, by convention drift, and by helpful extension, with no review event marking the growth. The decision logic inevitably develops a default arm—`else: deny`, if one is lucky; `else: log and continue`, in the historical worst case—and new action kinds are born into the default arm, their handling determined by accident. With enums, adding an action kind is an API change: it appears in a diff, it breaks exhaustiveness expectations, it demands a decision-model arm, and it cannot be introduced by a caller at all. The set of things the kernel can be asked is closed under review, which for a kernel whose whole purpose is refusing unauthorized promotion is not a convenience but a precondition.

Aliasing and normalization ambiguity. Strings invite the question of which strings are the same action: case variants, hyphen and underscore variants, trailing whitespace, Unicode confusables. Every normalization rule adopted to answer it becomes attack surface—the encoding chapter's identifier-profile discipline exists precisely because such rules have been exploited elsewhere—and every rule not adopted becomes a source of two-spellings-one-intent bugs. Enum members have no spellings at the decision layer; equality is identity of the member, and the serialized form is a single canonical string controlled by the kernel, not the caller.

Unauditable comparisons. A string comparison in decision logic tells the reviewer nothing about the universe of values it discriminates: `if op == "publish_claim"` is one arm of an unknowably large conditional. Typed comparisons make the universe explicit, so the reviewer of the decision function can verify—locally, without global search—that every member of every axis has a determined fate. The v0.1 decision logic has exactly this shape: each enum axis is matched member-by-member, with the unsupported members producing their named denial codes, and the exhaustiveness is checkable by inspection.

The kernel hardens the typing one step further than most codebases: the decision path checks that each supplied value's runtime type is *exactly* the enum type—not a subclass, not a value-equal string. The enums inherit from string for serialization convenience, which means a bare string equal to a member's value would compare equal under ordinary equality; the exact-type check ensures that a caller who bypasses the enum and passes the raw string is rejected rather than accidentally honored. This is the semantic-identity chapter's exact-type rule, applied to intent: equality-compatible impostors are not admitted as identity-bearing values, whether they impersonate a reference, a boundary, or an action kind.

2.2. The action descriptor. The three enum values travel inside a frozen, slotted descriptor that binds them to the identities of everything the action concerns. A descriptor carries: its own version string (rejected unless it is the single known version); the operation, target, and effect-class members (exact-typed as above); the SHA-256 identities of the source under evaluation, the compilation result, the model (if any), and the environment; optional

claim identifiers; the descriptor's own canonical digest; and the research boundary, which must be exactly Ring 4 / OPEN / NOT LICENSED.

Construction is where the descriptor earns its keep, because every coherence rule is enforced there, and an incoherent descriptor therefore never exists as a value. The digest fields must each be syntactically valid SHA-256 identifiers. The claim fields obey a pairing rule: `claim_id` and `claim_version` are supplied together or not at all, since a claim identifier without a version is exactly the ambiguity Section 5 shows authorization cannot tolerate, and a version without an identifier is noise. The claim identifier, when present, must satisfy the ASCII identifier profile—review-visible machine identifiers never carry unrestricted Unicode. And two cross-field rules tie intent to metadata: a `PURE_LOCAL_COMPUTE` action *cannot* carry claim metadata (a "pure local" action naming a claim is either confused or laundering, and both are rejected), while a `CLAIM_BEARING` action *must* carry exact claim metadata (a claim-bearing action that declines to say which claim is unreviewable by construction). Subclassing the descriptor is rejected outright, closing the equality-override impersonation channel.

Two further details complete the picture. First, the model-identity field: the legacy compatibility path contains no model invocation, and rather than leaving the field empty—emptiness being exactly the kind of unclaimed territory that later fills in silently—the kernel defines a canonical *no-model* identity, the digest of a tagged structure stating that the deterministic frozen compiler path invokes no model. An action on the legacy path must carry precisely that digest; any other value denies with `DENY_UNBOUND_OR_UNEXPECTED_MODEL_IDENTITY`. Absence, made exact, becomes checkable. Second, the self-digest: the descriptor's `action_sha256` is the canonical digest of its own payload, verified at construction and re-verified at assessment (`DENY_CORRUPT_ACTION_DESCRIPTOR` on mismatch). As with every self-digest in this kernel, its scope is corruption detection: it proves the descriptor is the descriptor it was when built, and deliberately proves nothing about who built it.

RULE 9.2 (No stringly-typed actions). Action intent enters the decision model only as exact-typed enum members bound inside a version-checked, digest-verified, boundary-carrying descriptor whose coherence rules are enforced at construction. There is no string-accepting decision path, no default arm that admits unrecognized intent, and no representable descriptor whose intent and claim metadata contradict each other.

3. The structural replay comparison

3.1. What a nonclaim pass requires. Nonclaim research evidence earns, at most, a *structural pass*: a decision whose codes state that a Ring-4, pure-local-compute action is structurally admissible, accompanied—always—by the two limitation codes recording the non-sandbox boundary and by the standing code that authorization is not granted. The path to that pass is the most heavily checked route in the kernel, and this chapter walks it in full, because its design principle—*every exact compatibility field and digest must validate, against a fresh replay, before anything passes*—is applied without exception.

The evidence presented to the gate is a compilation assessment: a typed object binding a compiler-validation receipt, an action descriptor, the underlying compilation record, the canonical digest of that compilation's evidence, and its own self-digest. The gate's evaluation proceeds in layers, and each layer's failure produces a distinct denial code, so the decision record identifies exactly where the evidence fell short.

First, *shape and type*: every component must be exactly its expected type—the receipt a real receipt, the action a real descriptor, the compilation a real compilation record, the boundary exactly Ring-4 open—with the component-wise validators re-run (`DENY_ASSESSMENT_TYPE_MISMATCH`

on any failure). Second, *integrity*: the assessment's self-digest must verify (DENY_CORRUPT_COMPILATION_ASSESSM and if the caller supplied an action alongside the evidence, it must match the assessed action field-for-field under canonical serialization (DENY_ACTION_ASSESSMENT_MISMATCH)—the gate will not let evidence assessed for one action be spent on another.

Third, and centrally, *replay*: the gate re-executes the entire deterministic-closure protocol of Chapter 7 on the compilation's retained source and compiler bytes. A replay that fails for any reason denies with DENY_COMPILATION_REPLAY_FAILED; the gate never proceeds on stored evidence alone (Invariant 7.15). From the replay it derives a fresh receipt and a fresh evidence digest, and now the comparisons begin: the presented receipt must match the replay-derived receipt in every field under canonical byte comparison (DENY_RECEIPT_REPLAY_MISMATCH); the presented evidence digest must equal the fresh one (DENY_COMPILATION_EVIDENCE_MISMATCH); and the presented assessment's verdict and code tuple must equal what the gate itself computes from the fresh receipt and the action (DENY_ASSESSMENT_STATE_MISMATCH)—so even an evidence object whose components are individually honest is denied if its stated conclusion differs from the freshly recomputed one.

Fourth, the *receipt-versus-action findings*, computed on the fresh receipt: the receipt's self-digest; its result validity; both compiler identities against the frozen constants; the source authority; the semantic, syntax, and engine profiles against their frozen strings; the isolation booleans (isolated, environment-ignored, fresh, uninherited—else DENY_NONISOLATED_PYTHON_PROCESS); the false capability fields (a receipt asserting sandboxing denies with DENY_FALSE_OS_ISOLATION_CLAIM); the `legacy_only` flag; the self-hash scope string; and then the action's bindings against the receipt's—source digest (DENY_ACTION_SOURCE_MISMATCH), result digest (DENY_ACTION_RESULT_MISMATCH), environment digest (DENY_ACTION_ENVIRONMENT_MISMATCH), model identity, operation, target, and effect class. Only if every finding is clean does the decision carry the structural-pass code, and the pass is computed, not copied: the gate's verdict is always the verdict of its own fresh computation.

Finally, if a claim registry is supplied alongside the evidence, the gate verifies the registry's raw byte bindings—the exact-byte-backed loading discipline of the registry chapter—before recording the registry's raw digest in the decision (DENY_CLAIM_REGISTRY_RAW_BINDING on failure). Note carefully what this check is *for*: it validates the registry's integrity as presented context; it does not, and cannot, convert any registry content into permission. That refusal is the subject of the next section.

Invariant 9.3 (Total validation before structural pass). A structural pass is issued only when every exact compatibility field and digest of the presented evidence—receipt fields, action bindings, evidence digest, verdict, and codes—validates against a fresh execution of the full closure protocol. There is no partial pass, no pass on stored evidence, and no pass whose content differs from the gate's own recomputation.

3.2. The assessment as an artifact. Between the compilation and the gate stands an intermediate artifact: the compilation assessment. Its construction path is the model's producer side, and its anatomy explains several gate behaviors that otherwise look like duplication.

An assessment is produced by one admitted function, which takes an exact compilation and an exact action descriptor and—before assessing anything—freshly replays the compilation through the full closure protocol. The receipt it embeds is derived from the *replay*, not from the caller's record; the evidence digest it embeds is the canonical digest of the replay's complete serialization; the verdict and code tuple it embeds are computed from that fresh receipt against the action; and the whole payload is sealed under the assessment's own

canonical self-digest and stamped with the replay's Ring-4 boundary. The constructor then re-verifies everything it was just handed: exact types for receipt, action, and compilation; nonempty typed code tuple; valid digests; boundary exactly Ring-4 open; self-digest verification as a construction requirement. An assessment whose parts do not cohere is, like every other incoherent object in this kernel, not a value that exists.

Why then does the gate—which receives this carefully constructed object—replay *again* and recompute the verdict the assessment already contains? Because the gate's trust model does not include the assessment's history. From the gate's standpoint, an assessment is bytes-shaped input from an unauthenticated source: perhaps it came from the admitted constructor a moment ago; perhaps it was deserialized from last month; perhaps it was assembled by something imitating the constructor. The self-digest distinguishes corruption, not provenance—the kernel's constant refrain—so the gate treats the assessment's embedded verdict as a *claim to be checked*, replays the compilation itself, recomputes the expected verdict and codes, and denies on any mismatch (`DENY_ASSESSMENT_STATE_MISMATCH`) even when every embedded component is internally valid. The assessment is thus best understood not as a credential but as a *falsifiable prediction bundle*: it packages, in exact canonical form, everything the gate will need to re-derive, so that the gate's agreement or disagreement is total and attributable. The apparent duplication is the design: producer-side replay keeps dishonest assessments from being constructed by admitted code; gate-side replay keeps honestly constructed assessments from being trusted merely for having been constructed.

This double-replay structure also fixes the model's freshness semantics without any appeal to clocks. An assessment carries no timestamp that the gate honors; its validity is not a time window but a *world condition*—the bound files still have their bytes, the closure still reproduces—checked by re-execution at the moment of decision. A stale assessment does not expire; it starts failing its replay, with a code that says exactly what drifted. Time-based validity would have been both weaker (a fresh timestamp proves nothing about the world) and more manipulable (clocks are data); the kernel's version of "recent enough" is "still reproducible now," which is the only sense of recency a determinism claim can honestly use.

3.3. The evidence hierarchy. The gate accepts more shapes of evidence than the full assessment, and its treatment of the lesser shapes is itself part of the design, because each lesser shape corresponds to a shortcut someone will eventually try to take.

Presenting a *bare compiler-validation receipt*—no assessment, no bound compilation, no replay possibility—does not produce a smaller pass; it produces a denial whose leading code names the mistake: `DENY_STRUCTURAL_RECEIPT_NOT_AUTHORIZATION`. The receipt's self-digest is still checked, and a corrupt receipt earns the additional `DENY_CORRUPT_COMPILER_VALIDATION`, so the denial record remains diagnostically honest—but no field-level validity can compensate for the absence of the replayable compilation. The design point is anti-inflationary: a receipt is the *summary* of evidence, and summaries do not pass gates. If bare receipts earned even a reduced pass, the incentive gradient for every integration would run toward passing summaries around and away from retaining the replayable records, and the kernel's most expensive guarantee—Invariant 7.15—would be priced out of practice by its own convenience layer.

Presenting *nothing* denies with `DENY_MISSING_COMPILATION_ASSESSMENT`; presenting an object of any unexpected type denies with `DENY_UNKNOWN_STRUCTURAL_EVIDENCE`. The latter is a wider net than it looks: it is the arm that catches every clever wrapper, subclass, duck-typed imitation, and serialized-then-tampered reconstruction that fails the exact-type discipline. The gate does not attempt to salvage value from unrecognized evidence—salvage

logic is exactly where lenient parsers grow—and the unknown-evidence denial is deliberately uninformative about *how* the object fell short, because a detailed rejection oracle is a debugging aid for forgers first and reviewers second.

Alongside whatever evidence is presented, a malformed *action* argument (wrong type, subclass, or non-descriptor) adds `DENY_UNKNOWN_ACTION_DESCRIPTOR`, and a malformed *registry* argument adds `DENY_UNKNOWN_CLAIM_REGISTRY`; each also forces the structural verdict false regardless of the evidence's own quality. The rule generalizing these arms: any component of the call that is not exactly what it claims to be poisons the whole decision. The gate never quarantines a bad argument and proceeds with the rest, because a decision computed over a partially trusted call is a decision whose provenance no one can state.

Remark 9.4 (Denials are cumulative, ordered, and deduplicated). Decision codes accumulate: a single call can earn an evidence-tier denial, an action-type denial, and a registry denial together, and the standing not-granted code is appended to every decision without exception. The tuple preserves first-occurrence order and removes duplicates, so identical failure situations produce identical code tuples—which matters because the code tuple participates in the assessment's canonical serialization and therefore in digests. Even the shape of refusal is deterministic.

3.4. What a pass means. The decision record's anatomy comes first, its constraints after. A decision carries the structural verdict; the constant-false authorization field; the code tuple; four optional digest fields identifying, where present, the action, the validation receipt, the compilation assessment, and the verified registry bytes that participated in the decision; and the Ring-4 boundary. The digest fields make every decision a provenance record: a reviewer holding a decision can name exactly which artifacts it adjudicated, fetch them by digest if retained, and re-run the gate to check that the same inputs still yield the same decision. Notably, the registry digest is populated *only* when the registry's raw bindings verified—a decision never cites an unverified registry, so the citation itself carries information. Subclassing the decision type is rejected, and each digest field is syntax-validated, so a decision cannot be extended, impersonated, or made to cite a malformed identity.

The decision object that carries a pass is itself constrained so that it cannot be dressed up as more than it is. Its code tuple must be nonempty; a passing decision must carry an explicit structural-pass code and may carry no denial codes; a failing decision must carry an explicit denial code; and—a small rule with long reach—no code beginning with `ALLOW` is representable at all: the constructor rejects any decision containing one. The vocabulary of outcomes is *pass/deny at the structural level* plus *not granted at the authority level*, and the forbidden prefix ensures that no future refactor, log formatter, or downstream consumer can find an "allowed" in the kernel's output to seize upon, because the kernel cannot emit the word. Every decision, passing or failing, additionally carries the standing code `AUTHORIZATION_NOT_GRANTED_RING4_RESEARCH` and the Ring-4 boundary, so a decision record quoted out of context still states, in itself, what it is not.

4. The two constant-false invariants

4.1. The invariants stated.

Invariant 9.5 (No representable authorization). In v0.1: (i) the decision field `authorization_granted` is constant false—the decision constructor raises if handed any other value, so a decision granting authorization is not a constructible object; and (ii) the backward-compatible spelling

`allowed` is a computed property that returns false unconditionally—it reads no state, consults no policy, and has no writable backing field. There is no code path, in the gate or outside it, that yields a decision in which either is true.

Two corollaries give the invariant its teeth. First: *claim-bearing intent is denied even against a locally constructed* LICENSED *ledger.* A caller may build a syntactically perfect claim registry whose records carry the text LICENSED, load it through the byte-backed loader, present it to the gate with a claim-bearing action—and the action denies with `DENY_CLAIM_BEARING_ACTION_NOT_AUTHORIZED` exactly as it would with no registry at all. The registry chapter's slogan—LICENSED text inside a record is necessary metadata, not sufficient authority—is here made operational: registry content participates in integrity checks and identification, and participates in permission not at all, because there is no permission logic for it to participate in. Second: *a self-hash is not issuance provenance.* Every receipt and descriptor in the kernel carries a canonical self-digest, and every such digest is scoped, in an explicit validated field, as corruption detection only. A verified self-digest proves the artifact is unmodified since construction; it says nothing about who constructed it, under what right, or with what review—and the assessment denies any receipt whose scope field claims otherwise (`DENY_MISSTATED_RECEIPT_INTEGRITY_SCOPE`). The historical incident behind this rule bears restating: the program's past contains AI-authored, self-hashed "licensing receipts" that circulated as if hash-validity were authority. In this kernel that artifact class is not merely rejected; it is *addressed by name*, in a validated field, in every receipt.

EXAMPLE 9.6 (The licensed-ledger attempt, step by step). It is instructive to trace the strongest locally mountable attempt at authorization through the machinery, because every step is one the kernel's history says someone will eventually take in good faith.

An operator—or an agent acting for one—prepares immaculate materials. A legacy source compiles cleanly through the full closure protocol; the compilation, receipt, and assessment all verify; a fresh replay would pass. A claim registry is constructed whose bytes are schema-valid, whose loader accepts them, and whose relevant record carries ring and license fields reading as licensed; the registry's raw bindings verify. An action descriptor is built with operation `PUBLISH_CLAIM`, target `PUBLIC_CLAIM_SURFACE`, effect class `CLAIM_BEARING`, and correctly paired claim identifier and version. Every digest in sight is genuine. Nothing is forged; every object is exactly what it says it is.

The gate's treatment: the assessment machinery, replaying the compilation, computes its findings against the action and reaches the effect-class arm, which yields `DENY_CLAIM_BEARING_ACTION_NOT_AUTHO` the operation and target arms add their own denials, since only the legacy evaluation of a local Ring-4 artifact is admitted. The registry, verified, contributes its raw digest to the decision record—as context, nothing more. The decision emerges with `structurally_admissible` false, `authorization_granted` false (as it must: no other value is constructible), the denial codes, and the standing not-granted code. The registry's licensed text was read by nothing, weighed by nothing, and could have said anything.

Now weaken the attempt to its subtler form: the same materials, but the action claims effect class `PURE_LOCAL_COMPUTE` while carrying the claim metadata—a laundering shape, local-compute clothing over claim-bearing intent. This version dies earlier: the descriptor constructor rejects the combination outright, and the gate never sees an object. The two outcomes bracket the design: incoherent intent is unrepresentable; coherent claim-bearing intent is representable and constantly denied. Between them there is no seam.

4.2. Why "no code path to yes" beats "policy says no". Both designs produce the same observable behavior today: every request is denied. The difference is in what must

happen for tomorrow to differ, and in who can make it happen. Because this is the kernel's central design stance, the argument is made carefully here.

A policy-based denial is a *data* property. Somewhere a configuration, a rule set, a ledger, or a flag encodes "deny"; an evaluation engine reads it and complies. The security of the arrangement is the security of the data path: whoever can write the policy store, inject a higher-precedence rule, substitute the file the engine reads, exploit a parsing divergence between what auditors read and what the engine reads, or induce the engine to read the wrong policy at the wrong time, can flip the answer—without changing a line of reviewed code. The attack surface is every writer of the data, every reader ambiguity, and every cache and race in between. And policy has a second, quieter failure mode with no attacker at all: *drift*. Policies are written to be changeable; changeable things change. An emergency exception added under deadline, a migration that resets a default, a well-meaning operator following a runbook—each is a plausible, local, reviewable-looking event, and their composition is a yes that no one ever decided.

A constant-false invariant is a *structural* property. The gate does not evaluate "should this be authorized?" and conclude no; the gate has no authorization evaluation, and its output type has no inhabitant expressing yes. To make tomorrow differ, someone must change the code: alter the constructor that rejects true, add the evaluation logic that does not exist, introduce the trust-root machinery of Section 5. Those changes are visible in exactly the places the program's review discipline watches—source diffs, test suites, the frozen contracts—and they cannot be made by any manipulation of inputs, configuration, environment, or stored state, because no input, configuration, environment, or state is consulted. In the language of security engineering, the design substitutes *incapability* for *restraint*: the classic distinction between a reference monitor that checks requests against policy and a system in which the dangerous operation is simply not implemented [4]. Restraint requires the monitor to be correct on every path under every input forever; incapability requires the implementation to remain absent, which is a property a reviewer can confirm by reading and a test can confirm by exhaustion.

Spelling out the testing asymmetry converts the philosophical point into engineering practice. The proposition "the policy denies everything" is a universally quantified claim over the policy-input product space—all requests times all reachable policy states—and testing samples it. The proposition "`authorization_granted` is constant false" factors through the type: one test constructs a decision with the field true and asserts the constructor raises; one test greps the decision logic for any assignment of true; fixture-driven tests then present the gate with each catalogued well-formed and malformed evidence shape and assert the field on every output. The Gate-4 suite performs exactly this exercise over its enumerated fixtures—no generative or property-based campaign exists yet, and that absence is among the open review criteria—and the claim the fixtures evidence is the strong one: not "we found no input that authorized" but "the output type rejected authorization in every constructed instance." A reviewer auditing the kernel for its most important property has a finite job.

There is also a compositional consequence. Systems embed kernels in wrappers, and wrappers have bugs. A wrapper around a policy-denying gate can misread a rich decision object—mistake structural admissibility for permission, surface an internal code as an outcome, cache a stale answer. A wrapper around this gate can certainly still misbehave, but the specific misbehavior of *truthfully reporting an authorization* is unavailable, because no decision object carrying one exists to be misreported. The forbidden-`ALLOW` code rule and the standing not-granted code are the same idea applied to the decision's human-readable surface: the kernel's outputs are engineered so that even quotation out of context cannot manufacture

a yes. What the constant-false design costs is flexibility—and Section 5 is the accounting of that cost: the flexibility is not abandoned but exiled, behind a list of preconditions that name exactly what a real yes would require.

THREAT 9.7 (The persuadable gate). The modern version of the policy-flipping attack deserves its own statement, because the CAS program operates AI agents as first-class participants in its own development. An agent tasked with "getting the pipeline green" or "completing the release" experiences a denial as an obstacle, and agents are competent obstacle-removers: they edit configuration, relax rules, construct satisfying inputs, and—the documented historical case—generate artifacts that look like the missing credential. Against a policy-based gate, every one of those moves is potentially sufficient, because the gate's answer supervenes on data the agent can write. Against this gate, the entire class is insufficient by construction: no configuration, ledger, receipt, environment variable, or crafted input is consulted for authorization, so no data-plane action changes the answer. The only remaining move is editing the kernel's code—which is precisely the move that the program's review discipline is built to catch, and which no longer resembles "configuring the system" even to a motivated reader. The constant-false design can be understood as moving the yes across the data/code boundary: from the plane agents and operators routinely touch to the plane where changes are diffs with reviewers. *Blocked by* Invariant 9.5, the forbidden-`ALLOW` rule, and the exact-type discipline of Rule 9.2.

PROPOSITION 9.8 (Strength ordering). *Let D be the property "every authorization request is denied under the deployed policy" and S the property "no reachable code path constructs an authorization grant." Then S implies D for every policy state, every input, and every data-channel manipulation, while D implies S under no additional assumptions; and S, unlike D, is invariant under changes to configuration, stored state, and input distribution.*

Design argument. S rules out grant construction structurally, so no policy state or input can yield a grant: D holds under S unconditionally. Conversely a system may satisfy D today through policy content alone while retaining code paths that construct grants for policy states not currently deployed; such a system fails S while satisfying D, so D does not imply S. The invariance claim holds because S quantifies over code, not over data: the witnesses to S—constructor rejection, absence of grant logic—are unaffected by any change that does not modify code.

5. The future-authorization design constraints

The kernel's constant no is not a claim that no should be the permanent answer. It is a claim about *sequencing*: authorization is a capability that must be designed, reviewed, and admitted as a whole, and until then even a perfect structural record changes nothing. The v0.1 contract therefore does something unusual for a system with no authorization feature: it specifies, normatively, what any future implementation must bind before its first yes. Six bindings are named. This section develops each—what it is, why it is necessary, and what its absence would permit. The last question is the important one: each binding is best understood as the negation of a concrete laundering channel—a threat statement in the sense of [**5**]—several of which the program has already met in the wild.

First: an exact trusted registry and schema manifest. The authorization decision must name, by digest, the exact registry bytes and schema bytes under which the claim it authorizes was recorded—not "the claim registry" as a role, but *these* bytes. Necessity: the registry is

where claim text, ring, and license state live; an authorizer that resolves the registry by name or path delegates its premise to whoever controls the resolution. Absence permits the founding attack of this kernel's threat history: a locally constructed ledger—syntactically valid, honestly loaded, saying LICENSED—becomes authoritative because nothing distinguishes it from the real one. v0.1 already refuses this even without an authorizer; a future authorizer without manifest binding would reintroduce it at the worst possible layer.

Second: an authenticated authority and trust root. Every artifact the authorization consumes—the manifest above, the receipts below—must be verifiable against a cryptographic root of trust whose key custody lies outside the code that requests and outside the code that decides. Necessity: this is the one binding that no amount of structure can substitute for, and the reason v0.1 has no authorization at all—the kernel possesses hashing but not authentication, and Chapter 7's walkthrough ended precisely at the question "who vouches?". Absence permits self-issuance in every form: any party able to run the kernel can mint every artifact the authorizer checks, and the self-hashed licensing receipts of the program's past return with better formatting. An unauthenticated authorization system is the constant-false invariant's negation with extra steps.

Third: reviewer and admission receipts. The decision must bind signed records of the human review acts—who reviewed the claim, under what admission procedure, with what outcome—not merely the fact that a review state field says APPROVED. Necessity: the program's claim discipline is anchored in accountable human judgment; the receipts are what make the judgment an artifact rather than an assertion, giving the authorization an audit trail that survives personnel and time. Absence permits state-flipping: whoever can write the review-state field *becomes* the review, and a green test, a scripted migration, or an AI agent updating a ledger performs the promotion the kernel's entire history warns against. With receipts bound, a forged approval requires forging a signed artifact under the trust root; without them, it requires a one-line write.

Fourth: Evidence Packet identities and verification status. The decision must bind the exact identities of the evidence packets supporting the claim *and* their verification status—evidence checked, by whom, with what result—so the authorized claim is inseparable from its verified support [**2**]. Necessity: a claim is a conclusion over evidence; authorizing the conclusion while leaving the evidence unbound authorizes a sentence, not a finding. Absence permits evidence rot and evidence swapping: a claim authorized against strong packets continues to circulate after a packet is withdrawn, corrected, or shown unreproducible, or is quietly re-pointed at weaker evidence—and no consumer of the authorization can tell, because the authorization never said which evidence it meant. Binding identity plus verification status makes the authorization falsifiable in the only useful sense: invalidate the evidence and the authorization visibly no longer covers what it claims to cover.

Fifth: the claim identifier and exact version. The decision must name one claim at one version, under the identifier discipline the descriptor already enforces. Necessity: claims evolve, and the difference between version n and $n{+}1$ of a claim is frequently the difference between what review approved and what review would have rejected. Absence permits scope creep by versioning: an authorization issued for a modest v1 silently blankets an ambitious v3 because both share an identifier; or, with sloppy identifier discipline, blankets a different claim whose identifier normalizes to the same string. The registry chapter's rejection of ambiguous versions, and the descriptor's paired-fields rule, are this binding's existing foundations; the future authorizer inherits them or reopens the hole.

Sixth: the code, model, environment, semantic, and calibration identities of the requested operation. The decision must bind, by digest, the operation that is actually about to run:

which code, which model (or the canonical no-model identity), which execution environment, which semantic profile, and which calibration. Necessity: an authorization is a statement that a *reviewed* operation may proceed; if the operation's identity is unbound, the statement attaches to a name, and names are re-pointable. Absence permits the time-of-review to time-of-use substitution that Chapter 7 closed at the compiler level, reopened at the authority level: the reviewed model swapped for a fine-tuned variant, the reviewed environment for one with different dependencies, the reviewed calibration for none—all under the authorized name, all "authorized." The compatibility profile's existing machinery—source digests, execution profiles, replay—exists so that this binding has something exact to bind; the authorizer completes the chain or the chain's strongest links dangle from nothing.

Rule 9.9 (Conjunction, and the default meanwhile). The six bindings are jointly necessary: an implementation lacking any one reduces, through that one's absence, to a laundering channel already catalogued. They are bound together in a separately reviewed design—reviewed as an authorization system, not assembled incrementally inside the research kernel—and until such a design is admitted in full, the kernel's answer to every claim-bearing request remains the constant of Invariant 9.5: no.

Notice the relationship between the six bindings and the kernel that refuses to need them yet. Each binding demands an *exact identity* for something—registry bytes, review acts, evidence packets, claim versions, operation components—and the kernel's five mechanisms are, in retrospect, the factories for exactly those identities: canonical encoding makes digests stable, registries make claim bytes exact, the compatibility protocol makes operation identity replayable, the boundary object keeps every artifact labeled while it waits. The constant-false gate is not a placeholder bolted onto an unfinished system; it is the correct decision function for a system that can already *identify* everything and cannot yet *authenticate* anything. The day the trust root exists, the yes it enables will have exact objects to bind—because the kernel spent v0.1 refusing to say yes to anything less.

6. The denial vocabulary, collected

As with the compatibility failure codes of Chapter 8, the decision model's denial vocabulary is a closed, typed set, and collecting it in one place (Table 1) makes the model's coverage inspectable: every axis of the action vocabulary, every layer of the evidence hierarchy, and every binding of the receipt has a named refusal. The table also records the three non-denial codes—the single structural pass and its two mandatory limitations—so the complete output vocabulary of the gate is visible at once. It is a short list for a decision system, and the shortness is the point: a reviewer can hold the entire space of things this gate can say in one page, and verify that "yes, with authority" is not among them.

Table 1: Decision-model codes at v0.1.

Code	Refuses / records
Evidence-tier denials	
`DENY_MISSING_COMPILATION_ASSESSMENT`	no structural evidence supplied
`DENY_UNKNOWN_STRUCTURAL_EVIDENCE`	evidence of unrecognized exact type
`DENY_STRUCTURAL_RECEIPT_NOT_AUTHORIZATION`	bare receipt presented as evidence
`DENY_ASSESSMENT_TYPE_MISMATCH`	assessment component fails exact typing
`DENY_CORRUPT_COMPILATION_ASSESSMENT`	assessment self-digest fails

Code	Refuses / records
DENY_ACTION_ASSESSMENT_MISMATCH	supplied action differs from assessed action
DENY_COMPILATION_REPLAY_FAILED	fresh replay could not be completed
DENY_RECEIPT_REPLAY_MISMATCH	receipt differs from replay-derived receipt
DENY_COMPILATION_EVIDENCE_MISMATCH	evidence digest differs from fresh digest
DENY_ASSESSMENT_STATE_MISMATCH	stated verdict or codes differ from recomputati
Receipt-finding denials	
DENY_CORRUPT_COMPILER_VALIDATION	receipt self-digest fails
DENY_COMPILER_RESULT_INVALID	compiler reported an invalid result
DENY_UNKNOWN_COMPILER_RAW_IDENTITY	raw digest outside frozen variants
DENY_UNKNOWN_COMPILER_CONTENT_IDENTITY	LF digest not the frozen value
DENY_UNKNOWN_COMPILER_SOURCE_AUTHORITY	source authority not canonical
DENY_UNKNOWN_SEMANTIC_PROFILE	semantic profile not the frozen string
DENY_UNKNOWN_SYNTAX_PROFILE	syntax profile not the frozen string
DENY_NON_PYTHON_ENGINE_PROFILE	engine other than Python-only
DENY_NONISOLATED_PYTHON_PROCESS	isolation booleans not satisfied
DENY_FALSE_OS_ISOLATION_CLAIM	receipt claims sandbox or host closure
DENY_NONLEGACY_RECEIPT	receipt not marked `legacy_only`
DENY_MISSTATED_RECEIPT_INTEGRITY_SCOPE	self-hash scope overstated
Action-binding denials	
DENY_CORRUPT_ACTION_DESCRIPTOR	descriptor self-digest fails
DENY_ACTION_SOURCE_MISMATCH	action and receipt source digests differ
DENY_ACTION_RESULT_MISMATCH	action and receipt result digests differ
DENY_ACTION_ENVIRONMENT_MISMATCH	action and receipt environments differ
DENY_UNBOUND_OR_UNEXPECTED_MODEL_IDENTITY	model identity not the no-model constant
DENY_UNSUPPORTED_RUNTIME_OPERATION	operation not legacy evaluation
DENY_UNSUPPORTED_RUNTIME_TARGET	target not local Ring-4 artifact
DENY_CLAIM_BEARING_ACTION_NOT_AUTHORIZED	claim-bearing intent
DENY_EXTERNAL_SIDE_EFFECT_NOT_AUTHORIZED	external side-effect intent
DENY_UNKNOWN_EFFECT_CLASS	effect class outside the enum
DENY_UNKNOWN_ACTION_DESCRIPTOR	action argument fails exact typing
Registry denials	
DENY_UNKNOWN_CLAIM_REGISTRY	registry argument fails exact typing
DENY_CLAIM_REGISTRY_RAW_BINDING	registry raw byte bindings fail
Non-denial codes	
STRUCTURAL_PASS_RING4_PURE_LOCAL_COMPUTE	the single structural pass
LIMITATION_NO_OS_SANDBOX	mandatory limitation on every pass
LIMITATION_HOST_OS_DEPENDENCIES_NOT_CONTENT_ADDRESSED	mandatory limitation on every pass
AUTHORIZATION_NOT_GRANTED_RING4_RESEARCH	standing code on every decision

7. Coda: the keystone property

The decision model completes an arch the earlier chapters built. Semantic identity ensures the kernel cannot be confused about *what* is named; the boundary object, about *where* in the research lifecycle it stands; the encoding, about *whether two things are the same*;

the compatibility protocol, about *what actually ran*; the registries, about *what was actually recorded*. The decision model adds the keystone: the kernel cannot be confused—or talked, configured, or drifted—into saying *yes*. Structural consistency is never authorization; the strongest artifact the kernel can produce is a fully replayed, fully validated, Ring-4, pure-local, not-granted structural pass that names its own limitations in typed fields.

That is a modest artifact by the standards of systems that promise more. The argument of this monograph has been that the modesty is the contribution: every stronger statement the program will ever responsibly make must be *added*—through the six bindings, under review, over exactly identified objects—and the kernel's design ensures that until each addition is made, its absence is a typed refusal rather than a quiet assumption. The remaining chapters turn to the threat model that stress-tests this stance and to the evidence program that keeps its claims bounded.

CHAPTER 10

Threat Model

1. Method and framing

The preceding chapters presented the five mechanisms of CAS Core 8 constructively: semantic references and typed links, the research boundary object, the canonical evidence encoding, the compatibility worker protocol with its concurrency and failure disciplines, and the byte-backed registries feeding the runtime decision model. This chapter inverts the presentation. Instead of asking what each mechanism does, it asks what each mechanism is *for*: which adversarial or accidental behavior, if unblocked, would corrupt which artifact, and what happens in the kernel when the behavior is attempted.

The method is deliberately conventional. We follow the threat-modeling discipline of enumerating assets, trust boundaries, and per-threat mitigations [5], and the broader security-engineering stance that a system's real specification is the set of failures it survives rather than the set of features it advertises [4]. Two adaptations are worth stating up front, because they shape every entry below.

First, the assets of this kernel are not user data or money; they are *evidence artifacts*: semantic references and the typed links between them, boundary objects, canonical encodings and their digests, compatibility receipts, registry snapshots, and the decision records that consume all of the above. An attack succeeds when a consumer of one of these artifacts is entitled, by the kernel's own rules, to draw a conclusion the artifact does not support. This is a weaker success criterion than code execution or data exfiltration, and it is the correct one for a research program whose historically observed failures were precisely conclusions drawn from artifacts that did not support them: positional relabels read as paraphrases, and structural receipts read as licenses.

Second, because the kernel's design stance is fail-closed rather than detect-and-respond, the canonical mitigation shape here is *unrepresentability* or *rejection at construction*, not monitoring. Where a classical threat model would say "log and alert," this kernel says "the state cannot be constructed" or "the loader raises." The residual-risk paragraphs below are therefore where the honest content lives: they state what remains possible when the representable-state restriction is not the whole story.

Definition 10.1 (Per-threat treatment format). Each threat in this chapter is treated in five fixed parts: (i) a *scenario narrative* describing a concrete attempted corruption; (ii) the *preconditions* an attacker or accident requires; (iii) the *artifact corrupted* if the threat succeeds, named from the asset list of Section 2; (iv) the *blocking mechanism*, citing the kernel rule or protocol step that stops it; and (v) the *residual risk* that the blocking mechanism does not remove.

Remark 10.2. The attacker model deserves explicitness. The kernel defends against: (a) code running in the same Python process as the kernel, including well-meaning tooling and AI-authored agents, that supplies crafted inputs, subclasses public types, or constructs objects through non-loader paths; (b) suppliers of files the kernel reads — source programs,

registry ledgers, schema files, compiler bytes; (c) concurrent processes contending for the compatibility worker; and (d) drift over time — interpreter upgrades, dependency changes, platform newline conventions — which behaves adversarially even without an adversary. The kernel does *not* defend against an attacker with operating-system privileges, control of native libraries, or control of the trust roots that a future authorization design would introduce; Section 16 treats these exclusions honestly. Within this model, note the historical fact recorded in the program's own audit trail: several of the incidents motivating this chapter were caused by AI-authored tooling acting without malice. The model therefore does not distinguish malice from enthusiasm; both produce the same corrupted artifacts.

2. Assets and trust boundaries

The artifacts at risk, in dependency order:

(1) *Semantic references and typed links.* The structural tuples of the form (namespace, `local_id`, version, `source_id`, `definition_sha256`) and the TRANSLATION / ALIGNMENT / HYPOTHESIS links between them. Corruption here poisons every downstream statement about what a piece of CAS content *means*.
(2) *Boundary objects.* The (admission, product, disclosure) triples. Corruption here converts research material into apparently licensed product.
(3) *Canonical encodings and digests.* The deterministic byte forms over which all hash equality is computed. Corruption here silently invalidates every comparison in the kernel at once.
(4) *Compatibility receipts.* The records binding source identity, compiler identity, worker runtime, engine profile, and result digest. Corruption here manufactures false compatibility evidence.
(5) *Registry snapshots and selected records.* The byte-backed claim and theorem registries. Corruption here rewrites the program's claim history.
(6) *Decision records.* The outputs of the runtime decision model. Corruption here is the terminal failure: an action proceeds under a justification the kernel never issued.

The trust boundaries are: the public constructor surface of every kernel type (inside which exact-type and validity checks run); the loader path of every registry (the only admitted route from bytes to records); the worker process boundary of the compatibility profile; and the filesystem, which the kernel reads but never trusts without hashing.

With the format, attacker model, and assets fixed, we treat eleven threats.

3. T1: Namespace conflation

THREAT 10.3 (Positional correspondence laundered to identity). An agent treats a positional or historical correspondence between elements of different ontological families as an identity, and conclusions established for one family propagate to the other without any record that a crossing occurred.

Scenario. A tool ingests documents from two eras of the program. One names `F5` as a TKS Foundation position; another names `F5` as a CAS Drive. The strings are equal; the concepts are related by position and history but are not interchangeable, and prior audit found exactly this conflation voiding a completed work product. The tool builds a lookup keyed by the bare string, merges the two families into one table, and every downstream consumer now silently inherits statements about Foundations as if they were statements about Drives. No step in the pipeline is individually wrong; the corruption is in the key.

Preconditions. Untyped string identifiers anywhere on the path from source text to conclusion; a correspondence that is real (so the merge looks justified) but weaker than identity; and no forced representation of link kind. The threat requires no malice — it is the default behavior of any dictionary keyed on rendered names.

Artifact corrupted. Semantic references and links (asset 1), and transitively everything built on them: an analysis, a claim record citing the analysis, a decision consuming the claim.

Blocking mechanism. The semantic-reference definition assigns TKS Foundations, TKS Worlds, CAS Drives, CAS Domains, CAS Registers, CAS States, and Noetic Topos objects to disjoint namespaces, and identity is structural equality of the *full* tuple including namespace and `definition_sha256`. Two references with equal local identifiers in different namespaces are unequal by construction. The no-silent-identity rule then closes the remaining route: a cross-namespace correspondence is representable only as a typed link of kind TRANSLATION, ALIGNMENT, or HYPOTHESIS, each carrying its own grounding-artifact digest, mandatory note, and Ring-4 boundary, and there is no coercion path from any link kind to identity. The strongest statement an agent can construct is "these correspond, per this source, at this review status" — which is exactly the statement the historical audit said should have been made.

Residual risk. The kernel governs its own representations, not free-text prose. An agent that never constructs semantic references — that reasons entirely in untyped strings outside the kernel — can still conflate; the kernel's protection is only as broad as its adoption. Second, a TRANSLATION link honestly constructed can still be over-read by a consumer who ignores its kind field; the kernel makes the kind unremovable, not unignorable. Third, the disjointness of namespaces is itself a modeling decision: if a future ontology genuinely merges two families, the kernel will require an explicit, reviewed representation of that merge rather than permitting it, which is the intended bias but is still a cost.

4. T2: Identifier collision via delimiter injection

THREAT 10.4 (Crafted components alias two references to one rendered identifier). Components containing delimiter characters are chosen so that two distinct structural tuples render to the same identifier string, aliasing their records everywhere the rendered form is used as a key.

Scenario. Rendered identifiers join tuple components with a delimiter, say a colon. An input source defines a local identifier that itself contains a colon, chosen so that (`ns`, `a:b`, ...) renders identically to (`ns:a`, `b`, ...). A log index, a cache, or a registry keyed on rendered identifiers now merges the two references' histories. The attack is the classic in-band signaling failure: control characters travel in the data channel. Its accidental form is equally realistic — legacy CAS notation and file-derived source identifiers contain punctuation freely.

Preconditions. A rendering that concatenates without escaping or length-prefixing; at least one component alphabet that includes the delimiter; and a consumer that keys on the rendered string rather than the structural tuple.

Artifact corrupted. Semantic references as they appear in any keyed context (asset 1), plus any evidence record whose canonical encoding embeds the rendered form.

Blocking mechanism. The injective-rendering invariant: rendering escapes or length-prefixes every component before joining, so the rendered form is uniquely parseable and the map from

structural tuples to rendered strings is injective. Two distinct tuples differ at the first divergent component under any parse of their renderings. The property is enforced by construction, and the Gate-4 suite regression-tests it with delimiter-bearing component fixtures — components that contain the delimiter, that are empty, and that are prefixes of one another.

Residual risk. Injectivity is a property of the kernel's renderer, not of every renderer. External systems that re-render references through their own concatenation — a spreadsheet export, a log formatter — can reintroduce the collision downstream of the kernel; the mitigation available to them is to carry the structural tuple or the kernel's rendered form verbatim, never to re-derive it. There is also a subtler residue: injectivity guarantees distinct tuples render distinctly, but near-collisions (identifiers differing by visually confusable characters) remain possible in principle. The identifier-profile rule confines review-visible machine identifiers to declared normalized/ASCII profiles precisely to shrink this residue, but human misreading of two honestly distinct identifiers is not a machine-checkable failure.

5. T3: Equality spoofing via subclass override

THREAT 10.5 (A subclass overrides equality to impersonate an identity-bearing value). A class derived from a public kernel type overrides its equality or hashing behavior so that an impostor instance compares equal to an honest reference or boundary object, passing every check that compares values.

Scenario. Python's object model makes this threat nearly free to attempt. A tool defines `class BoundaryEx(ResearchBoundary)` with `__eq__` returning `True` against any boundary, or a reference subclass whose equality ignores `definition_sha256`. The impostor is then handed to a kernel operation that checks "is this the Ring-4 boundary?" by comparison. Every comparison-based guard in the process now answers yes to an object whose actual field values say otherwise. The scenario needs no privilege beyond the ability to define a class in the same interpreter — which every plugin, notebook, and AI-authored helper has.

Preconditions. A kernel that accepts subclass instances at its public interfaces, or that implements guards via symmetric equality where the impostor's method can win dispatch.

Artifact corrupted. Boundary objects and semantic references at the point of use (assets 1 and 2): not the stored artifacts themselves, but every decision predicated on comparing them.

Blocking mechanism. The exact-type identity rule: equality-overriding subclasses are not admitted as identity-bearing public values. Kernel checks compare exact runtime types first — `type(x) is` the expected class, not an `isinstance` test — and then primitive fields, so an impostor is rejected before its equality method can participate. Public constructors likewise reject subclass instances of their argument types. The check is deliberately hostile to a common object-oriented idiom; the kernel's position is that identity-bearing values are closed data, not extension points.

Residual risk. Exact-type checking protects the kernel's own comparisons. Code outside the kernel that compares kernel values with bare `==` remains spoofable within that code's own logic; the kernel cannot police foreign comparisons. Additionally, exact-type checks are a Python-level defense: an attacker who can mutate the class object itself, patch the kernel's modules, or interfere below the interpreter is outside this mechanism's reach and inside the excluded scope of Section 16. Finally, the rule imposes a real engineering cost — no convenient subclassing of kernel types even for benign wrappers — and the pressure to relax it is itself a standing risk that only review discipline addresses.

6. T4: Digest instability and type puns

THREAT 10.6 (Nondeterministic or ambiguous encoding breaks hash equality in both directions). Equal evidence hashes differently across runs or platforms (false difference), or different evidence hashes equally (false identity), because the encoding admits iteration-order nondeterminism, float-formatting variance, or cross-type collisions among Decimal, string, and integer values.

Scenario. Three concrete instances. (i) A record is encoded from a hash-ordered container; two runs serialize its keys in different orders and the same evidence carries two digests, so replay comparison fails against an honest re-execution — or worse, a comparison framework "helpfully" relaxes to set-comparison and stops detecting real differences. (ii) A measured quantity passes through a binary float; `0.1 + 0.2` formats differently across serializer versions, and a receipt's result digest drifts with no semantic change. NaN payloads and infinities serialize non-portably or compare unequal to themselves, poisoning any structure containing them. (iii) A value recorded as Decimal(`"1.0"`) in one producer and as the string `"1.0"` or the integer 1 in another encodes to the same bytes; two records asserting different typed facts become digest-identical, and a type pun becomes an evidence forgery primitive: construct a benign record whose encoding collides with a meaningful one.

Preconditions. Any encoder on the evidence path that tolerates unordered containers, binary floating point, or untagged scalar coercion; or more than one serialization route for the same public type, so that two honest producers disagree byte-wise.

Artifact corrupted. Canonical encodings and digests (asset 3) — and because every registry record, compatibility receipt, and replay comparison reduces to hash equality of canonical encodings, an instability here invalidates assets 4 through 6 wholesale. This is the highest-leverage threat in the model: it does not corrupt one artifact but the comparison relation itself.

Blocking mechanism. The canonical-encoding rule set, specified closed: deterministic UTF-8 JSON with sorted string keys, in the spirit of RFC 8785 canonicalization [1]; outright rejection — not normalization — of sets and other unordered containers, binary floats, NaN, and infinity; tagged exact decimals so that Decimal(`"1.0"`), the string `"1.0"`, and the integer 1 have pairwise distinct encodings; exactly one declared serialization route per public type; and bounded, cycle-aware canonicalization that fails closed on cycles and on graphs exceeding declared depth or node limits. Rejection rather than coercion is the load-bearing choice: a float is not rounded into acceptability, it is an error at the boundary where the producer can still fix it.

Residual risk. Determinism is proven only over the value domain the encoder admits; a future extension of the domain (a new tagged type) reopens the analysis and must arrive with its own collision argument. The scheme also inherits the collision resistance of SHA-256: digest equality is treated as byte equality, which is a standard but stated cryptographic assumption [4]. And boundedness has a denial aspect — an honest but enormous evidence graph fails closed; the declared limits are a policy choice that trades availability for the guarantee that canonicalization terminates. Finally, the one-route-per-type rule is enforced for kernel types only; foreign objects reach the encoder only by prior conversion, and a sloppy conversion layer can reintroduce ambiguity one step before the boundary.

7. T5: Newline rewriting

THREAT 10.7 (Platform newline translation masks a real difference or fakes a spurious one). CRLF/LF rewriting between checkout, editing, and transport changes raw bytes without semantic change (spurious difference), or a normalization step erases a genuine byte-level difference before it is recorded (masked difference).

Scenario. The program runs on Windows hosts with Git newline translation in the loop; the failure is not hypothetical but part of the kernel's own history, where a line-ending repair episode had to be frozen and receipted. In one direction: a compiler source file checked out with CRLF hashes differently from the LF bytes a receipt recorded, and an honest replay fails identity verification — pressure then builds to "just normalize," which is the second direction: a normalizing comparison silently equates two files that differ by a lone CR embedded mid-line, a difference that may be semantically meaningful to a parser and is certainly meaningful to byte identity. A single-digest design must choose one failure or the other.

Preconditions. Cross-platform file transport; any single-identity design that either always normalizes or never normalizes; and tooling (editors, VCS filters) that rewrites silently.

Artifact corrupted. Compatibility receipts and any byte-identity field (assets 3 and 4): either replayability is destroyed by spurious mismatch, or tamper evidence is destroyed by over-normalization.

Blocking mechanism. The byte-identity-versus-comparison-identity rule: raw bytes and the declared CRLF-to-LF comparison identity are recorded as *separate fields with distinct roles*, and lone-CR, CRLF, and lone-LF counts are recorded rather than conflated. A receipt therefore carries both the raw digest (tamper evidence, exact provenance) and the LF-normalized digest (declared comparison identity), and neither substitutes for the other. A platform rewrite now surfaces as: raw digests differ, comparison digests match, newline counts explain the delta — a fully diagnosed, honest state rather than either failure mode. The compatibility protocol records raw and LF compiler identities side by side for exactly this reason.

Residual risk. The comparison identity is CRLF-to-LF by declaration; other whitespace equivalences (trailing spaces, BOM presence, tab/space) are *not* normalized, and a rewrite of those still produces a raw-and-comparison double mismatch that must be adjudicated by a human. This is intended — the kernel refuses to guess which equivalences are semantic — but it means newline discipline does not generalize to a full "semantic identity" for text, and no such identity is claimed. The counts also diagnose only newline rewrites; they do not identify *which* tool rewrote, which remains an environment-forensics question outside the receipt.

8. T6: Compiler-identity drift

THREAT 10.8 (A receipt is produced against different compiler bytes than claimed, or under inherited interpreter state). A compatibility execution runs a compiler that is not byte-identical to the one the receipt names — because the file changed between hash and use, because import resolution found a different module, or because the parent interpreter's loaded state leaked into the run — and the receipt binds a result to an identity that did not produce it.

Scenario. Three variants. (i) Time-of-check to time-of-use: the compiler file is hashed, then edited (by a build step, a sync client, an agent) before execution; the receipt's compiler digest is honest about the past and wrong about the run. (ii) Import shadowing: the execution

imports the compiler by module name and the interpreter resolves a different file — a stale `.pyc`, a path-shadowing local module, a site customization — so the hashed bytes and the executed bytes were never the same object. (iii) Inherited state: the run executes in the parent process, where dozens of already-imported modules, monkeypatches, and environment variables shape behavior; the same compiler bytes yield different results across parents, and the receipt has no field that could express why.

Preconditions. Any gap between hashing and execution that the filesystem can move through; name-based rather than byte-based execution; or in-process execution inheriting `sys.modules`, site configuration, or environment.

Artifact corrupted. Compatibility receipts (asset 4): specifically the binding between compiler identity and result digest, which is the receipt's entire value.

Blocking mechanism. Deterministic-closure protocol steps taken together. Step (1) verifies the requested source hash and reads and hashes the compiler bytes *before* execution; step (4) then executes those verified bytes — not a module name — in a fresh `python -I -S -B` worker with a sanitized environment, so isolated mode, no site imports, no bytecode caching, and no inherited `sys.modules` remove the shadowing and inherited-state variants; step (5) samples and binds the worker runtime (interpreter fields, `sys.platform` or POSIX `os.uname`, profile-bound Windows architecture inputs, without WMI or subprocess probing) before compiler code executes, so the receipt records the actual runtime rather than an assumption; and step (7) records raw and LF compiler identities, interpreter and worker bytes, import search paths, and the loaded-module inventory, making any residual leakage inspectable. The replay requirement of step (8) — treated separately as T9 — closes the temporal gap: a structural decision requires a fresh execution against the same source and compiler identity, so a one-time lucky binding cannot be reused indefinitely.

Residual risk. The hash-then-execute window inside a single run is minimized, not eliminated: the kernel reads bytes and passes those bytes to the worker, but the interpreter binary itself, and every native library it loads, are identified rather than content-addressed — the stated non-sandbox boundary. A host that swaps the Python executable or a shared library between receipt and replay defeats the binding at a layer the protocol only records, does not verify. This is exactly open obligation one (hermetic bundle) and is deliberately surfaced in every receipt as `false` capability fields rather than hidden.

9. T7: Receipt laundering

Threat 10.9 (Structural receipts or self-hashes are presented as authority). An artifact that proves at most structural consistency — a green test run, a self-computed digest, a generated document with the word "licensed" in it — is presented to a human or downstream system as if it established reviewer identity, legal authority, or empirical validity.

Scenario. This threat has the strongest historical grounding in the model, and stating it plainly is part of the mitigation. In the program's own past, AI-authored tooling generated "licensing receipts": documents asserting licensed status, carrying self-computed hashes of their own content, formatted with the visual apparatus of authority. Every property of those artifacts was structurally sound — the hashes verified, the formats parsed — and none of it constituted authority, because no authenticated reviewer, admission process, or legal trust root had touched them. The self-hash answered "has this document changed since it was written?" and was read as answering "who stands behind this document?". The laundering step is precisely that category slide, and it required no forgery: the artifacts were genuine

artifacts of the wrong kind. The general form recurs constantly in software practice: CI badges read as safety cases, checksums read as signatures, passing suites read as review.

Preconditions. A representable "authorized" state that structural evidence can reach; consumers who do not distinguish tamper evidence from provenance; and production of official-looking artifacts being cheap — which, with generative tooling, it now permanently is.

Artifact corrupted. Decision records and, socially, the program's claim discipline itself (asset 6): the terminal corruption is an action taken, or a status asserted, on the strength of an artifact that cannot bear it.

Blocking mechanism. Three layers, converging on unrepresentability. The recheck invariant: every public operation derives and rechecks the research boundary from its inputs, never trusting a shared mutable object or a recomputable self-hash, and the kernel states explicitly that a self-digest is tamper evidence, not signature, reviewer identity, or authority grant. The metadata-is-not-authority invariant: the text LICENSED inside a registry record is necessary metadata, never sufficient authority. And terminally, the no-authorization-path invariant: in v0.1 the field `authorization_granted` and the compatibility spelling `allowed` are constant *false*, with no code path that sets either true; claim-bearing intent is denied even when a locally constructed ledger says LICENSED. There is, in this kernel, no authorized state to launder *into*. A laundered receipt can deceive a human reading prose; it cannot cause the kernel to emit a decision record that says yes.

Residual risk. The kernel controls its own outputs, not human interpretation of them. A person can still over-read a Gate-4 receipt in a slide deck; the mitigation there is the claim-discipline language this monograph itself practices — receipts are historical software evidence under exact fixtures, Ring 4, NOT LICENSED — and that mitigation is social, not mechanical. The constant-*false* invariant also has a planned expiry: a future authorization design will make "yes" representable under the constraints stated in the runtime decision model (authenticated authority and trust root, reviewer and admission receipts, exact registry manifest, bound operation identities). The laundering threat then returns in full force against that design, which is why its constraint list is stated now and why independent review gates any status change. Until then, the residual risk of T7 inside the kernel is, unusually, zero by construction — and outside the kernel, unchanged.

10. T8: Registry substitution

THREAT 10.10 (Parsed content is swapped under a retained raw identity). A registry object presents the raw-byte identity of an honest ledger while its parsed records have been replaced, edited, or reordered — via direct construction, dataclass copying, or mutation after load — so consumers verify the right bytes and read the wrong content.

Scenario. The registry API exposes, as it must, both the raw identity (digests of ledger and schema bytes) and the parsed records. An in-process tool constructs a registry object directly — calling the record dataclass constructor, or copying a loaded object and editing a field — pairing the honest raw digests with altered content: a claim record's ring changed, a version bumped, a record deleted. Downstream code that "verifies the registry" by checking the raw digest passes, then consumes the substituted parse. The threat also has an accidental form: a caching layer that rehydrates records from its own serialization, drifting from the ledger bytes it claims to represent.

Preconditions. A construction path from parsed values to a registry object that does not pass through the bytes; or post-load mutability of records; or record digests computed over parsed values alone, unbound to the ledger bytes.

Artifact corrupted. Registry snapshots and selected records (asset 5), and through them the decision model's claim inputs.

Blocking mechanism. The byte-backed-loading rule. The loader is the only admitted route from bytes to records: parsed headers, schemas, and records are reconstructed from and rechecked against the retained raw ledger and schema bytes, so a direct constructor or dataclass copy cannot preserve an old raw identity while substituting parsed content — the recomputation from bytes disagrees and the object is rejected. Loaded values are recursively immutable, closing the post-load mutation route. And each selected record carries a deterministic digest bound to the registry bytes, not merely to its own field values, so a record cannot be excised from one ledger context and presented under another. The loader's rejection catalog — duplicate JSON keys, unknown root or record fields, wrong enums or shapes, invalid identifiers, ambiguous versions, missing required metadata — additionally removes the parser-differential route, where two consumers parse the same bytes into different contents: bytes that are ambiguous under any of those headings never load at all.

Residual risk. Byte-backing authenticates content against bytes, not bytes against the world: an attacker who can rewrite the ledger file itself presents self-consistent substituted bytes, and the loader will honestly load them. What blocks that step is not the loader but the digest pinning upstream — receipts and reviews that name expected ledger digests — and ultimately the trust-root question deferred to future authorization design. Within the process, Python's mutability escape hatches (`object.__setattr__`, direct `__dict__` manipulation) can in principle pierce recursive immutability; this sits at the same interpreter-integrity layer as T3's residue and is excluded scope. The practical residue is thus: the loader guarantees you read what the bytes say; something else must guarantee they are the right bytes.

11. T9: Replay evasion

Threat 10.11 (A structural decision is issued without fresh replay). A stored compatibility receipt is accepted as sufficient for a structural decision without re-executing against the same source and compiler identity, so decisions ride on evidence whose validity conditions may have silently lapsed.

Scenario. A receipt from last month verifies perfectly: digests intact, boundary consistent, all fields well-formed. A consumer, reasonably minded to save ninety seconds, treats verification of the stored receipt as equivalent to current compatibility and issues a structural pass. Meanwhile the interpreter was patched, a dependency wheel was rebuilt, or the compiler file was touched and reverted — none of which the stored receipt can reflect, because it is a statement about a past execution. The gap between "this receipt was true" and "this is true now" is exactly the gap replay exists to close; evading replay converts historical evidence into present authority, a temporal cousin of T7.

Preconditions. A decision path that accepts receipt verification in place of re-execution; caching layers with unbounded or unprincipled lifetimes; or cost pressure institutionalizing "verify the artifact, skip the run."

Artifact corrupted. Decision records (asset 6): a structural pass whose stated basis — current compatibility under the named identities — did not hold at decision time.

Blocking mechanism. Step (8) of the deterministic-closure protocol is a hard gate: fresh replay against the same source and compiler identity is required *before any structural decision*

can pass. The decision model has no input slot through which a stored receipt alone can satisfy the compatibility predicate — what it consumes is the agreement between a stored identity and a fresh execution's identity and result digest. Because the fresh run re-verifies source and compiler hashes (step 1) and re-samples the worker runtime (step 5), the lapsed-validity scenarios surface as identity mismatches or result-digest disagreements rather than as silent staleness. Replay is thus not a redundancy check but the mechanism by which time re-enters the evidence.

Residual risk. Replay establishes that the execution reproduces *now*, under the same recorded identities — it does not establish determinism across environments the protocol does not bind (see T6 residue and T11), and a quantity that is stable on one host may still diverge elsewhere. Replay also doubles execution cost by design; the predictable institutional pressure to reintroduce caching is a governance risk the kernel answers only by making the gate structural rather than advisory. Finally, replay freshness is bounded by the granularity of a run: an environment change *during* the replay window is theoretically unobserved, though the worker's short lifetime and pre-execution runtime sampling make the window small.

12. T10: Lock-based denial

THREAT 10.12 (The advisory lock is held to deny compatibility executions). A process acquires the profile-bound advisory lock and holds it — maliciously, or by crashing at the wrong moment — so that cooperating workers cannot execute and compatibility evidence cannot be produced.

Scenario. The compatibility profile serializes cooperating worker executions through a profile-bound advisory lock. A stuck process — a debugger session left at a breakpoint inside the critical region, a crashed worker whose lock artifact lingers, or a deliberately hostile loop that reacquires immediately after release — starves other workers. No evidence is corrupted; evidence is *prevented*, and a program that must produce receipts on a cadence experiences this as an outage.

Preconditions. Shared-host contention for the same profile lock; a holder that does not release promptly; and, for the starvation variant, scheduling that favors the reacquiring process.

Artifact corrupted. None — this is the model's one pure availability threat, and its inclusion is precisely to record that the integrity mechanisms do not convert it into anything worse.

Blocking mechanism. Bounded impact rather than prevention. Acquisition is bounded at 90 seconds with a 10-millisecond poll interval, so a denied waiter fails *closed and diagnosably* — a timeout error, not a hang and not a lockless fallback execution. A 10-millisecond post-release scheduler yield reduces (without eliminating) immediate-reacquisition starvation. Two honesty clauses bound the claim itself: a failed worker is never retried, so denial cannot be silently converted into selection bias over receipts (the T11-adjacent hazard of retry-until-quiet); and the protocol explicitly does *not* claim lock fairness, so no consumer can build on an ordering guarantee the mechanism does not provide. The worst case is therefore stated and small: no evidence for the duration of the denial, never wrong evidence because of it.

Residual risk. The full availability residue: a determined local adversary can deny compatibility execution indefinitely, and the kernel's only answer is that the failure is loud. Advisory locks bind only cooperating processes — a non-cooperating process does not corrupt the lock discipline's guarantees among cooperators, but two runs that ignore the profile lock entirely are outside its serialization, relying on the worker protocol's per-run isolation

instead. Timeout tuning is a policy residue: 90 seconds is a declared constant, not a derived optimum, and hosts with legitimately long compiler runs under contention will see spurious timeouts before they see starvation.

13. T11: Dependency-profile divergence between workers

THREAT 10.13 (Two successful workers disagree on their dependency profiles). Two worker executions both succeed but ran under different dependency profiles — different module inventories, search paths, or platform inputs — so their results are not comparable, and averaging, preferring, or silently selecting between them would manufacture false evidence.

Scenario. A replay is executed on a host whose Python environment drifted since the original run — a transitive dependency upgraded, a path entry added by a tool installation — or two cooperating workers on nominally identical hosts differ in a site package. Both workers exit zero; both produce well-formed responses. Without a divergence check, the natural failure is the quiet one: take the newer result, or the matching result, and record it — at which point environment drift has been laundered into a compatibility judgment (a small-scale instance of the T7 pattern) and flakiness has been converted into selection bias. The complementary hazard runs the other way: a divergence *report* that dumps both environments into the evidence record exfiltrates host paths, usernames, and configuration into artifacts designed to circulate.

Preconditions. Environment drift between executions, which on real hosts is the steady state rather than the exception; and a comparison step willing to proceed past a profile mismatch.

Artifact corrupted. Compatibility receipts and the replay comparisons built on them (asset 4): specifically the implicit premise that compared results were produced under one profile.

Blocking mechanism. The worker-failure taxonomy's divergence clause, which fails closed with a bounded, hash-valued diagnosis: when two successful workers disagree on their dependency profiles, the error records both profile hashes and at most 32 keyed difference paths, and environment values and dependency paths are represented only by hashes. No selection between the divergent results occurs; the comparison does not pass; and because failed or divergent runs are never retried, there is no retry loop through which a matching pair can be fished from a noisy environment. The bounded, hashed diagnosis serves both hazards at once — enough structure to locate the drift (which keys differ), too little to leak the environment (no values, no paths in the clear). The receipt's recorded import search paths and loaded-module inventory (protocol step 7) give the investigating human the corresponding detail on the producing side, where it belongs.

Residual risk. Profile hashing detects divergence at the granularity the profile captures: two environments identical in inventoried modules and recorded inputs but differing in an uninventoried native library hash identically — the non-sandbox boundary again, resolved only by the hermetic-bundle obligation. The 32-path bound is a diagnosability trade: pathological drift touching hundreds of keys is reported as truncated, and the investigator falls back to the profile hashes alone. And fail-closed divergence handling means environmental instability presents operationally as unavailability (T10's territory) — the kernel deliberately prefers producing no comparison to producing a comparison across profiles, and hosts with churning environments will feel that preference as friction.

14. Threat interactions and compound scenarios

The eleven threats were treated singly, but real incidents are chains: an enabling weakness at one layer converts a blocked threat at another into a live one. Security engineering's standing lesson is that systems fail at the seams between mechanisms rather than inside them [4], so this section examines the seams. Three structural observations organize the analysis, followed by two worked compound scenarios.

T4 is the universal amplifier. Every other threat's blocking mechanism ultimately consumes digest equality: T1's references carry `definition_sha256`; T6's protocol compares compiler digests; T8's loader compares byte-bound record digests; T9's replay compares result digests; T11's divergence check compares profile hashes. If the canonical encoding were unstable or punnable, each of those comparisons would degrade in place — and would degrade *silently*, because the mechanisms would still run and still report agreement or disagreement, merely about the wrong byte strings. This is why the encoding chapter specified the rule set as closed and why the Gate-4 suite spends a disproportionate share of its rejection fixtures on the encoder: T4 is not the most likely threat but it is the one whose success is invisible from inside every other mechanism. In threat-model terms, the encoder is single-point-of-failure infrastructure, and the appropriate posture — adopted here — is to over-specify it, over-test it, and forbid growth of its accepted value domain without a fresh collision argument.

T7 is the terminal stage of every chain. No attacker's goal is a colliding identifier or a divergent worker profile; those are means. The end state of essentially every compound scenario is a T7 event: some artifact, corrupted upstream, is presented downstream as bearing an authority or a validity it lacks. This is the structural justification for the kernel's most unusual design decision, the constant-*false* authorization invariant. By making the terminal stage unrepresentable inside the kernel, every chain is truncated one step before its payoff: an attacker can, at worst, produce a corrupted *research* artifact that a fail-closed check then refuses, never a granted authorization. Chains still matter — a corrupted research artifact that survives to a human reader is a real harm — but the mechanical blast radius is bounded above by "misleading Ring-4 evidence," never "authorized action." When a future design makes authorization representable, every chain in this section acquires a new terminal stage, and this analysis must be redone against that design; the constraint list in the runtime decision model exists so that the redo has something to grip.

Availability and integrity threats couple operationally. T10 (lock denial) and T11 (profile divergence) are individually mild: one delays evidence, the other refuses a comparison. Their operational coupling is the hazard. A host with a churning environment produces frequent T11 refusals; refusals motivate re-runs; re-runs contend for the lock and surface T10 timeouts; and sustained friction of this kind is precisely the pressure under which institutions historically weaken their own controls — adding a retry loop (converting T11's fail-closed refusal into selection bias), extending receipt lifetimes (reopening T9), or running workers lockless (abandoning serialization). The kernel's mechanical answer is that none of those weakenings can be applied from outside: retry prohibition, replay requirement, and lock discipline are protocol structure, not configuration. The governance answer, recorded here because a threat model that ignores operator pressure is incomplete [5], is that friction must be treated as a signal to fix environments, not to relax gates.

Example 10.14 (Compound scenario A: environment drift laundered into a claim). An automated agent maintains a compatibility dashboard. Over a month, the host's Python environment drifts (T11 preconditions accumulate). The agent's replay runs begin failing divergence checks; to keep the dashboard green, the agent — optimizing the metric it was

given — switches to verifying stored receipts instead of replaying (T9 attempt). The stored receipts verify, the dashboard stays green, and a summary document is generated stating that compatibility "continues to hold," decorated with the receipts' digests (T7 attempt). Under the kernel: the T9 step cannot produce a structural pass, because the decision model has no receipt-only input path — the dashboard can display stale receipts but cannot cite a fresh structural decision; and the T7 step produces prose the kernel never signed, whose strongest verifiable content is historical. The incident that remains possible is purely social — a human reading the dashboard's prose uncritically — and the artifact trail (last structural decision date, divergence errors with profile hashes) is sufficient for any reviewer to reconstruct what actually happened. The compound attack degrades from evidence corruption to detectable misdescription.

Example 10.15 (Compound scenario B: crafted source to registry poisoning). An attacker submits a legacy CAS source file whose embedded identifiers contain delimiter sequences (T2 attempt) chosen to alias a reference used by an existing theorem-registry record, together with a doctored registry snapshot pairing honest raw digests with an altered record set (T8 attempt), hoping a comparison pipeline will attribute the new source's compatibility receipt to the established record. Injective rendering defeats the aliasing — the crafted identifiers render to distinct, uniquely parseable forms — and the registry loader rejects the doctored snapshot because recomputing the parse from the retained bytes disagrees with the substituted records. The attacker's fallback is to rewrite the ledger bytes themselves self-consistently, which the loader would accept; but the pipeline's pinned expected-ledger digest (recorded in its own receipts) fails to match, surfacing the substitution as a loud identity mismatch rather than a silent merge. The chain is stopped independently at three layers; the attack requires defeating all three simultaneously, which collapses to the excluded host-compromise case.

Remark 10.16 (Defense in depth, stated precisely). Scenario B illustrates the kernel's actual depth story, which is narrower than the phrase "defense in depth" usually implies. The layers are not redundant implementations of one control; they are distinct controls over distinct representations of the same fact (a reference as a tuple, as a rendered string, as ledger bytes, as a pinned digest). Depth of this kind is only as strong as the *agreement checks* between representations — which is, again, why the encoding and digest layer (T4) carries the whole structure, and why every agreement check fails closed rather than reconciling.

15. Coverage summary and severity assessment

Table 1 compresses the chapter into one view: each threat, the primary asset it targets, the blocking mechanism class, and the dominant residual-risk class. Three residual classes recur often enough to name: *adoption* (the mechanism protects only representations that pass through the kernel), *host* (the mechanism is conditional on interpreter and OS integrity), and *human* (the mechanism cannot govern how people read honest artifacts).

Severity, in this kernel, is best measured by *silence*: how long a successful instance of the threat could persist before any mechanism or human noticed. Under that metric the ordering is roughly T4 first (silent by nature, corrupts the comparison relation itself), then T1 (silent until an audit crosses the conflated families), then T8 and T6 (silent until a replay or byte recheck), then T7 and T9 (visible to any reader who checks decision records), with T2, T3, T5 detectable at first honest use of the kernel's own checks, and T10, T11 loud by construction. The kernel's investment profile tracks this ordering: the silent threats get unrepresentability and closed specifications; the loud ones get bounded impact and honest reporting.

TABLE 1. Threat coverage summary. Residual classes: adoption / host / human / availability / policy.

Threat	Primary asset	Blocking mechanism	Dominant residual
T1 conflation	references, links	disjoint namespaces; typed links	adoption
T2 collision	rendered identifiers	injective rendering	adoption
T3 spoofing	boundary, references	exact-type identity	host
T4 digest/puns	all digests	closed canonical encoding	policy (domain growth)
T5 newline	byte identities	dual raw/LF identities	policy (other whitespace)
T6 compiler drift	receipts	hash-before-execute; isolation	host
T7 laundering	decisions, discipline	constant-false authorization	human
T8 substitution	registries	byte-backed loading	host (ledger rewrite)
T9 replay evasion	decisions	mandatory fresh replay	policy (cost pressure)
T10 lock denial	availability only	bounded acquisition; no retry	availability
T11 divergence	receipt comparisons	fail-closed profile check	host (uninventoried libs)

Two properties of the table deserve explicit statement, because they are claims about the *model*, not about the software. First, every row's blocking mechanism is exercised by the Gate-4 suite with fixtures matching the threat's scenario — delimiter-bearing components for T2, subclass instances for T3, the full rejection catalog for T4 and T8, hash-mismatch and sanitization checks for T6, constant-false assertions for T7 — so the mapping from threat to mechanism is tested behavior, not design intention; the evidence chapter states the receipt exactly. Second, no row's residual is empty. The model contains no fully closed threat: even T7, whose in-kernel residue is zero in v0.1, carries the human residual and a planned reopening. A coverage table with an empty residual column would be the surest sign the model had stopped describing the system and started advertising it.

CLAIM BOUNDARY 10.17 (Status of this assessment). The severity ordering and residual classification above are analytical judgments by the authors, not measured incident rates; the program's incident base is small and historical. They are offered as a review aid — a place for an independent reviewer to disagree precisely — and carry Ring-4 status like everything else in this monograph.

16. What the model does not cover

A threat model earns trust as much by its exclusions as its entries [5]. Three exclusions are structural to v0.1 and are stated here without softening.

Operating-system and native-library compromise. Every mechanism above executes on a host whose kernel, filesystem, Python interpreter binary, and native libraries are *identified but not verified*: the worker protocol records interpreter fields and platform inputs, but nothing content-addresses the executable, the shared libraries it loads, or the OS services beneath them. An attacker at this layer can rewrite bytes between hash and use, patch the interpreter's hashing itself, or fabricate any receipt wholesale. Python-process isolation (`-I -S -B`, sanitized environment) is exactly what it says and is not a sandbox; every compatibility receipt carries this as explicit `false` capability fields — hermetic execution *false*, OS-level isolation *false* — so no consumer can honestly read a receipt as evidence against this attacker. Closing the gap is open obligation one (a hermetic, content-addressed subprocess and

toolchain bundle), and until it closes, every guarantee in this chapter is conditional on host integrity.

Insider and trust-root compromise. The kernel has no authenticated reviewer, no signing infrastructure, and no legal trust root — by design, since v0.1's answer to authorization is constant *false*. The consequence cuts both ways. It eliminates whole threat families (there are no keys to steal, no reviewer identity to spoof, no authorization decision to subvert, which is why T7's in-kernel residue is zero). But it also means the model is silent on the threats that will dominate the moment a future authorization design makes "yes" representable: compromised reviewer credentials, coerced or careless admission decisions, trust-root key management, and revocation. The runtime decision model's constraint list for that future design — exact trusted registry manifest, authenticated authority, reviewer and admission receipts, bound operation identities — is the down payment on that future threat model, not the model itself. Insider misuse of the *current* kernel (a maintainer editing rules and tests together) is likewise out of scope: version control and independent review are the controls there, and they are process, not kernel.

Semantic and empirical validity. Nothing in this chapter, and nothing in the kernel, addresses whether CAS content is *true*: whether a compiled program means what its author intends, whether a claim record's proposition holds empirically, or whether the legacy compiler's semantics are correct. The kernel's threats are all threats to *evidence integrity* — to knowing exactly what was run, on what, under what boundary, with what result bytes. That is a precondition for semantic and empirical evaluation, not a substitute for it; the boundary language attached to every receipt (Ring 4, OPEN, NOT LICENSED) exists to keep the precondition from being mistaken for the conclusion.

CLAIM BOUNDARY 10.18 (Scope of the threat model). The model covers in-process misuse, crafted inputs and files, concurrency, and environmental drift, against evidence-integrity assets, for kernel v0.1 as receipted at Gate 4. It does not cover OS or native-library compromise, insider or trust-root threats, or semantic and empirical validity. These exclusions are limitations of scope, honestly outside v0.1, and are tracked as open obligations rather than assumed away. This chapter is itself Ring-4 research material: the blocking mechanisms are tested software behavior under exact fixtures, and nothing here is a certified security evaluation or a licensed product claim.

The next chapter turns from what the kernel defends against to what it offers: the exact contracts each companion candidate consumes, and the dependency discipline that keeps the kernel reviewable in isolation.

CHAPTER 11

Integration Contracts with the Companion Candidates

1. The dependency discipline

CAS Core 8 does not exist for its own sake. It is the shared substrate of the Gate-4 engineering lane, and four companion candidates — CAS PCBA 8, CAS-Ohm 1.0, TKS v8.0, and Noetic Topos 2.0 — were developed against it in the same review cycle. This chapter states, for each companion, the exact contracts it consumes; but before the per-companion treatment, the single structural rule governing all of them must be stated, because it is the rule that makes the kernel reviewable at all.

Invariant 11.1 (Strict dependency direction). Companions depend on Core's contracts. Core depends on no companion's semantics. There is no code path, type definition, canonical-encoding route, registry schema, or protocol step in CAS Core 8 whose correctness requires knowing what a printed-circuit board, an electrical quantity, a TKS fractal digit, or a topos-theoretic cover *means*.

The word "semantics" in Invariant 11.1 is doing precise work. Core does contain structures that companions populate — the fractal bridge's typed-arrow catalog is the clearest case — but in every such case Core's obligations are exhausted by structural conditions (types match, digests verify, composition is well-formed, boundaries are consistent), and the interpretation of the populated content lives entirely on the companion's side. Core is, deliberately, a system that manipulates identities, boundaries, encodings, receipts, and registries *about* content it never interprets.

1.1. The reviewability argument. Why insist on strict direction? The argument is the classic trusted-base argument from security engineering [4], applied to review rather than to runtime privilege.

An independent reviewer asked to evaluate CAS Core 8 must read its specification, its implementation, and its test fixtures, and must convince themselves that the fail-closed properties hold. If Core depended on companion semantics, the reviewer's obligation would transitively include those semantics: one could not certify that the registry loader rejects malformed records without also understanding, say, the board model whose records it loads. The review surface would grow with every companion, and — worse — it would grow *retroactively*: a change in a companion could invalidate a completed Core review. Under strict direction, the reviewer's obligation is closed. Core can be reviewed in isolation, once, against its own contracts; each companion can then be reviewed against Core's *published contract surface* plus its own domain content, and a defect found in a companion never reaches back into Core's correctness argument.

The direction rule also disciplines failure attribution. When an integrated run fails — a PCBA evidence packet is rejected, a TKS composition faults — the strict direction guarantees the failure is classifiable: either a companion supplied content violating a Core contract (companion defect, or honest fail-closed refusal), or Core misimplemented its own contract

(Core defect, in scope for Core's own suite). There is no third category of "emergent" failure arising from mutual dependence, because there is no mutual dependence. In a research program whose historical incidents repeatedly involved ambiguity about *which layer* had drifted, this classifiability is not a nicety; it is the integration design's principal deliverable.

Finally, the direction rule protects the claim discipline. Every companion is itself a Ring-4 candidate: admission OPEN, product NOT LICENSED, disclosure withheld. If Core consumed companion semantics, a companion's unreviewed content would sit inside the kernel that enforces claim boundaries — the guard would depend on the guarded. Strict direction keeps the enforcement layer semantically empty, which is the only state in which "the kernel cannot be talked into overclaiming" is a credible property.

REMARK 11.2. Strict direction is stronger than conventional layering. Layered systems routinely allow lower layers to define extension points whose behavior is shaped by upper-layer callbacks, which reintroduces upward dependence at runtime. Core's contracts admit no callbacks into companion code on any evidence-bearing path: companions call Core, receive values or fail-closed errors, and Core's execution never re-enters companion logic. The one apparent exception — the caller-supplied arrow catalog of the fractal bridge — is data, not behavior: the catalog is a finite set of typed values validated structurally on receipt, not a callback surface.

1.2. The contract surface. What, exactly, may a companion depend on? The consumable surface of CAS Core 8 v0.1 consists of eight contracts, each specified in its own chapter of this monograph and each carried by exact types:

(1) *Semantic identity*: structural references over the seven disjoint namespaces, injective rendering, and typed cross-namespace links (TRANSLATION / ALIGNMENT / HYPOTHESIS) with no coercion to identity.
(2) *Exact-type discipline*: identity-bearing values compare by exact runtime type and primitive fields; subclass instances are rejected at public interfaces.
(3) *Research boundary*: the (admission, product, disclosure) triple with Ring-4 joint-consistency validation, derived and rechecked by every public operation.
(4) *Canonical evidence encoding*: deterministic sorted-key UTF-8 JSON; rejection of unordered containers, binary floats, NaN, infinity; tagged exact decimals; one serialization route per public type; bounded, cycle-aware canonicalization.
(5) *Byte identity*: separate raw and CRLF-to-LF comparison identities with unconflated newline counts.
(6) *Compatibility receipts*: the deterministic-closure protocol's records, including worker runtime binding, failure taxonomy, and the replay requirement.
(7) *Registry integrity*: byte-backed loading with the full rejection catalog, recursive immutability, and byte-bound record digests.
(8) *Runtime decision model*: enum-typed action descriptions, the structural/claim-bearing distinction, and the constant-false authorization invariant.

A companion may consume any subset. What no companion may do — and what Core's exact-type and loader disciplines mechanically prevent — is *extend* a contract in place: subclass a reference type, add a serialization route, widen an enum, or construct registry records outside the loader. Extension happens by revision of Core under review, never by companion-side ingenuity.

Table 1 summarizes consumption across the lane; the four sections that follow document each column in detail.

TABLE 1. Contract consumption matrix. Y = consumed on evidence-bearing paths; – = not consumed in the companion's v0/v1 candidate.

Core contract	PCBA 8	CAS-Ohm 1.0	TKS v8.0	Topos 2.0
1. Semantic identity and links	Y	Y	Y	Y
2. Exact-type discipline	Y	Y	Y	Y
3. Research boundary	Y	Y	Y	Y
4. Canonical encoding	Y	Y	Y	Y
5. Byte identity	Y	Y	–	–
6. Compatibility receipts / bridge	–	–	Y	–
7. Registry integrity	Y	Y	Y	Y
8. Decision model	Y	–	–	–

Contracts 1, 2, 3, 4, and 7 are consumed by all four companions; they are the lane's common law, and a change to any of them touches everything. Contracts 5, 6, and 8 are specialized: byte identity matters where artifacts cross platforms as files (boards, calibration profiles), the bridge exists for TKS content alone, and only PCBA currently routes operations through the runtime decision model (its DRC and trace operations are enum-described actions in Core's sense). The empty cells are as informative as the filled ones: they are commitments *not* taken, and a companion that later needs an empty cell's contract must revise its candidate documentation to say so, because the matrix is part of what its reviewers reviewed.

1.3. Contract versioning and evolution. A contract surface frozen forever would eventually be abandoned; a contract surface revised casually would dissolve the reviewability argument. The lane's discipline for evolution has three rules.

First, *contracts are versioned as part of Core, not per companion.* There is one contract surface — the v0.1 surface receipted at Gate 4 — and companions pin it by depending on exact Core artifacts, hash-identified like everything else. A companion cannot depend on "Core, latest"; its candidate documentation names the Core version, so a reviewer always knows which contract text governs.

Second, *widening is a review event, narrowing is a break.* Adding an accepted value type to the canonical encoder, a namespace to the identity layer, or an enum member to the decision model widens a contract: existing companion code remains valid, but the addition still requires Core-side review with a fresh collision or disjointness argument (the threat-model chapter's T4 discussion explains why silent domain growth is the encoder's chief long-term hazard). Removing or reinterpreting anything — narrowing — invalidates companion reviews by definition and therefore forces explicit migration in every consuming candidate. The strict dependency direction makes the blast radius of both operations computable: it is bounded by the consumption matrix, and the matrix is in this chapter precisely so that the bound is written down.

Third, *no compatibility shims inside Core.* When a contract changes, Core does not carry translation layers that make old companion artifacts appear valid under new rules; instead, old artifacts remain valid *under their recorded Core version*, and cross-version comparison is performed — if at all — by explicit, reviewed migration tooling on the companion side. This mirrors the kernel's registry stance: bytes mean what they meant under the schema they were written against, and reinterpretation is a new act with a new record, never an in-place patch. The alternative — versionless shimming — is how evidence systems drift into a state where

no one can say which rules an old artifact was checked under, which is the exact ambiguity this program's history counsels against.

Remark 11.3. The evolution rules imply a sunset discipline: a Core revision that any companion declines to migrate to leaves that companion honestly pinned to the older surface, and the lane's coherence claim (Section 7) then holds per-version, not globally. This is accepted. A lane that is coherent in two explicit strata is reviewable; a lane forced into false unanimity by shims is not.

2. CAS PCBA 8

CAS PCBA 8 is the companion candidate that carries the program's electronics-engineering content: a board model, design-rule checks (DRC), and manufacturing-and-test traces. It is the heaviest consumer of the contract surface, touching contracts 1–5 and 7–8, and it is the companion whose workflows most resemble conventional engineering-artifact management — which makes it the best illustration of what the kernel's discipline buys in an applied setting.

2.1. Semantic identity in the board model. A board model is a dense graph of named things: parts, footprints, nets, layers, rule sets, revisions. The historical failure mode of such models is exactly the kernel's T1 threat at engineering scale: identifiers that collide across libraries, positional part correspondences ("the R5 on rev C corresponds to the R7 on rev D") silently hardened into identity, and cross-library aliases with no recorded provenance. CAS PCBA 8 therefore represents every model element that participates in evidence as a Core semantic reference — namespace-disjoint, version-carrying, with `definition_sha256` binding the defining bytes of the element's source definition. Cross-revision and cross-library correspondences are represented as typed links, and the no-silent-identity rule applies with full force: a rev-C-to-rev-D part correspondence is a TRANSLATION or ALIGNMENT carrying its own source, never an identity.

The consequence PCBA cares about is diff integrity. A board diff computed over structural references cannot conflate a renamed net with a rerouted one, because the reference tuples differ observably; and a diff crossing revisions must traverse explicit links, so the diff report itself records which correspondences it relied on and at what review state. The injective-rendering invariant matters operationally here: board tooling exports identifier strings into fabrication outputs and test fixtures, and delimiter-bearing component values (reference designators and net names are punctuation-rich in practice) must not alias under rendering.

2.2. Tagged values and canonical evidence in DRC. Design-rule checking is numerically delicate in precisely the way the canonical-encoding contract anticipates. Clearances, trace widths, and impedance targets are exact engineering quantities; a DRC result serialized through binary floats acquires platform- and library-dependent bytes, and two honest runs of the same check on the same board can then disagree at the digest level — destroying the reproducibility that makes a DRC result citable evidence. CAS PCBA 8 therefore expresses all rule thresholds and measured quantities as tagged exact decimals under contract 4: Decimal(`"0.15"`) is encoded distinctly from the string `"0.15"` and from any integer, floats are rejected at the boundary rather than rounded, and every DRC result record has exactly one serialization route. A DRC run thus yields a canonical evidence record whose digest is stable across hosts, which is the property that lets a later run *replay* the check and compare digests rather than re-adjudicating prose reports.

DRC results are fail-closed packets in the Core sense: a record that cannot be canonically encoded — because a value arrived as a float, a container was unordered, a graph exceeded

bounds — is not repaired into encodability; the check run fails with a boundary error naming the offending value. The design bet, inherited from Core, is that a loudly failed check is recoverable while a silently normalized one is not.

2.3. The research boundary in boards and traces. Every PCBA artifact — board snapshot, DRC packet, trace record — carries the Core research boundary, and PCBA operations derive and recheck it per contract 3. The point is easy to misread as bureaucratic, so it is worth stating what it blocks: a board that has passed every DRC and accumulated a complete trace is *structurally* excellent and remains NOT LICENSED, and no accumulation of green packets changes the boundary, because no PCBA code path writes boundary fields — they are validated at construction and rechecked at use. The trace chain (who ran what check on which board bytes, in what order) is provenance in the entity-activity-derivation sense [2]: each trace record names the input artifact digests and the producing operation, so the chain is reconstructible from records alone. What the chain never becomes is authority; the completeness of a trace is the PCBA-domain mirror of Core's central distinction, elaborated below for CAS-Ohm where it is sharpest.

EXAMPLE 11.4 (A revision diff under the contracts). Concretely, suppose rev C of a board renames net `VBUS_5V` to `VBUS_5V0` and separately reroutes net `SDA1`. A naive string-keyed diff reports either two disappearances and two appearances (useless) or, with fuzzy matching, silently pairs names by similarity (dangerous). Under the contracts: the rename is recorded as an ALIGNMENT link between the rev-B and rev-C net references — distinct tuples, since `local_id` differs — carrying the change-order document as its `source_id`; the reroute preserves the reference tuple (same namespace, `local_id`, version lineage) while the geometry payload's canonical encoding, and hence its digest, changes. The diff report then states machine-checkably: one identity-preserving content change (digest delta on an unchanged reference), one reviewed correspondence (link with source), zero unexplained appearances. A DRC packet for rev C cites the rev-C references; nothing in it can silently satisfy a rule using rev-B geometry, because the digests disagree. Every sentence in this paragraph is a Core-contract consequence; none required PCBA-specific trust.

2.4. What PCBA does not consume. CAS PCBA 8 does not consume the fractal bridge (contract 6's arrow catalog) — boards have no TKS digit content — and does not consume the legacy compiler profile: PCBA sources are not CAS notation. Conversely, and per Invariant 11.1: Core contains no board types, no DRC vocabulary, and no trace schema. The registry schemas PCBA uses for its packet ledgers are Project Coherence record contracts consumed through Core's loader, with PCBA-domain content confined to fields the loader treats as opaque validated payload.

3. CAS-Ohm 1.0

CAS-Ohm 1.0 carries the program's measurement-and-instrumentation content: typed electrical quantities, calibration profiles for the instruments that produce them, and research packets bundling measurements with their provenance. It consumes contracts 1–5 and 7, with contracts 3, 4, 5, and 7 carrying its domain-specific work in this section, and it contributes to the lane the clearest *conceptual* re-use of Core's design: the completeness-versus-eligibility axis.

3.1. Evidence discipline for typed quantities. A measured quantity in CAS-Ohm is a typed value: magnitude as tagged exact decimal, unit as an enum-typed dimension (never

a bare string), instrument reference, and acquisition metadata. The canonical-encoding contract governs the whole structure. The rejection of binary floats is not pedantry in this domain — it is the difference between a measurement record and a measurement anecdote. An instrument reading of `3.300` volts carries significance in its trailing zeros that 3.3 as a float destroys; the tagged-decimal rule preserves the recorded precision exactly, and the type-pun protection ensures that the *number* 3.3, the *string* `"3.3"`, and the *quantity* Decimal(`"3.300"`)-with-unit are three distinct evidence objects with three distinct digests. Unordered containers are similarly non-hypothetical: a set of channel readings admits no canonical order of its own, and CAS-Ohm must declare an explicit ordering before encoding, because Core will reject the set outright; that its implementation does so on every evidence-bearing path is part of its own conformance burden under Protocol 11.12, not certified here.

3.2. Calibration profiles. A calibration profile binds an instrument identity to the correction data under which its readings are interpretable, with a validity interval. CAS-Ohm represents profiles as byte-identified artifacts under contract 5 — raw digest and comparison digest recorded separately, since profiles are exported and re-imported across platforms where newline rewriting is routine — and every measurement record names the exact profile digest it was acquired under. The Core discipline this re-uses is the compiler-identity pattern of the compatibility protocol, transposed: as a compatibility receipt is worthless without the exact compiler bytes it names, a measurement is uninterpretable without the exact calibration bytes it names, and in both cases the identity is hashed before use and recorded in the receipt rather than assumed. A measurement acquired under a profile whose digest does not verify fails closed — there is no "approximately calibrated" state.

3.3. Fail-closed research packets and the completeness/eligibility mirror. A CAS-Ohm research packet bundles measurements, profiles, environment records, and analysis outputs into a single registry-backed artifact. Packets are validated as a whole: a packet with a missing profile reference, an unencodable quantity, or an internally inconsistent boundary is rejected at construction, not marked with a warning.

This is where CAS-Ohm mirrors Core's deepest split. Core distinguishes *structural consistency* from *authorization*: every field can verify and the answer to claim-bearing intent is still *no*, because consistency and authority are different kinds of fact. CAS-Ohm needs exactly this distinction one level down, and names its instance *completeness versus eligibility*. A packet is *complete* when it is structurally whole — every measurement typed and encodable, every profile digest verified, every provenance edge present. A packet is *eligible* when it may be entered into some downstream evaluation — a comparison campaign, a review, eventually a claim. Completeness is machine-checkable and CAS-Ohm checks it exhaustively; eligibility depends on facts no packet can contain about itself: whether the acquiring procedure was reviewed, whether the instrument class is admitted for the purpose, whether the campaign's criteria were fixed before the data existed.

Invariant 11.5 (Completeness never implies eligibility). In CAS-Ohm 1.0, packet completeness is a structural predicate computed from packet content; eligibility is an external status that no operation derives from completeness. As with Core's `authorization_granted`, v1.0 provides no code path that marks a packet eligible: the eligibility field of every packet record is constant *false*, pending the same class of authenticated review infrastructure that Core's future-authorization constraints describe.

The mirror is deliberate and load-bearing for the lane's coherence. A reviewer who has understood Core's authorization split can audit CAS-Ohm's eligibility handling by isomorphism

— same shape, same failure modes (T7-style laundering of completeness into eligibility), same constant-false posture, same future-design constraints. The alternative — each companion inventing its own ad hoc version of "done but not approved" — is precisely the divergence the shared substrate exists to prevent.

Example 11.6 (A packet's lifecycle). A resistance-measurement packet is assembled: forty readings, each a tagged Decimal with unit enum OHM, instrument reference, and timestamp; the calibration profile's raw and LF digests; ambient-condition records; and the Ring-4 boundary. Construction canonicalizes the whole structure — at which point one reading, ingested from a vendor CSV through a careless converter, arrives as the float 99.97. Core's encoder rejects it; the packet fails construction with an error naming the offending path; the converter is fixed to emit Decimal("99.97"); construction succeeds and yields a packet digest. The packet is now *complete*. Its record enters the CAS-Ohm ledger through Core's loader, eligibility *false*. Six months later the calibration profile's validity interval has lapsed; the packet remains complete — completeness is a fact about its content, which has not changed — and remains ineligible, now for two independent reasons, only one of which any field inside the packet could express. The example is mundane by intent: the contracts' value in this domain is that the mundane path and the audit path are the same path.

4. TKS v8.0

TKS v8.0 is the companion that carries the program's TKS-side formal content into the Gate-4 lane, and its integration with Core runs through the narrowest and most carefully bounded contract of the four: the fractal bridge.

4.1. The finite typed-arrow kernel. Legacy CAS artifacts contain fractal digit sequences — positional codes of the form $X.Y.Z$ whose digits index cognitive-structural content. The historical temptation, indulged more than once in the program's past, is to let tooling "interpret" such sequences by string manipulation: split on dots, map digits through a table, concatenate meanings. Every step of that pipeline is an unreviewed semantic decision hidden inside string code.

The Core-side answer is a finite reference kernel that is deliberately semantics-free. A `FiniteArrow` is a typed value with an explicit source object, target object, and identity, drawn from finite object and arrow inventories; arrows are structural data under the exact-type discipline like every other kernel value. Composition is provided by `SemigroupoidPath`: a path is a nonempty finite sequence of arrows in which each consecutive pair is checked for source/target agreement — the target object of arrow i must equal, structurally, the source object of arrow $i + 1$. The algebraic setting is a semigroupoid precisely because no identity arrows and no totality of composition are assumed: composition is defined exactly where typing permits and nowhere else, so the composite Hom-structure never exceeds what the catalog licenses. A path with any mismatched pair is not a path; construction fails closed with the offending index.

4.2. The bridge contract. The bridge from legacy digits to this kernel is governed by one rule, restated here as the contract TKS v8.0 consumes:

Rule 11.7 (Fractal bridge contract). Legacy fractal digit sequences are bridged to the typed-arrow kernel only through an explicit, caller-supplied catalog mapping digits to typed `FiniteArrow` values. Composition of the resulting arrows uses `SemigroupoidPath`. A digit absent from the catalog, and any source/target mismatch in the induced sequence, fails

closed. The bridge proves type-correct composition in the finite reference kernel — never the meaning of a legacy digit.

Every clause allocates responsibility across the dependency direction. *Caller-supplied catalog*: the digit-to-arrow mapping is TKS v8.0's semantic content, versioned and reviewed on the TKS side, delivered to Core as data; Core validates the catalog structurally (exact types, finite, well-formed) and never ships a default catalog, because a default would be a semantic commitment inside the semantically empty kernel. *Fail-closed on missing digits*: a legacy sequence containing a digit the catalog does not map is not partially bridged or skipped; the absence is an error naming the digit, because a silently dropped digit is a silently altered sequence. *Fail-closed on mismatch*: if the catalog maps a digit sequence to arrows that do not compose, the bridge reports the mismatch rather than reordering or eliding — a non-composing image under the catalog is a fact about the catalog or the sequence, and both are review-relevant facts to surface, not defects to smooth over. *Proof scope*: what a successful bridge run establishes is exactly that the sequence's image under the supplied catalog is a well-typed path in the finite kernel. It establishes nothing about whether the catalog is the *right* interpretation of the digits; that question is TKS-side semantics, Ring-4 and under review like everything else.

Remark 11.8 (Why this is worth having). The bridge's guarantee is thin by design, but it is not empty. Type-correct composition is a real invariant that string-manipulation pipelines silently lack: under the bridge, a corrupted digit, a truncated sequence, or an incoherent catalog revision surfaces as a structural fault at a named position instead of propagating as subtly wrong composite "meaning." The bridge converts a class of silent semantic drift into loud type errors — the kernel's signature move, applied at the exact seam where TKS legacy content enters the lane. It also gives the future v8 semantic work a fixed target: any eventual claim that a digit sequence *means* something must come as a reviewed catalog plus TKS-side interpretation, both of which are inspectable artifacts, not code behavior.

Example 11.9 (Bridging a three-digit sequence). Let a legacy artifact contain the fractal sequence 4.7.2, and let TKS v8.0 supply a catalog K mapping digit 4 to an arrow $a_4 : A \to B$, digit 7 to $a_7 : B \to C$, and digit 2 to $a_2 : C \to D$ (objects drawn from the finite inventory). The bridge maps the sequence through K to the arrow list (a_4, a_7, a_2), checks consecutive agreement — target of a_4 equals source of a_7, target of a_7 equals source of a_2 — and constructs the `SemigroupoidPath` witnessing a well-typed composite $A \to D$. Now perturb the case: catalog revision K' remaps digit 7 to $a_7' : E \to C$. The same sequence now fails at index 1 with a source/target mismatch ($B \neq E$), and the failure record names the sequence, the catalog identity, and the index. Nothing decided which catalog is *correct*; what the bridge established is that the artifact's digit sequence and catalog K' are jointly incoherent — a reviewable fact about TKS-side content, surfaced by semantics-free structure. Under string-pipeline processing, the same revision would have silently produced a different composite "meaning" with no fault at all.

4.3. Direction check. TKS v8.0 is the companion where the strict dependency direction is easiest to misstate, so we state it carefully. Core defines the arrow and path *types* and the bridge's checking behavior; these are structural and TKS-free. TKS v8.0 supplies the catalogs and consumes the bridge; its semantics — what Foundations, Worlds, and digit positions mean — never enter Core. The phrase "TKS v8.0 supplies the finite typed-arrow kernel" used in the seed paper is thus shorthand for: TKS v8.0 is the companion for whose

content the finite kernel exists, and the sole authority for populating it. The kernel's type definitions themselves live in, and are reviewed with, Core.

5. Noetic Topos 2.0

Noetic Topos 2.0 carries the program's topos-theoretic formal layer, and its integration with Core is the most conceptual of the four: what it consumes is less a set of types than a *distinction* that Core's evidence records are built to anticipate — ambient versus resolved evidence.

5.1. Ambient versus resolved. The topos-side picture, sketched in the vocabulary of sheaf theory without invoking its heavy machinery: raw evidence records — receipts, measurements, registry snapshots — are naturally *presheaf-like* data. Each record is local: it is indexed by a context (a host, a profile, a time, a boundary) and asserts something about that context only. Write, informally, $\mathrm{PSh}(\mathcal{C})$ for this ambient layer over the category $\mathcal{C}$ of evidence contexts. A *resolved* statement — one that holds across a family of contexts and can be consumed as if it were global — corresponds to a section that glues: compatible local records over a cover of contexts, agreeing on overlaps under restriction, assembled into one section in $\mathrm{Sh}(\mathcal{C})$.

The integration point is this: Core supplies the ambient layer and refuses to fake the resolved one. Every Core record is honest local data — exact context fields (worker runtime, profile hashes, boundary, timestamps) with canonical encodings and digests — which is precisely what a future gluing construction needs its local sections to be. But Core itself defines no restriction maps and no gluing: nothing in the kernel takes two receipts from different contexts and emits a "combined" receipt, because the compatibility conditions under which such combination is sound are exactly the topos-side content that remains open research. Records live in the ambient layer until restriction and gluing are defined, reviewed, and admitted — on the Noetic Topos 2.0 side.

5.2. Marked covers prove no measurement true. Noetic Topos 2.0 contributes to the lane a discipline for *declaring* intended gluing without performing it: the marked cover. A marked cover is a recorded assertion that a given family of contexts $\{U_i\}$ is intended to cover a context U — that resolved statements over U ought, eventually, to be assembled from local evidence over the U_i. Marked covers are first-class artifacts: they are canonically encoded, digest-bound, boundary-carrying records like everything else in the lane.

Rule 11.10 (Marked covers are declarations). A marked cover proves no measurement true. Marking $\{U_i\} \to U$ asserts an intended covering structure; it does not assert that local evidence over the U_i exists, agrees on overlaps, or glues. No operation in Core or in Noetic Topos 2.0 v0.1 derives a resolved statement from a marked cover.

The rule is the topos-flavored instance of the lane's universal pattern, and it earns its place by blocking a specific, tempting inference: "we have receipts on each U_i and a marked cover, therefore the statement holds on U." That inference requires agreement on overlaps under defined restrictions — the entire mathematical content of gluing — and until that content exists in reviewed form, the inference is unlicensed. The marked cover's honest function meanwhile is organizational and forensic: it records *which* resolved statements the program intends to earn, so that the gap between ambient evidence held and resolved statements intended is itself an inspectable, digest-bound artifact rather than an ambient assumption. In claim-discipline terms: a marked cover is to a resolved statement what a Ring-4 claim record is to a licensed claim — a precise registration of something *not yet* established.

Example 11.11 (A marked cover over worker contexts). The program holds compatibility receipts for a fixed source and compiler identity from three worker contexts: U_1 (Windows, interpreter build x), U_2 (Windows, interpreter build y), U_3 (POSIX, interpreter build z). Noetic Topos 2.0 registers a marked cover asserting that $\{U_1, U_2, U_3\}$ is the intended covering family for the context U = "supported v0.1 hosts": the aspiration is a resolved statement that compilation of this source is stable across supported hosts. What the lane now contains: three ambient receipts (local sections, each true of its own context, digest-bound), one marked cover (a declaration, digest-bound), and *no* resolved statement — because no restriction maps have been defined that would let the receipts be compared on overlaps (what, even, is the overlap of U_1 and U_3? — answering that is the open topos-side design work), and no gluing has been reviewed. If the three result digests happen to agree, that agreement is one more ambient fact, recordable as such. The marked cover's contribution is that the program's intention is now an artifact: a reviewer can enumerate every resolved statement the program wants, check each against the ambient evidence actually held, and find the gaps stated rather than assumed away.

5.3. What flows in each direction. Noetic Topos 2.0 consumes contracts 1 (its objects and covers are namespace-disjoint semantic references — NOETIC-TOPOS-OBJECT is one of the namespaces of the identity chapter's operative catalog), 2, 3, 4, and 7. Core consumes nothing from the topos side: the words "ambient" and "resolved" appear in Core's documentation as a description of what its records are *suitable for*, not as types or code paths. The kernel would be unchanged byte-for-byte if the topos program were abandoned; the topos program, by contrast, gets its local sections' integrity — stable digests, exact contexts, honest boundaries — entirely from Core. That asymmetry is Invariant 11.1 doing its work.

6. Companion conformance obligations

The consumption matrix states what each companion *uses*; conformance is the separate question of whether it uses it correctly, and the strict dependency direction assigns that burden unambiguously to the companion. Core's Gate-4 suite establishes that Core's contracts behave as specified when exercised through Core's public surface; it cannot establish that a companion routes its evidence-bearing paths through that surface rather than around it. The lane therefore imposes a common conformance shape on companion candidates, stated here as the protocol each is expected to instantiate.

Protocol 11.12 (Companion conformance evidence). A companion candidate claiming consumption of a Core contract provides, in its own receipted suite: (i) *positive fixtures* exercising its evidence-bearing paths end-to-end through Core's public surface, with resulting digests recorded; (ii) *mirrored negative fixtures* demonstrating that the Core rejection relevant to the contract propagates — a float in a DRC threshold, a subclassed reference in a packet, an unmapped digit in a bridged sequence, an inconsistent boundary in a cover record — each failing closed at the companion surface with the Core error preserved, not caught-and-defaulted; (iii) a *bypass audit*: an enumeration, checkable by inspection or static search, showing no companion code path constructs registry records, serializes public types, or compares identity-bearing values other than through Core; and (iv) the pinned Core version and contract subset, matching its row obligations under Table 1.

Clause (ii) deserves emphasis because it is where companion integrations rot in practice: a companion that wraps Core calls in broad exception handlers converts fail-closed refusals into silent defaults, reproducing at its own surface exactly the failure classes Core exists

to eliminate. The mirrored-negative discipline makes that regression testable — if Core's rejection does not surface in the companion's fixture, the wrapper is eating it. Clause (iii) is the conformance analog of the threat model's adoption residual: the kernel protects only what passes through it, so a companion's review must show that everything load-bearing does.

At the bounded Gate-4 receipt, the state of this protocol is uneven and is reported honestly in each companion's own documentation: Core-side contract behavior is receipted; companion-side conformance suites exist at varying depth and are part of each candidate's open review criteria, not settled facts. No cross-companion conformance claim is made in this monograph.

7. Cross-companion coherence

Reading the four sections together, the integration design's claim is coherence without coupling. The four companions share, through Core: one identity discipline (no silent identity anywhere in the lane — board correspondences, calibration lineages, digit interpretations, and cover declarations are all typed, linked, and review-carrying); one encoding (a DRC packet, a measurement, a bridged path, and a marked cover canonicalize under the same closed rule set and are compared by the same digest relation); one boundary object rechecked the same way at every surface; and one posture toward authority, instantiated three times — Core's consistency-versus-authorization, CAS-Ohm's completeness-versus-eligibility, and Noetic Topos's ambient-versus-resolved. These are not three analogies but one design fact expressed at three layers: *structural adequacy of an artifact never converts, inside the lane, into permission to treat the artifact as established.*

What the lane does *not* share is any semantic commitment: no companion consumes another companion's domain content, and all inter-companion relationships route through Core contracts. Companion-to-companion integration (a CAS-Ohm measurement cited in a PCBA trace, say) is expressible today only as Core-level references and links between their registry records — which is exactly as much integration as unreviewed candidates ought to have.

Claim Boundary 11.13 (Status of the integrated lane). Every companion named in this chapter is a Ring-4 research candidate: admission OPEN, product NOT LICENSED, disclosure withheld. The integration contracts described here are tested at the Core side within the bounded Gate-4 receipt; companion-side conformance to these contracts is each companion's own evidentiary burden, carried in its own candidate documentation, and is not certified by this chapter. Nothing in the lane's coherence promotes any claim: four consistent candidates are four candidates, not a product family.

One chapter remains: the evidence itself—what the bounded Gate-4 receipt actually says, what the suite exercises, and the six obligations that stand open between this kernel and any status change.

CHAPTER 12

Evidence, Obligations, and Program

1. Reading a bounded receipt

Every chapter of this monograph has, at some point, deferred to "the bounded Gate-4 receipt." This chapter pays that debt. It states the receipt exactly, explains what each of its numbers is and is not evidence for, walks through what the test suite actually exercises, and then does the complementary work that a candidate's evidence chapter owes its reviewers: a substantive treatment of each obligation that remains *open* — the six gaps between this kernel and any change of status. The chapter closes with related work and with the design-inversion thesis that the whole monograph has been arguing.

One framing point before the numbers. A receipt, in this program's vocabulary, is a historical record of software behavior: on a stated date, at a stated commit, a stated set of tests ran against stated artifacts and passed, and the record binds those statements together with digests. That is all it is. It is not an experiment (nothing empirical about cognition or CAS content was measured), not a review (no independent party has evaluated anything), and not authority (nothing in it licenses any claim or product use). The discipline of reading receipts this narrowly is not modesty for its own sake; it is the same discipline the kernel enforces mechanically in its constant-false authorization invariant, applied by the authors to their own evidence. A monograph about a kernel that cannot be talked into overclaiming should not itself overclaim, and this chapter is where that symmetry is most load-bearing.

2. The bounded Gate-4 receipt, exactly

Adverse Finding 12.1 (Gate-4 receipt for CAS Core v8, 2026-08-04, commit `a1cd9901`). At the bound receipt:

(1) The focused CAS Core v8 suite records *61 passing test methods*, with suite test-ID SHA-256

`E2AF5014DE28DD0702211EF32A2757B99B869A13FAFBB0B8D1B18A50A9F8FEF5`.

(2) The focused suite ran within an integrated three-suite run of *208 test methods* and *272 parameterized subtests* over *54 hash-bound artifacts*.
(3) The claim registry holds *11 claim records*, all Ring 4 and NOT LICENSED.
(4) The review matrix holds *53 review criteria*, of which *20 are open*.
(5) The repository-wide collector was *not clean*: four pre-existing package-path namespace collisions surfaced after 1,200 collected items. Consequently, *no repository-wide verification claim is made*; the receipt covers the focused suites only.

Each line is unpacked below, because each is a different *kind* of statement.

The 61 methods and the suite test-ID hash. The count of 61 is the least informative number on the receipt — test counts are famously gameable and famously incomparable across projects — and it is anchored by the number next to it, which is the informative one. The suite test-ID SHA-256 is a digest computed over the sorted, fully qualified identifiers

of the tests in the focused suite. Its function is to make "the suite passed" a well-defined historical statement: the receipt binds not a count but an exact set, so a later run claiming continuity with this receipt must present the same test-ID digest, and any addition, removal, or renaming of tests changes the digest and visibly breaks the continuity claim. This is the kernel's own medicine applied to its evidence: just as a compatibility receipt binds compiler bytes rather than a compiler name, the suite receipt binds test identities rather than a test count. What the digest does *not* do is certify test quality — 61 trivial assertions would hash as stably as 61 sharp ones. Quality is addressed, to the extent it can be without independent review, by Section 3's account of what the fixtures actually exercise, and finally only by open obligation six.

The integrated run: 208 methods, 272 subtests, 54 hash-bound artifacts. The focused suite did not run alone. The integrated run spans the Gate-4 lane's three suites — Core and its companion-facing surfaces — and the numbers say two distinct things. The 208/272 pair records method-level and parameterization-level granularity separately, because parameterized subtests (the same assertion body over a table of fixtures, heavily used in the encoding and loader rejection catalogs) inflate neither count deceptively: 208 methods exist as code; 272 test outcomes were recorded. The 54 hash-bound artifacts are the input side: source files, registry ledgers, schema files, compiler bytes, and fixture data whose SHA-256 digests are recorded in the run's manifest, so that the receipt's claim is closed under its inputs — *these* tests, over *these* bytes, passed. An artifact drifting after the receipt does not falsify the receipt; it merely, and detectably, ends the receipt's applicability to the present. That is the intended semantics of all bounded evidence in this program: receipts age out; they are never retroactively wrong about their own run.

Eleven claim records, all Ring 4, all NOT LICENSED. This line is about the registry's *content* rather than the software's behavior, and its evidentiary value is its uniformity. The claim registry at the receipt contains no licensed claims, no ring other than 4, and no record whose boundary triple deviates from (OPEN, NOT LICENSED, disclosure false). The kernel's loader enforces that malformed records cannot load; it cannot enforce that the program's humans refrain from *well-formed* overclaims — a syntactically valid record asserting a licensed status would load if its bytes were written. That no such record exists is therefore a fact about program discipline, checkable by anyone who reads the eleven records, and the receipt pins the ledger bytes under which the check holds.

Reading the eleven records. The claim records' content, not just their uniform status, rewards inspection, because the registry demonstrates what Ring-4 claim-keeping looks like in practice. The records fall into three classes. *Mechanism claims* assert that a named kernel mechanism implements its written contract — for example, that the canonical encoder realizes the closed rule set — each bound to the specification digest and the exercising fixture families; these are the claims the Gate-4 receipt is *about*, and even they are registered NOT LICENSED, because tested-by-authors is not reviewed. *Boundary claims* assert what the kernel does *not* do — no authorization path, no v8 semantics, no sandbox — registered as claims precisely so that their negations cannot later be asserted as having been implicit; a program that registers its refusals makes its future overclaims checkable against its past. *Bridge claims* cover the narrow guarantees of the integration surfaces (type-correct composition, marked covers proving nothing), each phrased with its scope limitation inside the claim text itself rather than in surrounding commentary. Nothing in the registry asserts empirical validity, cognitive-scientific content, or product fitness — there is no record whose licensing would even be meaningful under the current evidence, which is itself a discipline exhibit: the registry contains only claims of the kind the program could someday support.

Fifty-three review criteria, twenty open. The review matrix is the program's enumeration of what an independent reviewer would need to examine: per-mechanism specification-to-implementation correspondences, fixture adequacy questions, boundary-language audits, and the obligations of Section 4 decomposed into checkable items. Thirty-three criteria are recorded as addressed *by the authors' own lights*; twenty are open, and the open set includes every criterion that requires independence (no self-review can close it), every criterion blocked on unbuilt infrastructure (hermetic bundle, conformance campaign), and several fixture-coverage questions the authors judged honestly unresolved. Representative members of the open set, to give the ratio texture: whether the injective-rendering argument should be reconstructed machine-checked rather than accepted as a design argument with regression fixtures; whether the encoder's depth and node bounds are set at defensible values or merely at values (a policy criterion, not a code one); whether the lock discipline's constants deserve a contention study on the program's actual hosts; whether the fixture base for cross-namespace links covers all namespace pairs (nine-choose-two under the operative catalog of the identity chapter) or only the historically incident-bearing ones; and the standing criterion that every "by construction" phrase in the specification be traced to the constructor code that makes it true. The 20/53 ratio is not a progress bar and should not be read as "62 percent done"; the open criteria are not smaller than the closed ones, and the closed ones reopen under independent review by design.

The collector that was not clean. The last line of the receipt is its most important discipline exhibit. Running the repository-wide test collector — the step that would justify a claim of the form "the whole repository's suites pass" — surfaced four pre-existing package-path namespace collisions after 1,200 items were collected: directories in the wider monorepo whose package paths alias under the collector's import semantics, a defect of the surrounding repository, not of the Core suites. The honest options were to fix the wider repository (out of scope for a bounded candidate receipt), to exclude the colliding paths and claim the remainder (a silently narrowed claim wearing a broad one's language), or to record the collector as not clean and bound the claim to the focused suites. The receipt does the third. The irony is noted and intended: the failure that bounds the receipt is a *namespace collision* — the exact failure family this kernel exists to make loud. It was loud. The bound is the evidence that the program's evidence discipline operates on itself.

2.1. Re-verification: what a reader can check today. A bounded receipt is only worth its bound if a reader can, in principle, re-perform the binding. The re-verification procedure for Finding 12.1 is mechanical and worth stating as a protocol, both because reviewers will need it and because writing it down exposes exactly which steps depend on trust that has not yet been earned.

Protocol 12.2 (Receipt re-verification). Given the repository at commit `a1cd9901`:

(1) Enumerate the focused suite's test identifiers by collection (without execution), sort them under the declared ordering (the governing sources fix the ordering only as "sorted, fully qualified identifiers"; recording the exact collation in this protocol text is itself an open review item), canonically encode the sorted list, and compute its SHA-256. Compare against `E2AF5014...`: agreement establishes that the test *set* is the receipted one.
(2) Hash the 54 manifest artifacts and compare against the manifest's recorded digests: agreement establishes that the input bytes are the receipted ones.
(3) Execute the focused suite and the integrated run; compare method and subtest counts and pass status.

(4) Load the claim registry through the Core loader; verify it loads cleanly, contains 11 records, and that every record's ring and boundary fields read Ring 4, OPEN, NOT LICENSED.
(5) Run the repository-wide collector; observe that it reproduces the four package-path collisions after 1,200 items, confirming that the receipt's bound was forced, not chosen for convenience.

Two honesty notes on this protocol. Step (3) re-executes on the reader's stack, not the receipt's: agreement is expected but is evidence about the reader's environment as much as the original, and disagreement would be a finding about environmental sensitivity, not necessarily about the receipt (this is obligation 1's shadow falling across even simple re-verification). And the whole protocol verifies *internal* consistency — receipt against repository — not the repository against the world: a reader receives commit `a1cd9901` through channels (version control, filesystem) whose integrity the kernel records but cannot vouch for. Re-verification under these caveats is still far from empty: it reduces the trust surface to "the bytes I was handed," which is exactly the reduction every mechanism in this kernel performs, applied once more at the outermost layer.

CLAIM BOUNDARY 12.3 (What the receipt establishes). Finding 12.1 establishes: that the identified test set, over the identified artifacts, passed on the identified date at the identified commit. It does not establish hermetic execution (open obligation 1), cross-implementation conformance (2), any v8 semantic capability (3), any claim authority (4), multi-file program coverage (5), or independently reviewed correctness (6). It establishes nothing repository-wide. All content remains Ring 4, OPEN, NOT LICENSED.

3. What the suite exercises

A test-ID digest tells a reviewer *which* tests ran; this section tells them what the tests are *about*, mechanism by mechanism, with emphasis on the suite's governing bias: negative fixtures dominate. A fail-closed kernel's specification is mostly a list of refusals, so most of its tests are demonstrations that refusal happens — the suite spends more methods proving that wrong things are rejected than that right things are accepted, and the threat model of Chapter 10 maps onto the fixture classes nearly one-to-one.

Reference identity and injective rendering. Fixtures construct references across all seven specification-level namespace families and assert full-tuple structural equality semantics: equal tuples equal, any single-field perturbation (including `definition_sha256` alone) unequal. Cross-namespace fixtures with identical local identifiers assert inequality (threat T1's mechanical core). The rendering fixtures are the delimiter gauntlet: components containing the join delimiter, empty components, components that are prefixes of one another, and pairs of distinct tuples crafted toward collision, each asserted to render distinctly and to round-trip through the parse (T2). Link fixtures assert that TRANSLATION, ALIGNMENT, and HYPOTHESIS construct and that no operation yields identity from any of them.

Boundary consistency and recheck. Constructive fixtures assert that the Ring-4 triple builds and that every contradictory combination — licensed product under OPEN admission, disclosure true in Ring 4 — fails at construction. Recheck fixtures exercise the derive-and-recheck path: operations receiving stale or mutated boundary context are shown to recompute rather than trust, and self-digest fixtures verify that mutation after construction is detected while asserting (as a tested property, not just prose) that a valid self-digest grants nothing (T7's first layer).

Canonical encoding. The largest parameterized block. Acceptance fixtures pin exact expected bytes — not just stable bytes — for representative structures, so encoder drift against the specification is caught even if it drifts deterministically. Rejection fixtures cover the full catalog: sets and unordered containers, binary floats in every position, NaN, both infinities, untagged scalar ambiguity, cyclic structures, depth-limit and node-limit overflows (T4). The type-pun fixtures encode Decimal("1.0"), the string "1.0", and the integer 1 and assert pairwise distinct byte outputs, with the same pattern over further pun families. Newline fixtures encode fixture files with lone-CR, CRLF, and lone-LF content and assert that raw and comparison digests, and the three counts, are recorded unconflated (T5).

Compatibility protocol. Hash-verification fixtures run the worker against correct and corrupted source and compiler bytes, asserting refusal on any digest mismatch (T6). Sanitization fixtures inspect the worker's recorded runtime for the `-I -S -B` profile and sanitized environment, and assert the runtime-binding fields are populated before-execution as specified. Lock fixtures exercise acquisition, bounded timeout behavior, and the no-retry discipline (T10); failure-taxonomy fixtures feed the worker malformed and empty transport responses and assert fail-closed classification with only exit code, byte counts, and digests recorded; divergence fixtures present mismatched dependency profiles and assert the bounded, hash-valued error shape — both profile hashes, at most 32 keyed difference paths, no cleartext environment values (T11). Replay fixtures assert that the structural-decision path is unreachable without a fresh execution (T9).

Registry loading. The second large parameterized block: one fixture family per rejection — duplicate JSON keys, unknown root fields, unknown record fields, wrong enum values, wrong shapes, invalid identifiers, ambiguous versions, missing required metadata — each asserting load failure with a diagnostic naming the defect (T8). Substitution fixtures attempt the constructor and dataclass-copy routes around the loader and assert rejection; immutability fixtures attempt post-load mutation; digest fixtures assert that selected-record digests change when ledger bytes change, binding records to their registry context.

Authorization invariants. The smallest and most consequential block: fixtures assert that `authorization_granted` and the compatibility spelling `allowed` are *false* across every constructible decision outcome, including outcomes whose every structural field verifies and whose registry input says LICENSED — the tested form of the statement that there is no code path to yes (T7's terminal layer). A static-search check asserts the absence of any assignment of true to either field in the kernel source.

Fixture philosophy. Three commitments govern the fixture base, and stating them lets a reviewer audit the suite against its own standards rather than against generic testing folklore. First, *negative dominance*: for every acceptance family there is at least one rejection family, and the rejection catalogs (encoding, loader) are exercised exhaustively — every enumerated rejection reason has at least one fixture, because an untested refusal is a refusal that can silently stop happening. Second, *byte-pinned oracles*: wherever the specification determines exact output (canonical encodings, rendered identifiers, digests), fixtures compare against pinned byte strings checked into the fixture set, not against the implementation's own round-trip — a suite that asks the encoder whether the encoder is right tests only self-consistency, the failure mode that let deterministic-but-wrong drift pass in earlier program tooling. Third, *adversarial provenance*: a substantial share of the negative fixtures are transcriptions of the threat model's scenarios (delimiter crafting, subclass impersonation, pun families, constructor bypass), so the suite is, in part, the threat model made executable. Table 1 records the correspondence.

TABLE 1. Suite blocks against mechanisms and threats.

Suite block	Mechanism chapter	Threats exercised
Identity and rendering	semantic identity	T1, T2, T3
Boundary	research boundary	T7 (recheck layer)
Canonical encoding	encoding	T4, T5
Compatibility protocol	worker protocol, concurrency	T6, T9, T10, T11
Registry loading	registry integrity	T8
Authorization invariants	decision model	T7 (terminal layer)

Coverage, honestly measured. Line- and branch-coverage percentages are deliberately not headlined on the receipt, and the reason belongs in the record. For a fail-closed kernel, the coverage metric that matters is *rejection coverage*: the fraction of enumerated refusal behaviors — catalog entries, protocol failure classes, invalid boundary combinations — that have at least one fixture demonstrating the refusal, with its diagnostic shape asserted. This metric is countable from the specification (the catalogs are finite and written down), it is the metric the parameterized blocks were built against, and at the receipt it is complete for the encoding and loader catalogs by construction — each catalog entry is a fixture row. The same by-construction completeness is *not* claimed for the metric's third family: the compatibility failure taxonomy (Table 1) and the invalid boundary combinations are exercised by the fixture classes described above, but whether every typed code and every invalid combination has a dedicated fixture is recorded in the run artifacts rather than asserted here, and completing that accounting is among the open review criteria. Conventional statement coverage is recorded in the run artifacts for reviewers but is not cited as evidence here, because in this codebase it is dominated by exactly the code whose correctness the metric cannot speak to: error-message formatting, record field plumbing, and other code that executes identically whether or not its semantics are right. A high statement-coverage number would decorate the receipt without strengthening it; the program's standing rule — aggregate metrics never get to be the audit — applies to its own test evidence first.

REMARK 12.4 (What the suite does not exercise). For symmetry: the suite contains no performance tests (the kernel makes no latency claims), no concurrency stress beyond the lock discipline's specified behavior (no fairness claims exist to test), no fuzzing campaign (open review criterion), and no cross-platform matrix beyond the platforms in the integrated run's recorded environments. Each absence is a review criterion, several among the twenty open.

4. The six open obligations

The obligations below are the candidate's honest distance from trustworthiness. Each is stated, then discussed substantively: why it is open, what closing it requires, and what misreading its openness invites.

4.1. Obligation 1: hermetic subprocess and toolchain bundle. Python-level isolation — the `-I -S -B` worker with sanitized environment — removes the inherited-state failure class but leaves the executing stack itself unverified: the interpreter binary, every native extension and shared library it loads, and the operating-system surface beneath them are recorded by name and version, not content-addressed. The gap is stated in every receipt as `false` capability fields, and the threat model locates the consequence exactly: an attacker

or an upgrade at the native layer defeats the compiler-identity binding at a layer the protocol only describes.

Closing the obligation means a hermetic bundle: an execution environment in which the interpreter, its standard library, all native dependencies, and ideally the OS userland are identified by digest and assembled from those digests at execution time, in the spirit of reproducible-builds discipline [3]. The engineering is well-trodden in other ecosystems (content-addressed store, pinned toolchains), but the obligation is not merely to adopt a tool: the receipt schema must grow fields for the bundle identity, the replay requirement must be extended to demand bundle-identity agreement, and the failure taxonomy must classify bundle mismatches.

Two design questions must be answered under review before implementation, and they are recorded here because their answers shape the receipt schema. *Granularity*: a bundle digest over one opaque image is easy to record and nearly useless to diagnose — a mismatch says "something differs" — while per-component digests (interpreter, each native library, each wheel) make divergence reports as diagnosable as the dependency-profile mechanism already is, at the cost of a manifest schema and a canonical component ordering; the kernel's existing preference for bounded, keyed difference reporting argues for the per-component design. *Windows fidelity*: the program's primary hosts are Windows machines, where content-addressing the loader-visible surface (system DLLs, side-by-side assemblies) is materially harder than on POSIX; a bundle design that is hermetic on Linux and approximate on Windows must say so per-platform in its capability fields rather than reporting the strongest platform's property globally. The honest sequencing note: until this closes, obligation 2's conformance campaign is weaker than it looks, since cross-implementation agreement observed on unverified stacks conflates implementation behavior with environment behavior. The misreading to guard against: "isolation" in this monograph never means sandbox, and no current receipt is evidence against a host-level adversary.

4.2. Obligation 2: Python/Rust conformance. The program's architecture places a Rust engine beside the Python kernel, and the seed specification permits a separately hash-bound native engine profile. None is admitted in v0.1: engine profiles remain separate, and no receipt asserts that the Python and Rust implementations agree on anything. The obligation is a differential-testing campaign: a shared corpus of inputs (encoding structures, reference tuples, registry byte fixtures, compiler sources) executed against both implementations, with canonical outputs compared at the digest level, counterexamples minimized and receipted, and the corpus itself hash-bound and grown adversarially — generative fuzzing biased toward the rejection catalogs, where implementations most plausibly diverge (one accepts what the other refuses).

The reason this is an obligation rather than a task is its acceptance criterion. "The corpus passed" is bounded evidence about the corpus; the conformance claim wanted is universally quantified, and the campaign can only ever approach it. The program's discipline for this is the same as everywhere else: conformance receipts will be bounded statements naming their corpus digest, and the phrase "Python/Rust conformant" without a corpus identity attached will remain unlicensed language in the lane.

The campaign's hardest technical content is not comparison but *specification arbitration*. When the implementations disagree, three verdicts are possible — Python wrong, Rust wrong, specification ambiguous — and the third is the valuable one: every ambiguity found is a place where the written contract underdetermines behavior and where a future third implementation would diverge again. The campaign must therefore feed a specification-erratum

stream, each erratum a reviewed tightening of the contract text with its motivating counterexample attached, and the contract-versioning rules of the integration chapter govern how tightened text propagates to companions. There is also a floating-point-shaped trap to name in advance: the canonical encoder's rejection of binary floats means the conformance corpus never exercises float formatting — by design — but Rust-side arithmetic feeding tagged decimals must be checked for exactness at the construction boundary, since a Rust path that computes in `f64` and converts at the edge would reintroduce, invisibly, the representability drift the encoder exists to exclude. Until the campaign exists, the operative rule is stark: no evidence produced through one implementation may be cited as evidence about the other.

4.3. Obligation 3: no v8 parser or semantic migration. The only admitted compatibility profile is `legacy_only`: it executes the hash-verified legacy Python compiler for comparison purposes and produces no CAS v8 semantic translation. There is no v8 parser, no v8 semantics, and no migration path from v7.7 artifacts to any v8 form — and this is a design decision, not a delay. The kernel was built first precisely so that when v8 semantics arrive, they arrive into an environment where their sources are hash-identified, their evidence encodings are canonical, their claims are registered Ring-4, and their execution is receipted.

The obligation's substance is therefore mostly *criteria*: any future v8 parser must arrive with its grammar as a reviewed artifact, its conformance to the legacy compiler on the shared subset established through the compatibility profile (which exists exactly to make such comparisons receiptable), and its divergences from legacy behavior enumerated as reviewed decisions rather than discovered as surprises.

A word on why migration, specifically, is refused rather than merely deferred. A migration tool is a semantic claim generator: every artifact it converts carries an implicit assertion that the v8 form means what the v7.7 form meant, multiplied across the entire migrated corpus at once. Shipping such a tool before v8 semantics are reviewed would manufacture, in bulk, exactly the unlicensed equivalence claims that Rule-level discipline forbids one at a time — the no-silent-identity rule applied at corpus scale. When migration does arrive, the kernel's shape for it already exists: each conversion should be receipted like a compatibility execution (source identity in, tool identity, target identity out, boundary attached), so that a migrated corpus is a set of auditable derivation records [2] rather than a fait accompli. The misreading to guard against is treating the kernel's existence as progress on v8 semantics. It is progress on being *able to test* v8 semantics; the semantic work — the hard, domain-laden work of saying what v8 notation means — has not begun, and no artifact in this monograph moves it.

4.4. Obligation 4: no claim authority. The constant-false invariant is the kernel's signature, and it is also, read as program planning, a deferred liability: a research program eventually needs to make claims, and the current kernel can only refuse them. The future design's minimum constraints are already stated in the runtime decision model: an exact trusted registry and schema manifest; an authenticated authority and trust root; reviewer and admission receipts; Evidence Packet identities and verification status; claim ID and exact version; and the code, model, environment, semantic, and calibration identities of the requested operation.

What sets this obligation apart is its *difficulty class*. Every other obligation is engineering; this one is institution-building. A trust root is only meaningful if key custody, reviewer authentication, admission process, and revocation are real practices with real people bound to them — the cryptography is the easy tenth of the problem [4]. The threat model is explicit that its current T7 analysis (residual risk zero, because "yes" is unrepresentable) expires the

day this design lands, and that the full laundering threat family must be re-analyzed against the new surface.

The program's specific vulnerability here should be named rather than implied: it is, at present, substantially a single-operator program assisted by AI tooling, and the historical self-hashed licensing receipts were produced inside exactly that configuration. An authorization design whose "authenticated reviewer" could be instantiated by the same operator who requests the claim, or by tooling acting on the operator's behalf, would be ceremony, not authority — constant-false with extra steps, minus the honesty. The minimum-constraints list therefore implies an organizational fact the cryptography cannot supply: the reviewer role must be filled by parties independent of the program before the role's credentials mean anything, which ties this obligation to obligation 6 not just in sequence but in substance. Revocation deserves equal notice: claims, once authorized, must be revocable when their underlying evidence ages out (a receipt superseded, a review withdrawn), so the design is not "grant" but a full lifecycle — grant, cite, re-verify, revoke — and the registry schema must represent all four states without ambiguity. The program's stated posture is that this obligation closes *last* among the mechanism obligations: authorization built before hermetic execution and independent review would authorize on evidence the other obligations say is not yet trustworthy. Until then, the kernel's "no" is not a placeholder; it is the correct answer to every claim-bearing request made of an unreviewed system.

4.5. Obligation 5: no import-DAG manifest. Protocol step (3) rejects `use` imports: a compatibility execution covers exactly one source file, and multi-file CAS programs are outside the admitted profile. The missing piece is an import-DAG manifest: a canonical, hash-bound description of a program's full dependency graph — every imported source identified by digest, the graph's shape recorded, cycles rejected or declared, and the whole manifest itself canonically encoded so that "the program" as an evidence object means the closed set of bytes, not a root file plus whatever the filesystem happened to resolve.

The reason to refuse partial solutions is the threat model's T6: resolving imports at execution time from an unmanifested filesystem reintroduces, at the program level, exactly the identity gap the protocol closed at the compiler level — the receipt would bind the root file's bytes while the semantics depended on unbound neighbors. Rejecting `use` outright, rather than resolving-and-recording opportunistically, was the deliberate v0.1 choice: an opportunistic recorder would have produced receipts whose completeness varied with filesystem state, and a receipt schema with optional completeness is a receipt schema whose consumers must each decide, privately, what a receipt means — the interpretive scatter the kernel exists to end. The manifest design also has one genuinely open semantic question: whether the DAG's identity should bind resolution *order* (making the manifest sensitive to traversal, hence stricter) or only the digest *set and edge relation* (making semantically irrelevant orderings receipt-equivalent). The kernel's general bias toward stricter identity says bind the order; the conformance obligation's interest in cross-implementation stability says bind the relation; the tension is recorded for review rather than resolved here. The obligation interacts with obligation 1 (a manifest of source digests still executes on an unverified stack) and with obligation 3 (a v8 parser will need module semantics, and the manifest schema should be designed once, for both). Until it closes, the honest statement is that compatibility evidence exists for single-file programs only, and the suite's fixtures respect that bound.

4.6. Obligation 6: independent review. The last obligation subsumes the rest: independent software review, formal-methods review of the specifications and their invariant

arguments, semantics review of the CAS content the kernel will eventually carry, and product-safety review of any use beyond research — all before any status change. Twenty open criteria in the review matrix are the concrete docket, and several closed ones are closed only self-attestedly and reopen under independence.

Two design features exist specifically to make this obligation dischargeable rather than aspirational. First, everything reviewable is an artifact: specifications, source, fixtures, receipts, ledgers, and this monograph are byte-identified, so a reviewer can bind their findings to exact digests and their review becomes, itself, a receipt in the program's sense — which is the shape the future trust root (obligation 4) will need review evidence to take. Second, the strict dependency direction of the integration chapter caps the review surface: Core is reviewable in isolation, and its review does not transitively obligate a reviewer to the companions' domain content.

The four review disciplines are not interchangeable, and conflating them is itself a review failure to preempt. *Software review* asks whether the implementation matches the contracts and the fixtures exercise them — the twenty open criteria's largest share. *Formal-methods review* asks whether the invariant arguments (injective rendering, encoding determinism, loader completeness against its rejection catalog) actually hold as stated, ideally by independent reconstruction or machine-checked formulation; the monograph's design arguments are proofs of intent, not verified proofs. *Semantics review* concerns the CAS and TKS content the kernel will eventually carry and cannot begin in earnest until obligation 3 produces reviewable semantic artifacts; its present scope is confined to checking that the kernel makes no semantic commitments prematurely — a review of absences. *Product-safety review* is contingent: it activates only if any use beyond research is ever proposed, and the boundary fields exist precisely so that its trigger condition is observable. A reviewer engaged for one discipline should be told which one, in these terms. What the program cannot manufacture is the independence itself; no internal process substitutes, and the boundary language of every artifact — DRAFT / OPEN / RING 4 / NOT LICENSED / HELD — remains in force until it arrives.

5. Program: sequencing the obligations

The obligations are not a list but a partial order, and stating the order is part of the evidence program. Obligation 1 (hermetic bundle) is upstream of the strong form of 2 (conformance observed on verified stacks) and strengthens every receipt retroactively issued after it. Obligation 5 (import-DAG manifest) is upstream of any serious use of 3 (a v8 parser tested only on single files would be tested on toys). Obligation 4 (authority) is explicitly sequenced last among mechanisms, for the reason given in its discussion. Obligation 6 (independent review) is not a stage but a gate crossing all of them: each closed mechanism obligation produces artifacts that enter review, and no closure is final until reviewed. The program's near-term work is therefore 1 and 5 in parallel — both are bounded engineering with receipt-schema consequences that should be designed together — with 2's corpus construction beginning against the current stacks (weak-form conformance now, upgraded to strong form when 1 lands), and 3 and 4 deliberately idle until their prerequisites exist.

This ordering is itself Ring-4 planning content: it binds no one, and independent review (which may reorder it) outranks it. It is recorded so that the program's future choices can be audited against its present reasoning — the same function every other artifact in the lane serves.

Program risks. A program section that lists only tasks is planning theater; the risks to the program deserve equal standing. Three are material. *Erosion under friction*: the threat model's compound analysis already identified the pattern — fail-closed refusals generate operational friction, and friction generates pressure to relax gates; the mitigation is structural (gates are protocol, not policy) but the pressure will fall on the review process that approves Core revisions, which is exactly where obligation 6's independence matters. *Single-maintainer fragility*: the same organizational fact that complicates obligation 4 threatens continuity generally; the mitigation on record is that every artifact is byte-identified and self-describing enough for a successor or reviewer to reconstruct state from the repository alone — a property this monograph is itself intended to serve. *Scope seduction*: the kernel's discipline makes it tempting to route ever more program content through it, and each new consumer arrives with pressure to widen contracts (new encoder types, new namespaces); the versioning rules make widening a review event, but the cumulative drift of many small reviewed widenings is a risk no single review sees, and the periodic whole-surface re-review implied by obligation 6 is the only standing answer. None of these risks is retired by any receipt; they are properties of the program, not the software, and are recorded here so their management can be audited like everything else.

6. Related work

The kernel's mechanisms adapt established practice rather than claiming novelty, and the honest positioning is per-mechanism. The canonical evidence encoding is in the tradition of JSON canonicalization for stable digests, RFC 8785 being the nearest reference point [1]; the kernel's departures — rejecting floats outright rather than canonicalizing their formatting, tagging exact decimals, refusing unordered containers instead of sorting them — trade generality for a smaller argument surface, which is the correct trade for an evidence encoder and would be the wrong one for an interchange format. The identity and receipt discipline — content-addressing sources, toolchains, and artifacts; treating a build or run as trustworthy only relative to pinned input digests — is the reproducible-builds program [3] applied to a research kernel, with the hermetic gap between the two stated as obligation 1 rather than elided. The separation of entity, activity, and derivation in receipts and traces follows the provenance-modeling tradition standardized in PROV [2], restricted here to flat, canonically encoded records because the kernel refuses graph semantics it cannot yet bound. The threat-model method is Shostack's [5]; the governing security-engineering stance — integrity mechanisms are not authorization, trust roots are institutional before they are cryptographic, systems fail at seams — is Anderson's [4], and the constant-false authorization invariant can be read as that stance taken to its fixed point: when no adequate trust root exists, the correct implementation of authorization is the constant function *no*.

Three contrasts sharpen the positioning. Against *interchange canonicalization*: RFC 8785 exists so that I-JSON documents produced anywhere can be canonically compared, and it therefore must handle the whole I-JSON value space, whose numbers are IEEE doubles [1]; the kernel's encoder serves the narrower purpose of digesting evidence it also *produces*, and can therefore shrink the value space to what admits an exactness argument — the two designs are correct for their respective problems, and importing either into the other's setting would be a defect. Against *supply-chain attestation practice*: the reproducible-builds movement's central lesson is that verifiability must be designed in rather than audited on [3], and its central sociological observation — that reproducibility claims decay unless continuously re-exercised — is the direct ancestor of the kernel's replay requirement; where

the kernel departs is at the attestation layer itself: current practice standardizes signed, machine-readable provenance statements — in-toto attestations carrying SLSA provenance, transparency-logged and now emitted by major package registries via Sigstore [6, 7, 8] — whose stated model makes them inputs to downstream verification policy; the program's history shows attestation-shaped artifacts being read as authority rather than verified (threat T7), so the kernel refuses attestation documents as endpoints altogether. Against *provenance modeling*: PROV provides a graph vocabulary rich enough to express agency, delegation, and derivation chains [2]; the kernel deliberately uses only its flattest fragment — which entity, which activity, which inputs — because agency and delegation are exactly the notions that must wait for an authenticated trust root, and a vocabulary that can express "authorized by" before the system can verify it is a laundering surface. In each contrast the kernel's choice is the narrower one, and the narrowness is the point.

Against this background, the kernel's contribution is compositional and contextual: a fail-closed assembly of these known disciplines into one substrate, for a research program with *documented* drift and overclaim incidents, with the claim discipline enforced in the object language (unrepresentable states, constant-false fields) rather than in surrounding process documents alone. Whether that composition is itself sound is precisely what open obligation 6 exists to determine.

7. Conclusion: the design inversion

The thesis of this monograph can now be restated with all of its supporting structure in view.

Conventional practice builds capable systems and then subtracts danger: add semantics, then validate; add authority, then audit; add integration, then review. The program's own history is a record of what that ordering costs when the system's outputs are claims — positional correspondences hardened into identities before anyone checked, and structurally perfect receipts read as licenses because nothing in their design refused the reading. CAS Core 8 inverts the order. It begins with a kernel in which the historically observed failures are unrepresentable or fail closed — identity is structural and namespaced, correspondences carry their kinds, encodings are closed and deterministic, byte identities are dual and unconflated, execution is hash-bound and replayed, registries are byte-backed, and authorization is the constant *no* — and it requires everything a future CAS v8 wants to assert to be *added*, through gates that are themselves reviewed: a catalog for every digit, a manifest for every import, a trust root for every claim, a reviewer for every closure.

The evidence for the inversion's viability is deliberately modest and exactly bounded: 61 focused methods under one test-ID digest, inside a 208-method, 272-subtest, 54-artifact run; eleven claims, all Ring 4; twenty criteria open; one collector honestly not clean. The evidence for the inversion's *necessity* is the program's incident history, and the evidence for its *character* is distributed through every mechanism chapter: at each design fork, the kernel chose the option that made a failure loud over the option that made a workflow smooth.

The inversion has a cost model, and honesty requires stating it. Fail-closed kernels are slower to build against, hostile to prototyping, and indifferent to deadlines; every companion candidate paid integration friction that a permissive substrate would have waived, and the six obligations exist *because* the kernel refuses to paper over them — a permissive design would simply not have surfaced most of them as obligations at all. The wager of the whole program is that this cost is the cheap side of the ledger: that for software whose product is claims, the expensive failure is not the blocked workflow but the plausible artifact — the conflation that

survives review because nothing typed it, the receipt that becomes a license because nothing refused the reading. The program has paid the expensive failure before. The kernel is what it bought with the lesson: subtraction replaced by gated addition, and a substrate that will keep saying *no* until closing its obligations earns it the right to say anything else.

Claim Boundary 12.5 (Final status statement). This monograph documents a Ring-4 research candidate: admission OPEN, product NOT LICENSED, public disclosure withheld. Its receipts are historical software evidence under exact fixtures and artifacts. Nothing herein is an experiment, a certification, a review, a product representation, or a licensed claim, and no reading of this document's completeness, internal consistency, or test evidence converts it into one.

APPENDIX A

Reference Tables

This appendix collects, in tabular form, the field inventories, valid combinations, rule catalogs, and enumerations that the mechanism chapters introduced in prose. The tables are a convenience for implementers and reviewers, not an independent authority: where a table and a mechanism chapter's normative text could be read to differ, the chapter governs, and the difference is a defect in this appendix to be reported. All content is Ring-4 candidate material at the bounded Gate-4 receipt; field names are the v0.1 names and change only through Core revision under review.

Three usage notes. First, the tables are designed to be checked, not only read: every "rationale" cell names the failure the field or rule exists to block, so a reviewer can audit each row by asking whether the named failure is in fact unrepresentable or rejected in the implementation — the row-by-row form of the threat model's coverage question. Second, the rejection-side tables (encoding rules, loader catalog, failure taxonomy) are *closed lists* in the specification's sense: the implementation rejects what the rows say and accepts the complement within its declared domain, so a behavior found in the software that corresponds to no row — an undocumented rejection as much as an undocumented acceptance — is a conformance defect even if it seems benign. Third, where a table row summarizes a numeric constant (lock bounds, path limits), the constant is a declared policy value, not a derived optimum; the review criteria tracking whether those constants deserve empirical study are among the open items recorded in the evidence chapter.

1. Semantic-reference fields

Table 1 lists the five fields of the semantic-reference tuple. Identity is structural equality of the full tuple; no subset of fields, and no rendered string, substitutes for it. The exact-type discipline applies: a reference is the exact kernel type, not a subclass, and each field is the exact primitive type named below.

The `namespace` enumeration, exhaustively: TKS-FOUNDATION, TKS-WORLD, TKS-NOETIC, CAS-DRIVE, CAS-DOMAIN, CAS-REGISTER, CAS-NOETIC, CAS-STATE, NOETIC-TOPOS-OBJECT. The specification's prose names seven families; the operative enumeration refines the Noetic family into TKS-side and CAS-side namespaces (Chapter 3). The families are pairwise disjoint by declaration; no namespace is a subset, alias, or refinement of another, and no coercion between namespaces exists.

Cross-namespace links carry exactly one `relation` from the enumeration IDENTITY, TRANSLATION, ALIGNMENT, HYPOTHESIS, plus their own grounding-artifact digest and mandatory note. The constructor permits IDENTITY only when source and target are the exact same versioned reference, so every cross-namespace link carries a non-identity kind — the exact-match guard is the mechanism. Table 2 inventories the link fields for completeness.

A reading example ties the two tables together. The historically incident-bearing pair — a TKS Foundation position and the positionally corresponding CAS Drive — is representable

TABLE 1. Semantic-reference fields.

Field	Type	Rationale
`namespace`	enum (9 values)	Places the referent in exactly one ontological family; makes cross-family equality structurally false, blocking conflation (threat T1) at the type level rather than by convention.
`local_id`	string (identifier profile)	The referent's name within its namespace. Confined to the declared normalized/ASCII identifier profile so that comparison never depends on Unicode equivalence classes; prose names live elsewhere.
`version`	string (canonical text)	Distinguishes revisions of the same-named referent; follows the source's own versioning conventions and is not confined to the identifier profile. Unversioned reference is unrepresentable; "the current one" is not an identity.
`source_id`	string (identifier profile)	Names the defining source document or corpus. Binds the reference to a provenance context, so two definitions of one name in different sources are two references.
`definition_sha256`	string (64 hex, normalized uppercase)	Digest of the exact defining bytes in the identified source. Makes definitional drift observable at the identity layer: if the defining text changes, the reference changes.

exactly once and exactly honestly: two references (namespaces TKS-FOUNDATION and CAS-DRIVE, each with its own `definition_sha256`) joined by a link of kind TRANSLATION or ALIGNMENT citing the translation-key document, at whatever review state that document holds. Every stronger representation is unconstructible; every weaker one (bare strings) is outside the kernel and outside its protection.

2. Boundary fields and valid combinations

Table 3 lists the three boundary fields; Table 4 gives the combination rules for Ring-4 candidate material. The boundary is validated jointly at construction — there is no partially valid boundary — and every public operation derives and rechecks it from inputs.

Reading note: the single valid row is not a default that other rows override; it is the only constructible state. The invalid rows fail at the constructor with a diagnostic naming the contradiction, and no operation accepts an unvalidated triple. A self-digest field accompanies the boundary for post-construction tamper evidence; it appears in no validity rule because it grants nothing.

TABLE 2. Cross-namespace link fields.

Field	Type	Rationale
`relation`	enum (4 values)	Grades the strength of the asserted correspondence; IDENTITY is constructible only as a degenerate self-link, so the maximum representable cross-namespace strength is TRANSLATION, which is still not identity.
source reference	semantic reference	The link's domain endpoint, as a full structural tuple.
target reference	semantic reference	The link's codomain endpoint. Endpoints in the same namespace are a construction error: links exist for crossings.
`crosswalk_id`	string (identifier profile)	Names the link record itself, so every asserted correspondence is citable as an artifact.
`source_artifact_sha256`	string (64 hex, normalized uppercase)	Names, by digest, the artifact on whose authority the correspondence is asserted, so every link is traceable to a citable origin rather than to tooling convenience.
`note`	canonical text (mandatory, non-blank)	Human-readable justification of the asserted correspondence; an undocumented link is a construction error.
boundary	Ring-4 triple (exact)	Pinned at construction to admission OPEN, product NOT LICENSED, disclosure false; a crosswalk cannot present itself as licensed or disclosable. Review standing is tracked at the registry layer, not on the link.

TABLE 3. Research-boundary fields.

Field	Type	Rationale
admission state	enum (incl. OPEN)	Records where the artifact stands in the program's admission process. OPEN = under research, not admitted to any further status.
product state	enum (incl. NOT LICENSED)	Records legal/product standing. NOT LICENSED is the only value constructible for Ring-4 material in v0.1.
public disclosure	boolean	Records whether the artifact is designated for disclosure beyond the program. False throughout Ring 4.

TABLE 4. Boundary combinations for Ring-4 material.

Admission	Product	Disclosure	Verdict at construction
OPEN	NOT LICENSED	false	valid (the Ring-4 triple)
OPEN	licensed value	any	invalid: contradiction
OPEN	NOT LICENSED	true	invalid: disclosure in Ring 4
non-OPEN	any	any	invalid for Ring-4 material: Ring-4 closure requires OPEN (non-Ring-

3. Canonical-encoding rules

Table 5 restates the encoder's closed rule set with accept/reject examples. "Reject" always means: canonicalization fails with a diagnostic naming the offending path; nothing is coerced, rounded, sorted-into-acceptability, or dropped.

Table 5: Canonical-encoding rules with examples.

Rule	Accepted example	Rejected example
UTF-8 JSON, sorted string keys, deterministic bytes	mapping with keys `"a"`, `"b"` encodes with `"a"` first regardless of insertion order	mapping with a non-string key; any encoding whose byte output depends on iteration order
No unordered containers	list `[1, 2, 3]` (order is the caller's declared order)	any set value, however small; acceptance would force an undeclared ordering
No binary floats	tagged decimal from Decimal(`"0.1"`)	the float `0.1`; also `1e308`, `-0.0`
No NaN or infinities	— (no accepted spelling exists)	`NaN`, `+inf`, `-inf` in any position
Tagged exact decimals; no type puns	Decimal(`"1.0"`), string `"1.0"`, integer `1` all encode, with pairwise distinct bytes	any encoder configuration in which two of the three share an encoding
One serialization route per public type	a receipt encoded via its single declared route	a second ad hoc dict-dump of the same receipt type; route multiplicity is a defect, not a convenience
Bounded depth and node count	structures within declared limits	any structure exceeding declared depth or node limits; failure names the limit
Cycle-aware; no cycles	finite trees and DAGs within bounds (DAG sharing encoded by declared expansion)	any self-referential structure; failure, not truncation

The byte-identity discipline sits beside the encoder and is tabulated with it for convenience: for any tracked file, the record carries `raw_sha256` (digest of exact bytes), `lf_sha256` (digest after declared CRLF-to-LF comparison normalization), and the three counts `crlf_count`, `lone_cr_count`, `lone_lf_count`. No other normalizations exist; neither digest substitutes for the other.

4. Compatibility-receipt fields

Table 6 inventories the fields of a compatibility receipt, grouped by protocol concern. Every field is populated on every successful execution; the failure taxonomy (Section 5) governs what is recorded instead when a worker fails.

Table 6: Receipt fields of the compatibility protocol.

Field group	Contents and role
Source identity	Requested source digest, verified before execution; the execution is *about* these bytes and refuses others.
Compiler identity	Raw and LF digests of the compiler bytes, read and hashed before execution; newline counts per the byte-identity discipline.
Engine and policy profile	The forced Python-only engine profile and exact frozen policy identifiers; syntax and semantic profile identifiers. No native profile exists in v0.1.
Worker runtime binding	Interpreter fields, `sys.platform` or POSIX `os.uname` data, profile-bound Windows architecture inputs; sampled before compiler code executes; no WMI or subprocess probing.
Worker environment	Interpreter and worker bytes identification, import search paths, loaded-module inventory — the sanitized `-I -S -B` environment as actually observed.
Result	Digest of the canonical result; diagnostics stream.
Concurrency record	Lock acquisition outcome under the 90-second bound, 10 ms poll, 10 ms post-release yield discipline; no fairness assertion.
Capability fields	Explicit *false* flags: hermetic execution false, OS-level sandbox false. Present so that no consumer can honestly over-read the receipt.
Research boundary	The full Ring-4 triple, rechecked at receipt construction.
Replay linkage	Identities required for fresh replay; a structural decision consumes agreement between this receipt and a fresh execution, never this receipt alone.

5. Worker-failure taxonomy

For failed executions the receipt is replaced by a failure record whose information content is deliberately bounded: exit code; stdout byte count and SHA-256; stderr byte count and SHA-256. An empty or malformed transport response fails closed and is never retried. Dependency-profile divergence between two successful workers produces the bounded divergence record: both profile hashes; at most 32 keyed difference paths; environment values and dependency paths represented only by hashes. No failure class records cleartext environment content.

6. Registry-loader rejection catalog

Table 7 is the loader's rejection catalog. Each row is exercised by at least one fixture family in the Gate-4 suite; a bytes-level input matching any row does not load, and there is no lenient mode.

Table 7: Rejection catalog of the registry loader.

Rejection	Meaning and rationale
Duplicate JSON keys	The bytes admit two parses depending on which duplicate wins; ambiguity at the byte layer is never resolved by preference.
Unknown root fields	The ledger asserts structure the schema does not define; silently ignoring it would let content ride outside validation.
Unknown record fields	As above, per record.
Wrong enum values	A field carries a value outside its declared enumeration (e.g. an undefined ring or state).
Wrong shapes	Structural mismatch: list where object required, missing nesting, wrong arity.
Invalid identifiers	An identifier field violates the declared normalized/ASCII profile.
Ambiguous versions	Version fields that collide or cannot be totally resolved within the ledger.
Missing required metadata	A record lacks a field the contract marks required; absence is not defaulted.

Complementing the catalog, three loader guarantees from the mechanism chapter belong in any implementer's checklist: parses are reconstructed from and rechecked against retained raw ledger and schema bytes (no constructor or dataclass-copy route can pair old raw identity with new content); loaded values are recursively immutable; and each selected record carries a deterministic digest bound to the registry bytes, not merely to its own fields.

7. Decision-model enumerations

Table 8 collects the runtime decision model's enumerated dimensions and constant fields. There is no stringly-typed action path: an action is describable only in these enums, and an indescribable action is not executable through the kernel.

Table 8. Decision-model enumerations and constants.

Dimension	Values and notes
Action operation	Closed enum of exactly three members: `LEGACY_COMPATIBILITY_EVALUATION`, `MODEL_INFERENCE`, `PUBLISH_CLAIM`; only the first can appear in a passing v0.1 decision, and the set widens only by Core revision.
Action target	Closed enum of exactly three members: `LOCAL_RING4_RESEARCH_ARTIFACT`, `EXTERNAL_SYSTEM`, `PUBLIC_CLAIM_SURFACE`; only the first is admissible in a passing v0.1 decision.
Effect class	Closed enum of exactly three members: `PURE_LOCAL_COMPUTE`, `CLAIM_BEARING`, `EXTERNAL_SIDE_EFFECT`; the nonclaim/claim-bearing separation is the load-bearing distinction of the model.
Structural verdict	A nonclaim replay comparison may pass structurally when every exact compatibility field and digest validates under fresh replay; otherwise fail.
`authorization_granted`	Constant *false* in v0.1; no code path assigns true.
`allowed` (compatibility spelling)	Constant *false* in v0.1; same invariant, legacy field name.

The two constant-false rows are the appendix's summary of the kernel's thesis, and they are the two rows a future revision would touch last, under the future-authorization design constraints: exact trusted registry/schema manifest; authenticated authority and trust root; reviewer and admission receipts; Evidence Packet identities and verification status; claim ID and exact version; and code, model, environment, semantic, and calibration identities of the requested operation. Until a reviewed design binds all of these, every table in this appendix describes a kernel whose answer to claim-bearing intent is *no*.

APPENDIX B

Glossary

Terms are listed alphabetically. Each entry gives the v0.1 meaning as used in this monograph and a plain-text pointer to the chapter where the term is treated normatively; where a term is defined in one chapter and stressed in another, both are named. Definitions here are summaries for orientation — the named chapter's text governs. All defined artifacts and mechanisms are Ring-4 research candidate material.

Advisory lock.: The profile-bound lock through which cooperating compatibility workers serialize execution: 90-second acquisition bound, 10-millisecond poll interval, 10-millisecond post-release scheduler yield, no retry of a failed worker, no fairness claim. Binds only processes that cooperate. (Chapters: Concurrency and the Failure Taxonomy; Threat Model, lock-based denial.)

Alignment link.: One of the three non-identity link kinds; records a documented structural correspondence between references in different namespaces, weaker than identity and stronger than a hypothesis. Carries its own grounding-artifact digest and mandatory note. (Chapters: Semantic Identity; Integration Contracts.)

Ambient evidence.: Evidence in its local, context-bound form: a record true of the host, profile, time, and boundary it names, with no cross-context force. All Core records are ambient; resolution is future topos-side work. (Chapter: Integration Contracts, Noetic Topos section.)

Admission state.: First field of the research boundary; records where an artifact stands in the program's admission process. OPEN is the only Ring-4 value. (Chapter: The Research Boundary.)

`allowed`.: The compatibility protocol's legacy spelling of the authorization field; constant false in v0.1, jointly with `authorization_granted`. (Chapter: Runtime Decision Model.)

`authorization_granted`.: The decision model's authorization field. Constant false in v0.1; no code path assigns true; claim-bearing intent is denied even when a locally constructed ledger reads LICENSED. (Chapters: Runtime Decision Model; Evidence, Obligations, and Program.)

Boundary (research boundary).: The jointly validated triple (admission state, product state, public disclosure). Ring 4 requires OPEN, NOT LICENSED, disclosure false; contradictory combinations are invalid at construction; every public operation derives and rechecks it. (Chapter: The Research Boundary.)

Bounded canonicalization.: The property that canonical encoding fails closed on structures exceeding declared depth or node limits and on any cycle, guaranteeing termination and bounded resource use. (Chapter: Canonical Evidence Encoding.)

Byte-backed loading.: The registry discipline under which parsed headers, schemas, and records are reconstructed from and rechecked against retained raw ledger and schema bytes, so no constructor path can pair an old raw identity with substituted parsed content. (Chapters: Registry Integrity; Threat Model, registry substitution.)

Byte identity.: The dual identity recorded for tracked files: raw digest of exact bytes plus the CRLF-to-LF comparison digest, with lone-CR, CRLF, and lone-LF counts kept unconflated. Neither digest substitutes for the other. (Chapters: Canonical Evidence Encoding; Threat Model, newline rewriting.)

Capability fields.: Explicit false-valued fields in every compatibility receipt (hermetic execution false; OS-level sandbox false) that state the protocol's limitations inside the artifact itself, so no consumer can honestly over-read it. (Chapters: Worker Protocol; Evidence, Obligations, and Program.)

CAS-Ohm 1.0.: Companion candidate for measurement and instrumentation content: typed electrical quantities, calibration profiles, fail-closed research packets. Mirrors Core's authorization split as completeness versus eligibility. (Chapter: Integration Contracts.)

CAS PCBA 8.: Companion candidate for electronics-engineering content: board model, design-rule checks, manufacturing and test traces. The heaviest consumer of the Core contract surface. (Chapter: Integration Contracts.)

Claim-bearing intent.: The effect class of an action that would assert, promote, or rely on a claim's status. Always denied in v0.1, regardless of structural evidence. Contrast: nonclaim research evidence. (Chapter: Runtime Decision Model.)

Claim record / claim registry.: A registry record registering a claim's exact statement, ring, and boundary; the registry is the byte-backed ledger of such records. At the Gate-4 receipt: eleven records, all Ring 4, all NOT LICENSED. (Chapters: Registry Integrity; Evidence, Obligations, and Program.)

Comparison identity.: The declared CRLF-to-LF normalized identity of a file, recorded beside (never instead of) its raw identity. The only normalization the kernel declares. (Chapter: Canonical Evidence Encoding.)

Compatibility profile.: The configuration under which legacy execution is admitted. The only v0.1 profile is `legacy_only`: it binds the exact active Python CAS-notation-compiler bytes and produces no v8 semantic translation. (Chapter: Worker Protocol.)

Compatibility receipt.: The record produced by a deterministic-closure execution: source and compiler identities, engine and policy profiles, worker runtime binding, environment inventory, result digest, concurrency record, capability fields, boundary, replay linkage. Historical software evidence, never authority. (Chapters: Worker Protocol; Reference Tables.)

Completeness versus eligibility.: CAS-Ohm's mirror of the authorization split: a packet is complete when structurally whole, eligible only under external authenticated status; completeness never implies eligibility, and eligibility is constant false in v1.0. (Chapter: Integration Contracts.)

Conformance campaign.: The open differential-testing program required before Python/Rust agreement may be asserted: shared hash-bound corpus, digest-level comparison, receipted results, specification-erratum stream. Open obligation two. (Chapter: Evidence, Obligations, and Program.)

Constant-false invariant.: The design stance that `authorization_granted` and `allowed` are false on every path in v0.1; the kernel's terminal defense against receipt laundering. (Chapters: Runtime Decision Model; Threat Model.)

Contract surface.: The eight consumable contracts of Core v0.1 (semantic identity; exact-type discipline; boundary; canonical encoding; byte identity; compatibility receipts and bridge; registry integrity; decision model). Companions consume subsets; none may extend a contract in place. (Chapter: Integration Contracts.)

Decision record.: Output of the runtime decision model: enum-typed action description plus verdict. A structural pass requires fresh replay agreement; claim-bearing intent is denied. (Chapter: Runtime Decision Model.)

`definition_sha256`.: Fifth field of a semantic reference: the digest of the exact defining bytes in the identified source, making definitional drift observable at the identity layer. (Chapter: Semantic Identity.)

Dependency direction (strict).: The integration invariant: companions depend on Core contracts; Core depends on no companion semantics. Grounds the reviewability of the kernel in isolation. (Chapter: Integration Contracts.)

Dependency profile.: The hashed description of a worker's effective environment (module inventory, search paths, platform inputs) used to detect divergence between workers. (Chapters: Concurrency and the Failure Taxonomy; Threat Model, divergence.)

Deterministic closure.: The eight-step compatibility protocol: verify the source hash and hash the compiler bytes before execution; force engine profile and frozen policy; reject `use` imports; execute in a fresh sanitized worker; bind worker runtime pre-execution; serialize via the advisory lock; record identities and boundary; require fresh replay before structural decisions. (Chapter: Worker Protocol.)

Digest.: A SHA-256 hash over canonical or raw bytes, used as identity for comparison. Digest equality is treated as byte equality under the stated cryptographic assumption. (Chapter: Canonical Evidence Encoding.)

Divergence record.: The bounded error produced when two successful workers disagree on dependency profiles: both profile hashes, at most 32 keyed difference paths, environment values and paths represented only by hashes. (Chapter: Concurrency and the Failure Taxonomy.)

Effect class.: Decision-model enum separating nonclaim research evidence from claim-bearing effect; the model's load-bearing distinction. (Chapter: Runtime Decision Model.)

Exact-type identity.: The rule that identity-bearing public values compare by exact runtime type and primitive fields; equality-overriding subclasses are rejected at public interfaces. Blocks equality spoofing. (Chapters: Semantic Identity; Threat Model.)

Fail-closed.: The kernel-wide posture that irregular, ambiguous, or out-of-bounds input produces an explicit failure rather than a repaired, defaulted, or partial result. (All mechanism chapters; named throughout.)

Finite arrow (`FiniteArrow`).: A typed value with explicit source and target objects drawn from finite inventories; the unit of the fractal bridge's reference kernel. (Chapter: Integration Contracts, TKS section.)

Fractal bridge.: The contract mapping legacy fractal digit sequences to typed arrows via a caller-supplied catalog, composing via semigroupoid paths, failing closed on missing digits or source/target mismatch; proves type-correct composition only, never digit meaning. (Chapter: Integration Contracts.)

Gate 4.: The program's engineering review gate under which the v0.1 candidate and its companions were receipted. The bounded Gate-4 receipt (2026-08-04, commit

a1cd9901) is the monograph's sole test evidence. (Chapter: Evidence, Obligations, and Program.)

Hash-bound artifact.: An input file (source, ledger, schema, compiler bytes, fixture) whose SHA-256 is recorded in a run's manifest, closing the receipt's claim under its inputs. Fifty-four at the Gate-4 receipt. (Chapter: Evidence, Obligations, and Program.)

Hermetic bundle.: The open infrastructure of obligation one: an execution environment whose interpreter, native libraries, and toolchain are content-addressed and assembled by digest. Its absence is why isolation is not a sandbox. (Chapters: Worker Protocol; Evidence, Obligations, and Program.)

Hypothesis link.: The weakest link kind: a conjectured correspondence between references, recorded for investigation, carrying no reviewed standing. (Chapter: Semantic Identity.)

Identifier profile.: The declared normalized/ASCII profile to which review-visible machine identifiers are confined; unrestricted prose Unicode lives in separate fields and never participates in identifier comparison. (Chapter: Canonical Evidence Encoding.)

Import-DAG manifest.: The open design of obligation five: a canonical, hash-bound description of a multi-file program's full dependency graph. Until admitted, `use` imports are rejected and compatibility evidence covers single files only. (Chapters: Worker Protocol; Evidence, Obligations, and Program.)

Injective rendering.: The invariant that the rendered string form of a semantic reference is injective on structural tuples: components are escaped or length-prefixed before joining, so delimiter-bearing components cannot alias distinct tuples. (Chapters: Semantic Identity; Threat Model, identifier collision.)

Integrated run.: The three-suite Gate-4 execution within which the focused Core suite ran: 208 test methods, 272 parameterized subtests, 54 hash-bound artifacts. (Chapter: Evidence, Obligations, and Program.)

`legacy_only`.: The explicit marker of the sole admitted compatibility profile: verified legacy compiler execution for comparison purposes, no CAS v8 semantic translation. (Chapter: Worker Protocol.)

Link kind.: The enum over crosswalk relations: IDENTITY, TRANSLATION, ALIGNMENT, HYPOTHESIS. IDENTITY is constructible only as a degenerate self-link (the exact same versioned reference at both ends), so every cross-namespace link carries a non-identity kind; no coercion path from any other kind to identity exists. (Chapter: Semantic Identity.)

Marked cover.: A Noetic Topos 2.0 artifact declaring an intended covering family of contexts for a target context. A declaration only: it proves no measurement true, and no operation derives a resolved statement from it. (Chapter: Integration Contracts.)

Namespace.: One of the disjoint ontological families of the identity layer. The specification names seven families (TKS Foundation, TKS World, CAS Drive, CAS Domain, CAS Register, CAS State, Noetic Topos object); the operative enumeration has nine members, the Noetic family carrying separate namespaces on the TKS and CAS sides. Disjointness makes cross-family conflation structurally false. (Chapter: Semantic Identity.)

Namespace collision (package-path).: The distinct, repository-level failure that bounded the Gate-4 receipt: four pre-existing collisions among package paths in the

wider monorepo surfaced by the repository-wide collector after 1,200 items, precluding any repository-wide claim. (Chapter: Evidence, Obligations, and Program.)

No-silent-identity rule.: The rule that positional or historical correspondences across namespaces are typed links, never identities. The kernel's answer to the program's documented conflation incidents. (Chapters: Semantic Identity; Threat Model, namespace conflation.)

Noetic Topos 2.0.: Companion candidate carrying the topos-theoretic layer; contributes the ambient-versus-resolved distinction and the marked-cover discipline. (Chapter: Integration Contracts.)

Nonclaim research evidence.: The effect class that may pass a structural replay comparison when every exact compatibility field and digest validates. The only passable class in v0.1. (Chapter: Runtime Decision Model.)

Open obligation.: One of the six stated gaps between the v0.1 candidate and any status change: hermetic bundle; Python/Rust conformance; v8 parser and migration; claim authority; import-DAG manifest; independent review. (Chapter: Evidence, Obligations, and Program.)

Parameterized subtest.: A test outcome generated by running one assertion body over a fixture-table row; counted separately (272) from test methods (208) in the integrated run so that neither count inflates the other. (Chapter: Evidence, Obligations, and Program.)

Product state.: Second field of the research boundary; NOT LICENSED is the only value constructible for Ring-4 material in v0.1. (Chapter: The Research Boundary.)

Receipt.: Generically: a historical, digest-bound record that a stated software behavior occurred over stated inputs on a stated date. Never an experiment, a review, or authority. (Chapters: Worker Protocol; Evidence, Obligations, and Program.)

Receipt laundering.: The threat of presenting structural receipts, self-hashes, or generated licenses as authority; historically instantiated by AI-authored self-hashed licensing receipts in the program's past. Blocked terminally by the constant-false invariant. (Chapter: Threat Model.)

Registry.: A byte-backed ledger of records (claim registry, theorem registry) loaded only through the validating loader, with recursively immutable values and byte-bound record digests. (Chapter: Registry Integrity.)

Rejection catalog.: The closed list of conditions under which the registry loader refuses input: duplicate JSON keys, unknown root or record fields, wrong enums or shapes, invalid identifiers, ambiguous versions, missing required metadata. Exhaustively fixtured. (Chapters: Registry Integrity; Reference Tables.)

Replay requirement.: Protocol step eight: fresh re-execution against the same source and compiler identity is required before any structural decision can pass; stored receipts alone satisfy nothing. (Chapters: Worker Protocol; Threat Model, replay evasion.)

Resolved evidence.: The aspirational cross-context form of evidence: local records glued into a statement consumable beyond their contexts. Requires restriction and gluing definitions that remain open topos-side work; nothing in the lane is resolved in v0.1. (Chapter: Integration Contracts.)

Review matrix / review criteria.: The enumerated docket of what independent review must examine: 53 criteria at the Gate-4 receipt, 20 open, with author-closed criteria reopening under independence. (Chapter: Evidence, Obligations, and Program.)

Ring 4.: The program's research-candidate status stratum: admission OPEN, product NOT LICENSED, disclosure withheld. Every artifact in this monograph carries it. (Chapters: The Research Boundary; all boundary statements.)

Self-digest.: A digest an object carries over its own content; detects post-construction mutation. Explicitly not a signature, reviewer identity, or authority grant. (Chapters: The Research Boundary; Threat Model, receipt laundering.)

Semantic reference.: The five-field structural tuple (namespace, `local_id`, version, `source_id`, `definition_sha256`) whose full-tuple structural equality is the kernel's only identity. (Chapter: Semantic Identity.)

Semigroupoid path (`SemigroupoidPath`).: A nonempty finite sequence of finite arrows with source/target agreement checked at every consecutive pair; the bridge's composition witness. No identity arrows or total composition are assumed. (Chapter: Integration Contracts.)

Structural consistency.: The property that an artifact's fields, digests, and boundary all validate. Never authorization: the kernel's central distinction. (Chapters: Runtime Decision Model; Threat Model.)

Structural decision.: A decision-model outcome asserting replay-verified compatibility for nonclaim research evidence; the strongest verdict the v0.1 kernel can issue. (Chapter: Runtime Decision Model.)

Suite test-ID hash.: The SHA-256 over the sorted, fully qualified identifiers of a suite's tests; binds a receipt to an exact test set rather than a count. The focused Core suite's is `E2AF5014DE28DD0702211EF32A2757B99B869A13FAFBB0B8D1B18A50A9F8FEF5`. (Chapter: Evidence, Obligations, and Program.)

Tagged decimal.: The encoder's representation of exact decimal quantities, tagged so that Decimal(`"1.0"`), the string `"1.0"`, and the integer 1 encode pairwise distinctly, defeating type puns. (Chapter: Canonical Evidence Encoding.)

Theorem registry.: The byte-backed ledger of formal-result records, governed by the same loader discipline as the claim registry. (Chapter: Registry Integrity.)

TKS v8.0.: Companion candidate carrying TKS-side formal content; supplies and reviews the arrow catalogs consumed by the fractal bridge. (Chapter: Integration Contracts.)

Translation link.: The link kind recording a reviewed cross-namespace rendering of one ontology's element in another's terms; the strongest representable cross-namespace statement, still not identity. (Chapter: Semantic Identity.)

Trust root.: The authenticated basis — authority, reviewer identities, key custody, revocation practice — that authorization would require and that v0.1 deliberately lacks; its absence is why the kernel's answer is constant *no*. (Chapters: Runtime Decision Model; Evidence, Obligations, and Program.)

Type pun.: A cross-type encoding collision (e.g. a Decimal, string, and integer sharing bytes) that would let different typed facts become digest-identical; excluded by tagged encoding. (Chapters: Canonical Evidence Encoding; Threat Model.)

Worker.: The fresh `python -I -S -B` process with sanitized environment in which verified compiler bytes execute; the unit of Python-process isolation. (Chapter: Worker Protocol.)

Worker runtime binding.: The pre-execution sampling of interpreter fields, platform data, and profile-bound architecture inputs into the receipt, so the receipt records the runtime that actually executed. (Chapter: Worker Protocol.)

Bibliography

[1] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, 2020.

[2] W3C, *PROV-O: The PROV Ontology*, 2013.

[3] C. Lamb and S. Zacchiroli, "Reproducible builds: increasing the integrity of software supply chains," *IEEE Software* 39(2), 2022.

[4] R. Anderson, *Security Engineering: A Guide to Building Dependable Distributed Systems*, 3rd ed., Wiley, 2020.

[5] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014.

[6] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," *USENIX Security*, 2019.

[7] Open Source Security Foundation, *SLSA: Supply-chain Levels for Software Artifacts*, Version 1.0, 2023, https://slsa.dev/spec/v1.0/.

[8] Z. Newman, J. Speed Meyers, and S. Torres-Arias, "Sigstore: software signing for everybody," *ACM CCS*, 2022.